STUDY TEXTBOOK

# BOOK TITLE

*Subtitle*

First edition · 10 October 2026

# Contents

**Start here: from zero** **xi**

**1 Electricity, circuits and the workbench** **1**

1.1 What electricity is . . . 2
1.2 Current . . . 3
1.3 Resistance and the resistor . . . 3
1.4 A complete circuit — and the schematic as the real object . . . 5
1.5 Two conventions for direction . . . 7
1.6 The multimeter . . . 7
1.7 The workbench kit and the budget . . . 9
1.8 Worked example: how long does the battery last? . . . 11
1.9 Safety: what this book will and will not have you touch . . . 12
1.10 Common mistakes . . . 13
1.11 Chapter summary . . . 13
Exercises . . . 14
Notes . . . 15

**2 Ohm's law, power and the simulator** **17**

2.1 Ohm's law: a definition with a job . . . 18
2.2 Power: $P = VI$ and the quarter-watt ceiling . . . 19
2.3 When the line bends: nonlinear loads . . . 20
2.4 Resistor networks, concept only . . . 21
2.5 Reading a datasheet's headline numbers . . . 21
2.6 Your first ngspice netlist . . . 22
2.7 Why we simulate before we build . . . 24
2.8 Safety . . . 26
2.9 Common mistakes . . . 26
2.10 Chapter summary . . . 27

Exercises . . . 27
Notes . . . 28

**3 Series, parallel and Kirchhoff's laws** **30**
3.1 Series and parallel are about nodes . . . 31
3.2 Equivalent resistance . . . 31
3.3 The voltage divider . . . 32
3.4 The current divider . . . 33
3.5 KCL and KVL: bookkeeping, signed . . . 33
3.6 The node-voltage method . . . 34
3.7 Thévenin and Norton: networks compressed . . . 35
3.8 Measuring without breaking the circuit . . . 37
3.9 Common mistakes . . . 37
3.10 Chapter summary . . . 38
Exercises . . . 38
Hints and answers . . . 39
Notes . . . 40

**4 Capacitors, inductors and RC/RL transients** **41**
4.1 The capacitor: charge bucket, $i = C\,dv/dt$ . . . 42
4.2 The RC step response . . . 42
4.3 Charging vs discharging . . . 44
4.4 The inductor: the dual element . . . 45
4.5 RL transients and the flyback spike . . . 45
4.6 Where transients hide . . . 46
4.7 $\star$ Driven RLC: why things ring . . . 48
4.8 Safety notes . . . 48
4.9 Common mistakes . . . 48
4.10 Chapter summary . . . 49
Exercises . . . 50
Hints and answers . . . 51
Notes . . . 51

**5 Diodes** **52**
5.1 The junction diode: one curve, three models . . . 53

5.2 LEDs and forward voltage . . . 54
5.3 The rectifier: AC in, lumpy DC out . . . 54
5.4 The flyback diode: ch. 4's missing part . . . 56
5.5 The zener and the shunt regulator . . . 57
5.6 The diode in the wild: ESD and reverse-polarity guards . . . 59
5.7 Safety . . . 60
5.8 Common mistakes . . . 60
5.9 Summary . . . 61
Exercises . . . 61
Hints and answers . . . 62
Notes . . . 62
Notes . . . 62

**6 Transistors: the switch and the amplifier 64**
6.1 The MOSFET: a voltage-controlled switch . . . 65
6.2 The NMOS inverter, built by hand . . . 66
6.3 The static discipline: what makes a level a "1" . . . 67
6.4 CMOS: the pair that draws no power at rest . . . 67
6.5 The BJT: the current-controlled cousin . . . 69
6.6 The transistor as amplifier . . . 69
6.7 Driving a load: the switch, for real . . . 69
6.8 Logic families and voltage levels . . . 70
6.9 Common mistakes . . . 71
6.10 Summary . . . 72

**7 Op-amps and power supplies 74**
7.1 The op-amp contract . . . 74
7.2 The three circuits that cover 90% of use . . . 75
7.3 Worked example 1: buffering a weak sensor . . . 76
7.4 The comparator . . . 77
7.5 Worked example 2: gain of 10 — and the ceiling . . . 77
7.6 Worked example 3: a comparator that never rests . . . 79
7.7 Where op-amps bite . . . 80
7.8 Regulators: spending volts cleanly . . . 80
7.9 USB power, and decoupling revisited . . . 81

7.10 Mains power: why it stays boxed . . . 81
7.11 Common mistakes . . . 82
7.12 Exercises . . . 82
7.13 Summary . . . 83

**8 Numbers, bits and Boolean algebra 85**
8.1 Why two levels? . . . 86
8.2 Binary, hex, and two's complement . . . 86
8.3 Truth tables . . . 86
8.4 De Morgan and the universal gates . . . 87
8.5 Worked example 1: signed byte arithmetic, three ways . . . 89
8.6 Worked example 2: prove a simplification twice . . . 89
8.7 Worked example 3: the alarm circuit . . . 90
8.8 From expression to network . . . 91
8.9 Common mistakes . . . 92
8.10 Exercises . . . 92
8.11 Summary . . . 93

**9 Combinational circuits 94**
9.1 Multiplexers and decoders: the two routers . . . 94
9.2 Worked example 1: a 4:1 mux, gates then `assign` . . . 95
9.3 Adders: half, full, and ripple-carry . . . 95
9.4 Worked example 2: counting the delay . . . 96
9.5 Comparators, encoders, priority . . . 97
9.6 Worked example 3: seven-segment decoder . . . 97
9.7 The ALU preview and the road to a bus . . . 98
9.8 Common mistakes . . . 99
9.9 Exercises . . . 99
9.10 Summary . . . 100

**10 Sequential circuits and state machines 102**
10.1 Feedback and memory . . . 102
10.2 Worked example 1: the latch bug, live . . . 103
10.3 Setup, hold, registers, buses . . . 104
10.4 Counters . . . 104

10.5 Worked example 2: a 1 Hz tick from 27 MHz . . . 105
10.6 Finite state machines . . . 105
10.7 Worked example 3: the traffic-light FSM, end to end . . . 106
10.8 Worked example 4: Mealy answers a cycle early . . . 107
10.9 Worked example 5: the “101” detector, end to end . . . 107
10.10 Metastability and the two-flop rule . . . 108
10.11 Common mistakes . . . 108
10.12 Exercises . . . 109
10.13 Summary . . . 110

**11 Verilog I: describing hardware** **112**
11.1 HDL is not a program . . . 112
11.2 Modules, ports, wires and regs . . . 113
11.3 `assign`: combinational logic, permanently . . . 114
11.4 The two `always` recipes . . . 114
11.5 Blocking = vs non-blocking <= . . . 114
11.6 Parameters and a generate peek . . . 115
11.7 The testbench is a module too . . . 115
11.8 Icarus + GTKWave: the loop . . . 115
11.9 Reading a waveform . . . 116
11.10 Worked example 1: one mux, four legal ways and two wrong . . . 116
11.11 Worked example 2: a counter with a self-checking testbench . . . 117
11.12 Worked example 3: the traffic FSM, re-coded in this chapter’s style . . . 117
11.13 Worked example 4: the pipeline that ate a flop . . . 119
11.14 Worked example 5: one name, N flip-flops . . . 119
11.15 Summary . . . 120
11.16 Common mistakes . . . 120
Additional exercises . . . 121
Hints and answers . . . 122
Further reading . . . 122
Notes . . . 123
Notes . . . 123

**12 Verilog II: testbenches and the classic bugs** **124**
12.1 The anatomy of a clocked testbench . . . 125
12.2 $display, $finish and the severity family . . . 125
12.3 Worked example 1: a bench that reports, not just prints . . . 126
12.4 The classic bugs . . . 127
12.5 Worked example 2: the x that never leaves . . . 127
12.6 Worked example 3: the case that forgot a branch . . . 128
12.7 The race you cannot see . . . 129
12.8 Worked example 4: the same flip-flop, two benches . . . 129
12.9 Blocking in sequential, one line . . . 131
12.10 Self-checking patterns . . . 131
12.11 Worked example 5: the golden model catches a 1-bit bug . . . 131
12.12 Reading the wave . . . 132
12.13 Summary . . . 133
12.14 Common mistakes . . . 133
Hints and answers . . . 135
Further reading . . . 136
Notes . . . 137
Notes . . . 137

**13 The FPGA flow: from text to silicon** **138**
13.1 What an FPGA is . . . 139
13.2 The toolchain: five stages . . . 140
13.3 Hello, LED: blinky . . . 141
13.4 Counters, buses, and real area . . . 142
13.5 UART: talking to the outside world . . . 143
13.6 Constraints: telling the tools the truth . . . 144
13.7 On real hardware — and what we could not do here . . . 145
13.8 Common mistakes . . . 146
13.9 Summary . . . 149
Notes . . . 149

**14 An 8-bit ALU** **150**
14.1 What an ALU is, and the contract . . . 151
14.2 Choosing the operations . . . 151

14.3 Flags: semantics before code . . . 152
14.4 The design: compute all answers, then pick one . . . 153
14.5 The ALU in Verilog . . . 154
14.6 The testbench: vectors plus flags . . . 155
14.7 Onto the board . . . 157
14.8 Common mistakes . . . 158
14.9 Summary . . . 159
Additional exercises . . . 159
Hints and answers . . . 161
Further reading . . . 162
Notes . . . 163

**15 Registers, buses and the datapath** **164**
15.1 The register file . . . 165
15.2 The bus, and who may drive it . . . 166
15.3 Memory: where the program lives . . . 166
15.4 Datapath versus control . . . 169
15.5 The cycle: fetch, execute, write back . . . 169
15.6 Common mistakes . . . 172
15.7 Summary . . . 173
Additional exercises . . . 173
Hints and answers . . . 174
Further reading . . . 175
Notes . . . 176

**16 A small CPU that runs real programs** **177**
16.1 The ISA, frozen . . . 178
16.2 Fetch, decode, execute — in one cycle . . . 178
16.3 The decoder: a case statement with a job title . . . 180
16.4 The assembler . . . 181
16.5 Out the door: LEDs and UART . . . 182
16.6 What this CPU is missing . . . 183
16.7 Common mistakes . . . 184
16.8 Summary . . . 185
Additional exercises . . . 186

Hints and answers . . . 187
Further reading . . . 188
Notes . . . 189

**17 Breadboards and the real bench 190**
17.1 Anatomy: what the holes do . . . 190
17.2 Wiring that doesn't lie . . . 191
17.3 Three builds, honestly measured . . . 191
17.4 The debugging method . . . 193
17.5 Where breadboards stop being honest . . . 195
17.6 Common mistakes . . . 195
17.7 Summary . . . 196
Additional exercises . . . 197
Hints and answers . . . 198
Further reading . . . 199
Notes . . . 200

**18 KiCad: from schematic to a two-layer sensor board 201**
18.1 The pipeline . . . 202
18.2 The board spec . . . 202
18.3 Footprints: value versus body . . . 203
18.4 The two-layer stack . . . 204
18.5 Fab files: the Gerber contract . . . 205
18.6 Common mistakes . . . 206
18.7 Summary . . . 207
Additional exercises . . . 207
Hints and answers . . . 208
Further reading . . . 210
Notes . . . 210

**19 Microcontrollers and the road ahead 211**
19.1 An MCU is the same idea, shrink-wrapped . . . 211
19.2 The honest decision . . . 212
19.3 The sensor board, with an MCU . . . 213
19.4 What Arduino hides . . . 213

19.5 Soft cores: the CPU inside the FPGA . . . . . 214
19.6 The road ahead . . . . . 215
19.7 Common mistakes . . . . . 215
19.8 Summary . . . . . 216
Additional exercises . . . . . 217
Hints and answers . . . . . 217
Further reading . . . . . 218
Notes . . . . . 219

# Start here: from zero

You do not need to have built a circuit, studied physics, or touched a soldering iron. This book starts with what electricity *is* and ends with a small CPU you design yourself. What it does use is a little arithmetic with very large and very small numbers, so this chapter covers that, then shows how the book is laid out.

## Powers of ten and the prefixes you will see

Electronics uses numbers from billionths to millions, so we write them with a prefix. Memorise this table; it appears on every page of the book.

| Prefix | Symbol | Multiply by |
|---|---|---|
| mega | M | $10^{6}$ (one million) |
| kilo | k | $10^{3}$ (one thousand) |
| milli | m | $10^{-3}$ (one thousandth) |
| micro | $\mu$ | $10^{-6}$ (one millionth) |
| nano | n | $10^{-9}$ |
| pico | p | $10^{-12}$ |

So $4.7\,\mathrm{k\Omega} = 4700\,\Omega$, and $2.2\,\mu\mathrm{F} = 2.2 \times 10^{-6}\,\mathrm{F}$. Mixing prefixes is the commonest source of silly errors: 1 mA is 0.001 A, not 1000 A. Convert to base units (volts, amperes, ohms, farads) before you calculate, and convert back at the end.

## Rearranging a formula

Almost every calculation here is one formula used three ways. Take Ohm's law (Chapter 2), $V = IR$. If you want the current, divide both sides by $R$: $I = V/R$. For $V = 5\,\mathrm{V}$ and $R = 1\,\mathrm{k\Omega} = 1000\,\Omega$ this gives $I = 5/1000 = 0.005\,\mathrm{A} = 5\,\mathrm{mA}$. If you want the resistance, divide by $I$ instead: $R = V/I$. Whatever you do to one side of an equation, do to the other, and keep the units attached: volts divided by ohms gives amperes.

## A word about safety

Everything in the first half of the book runs on batteries and USB power, and every chapter says clearly what it will and will not ask you to touch. Mains electricity (the wall socket) can kill, and nothing in this book asks you to open or modify anything connected to it. Chapter 1 ends with the full safety statement; read it before you pick up a tool.

## How the books of this series fit together

| Book | Connection |
|---|---|
| 8 Physics for engineers | Explains the *why* behind the analogue chapters (fields, capacitance, induction); its "In engineering practice" sections point back here. |
| 7 Systems | Picks up exactly where Chapter 16 here ends: the custom 8-bit CPU is mapped to RISC-V and extended to a pipelined core. |
| 2 Discrete maths | The logic of Chapter 8 here (Boolean algebra) is the same as the logic in Book 2, Chapter 1. |

## How to use the boxes in each chapter

Each chapter opens with *After this chapter you can*, listing what you will be able to do. From Chapter 2, a *Before you start* box lists the chapters it relies on with a one-sentence recap. Chapters 1–7 are analogue (voltages and currents that vary smoothly); from Chapter 8 on, everything is digital (ones and zeros). You can start at Chapter 8 if you only care about the CPU, but Chapters 1–3 make the rest easier.

CHAPTER 1

# Electricity, circuits and the workbench

This chapter builds the vocabulary and the bench habits that every later chapter takes for granted. It answers four questions: what electricity *is* (as much as an engineer needs to know), what voltage, current and resistance measure, how to read a schematic and turn it into a shopping list, and which of the things on a workbench can hurt you. When you finish it you should be able to: translate a simple schematic into parts and measurements, use a multimeter for the three basic measurements without blowing its fuse, say precisely what this book will and will not ask you to touch, and check whether your computer already has the tools this book needs.

**Why this exists.** A processor, a phone charger and a sensor board are all circuits, and they all start from the same few ideas: voltage, current and resistance. If you want to design digital hardware or a CPU later in this book, you first need a working picture of what a wire and a battery actually do, and the habits (measure first, power off before changing anything) that keep both you and your parts safe.

**BEFORE YOU START**

Nothing. You do not need to have built a circuit or studied physics. The *Start here* chapter lists the arithmetic you will use (powers of ten, SI prefixes such as milli and kilo, and rearranging a formula).

**AFTER THIS CHAPTER YOU CAN**

- explain in plain words what voltage, current and resistance are;
- read a simple schematic and turn it into a parts list;
- use a multimeter for voltage, current and resistance without damaging it or yourself;
- work out how long a battery will last from its capacity and the load.

**HOW TO THINK ABOUT THIS**

Electricity is easier to understand if you picture water in pipes: voltage is the pressure, current is the flow, and resistance is how narrow the pipe is. The picture is not perfect, but it gives the right instincts. Before you touch a circuit, ask three questions: where does the current come from, where does it go, and what limits it?

## 1.1 What electricity is

Strip everything away and electricity starts with one observation: objects can push and pull on each other without touching. Rub a balloon on your hair and it will pick up scraps of paper from a table. Something invisible reached out and grabbed them. That something is *charge*: a basic property of matter, like mass, that some particles carry and others do not. Charge comes in two flavours that attract each other and repel their own kind, which we call positive and negative — the names are a bookkeeping device, not a value judgment. The electron carries one fixed small negative charge; the proton carries the same amount positive. That fixed amount, $e \approx 1.602 \times 10^{-19}$ coulombs, is the smallest packet of free charge we ever deal with. Ordinary matter contains roughly equal numbers of protons and electrons and so looks neutral; "charging" an object means unbalancing that equality slightly.

A charge creates a disturbance in the space around it — an *electric field* — and other charges feel that disturbance as a force. We almost never draw the field itself in this book, but it is what makes everything else work, so we define the quantity that packages it: *voltage* (also called *potential difference*). Voltage is the field's push measured as energy: the voltage between two points is the energy needed to move one unit of charge from one to the other,

$$V = \frac{W}{q}, \qquad 1 \text{ volt} = 1 \text{ joule per coulomb}.$$

A "9 V battery" is a device that gives each coulomb of charge 9 J of push between its terminals. Voltage is always measured *between two points*; the phrase "the voltage at this wire" is shorthand for "the voltage between this wire and whatever we agreed to call zero," and that agreed-zero is the *ground* node we draw at the bottom of every schematic.

*Remark* (What we are skipping). That paragraph is honest but thin. The full story is electromagnetic field theory — fields propagate, wires are not the whole circuit, and at high frequencies our tidy model breaks. For the circuits in this book (centimetres across, signals slower than tens of megahertz) the *lumped* model — every property lives inside components connected by ideal wires — is accurate to far better than the tolerance of any part you can buy. Where it stops being true (radio, very fast digital edges, mains safety) we say so at the point it matters rather than here. If you want the real derivations, a free calculus-based physics text covers charge, field and potential properly [5].

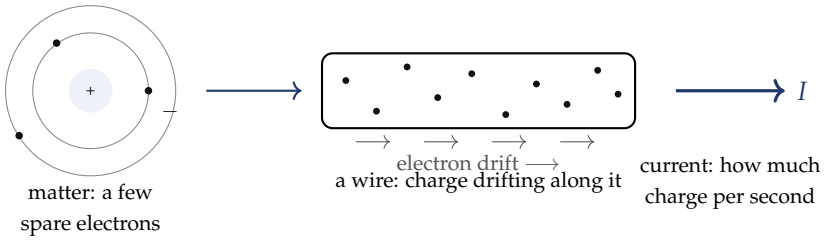



**Figure 1.1:** Read left to right: atoms are mostly neutral; a metal wire contains electrons that are free to drift; a battery (the field source) makes them drift in one direction. That drift *is* the current. Notice what is *not* shown: the battery does not inject electrons into an empty pipe — the electrons are already in the wire; the source only organises their motion.

## 1.2 Current

A metal wire is full of electrons that belong to no particular atom and drift randomly, as much left as right: no net motion, no current. Put a source across the wire — a field along it — and the random drift gains a small organised component in one direction. That net flow of charge is *current*:

$$I = \frac{\Delta Q}{\Delta t}, \qquad 1 \text{ ampere} = 1 \text{ coulomb per second}.$$

One amp is roughly $6 \times 10^{18}$ electrons per second squeezing through a cross-section of wire. You never count electrons; the meter does it for you by watching the field they drag with them.

It is worth calibrating the unit with real numbers. The LED we light in this chapter runs at about 20 mA. A phone charger supplies 1 A to 2 A. The LED strips in cheap room lighting draw a few hundred milliamps. A bolt of lightning peaks in the tens of kiloamps — which is why "it is only a little circuit" is a sentence that needs a voltage and a current attached before it means anything.

Two conventions matter immediately. First, current is a *signed* quantity with an arrow: we pick a direction for the arrow, compute or measure $I$, and a negative answer means the flow goes opposite to our arrow. Second — and this trips up every beginner — the arrow points the way *positive* charge would move, which is opposite to the actual electron drift. That historical quirk is Section 1.5's whole topic; for now just note that the arrow in Figure 1.3 points the engineering way.

## 1.3 Resistance and the resistor

Push charge through almost any material and it meets drag: the moving electrons scatter off the lattice and hand their energy back as heat. The measure of that drag is *resistance*,

$$R = \frac{V}{I}, \qquad 1 \text{ ohm} = 1 \text{ volt per ampere}.$$

A material with twice the resistance needs twice the voltage to push the same current. Copper wire has nearly none (milliohms per metre); the carbon film inside a quarter-watt *resistor* — the single most common component in this book — is manufactured to a precise value from fractions of an ohm to tens of megohms.

The relationship $V = I\,R$ is *Ohm's law*: one equation linking all three quantities, so that knowing any two fixes the third. We use it lightly this chapter (to estimate a battery's life and to *pose* the problem in Example 1.2), and Chapter 2 turns it into a working tool.

*Remark.* "Ohm's law" is really an empirical pattern, not a law of nature — it holds beautifully for resistors and not at all for LEDs, transistors or arcs. The components that break it are exactly the interesting ones; that is Chapters 5 and 6.

Resistor values are printed as coloured bands — a holdover from when components were too small to label. You need only the four-band reading: first two bands are digits, the third

| Colour | Digit (multiplier $10^n$) |
|---|---|
| Black | 0 |
| Brown | 1 |
| Red | 2 |
| Orange | 3 |
| Yellow | 4 |
| Green | 5 |
| Blue | 6 |
| Violet | 7 |
| Grey | 8 |
| White | 9 |
| Gold (4th band) | tolerance $\pm 5\%$ |
| Silver (4th band) | tolerance $\pm 10\%$ |

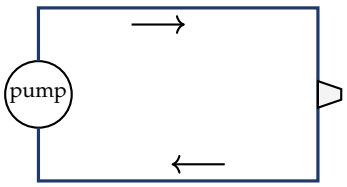
**Table 1.1:** The only colour-code facts this book needs: digit values and the two common tolerance bands. Brown–black–red–gold $= 1, 0, \times 10^2, \pm 5\% = 1\,\text{k}\Omega$. Notice you read *toward* the gap between the digit bands and the spaced-off tolerance band.

is the multiplier (power of ten), the fourth is tolerance. Table 1.1 gives the digits; Exercise 1.5 drills it.

Values span so many orders of magnitude that electronics runs on *engineering prefixes* in steps of $10^3$: milli m ($10^{-3}$), micro µ ("u" in plain ASCII, $10^{-6}$), nano n ($10^{-9}$), pico p ($10^{-12}$), kilo k ($10^3$), mega M ($10^6$). A "4k7" resistor is $4.7\,\text{k}\Omega = 4700\,\Omega$; a "100 nF" capacitor is $100\,\text{nF} = 0.1\,\mu\text{F}$. Datasheets, schematics and meters each pick different prefixes, so fluency is not optional. The chapter ships a tiny converter (Listing 1.3) you can check your mental arithmetic against.

**Exercise 1.1** (easy)**.** Convert: 0.05 A to milliamps; $4.7\,\text{k}\Omega$ to ohms; 0.000 000 1 F to nanofarads. Then run `prefix.py` and check.

Before leaving the three quantities, one deserved warning about how you will hear them explained elsewhere. The *water analogy* — voltage is pressure, current is flow, a resistor is a narrow pipe — is the standard pictorial model, and it is not useless: it correctly predicts that more pressure drives more flow and that pinching the pipe reduces it. Figure 1.2 puts it beside the honest version. It fails in exactly the places where intuition matters most (a cut wire does not "leak" current onto the bench; a battery does not inject anything) — which is why this book uses it as a first sketch and then abandons it. Exercise 1.12 asks you to catalogue the failures properly.




pump ≈ source
pressure ≈ voltage
constriction ≈ resistor
flow ≈ current

*the analogy*

**Where it breaks:** cut a pipe and the water pours out; cut a wire and charge simply stops. There is no "leak" of charge, nothing pushes sideways, and the pressure exists *between* two points, not inside one point. Keep the picture for intuition in this chapter; drop it the moment it argues back.

**Figure 1.2:** The water analogy, kept honest. Left: the picture as usually drawn (pump = battery, pipe = wire, constriction = resistor). Right: what each piece actually is — pressure is energy per charge, flow is charge per second, the "constriction" is energy lost to lattice collisions. The analogy survives Ohm's-law arithmetic and dies at the physics: a cut pipe leaks, a cut wire just stops.

## 1.4 A complete circuit — and the schematic as the real object

Current flows only in loops. Three things must exist for it to do so: a *source* that provides the push (a battery, a USB port, a power supply), a *conducting path* that carries the flow (wire, plus the components in the way), and a route *back to the source*. Break the loop anywhere and the current stops everywhere — instantly, throughout the whole loop. That is the entire operating principle of a switch, and it is the reason one broken wire can kill a whole circuit.

Figure 1.3 shows the smallest circuit that does something useful: a 5 V source, a resistor, and a light-emitting diode (LED). The schematic — the drawn map of symbols and wires — is not an illustration of the circuit; in every practical sense it *is* the circuit. The copper and plastic are just a heavy, solderable printout of the schematic. This is why we insist all circuits in this book be drawn properly (with `circuitikz`) rather than photographed.

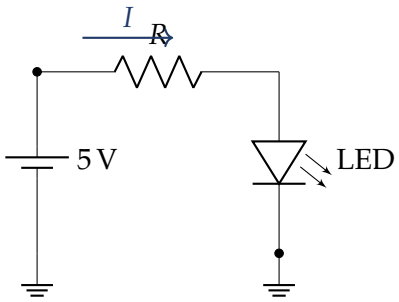
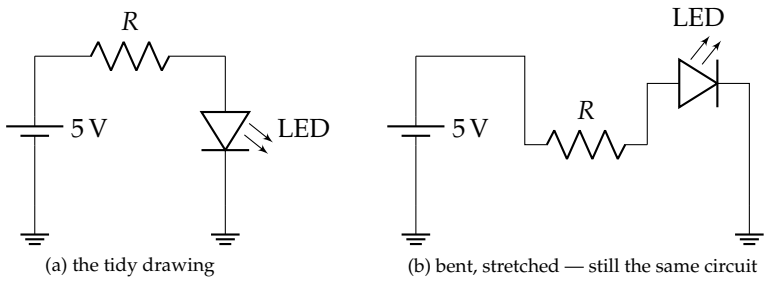



**Figure 1.3:** Our first real schematic. Read it as a loop, not a picture: leave the top of the source at +5 V, pass through $R$ (which will limit the current), through the LED (which lights if the current is right), down to the ground symbol and back into the bottom of the source. The arrow marks the *chosen* current direction (Section 1.5). Notice the LED symbol: diode triangle plus two small arrows = "the diode that emits."

Here is the subtle point beginners miss. The schematic records *connections*, not *locations*. Two points joined by a drawn wire (or marked with the same net name) are the same electrical object — the same *node*. Points merely near each other on paper are nothing. Figure 1.4

draws the same circuit twice: tidy, and stretched into an ugly pose. Electrically they are identical, because every node contains the same component leads. Reading schematics as *topology* — who is connected to whom, never where they sit — is the skill this section exists to install. We will reuse it mercilessly when schematics get big in Chapter 17.




(a) the tidy drawing (b) bent, stretched — still the same circuit

**Figure 1.4:** Same circuit, two drawings. To prove (a) ≡ (b), list what is attached to each node: node A holds source-+ and resistor lead 1; node B holds resistor lead 2 and LED anode; node C holds LED cathode and source−. The lists match, so the circuits are the same. The drawing changed; the *connections* — the only thing the circuit cares about — did not.

**Example 1.1** (From schematic to shopping list)**.** *Problem.* Turn Figure 1.3 into a parts order.

*Strategy.* Walk the schematic component by component, not wire by wire. For each symbol write the thing to buy and the value or rating written beside it. Wires and ground symbols buy nothing — they come free with the breadboard and cables.

*Solution.*

- one 5 V source — a USB cable or a 5 V supply;
- one resistor, value $R$ — value still undetermined (Example 1.2 is about exactly this gap);
- one LED, any colour, standard 5 mm or 3 mm package;
- means of connecting: breadboard + jumper wires (Section 1.7).

*Notice:* the schematic already told us everything except one number — which is the kind of gap a circuit diagram is supposed to leave for analysis.

**Example 1.2** (The LED question, posed on purpose)**.** *Problem.* The LED in Figure 1.3 wants about 2 V across itself and shines nicely at 10 mA, burns at 30 mA. The source is 5 V. What should $R$ be?

*Strategy.* Take stock of what each quantity does. The source fixes the total push: 5 V. The LED spends about 2 V of it (a fact of diode physics we take on trust until Chapter 5). The resistor must drop the rest, and the same current must pass through everything in a single loop. So the missing ingredient is a law that links the resistor's voltage share to the current through it — which is $V = IR$. We *have* that law already; what we are missing is practice using it and one bookkeeping rule (Kirchhoff's voltage law) that makes "the rest" precise.

*Solution.* Held for Chapter 2, where it becomes a one-line computation. The point of posing it now: you can already see that $R$ exists to dispose of a voltage difference and to choose a current — the two jobs resistors do all book long.

## 1.5 Two conventions for direction

Benjamin Franklin guessed the sign of charge wrongly, and by the time electrons were identified the convention was set: circuit arrows point the way *positive* charge would flow — *conventional current*. Electrons, being negative, drift the other way — *electron flow*.

Nothing about this is optional to get right, because everyone — every datasheet, every schematic, every simulator — uses conventional current. When someone (or a YouTube video) insists on electron flow, mentally flip every arrow; the numbers and the physics are unaffected. The failure mode is not choosing the "wrong" convention but switching conventions *mid-argument*: if you label an arrow one way and reason about electrons the other way, signs invert silently and the sums stop making sense. Pick conventional current and stay there; this book never leaves it.

**ARROWS ARE REFERENCE DIRECTIONS**

Draw the arrow whichever way is convenient. A positive computed value means flow along your arrow; negative means against it. You do not have to guess the "true" direction first — the sign will tell you.

## 1.6 The multimeter

The *multimeter* (or DMM, for digital multimeter) is the one instrument this book treats as essential. It measures the three quantities of Sections 1–3 and nothing else we need. Everything else in electronics is learned through it. Figure 1.5 shows a typical unit.

The three measurements differ in a way that matters physically, and the difference explains every DMM accident:

- **Voltage** is a *difference between two points*. To measure it you touch the two probes to the two points — the meter is in *parallel* with whatever is there. The circuit does not change; you added an observer, not a wire. Inside the meter is a very large resistance ($\sim 10\,\mathrm{M\Omega}$), so almost no current diverts into it. Voltage measurement is the safe, always-allowed move.
- **Current** is a *flow through a point*. To measure it you must put the meter *into* the path — break the circuit and bridge the gap with the probes, so all the circuit's current passes through the meter. Inside it is a very *small* resistance (a shunt). This is the measurement that changes the circuit and the one beginners fry meters with.
- **Resistance** is measured with the circuit *unpowered*: the meter sends out its own small test current and reads the resulting voltage. Measuring resistance on a live circuit gives wrong numbers at best.

**THE FUSE-BLOWER**

Red probe in the current jack + probes across a voltage source = a short circuit through the meter's (very small) input resistance. On most meters the mA jack is the fused one — a sacrificial fuse dies for your mistake; the 10 A jack is typically *unfused*, so the same

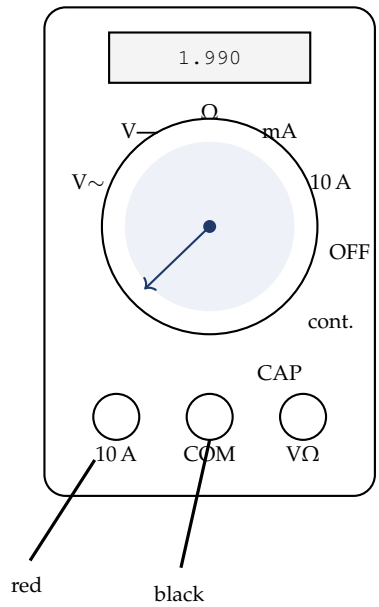



**Figure 1.5:** A typical multimeter. Three things to notice. (1) The dial divides the world into ranges: V ~ (AC volts — never used in this book), V- (DC volts), Ω (resistance), mA (current), 10 A (the big current range), plus extras (continuity beeper, capacitance). (2) Three jacks: COM is where the black probe always lives; VΩ is red for volts, ohms and small currents; 10 A is a separate, typically *unfused* path for large currents. (3) The screen reads the number *in the selected range's units* — "1.990" on the 2 V range means 1.990 V, on the mA range 1.990 mA.

mistake there can kill the meter itself, not to mention the circuit you shorted. Make a habit: *after* a current measurement, the red probe goes back to VΩ before anything else is touched. We will never again let it live in the current jack when idle.

**"OL" IS AN ANSWER**

On a resistance or voltage range, an "OL"/"1 " display means *over-range*, not zero and not broken: the value exceeds what that range can show (or the path is genuinely open). On the mA current jack by contrast, "OL" with a live circuit almost always means the range's fuse blew — that fuse exists precisely for this accident. Same screen, opposite meanings — know which.

**Checkpoint 1.1** *Predict before you measure.* A fresh 9 V battery reads ~9.4–9.6 V on a meter, not 9.000. An "empty" one reads ~7 V. *Before* every measurement this book asks you to make, write down your expected number and its range. If the meter disagrees wildly, the mistake is in the setup, not the meter — check the dial and the jacks first.

**Exercise 1.2** (medium)**.** You want to measure the current through the LED of Figure 1.3. State exactly what you physically change in the circuit, which range and jacks you use, and where each probe tip lands.

## 1.7 The workbench kit and the budget

This book is built to need almost nothing bought. Every analogue circuit is simulated in ngspice before (or instead of) being built; the entire digital half runs in simulation on free tools; the one hard purchase — a Tang Nano 9K FPGA board — is needed only from Chapter 13 and costs about US$20. But a minimum physical kit makes several chapters concrete, so Table 1.2 lists it with honest prices.

| Item | What it is for | Budget (USD) |
|---|---|---|
| Multimeter | every chapter's measurements; buy auto-ranging, fused | $15–30 |
| Breadboard + jumpers | solderless builds, ch17–19 | $5–8 |
| Resistor assortment (E24) | every analogue build | $4–6 |
| LEDs, a few colours | ch2, ch5, board bring-up | $2–4 |
| USB cable / 5 V supply | power for bench circuits | $3–5 |
| 9 V battery + clip | the battery-life experiment below | $2–3 |
| **Physical kit subtotal** | | **$31–56** |
| Tang Nano 9K (Sipeed, GW1NR-LV9) | FPGA chapters 13–16, required then | $18–23 [9] |

**Table 1.2:** The shopping list as a figure: what a minimal bench costs in 2026. Notice the split — the top block is a *recommended* bench; the FPGA board is the only *required* purchase, and only from Chapter 13. Indonesian availability/price for the board is marked unverified in the sources log; the global street price is US$18–23 [9].

Two things about this list deserve saying. First, a simulator never burns fingers and never needs resoldering — every analogue circuit in this book comes with an ngspice run you can do for free, and the experiments are structured so hardware is always a checkpoint, not a gate. Second, the board matters later: the custom CPU you design in Chapters 14–16 runs on real silicon, and the difference between "simulated" and "it blinks on a board" is the whole reason Part V exists.

The tools, unlike the kit, must be checked on your computer now. Listing 1.1 probes each command the book needs and reports what is installed; Listing 1.2 is its real output on a fresh machine — read the MISSING lines as a to-install list, not an error.

```
#!/bin/sh
# check_tools.sh -- verify the toolchain for this book is installed.
# Run: sh code/check_tools.sh
# Goal: every line says "found". Anything MISSING tells you what to
    install.

check() {
  # $1 = command to try, $2 = version flag, $3 = what it is for
  if command -v "$1" >/dev/null 2>&1; then
    ver=$("$1" $2 2>/dev/null | head -n 1)
    printf 'found %-22s %s\n' "$1" "$ver"
  else
    printf 'MISSING %-22s %s\n' "$1" "$3"
  fi
```

```
}

check iverilog -V "Verilog simulation (ch11+)"
check verilator --version "lint / fast simulation (ch12+)"
check yosys -V "synthesis (ch13+)"
check nextpnr-himbaechel --version "place-and-route, Gowin (ch13+)"
check gowin_pack --help "bitstream packer, Gowin (ch13+)"
check openFPGALoader --Version "programming the board (ch13+)"
check gtkwave --version "viewing waveforms (ch11+)"
check ngspice --version "analog simulation (ch2+)"
check kicad-cli version "PCB work (ch18)"
check python3 --version "scripts and vector generators"

echo "----"
echo "Missing tools come from the OSS CAD Suite (FPGA tools) and your"
echo "package manager (ngspice, KiCad). See code/README.md for
    versions."
```

**Listing 1.1:** `code/check_tools.sh` – reports which toolchain commands exist on your PATH.

```
MISSING iverilog Verilog simulation (ch11+)
MISSING verilator lint / fast simulation (ch12+)
MISSING yosys synthesis (ch13+)
MISSING nextpnr-himbaechel place-and-route, Gowin (ch13+)
MISSING gowin_pack bitstream packer, Gowin (ch13+)
MISSING openFPGALoader programming the board (ch13+)
MISSING gtkwave viewing waveforms (ch11+)
MISSING ngspice analog simulation (ch2+)
MISSING kicad-cli PCB work (ch18)
found python3 Python 3.9.6
----
Missing tools come from the OSS CAD Suite (FPGA tools) and your
package manager (ngspice, KiCad). See code/README.md for versions.
```

**Listing 1.2:** Real output of `check_tools.sh` on a machine before installing the toolchain. Every MISSING line names both the command and where it will first be needed.

The second small program converts between the engineering prefixes that meters and datasheets disagree about (Listing 1.3); its output is Listing 1.4. Neither program is mandatory — both exist so the chapter has real, runnable code like every other one.

```
#!/usr/bin/env python3
"""prefix.py -- metric-prefix converter used in Chapter 1 exercises.

Given a value like 0.0047 it prints engineering notation (4.7 m).
Why: datasheets and meters never agree on the prefix; being able to
shift between them fluently is the first practical skill.
"""

PREFIXES = [
    (1e9, "G"), (1e6, "M"), (1e3, "k"),
    (1.0, ""), (1e-3, "m"), (1e-6, "u"), (1e-9, "n"), (1e-12, "p"),
]
```

```
13
14 def to_eng(value):
15     """Format a plain number in engineering (prefix-aligned) notation."
       ""
16     if value == 0:
17         return "0"
18     for factor, sym in PREFIXES:
19         if abs(value) >= factor:
20             return f"{value / factor:g} {sym}".rstrip()
21     return f"{value:g}"
22
23 if __name__ == "__main__":
24     for v in [4700, 0.0047, 0.0000001, 20e-3, 3.3]:
25         print(f"{v:g} -> {to_eng(v)}")
```

**Listing 1.3:** `code/prefix.py` – metric-prefix helper used in the exercises.

```
4700 -> 4.7 k
0.0047 -> 4.7 m
1e-07 -> 100 n
0.02 -> 20 m
3.3 -> 3.3
```

**Listing 1.4:** Real output of `python3 code/prefix.py`. Compare the third line: $10^{-7}$ F is the ceramic-disc capacitor marked "104" you will meet in Chapter 4.

**Exercise 1.3** (medium)**.** Prove Figure 1.4(a) $\equiv$ (b) formally: write the contents of each node in each drawing and show the two tables are identical. (This is Exercise 1.5's harder sibling: the colour-code drill is mechanical; this one tests whether the *idea* of a node landed.)

## 1.8 Worked example: how long does the battery last?

We finish the technical content with a worked calculation of the sort this book does constantly — a back-of-envelope estimate that turns component numbers into an engineering decision.

**Example 1.3** (Battery life for a 20 mA LED)**.** *Problem.* You power the LED of Figure 1.3 from a 9 V battery instead of USB. How long will it run?

*Strategy.* Batteries store charge, rated in milliamp-hours: the product "mA $\times$ hours" the battery can deliver before it is flat. The naive estimate is capacity $\div$ current. Then derate: real batteries deliver less than nominal, especially at higher drain, so multiply by a real-world factor.

*Solution.* A small 9 V alkaline is rated about 550 mAh.

$$t \approx \frac{550\ \text{mAh}}{20\ \text{mA}} \times 0.7 \approx 19\ \text{hours}.$$

The 0.7 is honesty, not precision — datasheet curves for this drain sit near it. So: a day of continuous light, a week of occasional breadboard use. *Notice:* the answer came from one multiplication; the skill was knowing that mAh is a charge reservoir and that the LED current, not the voltage, sets the drain rate. The voltage instead decides *how many components* the battery can push through (Chapter 2's job).

## 1.9 Safety: what this book will and will not have you touch

Safety in a book like this is not a disclaimer page; it is a set of numbered rules you can recite. Here they are, all of them, for the whole book.

**THE LOW-VOLTAGE RULE**

Everything this book asks you to build runs at or below 9 V from batteries or a USB port. At these voltages, dry skin stops essentially all current — you cannot get a dangerous shock from the circuits in this book. What remains dangerous is *heat*: a shorted battery, a resistor asked to drop too much power, a reversed electrolytic capacitor. The rule: **low voltage only, and if anything gets hot, disconnect power first, then look.**

What about the numbers that *are* dangerous? Mains in Indonesia is 230 V at 50 Hz, fused at 10–16 A in the wall. Currents through the body stop being safe somewhere around 10–20 mA sustained (the exact threshold depends on path and person); mains can drive far more through muscle and heart. You do not need drama, just arithmetic: 230 V across the $\sim$1–10 k$\Omega$ of skin gives tens to hundreds of milliamps — over the threshold by an order of magnitude. The book's treatment of mains is therefore complete in one rule.

**THE WALL RULE**

If it plugs into the wall, we do not open it. Not the charger brick, not the supply inside it, not anything. This book never requires you to work on mains voltage; do not work on it without proper supervision and equipment.

Three more standing hazards, all quiet ones:

- **ESD** (electrostatic discharge): the spark you feel touching a doorknob is thousands of volts at trivial energy — harmless to you, fatal to the MOSFETs and the FPGA inside Chapters 6 onward. Handle boards by their edges; touch a grounded metal object first; keep ICs in their foam or bags until use.
- **Battery shorts**: an AA or 9 V shorted across its own terminals heats fast enough to burn. A lithium (LiPo) cell shorted or punctured can enter thermal runaway — fire, not warmth. Never let loose batteries share a pocket with keys or wire ends; never leave a LiPo charging unattended.
- **Hot solder and hot parts**: from Chapter 17 we solder. The iron lives in its stand; parts get 30 seconds before fingers. A dropped hot iron is caught by reflex — break the reflex now: let it fall.

**Checkpoint 1.2** *Rules audit.* Say these back without looking: the voltage ceiling for this book's builds; the one place the red probe may not rest; what to do when a component is hot; the four words of the wall rule. If any answer took more than a second, reread this section — it is the only one where the stakes are not academic.

## 1.10 Common mistakes

Each chapter ends the main text by naming the errors that actually hurt people. Four belong to this chapter.

**Reading the schematic as geography.** Positions on paper mean nothing; connections mean everything. Two drawn wires that cross without a junction dot do not connect; two points at opposite corners of the page joined by a net name are the same node. Re-doing Figure 1.4's node-listing exercise until it is automatic is the fix.

**Mixing current conventions mid-argument.** Choosing an arrow, then reasoning in electron flow, silently flips signs and produces $I = -20$ mA "errors" that are pure bookkeeping noise. Conventional current, always.

**Thinking the meter "uses up" what it measures.** A voltmeter adds a 10 MΩ observer in parallel — it diverts essentially nothing. An *ammeter*, inserted in series, is the opposite: a near-short in the path. The confusion between the two is exactly the fuse-blower tripwire of Section 1.6.

**Forgetting that a wire is a component too.** Ideal schematics hide resistance in wire and contacts; breadboards add ohms where fingers expect zero. When a measured number disagrees with a simulation in later chapters, the first suspect is the *connection*, not the math.

## 1.11 Chapter summary

| Quantity / idea | Symbol & unit | The one-line version |
|---|---|---|
| Charge | $q$, C | property of matter; electrons carry $-e$ |
| Voltage | $V$, V | energy per charge, between two points ($V = W/q$) |
| Current | $I$, A | charge per second through a point ($I = \Delta Q/\Delta t$) |
| Resistance | $R$, Ω | volts needed per amp pushed ($R = V/I$) |
| Node | — | all points joined by wire/name = one object |
| Conventional current | — | arrows point positive flow; electrons go the other way |
| Multimeter | V, A, Ω | parallel for V; series (break it) for A; power off for Ω |
| Metric prefixes | m, µ, n, p, k, M | $10^{-3}$ to $10^{6}$ in steps of $10^{3}$ |
| Battery capacity | mAh | charge reservoir; life ≈ capacity/current × 0.7 |
| Safety | — | ≤ 9 V builds only; never open wall-powered gear; ESD kills chips |

**Table 1.3:** Chapter 1 on a page. If a term in the left column does not produce the right column instantly, the section named in its row is the one to reread.

## Exercises

The exercises marked in the text were checkpoints; these stand alone. [easy] is one step, [medium] a small argument, [hard] an open problem.

**Exercise 1.4** (easy)**.** Convert each to engineering notation, then verify with `prefix.py`: 0.0022 A; 10 000 Ω; 0.000 000 047 F.

**Exercise 1.5** (easy)**.** Using Table 1.1, decode: (a) red–red–brown–gold; (b) yellow–violet–orange–gold; (c) brown–black–green–gold.

**Exercise 1.6** (easy)**.** For each measurement, name the dial position *and* the two jacks: (a) voltage of a 9 V battery; (b) current through the LED in Figure 1.3; (c) resistance of an unmounted 1 kΩ resistor.

**Exercise 1.7** (easy)**.** How many distinct nodes does Figure 1.3 contain? Name them. (Count the ground node; it is one node.)

**Exercise 1.8** (medium)**.** Two schematics are described in words. A: "source, then LED, then resistor, then ground." B: "source, then resistor, then LED, then ground." Same circuit or different? Justify with node lists — this has a correct electrical answer, not a taste answer.

**Exercise 1.9** (medium)**.** Two AA alkaline cells in series make a nominal 3 V, ∼2400 mAh pack. Recompute Example 1.3 for a 15 mA draw, including the 0.7 derating. Then explain *why* the derating exists — what is the battery losing to? (Hint: a battery is not an ideal voltage source; it has resistance inside it, called internal resistance, and real capacity falls as drain rate rises.)

**Exercise 1.10** (medium — derive)**.** An LED datasheet says: "$V_F = 2.1$ V (typical), $I_F$ max = 30 mA." To pick $R$ for the circuit of Figure 1.3 with a 5 V source and a target of 15 mA, list each quantity Ohm's law needs, state which of them the datasheet gives, and write the one equation you are still a step short of solving. (You are deriving the shopping list of Chapter 2, not solving it.)

**Exercise 1.11** (medium)**.** Your meter on the 10 A current setting shows "OL" for every circuit you try. Give two explanations consistent with the display — one in the circuit, one in the meter — and a test that separates them.

**Exercise 1.12** (hard — essay)**.** The water analogy (voltage = pressure, current = flow, resistor = narrow pipe) is everywhere, and it is wrong in instructive ways. Write one page: identify *two* concrete places it misleads (suggestions: what happens when a pipe "ends" vs. a wire ends; what a battery actually does to electrons already in the wire; what "pressure between two points" means for a hanging wire with no return path), and for each give the correct lumped-model statement instead. Marking rubric in the hints.

**Exercise 1.13** (hard — design)**.** You have $40 for the kit of Table 1.2, and the breadboard and multimeter are non-negotiable. Choose the rest and defend the two items you left out: for each, name a chapter capability you sacrificed and the simulation that partially covers it. (There are at least three defensible answers; grade is on the justification matching the book's structure.)

### Hints and selected answers

**1.4** 2.2 mA; 10 kΩ; 47 nF.

**1.5** (a) 220 Ω; (b) 47 kΩ; (c) 1 MΩ; all ±5%.

**1.6** (a) DC V range, COM + VΩ, probes across the battery (parallel). (b) mA or 10 A range, COM + current jack, *break* the loop and bridge with probes (series). (c) Ω range, COM + VΩ, resistor unpowered.

**1.7** Three: source+/resistor; resistor/LED anode; LED cathode/ground/source−.

**1.8** Same circuit. In a single loop, element order does not enter the node table — every element sits between the same two neighbours' nodes. Node lists identical ⇒ identical.

**1.9** $2400/15 \times 0.7 = 112$ h. The battery's internal resistance converts part of its energy to heat inside itself, and effective capacity shrinks as current rises — the mAh rating assumes a gentle drain.

**1.10** Needed: resistor voltage $V_R$ and target current $I$. Given: $V_F \approx 2.1$ V, source 5 V ⇒ $V_R = 5 - 2.1 = 2.9$ V; $I = 15$ mA. Missing equation solved next chapter: $R = V_R / I$.

**1.11** Either the circuit's current path is open (a break somewhere) or the meter's mA-range fuse is blown — the fuse lives on the current-measurement path, not the (unfused) 10 A jack. Test: measure the meter's own resistance between COM and the mA jack — near 0 ⇒ path good, OL ⇒ blown fuse.

**1.12** Rubric: each failure must (1) state the wrong prediction the analogy makes, (2) name the lumped fact that replaces it, (3) fit in two sentences. Examples that pass: "pipe ends ⇒ leak" vs. open circuit ⇒ current simply stops; "pump adds water" vs. source reorganises drift of electrons already present.

**1.13** Strong answers drop, e.g., the 9 V battery (the USB supply covers every experiment) and spare LEDs (one suffices for ch2), and note ngspice covers the battery-life check in simulation.

## Notes

1. T. R. Kuphaldt, *Lessons in Electric Circuits*, Vol. I "DC", chapters 1–3 (static electricity, conductors/insulators/electron flow, electric safety). The sequencing theory-then-safety is inherited from here; the safety facts are restated in our terms. Free online [2].
2. OpenStax, *University Physics* Vol. 2 (Ling, Moebs, Sanny et al.): the charge, field and potential definitions that Section 1.1 compresses; the reference for readers who want the skipped derivations [5].
3. SparkFun Electronics, "How to Use a Multimeter": the dial/jack anatomy Figure 1.5 redraws, and the fuse/jack warning [6].
4. SparkFun Electronics, "How to Use a Breadboard": internal connection topology used later in ch17; cited here because the "rails are not joined" trap is first mentioned in Section 1.7 [7].
5. The water analogy's honest limits are standard physics-teaching fare; OpenStax vol. 2's treatment of potential is the correct statement [5]. (Exercise 1.12 asks you to produce it, not quote it.)

6. Tang Nano 9K specifications and the open-source toolchain for it (yosys, nextpnr-himbaechel, `gowin_pack`, openFPGALoader) are documented by Project Apicula; the street price is marked [unverified] per the decisions log [9].

**WATCH INSTEAD**

- *Introduction to circuits and Ohm's law* (Khan Academy, 11:39) — the circuit as a loop of charge and energy, before the formulas. `https://youtu.be/NYQH1nK9PQ4` Listed in Book 8 (ch12), accessed 2026-10-11.

CHAPTER 2

# Ohm's law, power and the simulator

Chapter 1 gave you the nouns: voltage, current, resistance, schematic, node. This chapter gives you the two verbs — compute and simulate — and settles the debt from Example 1.2 by actually choosing the LED's resistor. When you finish it you should be able to: use $V = IR$ and $P = VI$ in every algebraic form, check whether a real resistor survives the power you ask of it, read the headline numbers on a component datasheet, write and run your first *ngspice* netlist (the simulator every analogue chapter uses), and say honestly when Ohm's law applies and when it does not.

**BEFORE YOU START**

Chapter 1: voltage, current, resistance and how to read a schematic. *Recap:* voltage is the push on charge, current is the flow of charge, and resistance is how much a part opposes that flow.

**AFTER THIS CHAPTER YOU CAN**

- use Ohm's law and $P = VI$ to find any one quantity from two others;
- pick a resistor's power rating with a safety margin;
- read the headline numbers on a datasheet;
- write and run a first ngspice netlist and check it against a hand calculation.

**HOW TO THINK ABOUT THIS**

Ohm's law is a relationship between three quantities, so knowing any two gives the third. The habit worth forming is to write the formula with units and let the units check your work. A simulation is a cheap experiment: use it to test your prediction before you build.

## 2.1 Ohm's law: a definition with a job

In Chapter 1 resistance was introduced as $R = V/I$ — volts per amp, measured in ohms. Turned around, that is *Ohm's law*:

$$V = I\,R \qquad \text{and the two shadows} \qquad I = \frac{V}{R}, \quad R = \frac{V}{I}.$$

For the components this law describes — resistors, wire, most fixed loads — it says the current is *proportional* to the voltage. Double the push, double the flow; the ratio never changes. That constancy is the content: a resistor is a part whose $V$-$I$ graph is a straight line through the origin, and $R$ is its slope. Components for which that is true are called *linear* or *ohmic*. Most of the interesting ones are not — Section 2.3 is about that.

The law earns its keep on one more fact we will prove properly in Chapter 3: in a single loop, the source's voltage is *spent* across the elements in the path — the shares always add back to the supply. If the 5 V battery in Figure 1.3 pushes and the LED keeps about 2 V for itself, the resistor is left holding the other 3 V. That sentence is the entire solution to the problem we posed in Chapter 1 and could not yet solve.

**Example 2.1** (The LED question, answered twice)**.** *Problem.* Drive an LED from 5 V and, separately, from 3.3 V. Datasheet headline: $V_F \approx 2\,\text{V}$ at the intended current, maximum $I_F = 30\,\text{mA}$. Target 15 mA.

*Strategy.* The resistor does two jobs: it drops whatever voltage the LED does not take, and Ohm's law then converts that drop into the current everything in the loop shares. Compute $R$, pick the nearest standard E24 value, then check the corners: source ±5%, resistor ±5%, $V_F$ anywhere in 1.8–2.2 V.

*Solution,* 5 V. $R = (5-2)/0.015 = 200\,\Omega$ — itself an E24 value, so the nominal build gives exactly 15 mA. Corners:

$$I_{\text{min}} = \frac{5 \times 0.95 - 2.2}{200 \times 1.05} = \frac{2.55}{210} = 12.1\ \text{mA}, \qquad I_{\text{max}} = \frac{5 \times 1.05 - 1.8}{200 \times 0.95} = \frac{3.45}{190} = 18.2\ \text{mA}.$$

Every combination is legal: nowhere near 30 mA, brightness variation invisible.

*Solution,* 3.3 V. $R = (3.3-2)/0.015 = 86.7\,\Omega \rightarrow$ choose E24 91 Ω (nominal 14.3 mA). Corners:

$$I_{\text{min}} = \frac{3.3 \times 0.95 - 2.2}{91 \times 1.05} = 9.8\ \text{mA}, \qquad I_{\text{max}} = \frac{3.3 \times 1.05 - 1.8}{91 \times 0.95} = 19.3\ \text{mA}.$$

*Notice:* the lower the headroom over $V_F$, the wider the proportional spread — 3.3 V designs run 10–19 mA where 5 V ran 12–18. Sensitivity analysis is not pedantry; it is how you find the designs that fail only on Tuesdays.

Example 2.1 ends with a part number: "200 Ω." The schematic knows that; the beige cylinder in your hand does not carry a number at all — it carries paint. Figure 2.1 shows how to read it.

**Exercise 2.1** (easy)**.** Compute: (a) $I$ for 9 V across 330 Ω; (b) $R$ needed for 10 mA from 3.3 V (no LED); (c) the voltage a 1 kΩ resistor drops at 2 mA.

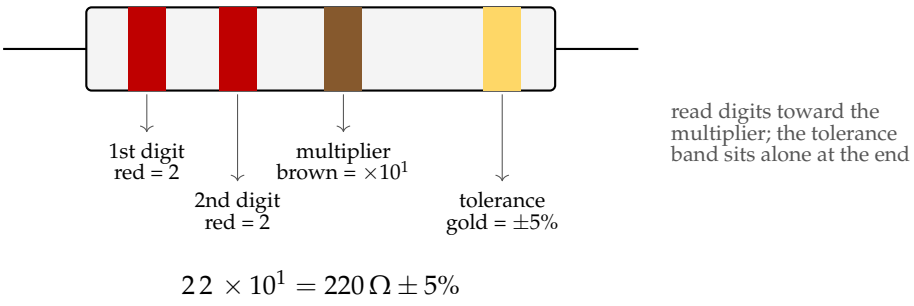



**Figure 2.1:** The 4-band resistor code: two significant digits, a power-of-ten multiplier, and a tolerance band set apart at the end. Read digits toward the multiplier. Notice that the bands encode the *nominal* value only — the true part lives anywhere inside the tolerance band, which is why Example 2.1 computed corners, not just the centre.

**Example 2.2** (First attempt: an LED straight across the supply)**.** You have a 5 V supply and a red LED rated for 20 mA that drops about 2 V when it conducts. The obvious circuit is: connect the LED across the supply.

**What goes wrong.** Nothing in that loop limits the current. The LED drops 2 V, so something must absorb the remaining 3 V; with no resistor, the only things left are the tiny resistance of the wires and the LED's own internal resistance. The current climbs far above 20 mA and the LED overheats. The fix is to let a resistor absorb the extra 3 V at the current you want. By Ohm's law, $R = \Delta V / I = 3\,\text{V}/0.020\,\text{A} = 150\,\Omega$, and the resistor dissipates $P = VI = 3 \times 0.020 = 0.06\,\text{W}$, well under the quarter-watt rating of a standard resistor.

## 2.2 Power: $P = VI$ and the quarter-watt ceiling

Voltage is energy per charge; current is charge per second. Multiply them and the units do the talking:

$$P = V\,I, \qquad 1\ \text{watt} = 1\ \text{joule per second}.$$

Every element in a circuit either supplies this power (sources, negative in the bookkeeping convention) or spends it (everything else). A resistor spends it as heat — substituting $V = IR$ gives the two forms you will use constantly:

$$P = I^2 R = \frac{V^2}{R}.$$

The second form has a counterintuitive lesson: *less* resistance across a fixed voltage means *more* dissipation. A 1 Ω resistor across 5 V dissipates 25 W — a space heater, if it survives.

Components are sold with a *power rating*: the continuous dissipation their package can shed before it overheats. The jellybean through-hole resistor of this book is rated 0.25 W (a quarter watt). Ratings are not suggestions — Example 2.3 shows the failure mode, and Section 2.8 the safety one.

**Example 2.3** (Can a 1/4 W resistor carry 100 mA?)**.** *Problem.* A design wants 100 mA through a 47 Ω resistor. Is the standard 0.25 W part adequate?

*Strategy.* The rating is about heat, so compute the dissipation with the form that uses the given quantities: $I$ and $R$ are known, so $P = I^2 R$.

*Solution.* $P = (0.1)^2 \times 47 = 0.47\,\text{W} > 0.25\,\text{W}$. **Fails** — by nearly a factor of two. A 0.5 W part barely passes ($0.47 < 0.5$), and engineering practice is to run resistors at no more than half to two-thirds of their rating (the *derating* habit: ratings assume ideal mounting, cool air, and a young part). So the correct order is a 1 W resistor — or better, a design that does not need 100 mA.

*Notice:* nobody warned you until you computed. Power ratings are invisible on the schematic; checking them is a habit, not a step.

**Exercise 2.2** (medium)**.** Repeat the check for Example 2.1: the 200 Ω resistor at 15 mA dissipates how much? Does 0.25 W suffice with derating margin? Then the worst-corner case: 190 Ω at 18.2 mA.

## 2.3 When the line bends: nonlinear loads

Here is the sentence to un-learn early: "the LED's resistance is 2 V/20 mA = 100 Ω." That number exists, but it is not a resistance — it is one point on a curve, and the curve is nothing like a line.

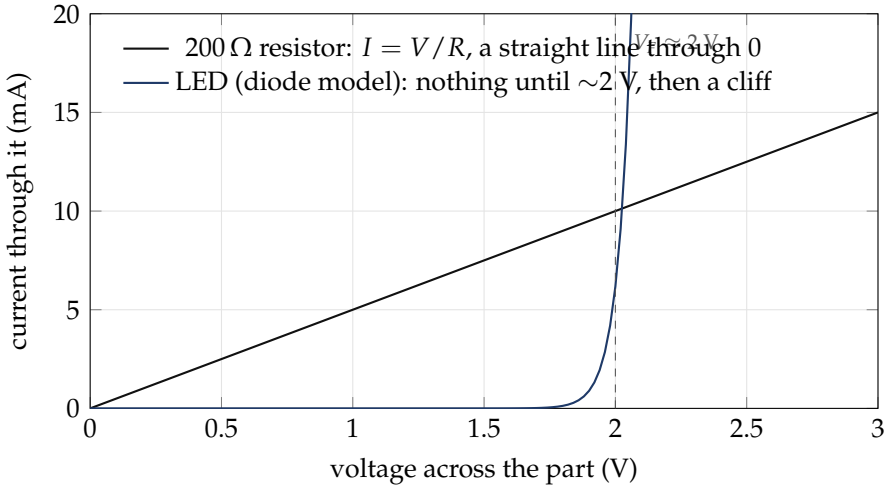



**Figure 2.2:** A resistor's $V$-$I$ characteristic against an LED's, from a real ngspice sweep of the diode model used in this chapter's netlists. Notice the two shapes this chapter keeps separate: the resistor's straight line through zero (Ohm's law is its *definition*) and the LED's dead-flat-then-cliff curve (Ohm's law has no foothold at all). Below $V_F$ the LED conducts almost nothing; above it, the current explodes — which is why the series resistor is not optional.

A *nonlinear* element is one whose $V$-$I$ relationship is not a line through the origin. The LED is the tamest case: a *forward voltage* $V_F$ it holds nearly constant (about 2 V for the red indicator type this book uses), and a current that rises exponentially once you exceed it. $V = IR$ cannot describe that — the ratio $V/I$ is different at every current. What survives is the *method*: the LED's curve still passes through every point the circuit forces it through; "about 2 V" is simply the datasheet's compression of that curve into one usable number. Chapter 5 unpacks the device physics; for now, treat $V_F$ as a fact with a tolerance.

*Remark.* This is also why the LED *needs* its resistor. A perfect 2 V source across a 2-V LED is a knife-edge: a degree of temperature or a millivolt of mismatch and the exponential current

is unbounded. The resistor converts the knife-edge into a gentle slope (Figure 2.5): above the knee the current is set by $(V_{\text{supply}} - V_F)/R$ — Ohm's law reappearing once the diode's share is paid.

## 2.4 Resistor networks, concept only

Resistors almost never appear alone. Two facts carry you until Chapter 3's formal treatment:

- **In series**, the same current threads all of them, and their resistances add: $R_{\text{eq}} = R_1 + R_2 + \cdots$ This is why the divider in Figure 2.4 splits the supply: two equal shares of the total resistance drop equal shares of the voltage.
- **In parallel**, they share the voltage and split the current — two 1 kΩ resistors across the same nodes behave as 500 Ω. More paths, less total opposition.

Chapter 3 turns these into Kirchhoff's laws and the divider formula; today you need only the intuition and, in the exercises, a netlist that checks it.

## 2.5 Reading a datasheet's headline numbers

Datasheets are long; the skill is knowing which three lines you actually need for a given decision. For an LED, these:

| Line | What it means | Typical (5 mm red LED) |
|---|---|---|
| $V_F$ (forward voltage) | voltage it keeps for itself at working current | 1.8–2.2 V, "typ 2.0" |
| $I_F$ max | current you must stay under, always | 20–30 mA |
| $P_D$ (power dissipation) | heat ceiling, $V_F \times I_F$ | ∼100 mW |
| viewing angle, $\lambda$ | optics, not electricity | 20–30°, 620–630 nm |

**Table 2.1:** The three electrical numbers that decide an LED circuit, and the optics rows you may ignore for now. Notice "typ" vs "max": *typical* is marketing's average; *max/min* are the corners your design must survive — Example 2.1 used the $V_F$ range, not the typ value, and that choice is why its corners computed to something honest.

The discipline generalises: for every component in this book, the first datasheet pass asks only *what does it want* (ratings to satisfy) and *what does it do to you* (ratings to stay under). Everything else is a second read.

## 2.6 Your first ngspice netlist

All the arithmetic so far assumed a 2 V LED. How wrong is that assumption — 1%, 30%? The way engineers ask such questions without burning parts is *simulation*: a program that builds the equations of your circuit and solves them exactly as you would, minus the arithmetic slips. This book uses *ngspice*, the free, open-source descendant of Berkeley SPICE — the same engine underneath the expensive packages.

First, proof it is installed — the Chapter 1 script again, now with `ngspice` present (Listing 1.1).

```
MISSING iverilog Verilog simulation (ch11+)
MISSING verilator lint / fast simulation (ch12+)
MISSING yosys synthesis (ch13+)
MISSING nextpnr-himbaechel place-and-route, Gowin (ch13+)
MISSING gowin_pack bitstream packer, Gowin (ch13+)
MISSING openFPGALoader programming the board (ch13+)
MISSING gtkwave viewing waveforms (ch11+)
found ngspice ** ngspice-47 : Circuit level simulation program
MISSING kicad-cli PCB work (ch18)
found python3 Python 3.9.6
```

**Listing 2.1:** Real output of `sh code/check_tools.sh` on this machine after `brew install ngspice`: the line that matters reads *found*. (The script's version grab was also fixed: ngspice prints a decorative banner first, so it now reports the first line containing a digit.)

A SPICE *netlist* is a text file that is to a schematic what a program is to pseudocode. Listing 2.2 is the entire simulation for the circuit of Figure 1.3 — and Listing 2.3 is its real output on this machine.

```
* ch02_led.cir -- operating point of the 5 V, R, LED circuit
* Worked example 2.1 target: about 15 mA through the LED
V1 1 0 DC 5
R1 1 2 200
D1 2 0 LEDMOD
.model LEDMOD D (IS=1e-19 N=2 BV=100)
.op
.end
```

**Listing 2.2:** `code/ch02_led.cir` – every line earns its place: a comment, one source, one resistor, one diode, a model, an analysis request, `.end`.

Read it like ngspice does. The first line is always a title/comment. Each element line is *name*, *node+*, *node-*, *value*: `V1 1 0 DC 5` puts a 5 V source between nodes 1 and 0; `R1 1 2 200` puts 200 Ω between nodes 1 and 2; `D1 2 0 LEDMOD` puts a diode from node 2 to ground, using the parameter set named `LEDMOD` that the `.model` line defines (its `IS` and `N` set the curve you saw in Figure 2.2). *Node 0 is ground, always* — the reference every voltage is quoted against. Finally `.op` asks for the DC *operating point*: solve the circuit once, no sweeps.

```
Node Voltage
---- -------
V(2) 2.045101e+00
V(1) 5.000000e+00
```

```
      Source Current
      ------ -------

      v1#branch -1.47745e-02

Diode: Junction Diode model
       vd 2.0451
       id 0.0147745

Resistor: Simple linear resistor
       i 0.0147745
       p 0.0436571

Vsource: Independent voltage source
       i -0.0147745
       p -0.0738725
```

**Listing 2.3:** Real output of `ngspice -b code/ch02_led.cir` (model parameter tables omitted; the numbers shown are verbatim). Three things to find: the LED's actual $V_F$, the loop current, and the sign of the source current.

Three readings, all loaded. **(1)** $v(2) = 2.045\,\text{V}$ — the model's $V_F$, not the assumed 2.0 — and $i = 14.77\,\text{mA}$, not 15. The 1.5% gap is the difference between an assumption and a model. **(2)** Every element reports the same current — the single loop really does share one flow. **(3)** The source's current is *negative*: SPICE measures current into the positive terminal, and a source delivers current, so its number comes out negative — convention, not error. Its power is likewise negative: it is supplying the $+73.9\,\text{mW}$ the resistor ($+43.7\,\text{mW}$) and LED (the rest) spend.

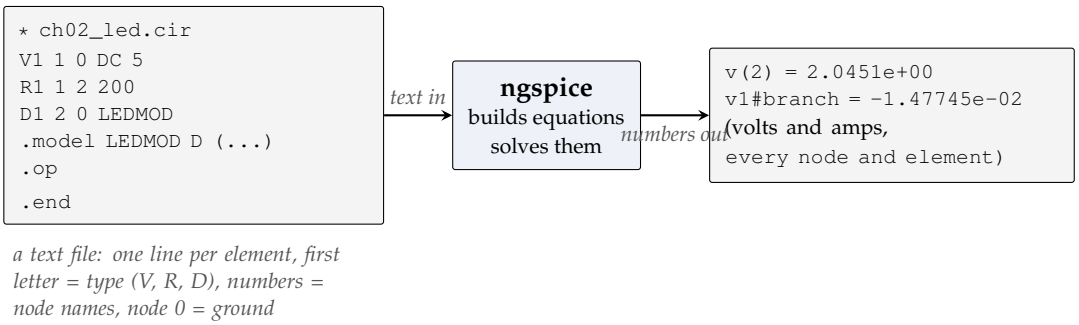



**Figure 2.3:** The simulation loop in one picture: you write a text file of nodes and elements, ngspice builds and solves the circuit equations, and it reports every node's voltage and every element's current. The schematic and the netlist are the same circuit in two languages — matching them is a skill the exercises drill.

**Checkpoint 2.1** *Predict before you run.* For Figure 2.4 — 9 V, two 330 Ω resistors in series — write down $v(2)$ and the loop current before simulating. Then write the netlist (three element lines) and run `.op`. Expect: $v(2) = 4.5\,\text{V}$, $i = 9/660 \approx 13.6\,\text{mA}$.

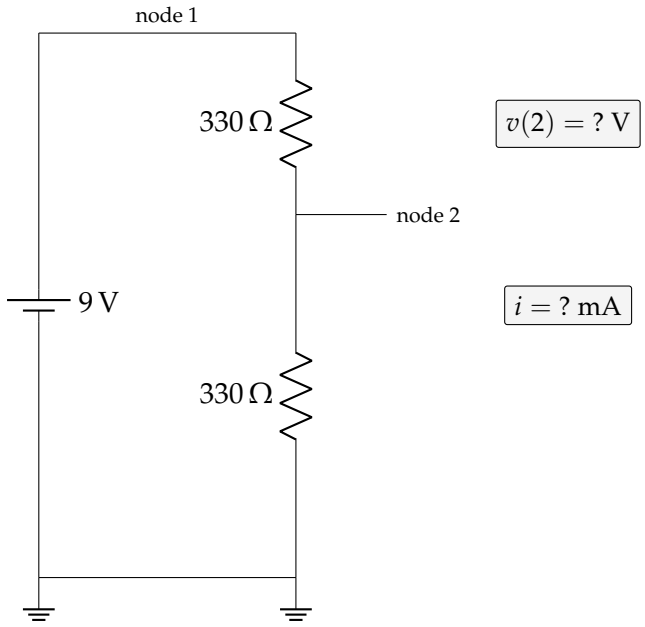



**Figure 2.4:** Checkpoint 2.1: cover the answer, compute both blanks by hand, then run the netlist. The point of the exercise is not the numbers — it is proving to yourself that two equal resistors split a supply evenly.

**Exercise 2.3** (medium)**.** Write the netlist for Figure 2.4 yourself (title, three elements, `.op`, `.end`), run it, and paste the three numbers you predicted. If they disagree, find whether the error is in your arithmetic or your netlist — both happen.

## 2.7 Why we simulate before we build

The honest answer is not "it is easier" — it is that a simulator makes *asking questions* free. Changing $R$ from 200 to 100 costs an edit and a keystroke; on a breadboard it costs a resistor and a resteadied hand. The sweep commands below are the cheapest form of one question every designer asks: *what does this circuit do across a range of conditions?*

```
* ch02_sweep.cir -- dc sweep: LED-circuit current vs supply voltage
V1 1 0 DC 0
R1 1 2 200
D1 2 0 LEDMOD
.model LEDMOD D (IS=1e-19 N=2 BV=100)
.control
dc v1 0 5 0.05
let iload = -i(v1)
set filetype=ascii
wrdata ch02_sweep.dat iload
.endc
.end
```

**Listing 2.4:** `code/ch02_sweep.cir` – a `.dc` sweep walks the source from 0 to 5 V and records the loop current at each point.

The `.control` block runs the `.dc` analysis: sweep `v1` from 0 to 5 in steps of 0.05, create a vector for the current (negated so it reads positive — the source-current sign convention of Listing 2.3), and write the 101 points to a data file. Listing 2.5 shows real lines from it, and Figure 2.5 plots the whole run.

```
0.00000000e+00 -1.60434623e-36
...
1.95000000e+00 4.35823416e-04
2.00000000e+00 6.02192794e-04
2.05000000e+00 7.83622176e-04
...
5.00000000e+00 1.47744954e-02
```

**Listing 2.5:** Real excerpt of `figures/ch02/ch02_sweep.dat` (voltage, then current in amps): first line, the knee region, and the endpoint. The endpoint's 14.77 mA is *exactly* the `.op` answer — same equations, same result.

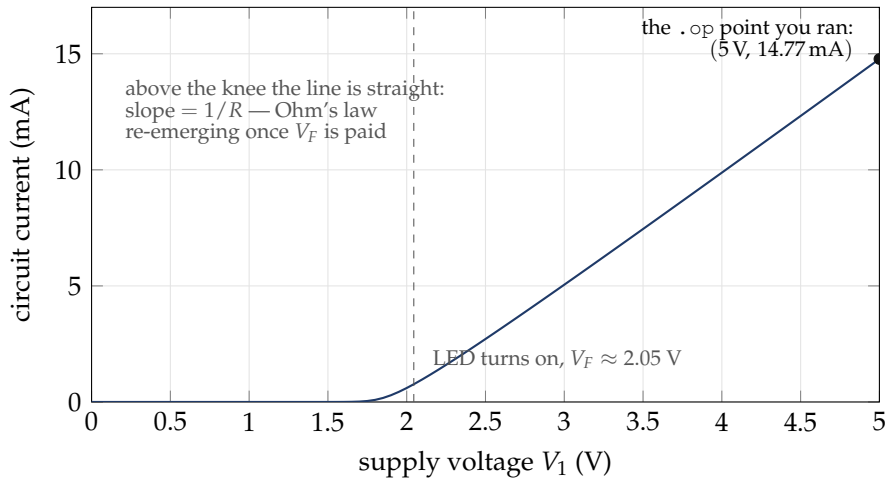



**Figure 2.5:** The full `.dc` sweep, plotted from the real simulation output. Notice the two regimes: dead flat below ∼2 V (the LED is off and almost nothing flows anywhere), then a straight ramp — the resistor has taken control. The marked point is the `.op` you ran: one point of this curve computed a different way, agreeing to the last digit.

**Example 2.4** (One circuit, three ways)**.** *Problem.* What current does the Figure 1.3 circuit actually draw at 5 V?

*Strategy.* Compute it three independent ways — assumption, model, curve — and require them to agree within stated tolerances. When estimates from different machinery agree, the answer is robust; when they do not, the disagreement tells you which assumption is weakest.

*Solution.*

1. **By hand** (the $V_F = 2$ assumption): $I = (5 - 2)/200 = 15\,\text{mA}$.
2. **By `.op`** (a model of the real diode curve): 14.77 mA — the model's $V_F$ came out 2.045 V, and $(5 - 2.045)/200 = 14.77\,\text{mA}$ exactly, because SPICE just does the same arithmetic with a sharper input.
3. **By `.dc` sweep** (the whole behaviour): Figure 2.5 returns the same 14.77 mA at $V = 5$ and shows the number is on the line's flat, boring stretch — no cliff-edge nearby.

*Notice:* the hand answer was 1.5% high — inside any tolerance the circuit will ever see. Simulation did not replace the estimate; it *priced* it. That is what the simulator is for in this book.

## 2.8 Safety

Chapter 1's rules stand: low voltage only, USB power for everything here. This chapter adds one line.

**RESISTORS GET HOT — CHECK THE RATING**

A resistor dissipating over its rating does not fail politely; it browns, smells, and can burn a finger or a board. Before powering anything you build, run the $P = I^2R$ or $V^2/R$ check of Example 2.3 on the stressed parts. In this chapter all dissipation is milliwatts — the rule is training for when it is not.

And a quieter one: **USB power only** for these experiments. A USB port is current-limited by design (typically a few amps, and it will simply shut off or sag rather than push dangerous current into a short). Do not substitute a bare bench supply or a larger battery until you have a reason — the limiting is the safety.

## 2.9 Common mistakes

**Applying $V = IR$ to nonlinear loads.** The LED has no resistance in the Ohm's-law sense: its $V/I$ ratio changes at every point (Figure 2.2). The fix is not a bigger formula, it is the right model — a $V_F$ fact plus a series resistor that re-linearises the loop.

**Ignoring power ratings until the resistor browns.** Schematics do not draw heat. A 47 Ω resistor at 100 mA is a 0.47 W part inside a 0.25 W package, and nothing on the schematic page will warn you. Example 2.3 is the three-line check; run it every time.

**Treating a `.op` number as exact.** Simulation solves the model you gave it — not the part in your hand. The model said 14.77 mA; a real board with a ±5% resistor and a loose $V_F$ range delivers anywhere in Example 2.1's computed band. Simulate for behaviour and estimates; measure for truth.

**Netlist nodes that don't match the schematic.** Node numbers in a netlist are names, not positions — but they must connect the same things the wires do. Every missing wire in a netlist is a component floating in space; every shared number is an invisible solder joint. Diff the node list against the schematic before running.

## 2.10 Chapter summary

| Quantity / idea | Form | The one-line version |
|---|---|---|
| Ohm's law | $V = IR$ | defines $R$; holds for linear parts only |
| Power | $P = VI = I^2R = V^2/R$ | energy per second; resistors spend it as heat |
| Power rating | e.g. 0.25 W | package's heat ceiling; derate 2× |
| Nonlinear (LED) | $V_F$, $I_F$ max | no $R$; a cliff curve + a fact from the datasheet |
| Series / parallel | add / share | series: one current, voltages split; parallel: one voltage, currents split |
| Datasheet | typ vs max | satisfy its wants, respect its limits |
| Netlist | name node+ node- value | text schematic; node 0 is ground |
| `.op` | operating point | one solved state: every $v$ and every $i$ |
| `.dc` | sweep | solve repeatedly over a range — the free question |
| Sign of source $i$ | negative | SPICE convention: current into the + terminal |
| Safety | — | rating check first; USB's current limit is a feature |

**Table 2.2:** Chapter 2 on a page: two formulas, one safety habit, and a simulator that answers what-if questions for free.

## Exercises

**Exercise 2.4** (easy)**.** (a) Current through 470 Ω at 5 V. (b) Voltage across 2.2 kΩ at 3 mA. (c) $R$ for 8 mA from 9 V.

**Exercise 2.5** (easy)**.** Dissipation of: 330 Ω at 13.6 mA; 100 Ω at 5 V; 1 kΩ at 20 mA. Which exceed 0.25 W?

**Exercise 2.6** (easy)**.** A datasheet lists $V_F$ typ 2.0 V, max 2.4 V at $I_F$=20 mA, and $I_F$(abs max)=30 mA. Which number do you design to, and which do you fear?

**Exercise 2.7** (easy)**.** In the netlist `R1 3 7 470; D2 7 0 LEDMOD`, name each element, the nodes it joins, and where ground is.

**Exercise 2.8** (medium — derive)**.** Starting from $P = VI$ and $V = IR$, derive $P = I^2R$ and $P = V^2/R$. Then derive the useful inverse: for a power ceiling $P$ at fixed $V$, the *smallest* allowed resistance is $R \geq V^2/P$ — watch which way the inequality flips when you clear a fraction.

**Exercise 2.9** (medium)**.** Redo Example 2.1 at 5 V but with $R = 220\,\Omega$ (the more common bin). Compute the nominal current and the tolerance band; decide whether it is still a good choice.

**Exercise 2.10** (medium)**.** Write (don't copy) a netlist for a 9 V source and two 1 kΩ resistors in series; predict $v(2)$ and $i$; run `.op` and compare.

**Exercise 2.11** (medium)**.** The kit resistor red-red-brown-gold is nominally 220 Ω±5%. Give the true-value range. A second resistor marked yellow-violet-red-gold: nominal and range.

**Exercise 2.12** (hard)**.** Two LEDs in parallel share one series resistor on 5 V. Explain why this is a trap: the parts are never identical, so one hogs current, gets hotter, takes even more current — a runaway split. What does a netlist with two slightly different `IS` values show? (Run it.)

**Exercise 2.13** (hard — design)**.** The supply is unknown, somewhere between 3.3 and 5 V. Choose a resistor and justify it so the LED current stays within 10–20 mA across the whole supply range with $V_F \in [1.8, 2.2]$, or prove the spec is impossible.

### Hints and selected answers

**Easy 2.1** 10.6 mA; 6.6 V; 1.125 kΩ (E24: 1.1 kΩ).

**Easy 2.2** 61.2 mW ok; 250 mW at the line; 400 mW over.

**Easy 2.3** Design current to the typ/$V_F$ band; the 30 mA abs-max is the cliff edge you never plan to visit.

**Easy 2.4** R1 between nodes 3–7 (470 Ω); D2 between 7–0; node 0 is ground.

**Med (derive)** $P = VI$, $V = IR \Rightarrow P = (IR)I = I^2R$; $I = V/R \Rightarrow P = V \cdot V/R = V^2/R$; solving $V^2/R \leq P$ for $R$ gives $R \geq V^2/P$ — careful: the *minimum* resistance is set by the rating, not the maximum.

**Med (220)** $I_{\text{nom}} = 3/220 = 13.6$ mA; band $\approx$ 11.0–16.6 mA; still good, slightly dimmer.

**Med (1k+1k)** $v(2) = 4.5$ V, $i = 4.5$ mA — a divider preview.

**Med (colours)** 220 Ω: 209–231 Ω; yellow-violet-red-gold $= 47 \times 10^2 = 4.7$ kΩ: 4465–4935 Ω.

**Hard (two LEDs)** With IS mismatched by 10%, the hog takes $\sim 2\times$ the other's current — and the gap widens with heat. Per-LED resistors is the fix.

**Hard (design)** $I_{\max}$ occurs at 5 V, $V_F$=1.8, $R_{\min}$: need $R \geq (5.25 - 1.8)/0.02 = 172.5\,\Omega$; $I_{\min}$ at 3.135 V, $V_F$=2.2, $R_{\max}$: need $R \leq 0.935/0.01 = 93.5\,\Omega$. Impossible as stated — the answer is a smaller tolerance or two supply rails; the exercise is the proof.

## Notes

1. T. R. Kuphaldt, *Lessons in Electric Circuits*, Vol. I "DC", chapters 2 (Ohm's law) and 1.6 (power): the $P = VI$ forms and the ratings habit; its own SPICE introduction validates our netlist-first order [2].
2. OpenStax, *University Physics* Vol. 2: the derivation of $V = IR$ for ohmic materials and where non-ohmic devices break it — Section 2.3's correct statement [5].
3. ngspice manual, `.op`/`.dc` analysis and `.control` scripting: the exact directives of Listings 2.4; version 47 pinned in `code/README.md` [8].
4. The E24 preferred-value series and colour code are industry standard; the decoding drill mirrors Table 1.1 [2].
5. The LED model parameters (`IS`, `N`) are chosen so the simulated knee lands near a real red LED's $V_F$; they are teaching values, not a vendor's characterised model — the honesty point of Example 2.4.

**WATCH INSTEAD**

- *Introduction to circuits and Ohm's law* (Khan Academy, 11:39) — $V = IR$ and power in a simple loop. https://youtu.be/NYQH1nK9PQ4 Listed in Book 8 (ch12), accessed 2026-10-11.

CHAPTER 3

# Series, parallel and Kirchhoff's laws

Chapter 2 gave you one rule, $V = IR$, and one tool, ngspice. This chapter connects resistors into *networks* and asks the obvious question: what is the voltage *here*, when the circuit has more than one place to look? You will learn the two bookkeeping laws every engineer uses without thinking (Kirchhoff's), the voltage divider — the single most-used fragment of analogue circuitry in this book — a systematic way to solve any linear network (node voltages), and a compression trick (Thévenin) that turns a whole network into one source and one resistor. By the end you can take any resistor circuit this book will build, write its equations by hand in a few lines, and check the answer in the simulator before reaching for a wire.

**BEFORE YOU START**

Chapter 2: Ohm's law and power. *Recap:* $V = IR$. Two parts are in *series* when the same current must pass through both; in *parallel* when they share the same two ends and so the same voltage.

**AFTER THIS CHAPTER YOU CAN**

- find the equivalent resistance of series and parallel networks;
- design a voltage divider and know what loading does to it;
- apply Kirchhoff's current and voltage laws to a circuit with several loops;
- compress a network to a Thévenin equivalent.

**HOW TO THINK ABOUT THIS**

Every complicated network is built from two arrangements: parts sharing the same current (series) and parts sharing the same voltage (parallel). The two conservation laws say what flows into a junction flows out, and what rises around a loop falls back. If you can account for current and voltage like money in a ledger, any circuit becomes solvable.

## 3.1 Series and parallel are about nodes

**Definition 3.1** (Series)**.** Two elements are in *series* when they share exactly one node and nothing else connects to that node. Whatever current enters one must leave through the other — there is nowhere else for it to go.

**Definition 3.2** (Parallel)**.** Two elements are in *parallel* when both of their terminals connect to the same pair of nodes. They therefore see the same voltage.

The definitions are about *nodes*, not geometry. Figure 3.1 shows one circuit — $R_1$ in parallel with $R_2$ — drawn four ways. Beginners call (c) and (d) "different circuits" because the resistors point differently or sit in a triangle. They are the same circuit: in all four drawings every $R_1$ terminal and every $R_2$ terminal lands on the same two nodes. The schematic is the topology (Section 1.4); squint at connections, not shapes.

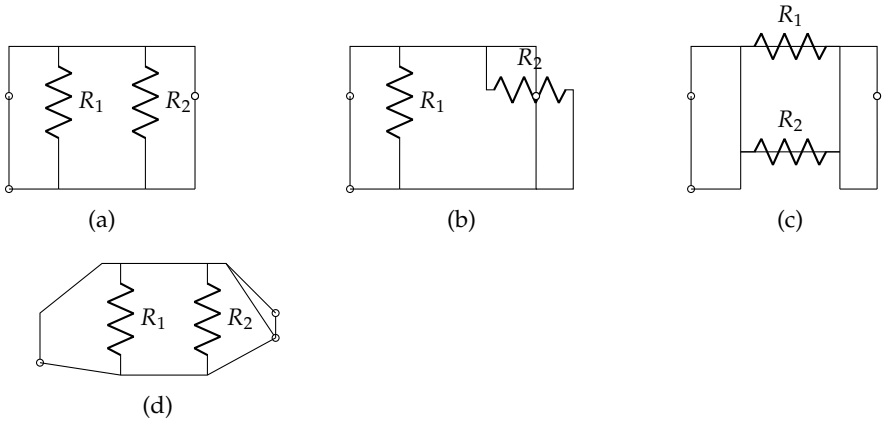



**Figure 3.1:** One parallel circuit, four drawings. In every panel each end of $R_1$ shares a node with an end of $R_2$ and nothing in between splits them. If you can label only two distinct nodes, the resistors are in parallel — however the wires are bent.

**Exercise 3.1** (easy)**.** In Figure 3.1(b), trace from the top of $R_1$ to the top of $R_2$ without lifting your pen. You never pass through a component — so those ends are one node. Do the same for the bottoms.

## 3.2 Equivalent resistance

A network of resistors between two terminals behaves, at those terminals, like one resistor called the *equivalent resistance* $R_{\text{eq}}$. Two cases cover nearly everything.

**Series:** the same current crosses both, voltages add (KVL), so $R_{\text{eq}} = R_1 + R_2$. **Parallel:** the same voltage sits across both, currents add (KCL):

$$\frac{1}{R_{\text{eq}}} = \frac{1}{R_1} + \frac{1}{R_2} \qquad \Longleftrightarrow \qquad R_{\text{eq}} = \frac{R_1 R_2}{R_1 + R_2} \quad \text{("product over sum").}$$

For $n$ resistors, series just sums; parallel sums the reciprocals. Reduce any network by repeatedly collapsing the two-terminal pieces. Sanity checks that catch almost every slip: in parallel, $R_{\text{eq}}$ is *smaller than the smallest* resistor (more paths, more current); in series, $R_{\text{eq}}$ is larger than the largest.

**Exercise 3.2** (easy)**.** Reduce this to one resistor: 1 kΩ in parallel with 2 kΩ, in series with 470 Ω. (Answer: $667 + 470 \approx 1.14\,\text{k}\Omega$.)

## 3.3 The voltage divider

Two series resistors across a source split the voltage in proportion to their resistances. With $R_1$ on top and $R_2$ to ground as in Figure 3.2,

$$v_{\text{out}} \;=\; V_{\text{in}}\,\frac{R_2}{R_1 + R_2}.$$

One line of KVL derives it: the current is $V_{\text{in}}/(R_1 + R_2)$, and the output is that current times $R_2$. This fragment appears dozens of times in this book — scaling sensor signals, setting logic levels, biasing transistors. Learn it once, recognise it everywhere.

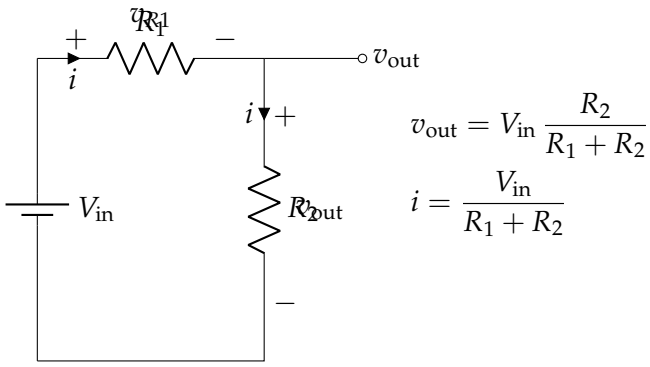



**Figure 3.2:** The voltage divider. The output is taken *at the middle node*; $R_2$ is the resistor *to ground*. Notice the formula needs the same current through both — the moment something draws current out of the middle node, $v_{\text{out}}$ falls (Section 3.7 quantifies it).

**Example 3.3** (A 3.3 V logic level from a 5 V rail)**. Strategy.** The ratio sets the voltage; the absolute values set how much current the divider wastes. Pick a ratio from the E24 ladder, then choose a magnitude that burns little current but still makes the output stiff compared to anything the logic input draws (chip inputs want microamps or less).

**Solution.** We need $R_2/(R_1 + R_2) = 3.3/5 = 0.66$, i.e. $R_1 : R_2 \approx 1 : 1.94$. E24 gives a clean hit: $R_1 = 1\,\text{k}\Omega$, $R_2 = 2\,\text{k}\Omega$, ratio $2000/3000 = 0.667$. Then $v_{\text{out}} = 5 \times 0.667 = 3.33\,\text{V}$ and the standing current is $5/3000 = 1.67\,\text{mA}$ — under 10 mW from the rail. Tolerance corners (5 % parts): worst high is $5 \times 2.1/3.05 = 3.44\,\text{V}$, worst low is $5 \times 1.9/2.95 = 3.22\,\text{V}$ — a 3.3 V input typically accepts down to about 2.5 V as "high", so both corners pass.

**Check.** Listing 3.2 runs it. Notice the simulator takes the question "what are all node voltages and branch currents" literally — and answers 3.33 V and 1.67 mA exactly as derived.

```
* ch03_divider.cir -- 5 V -> 3.3 V divider, operating point
V1 1 0 DC 5
R1 1 2 1000
R2 2 0 2000
.op
.end
```

**Listing 3.1:** `code/ch03_divider.cir` — the divider as a netlist.

```
V(2)  3.333333e+00
V(1)  5.000000e+00
v1#branch -1.66667e-03
```

**Listing 3.2:** ngspice output of `code/ch03_divider.cir`, verbatim (simulation S1).

## 3.4 The current divider

Parallel is the mirror image. Two parallel resistors share a voltage and split an incoming current *inversely* to their resistances — more current takes the easier path:

$$i_2 = I\,\frac{R_1}{R_1 + R_2}.$$

Note the twist: the current through $R_2$ is proportional to the *other* resistor. (The clean derivation uses conductances $G = 1/R$: in parallel, conductances add and current splits in proportion to $G$, i.e. $i_2 = I\,G_2/(G_1 + G_2)$ — same thing, one less fraction to fumble.)

## 3.5 KCL and KVL: bookkeeping, signed

**Definition 3.4** (Kirchhoff's current law)**.** The sum of currents *into* a node is zero — or equivalently, what flows in flows out. Charge is conserved and nodes store none of it.

**Definition 3.5** (Kirchhoff's voltage law)**.** The sum of signed voltage changes around any closed loop is zero. Voltage is a potential difference; returning to the same node must return the same potential.

The laws are trivial. The discipline is the *signs*. Pick a direction and label every drop before writing equations — Figure 3.3 marks the convention this book uses: a drop is $+$ when your loop direction enters the $+$ labelled terminal. Then the equation writes itself. Sign errors, not concept errors, are what actually go wrong (Section 3.9).

**Checkpoint 3.1** In Figure 3.3 (left), $v_1 = 3\,\mathrm{V}$. Write the KVL sum and find $v_2$ without computing any current. (Answer: $9 - 3 - v_2 = 0 \Rightarrow v_2 = 6\,\mathrm{V}$.)

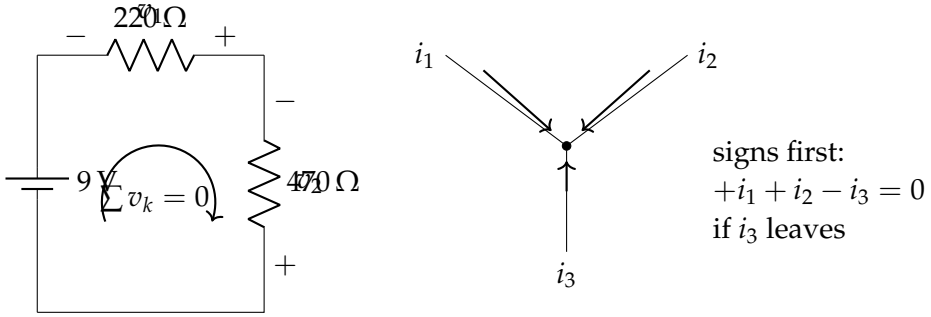



**Figure 3.3:** Left: KVL on the series loop — walking clockwise, $+9 - v_1 - v_2 = 0$. Right: KCL at a node — assign each current a direction *first* (here $i_3$ defined leaving), then sum algebraically: $i_1 + i_2 - i_3 = 0$. Negative answers just mean "the arrow was guessed backwards", which is fine.

## 3.6 The node-voltage method

For a network that will not collapse to one $R_{\text{eq}}$, do this instead: pick a ground, name every other node voltage $v_1, v_2, \ldots$, and write KCL at each node with each current expressed via Ohm's law. One equation per node, solved together — that is the entire method, and it is exactly what the simulator does internally.

**Example 3.6** (Two sources, one node: by hand then by ngspice)**. Strategy.** Node 3 joins three resistors to two sources. Write KCL there; the branch currents are $(v_3 - 12)/100$, $(v_3 - 6)/220$ and $v_3/470$. One unknown, one equation.

**Solution.**

$$\frac{v_3 - 12}{100} + \frac{v_3 - 6}{220} + \frac{v_3}{470} = 0 \implies v_3\left(\tfrac{1}{100} + \tfrac{1}{220} + \tfrac{1}{470}\right) = \tfrac{12}{100} + \tfrac{6}{220}$$

$$v_3 = \frac{0.1473}{0.01667} \approx 8.83\,\text{V}.$$

Notice the form: *weighted average*. $v_3$ lies between 6 and 12 V, pulled toward the 12 V source because its 100 Ω link is the strongest (lowest resistance).

**Check.** Listing 3.4 — ngspice agrees to the fourth decimal, and reports the branch currents $-31.7$ mA and $+12.9$ mA that the hand work predicts ($\frac{8.83-12}{100} = -31.7$ mA; $\frac{8.83-6}{220} = +12.9$ mA; both plus $v_3/470 = 18.8$ mA sum to zero — KCL, closing the loop on itself).

```
* ch03_twoloop.cir -- two sources feeding one node through resistors
V1 1 0 DC 12
V2 2 0 DC 6
R1 1 3 100
R2 2 3 220
R3 3 0 470
.op
.end
```

**Listing 3.3:** `code/ch03_twoloop.cir` — the two-source network of Example 3.6.

```
V(3)  8.832947e+00
V(2)  6.000000e+00
```

```
V(1)  1.200000e+01
v1#branch -3.16705e-02
v2#branch 1.287703e-02
```

**Listing 3.4:** ngspice output of `code/ch03_twoloop.cir`, verbatim (simulation S2).

## 3.7 Thévenin and Norton: networks compressed

**Definition 3.7** (Thévenin equivalent)**.** Any linear network viewed from two terminals behaves exactly like one voltage source $V_{\text{th}}$ in series with one resistor $R_{\text{th}}$: $V_{\text{th}}$ is the open-circuit voltage at the terminals, and $R_{\text{th}}$ is the resistance looking in (all independent sources zeroed) — equivalently $R_{\text{th}} = V_{\text{th}}/I_{\text{sc}}$ where $I_{\text{sc}}$ is the short-circuit current.

"Behaves exactly like" means *for every possible load*: the terminal's voltage-versus-load-current line is identical. Figure 3.4 shows the picture; Figure 3.5 proves it on the divider — sweep a load current through the original and through the model and the lines coincide. The *Norton equivalent* is the same idea drawn the other way: a current source $I_{\text{sc}}$ in parallel with $R_{\text{th}}$, interchangeable by $V_{\text{th}} = I_{\text{sc}}R_{\text{th}}$. Worked use of it: our divider's Norton form is 5 mA in parallel with 667 Ω. Ask how much current a 2 kΩ load draws and the current divider answers in one line: $5\,\text{mA} \times 667/(667 + 2000) = 1.25\,\text{mA}$ — the same answer KCL gives on the full circuit ($v_{\text{out}} = 2.5\,\text{V}$), with the network replaced by two numbers. Norton is the natural form whenever the quantity you care about is current — sensors, current-output devices, parallel paths.

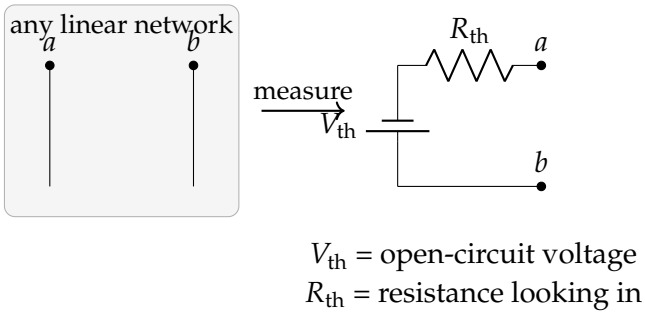



**Figure 3.4:** The Thévenin picture: whatever is inside the left box — any number of sources and resistors — the terminals see only $V_{\text{th}}$ and $R_{\text{th}}$. Two measurements determine it: open-circuit voltage, and either short-circuit current or resistance-with-sources-zeroed.

**Example 3.8** (The divider, compressed — and when "3.3 V" lies)**. Strategy.** Compress the Example 3.3 divider ($R_1 = 1\,\text{k}\Omega$, $R_2 = 2\,\text{k}\Omega$ on 5 V) into a Thévenin model, then ask it a loaded question.

**Solution.** $V_{\text{th}}$ = open-circuit output = 3.33 V. Looking in with the 5 V source zeroed (shorted): $R_1$ and $R_2$ are both from the output node to ground ⇒ $R_{\text{th}} = 1\|2\,\text{k}\Omega = 667\,\Omega$. Now attach a 1 kΩ load: the model predicts $v_{\text{out}} = 3.33 \times 1000/1667 = 2.0\,\text{V}$ — the "3.3 V rail" collapsed by 40 %. The simplification does not lie; the lie was treating an unstiff divider as a voltage source. The load-line sweep in Figure 3.5 (from the Listings below) shows the model and the original agreeing to seven digits: slope $-667\,\Omega$, intercept 3.33 V, zero crossing at $I_{\text{sc}} = 5\,\text{mA} = 3.33/0.667$.

**Rule of thumb.** A divider feeding a load wants $R_{\text{th}} \ll R_{\text{load}}$ — a factor of 10 keeps the sag under about 9 %.

```
* ch03_thevenin.cir -- load-line sweep: original divider
I1 2 0 DC 0 * load current pulled OUT of node 2
V1 1 0 DC 5
R1 1 2 1000
R2 2 0 2000
.control
dc i1 0 0.01 0.0005
let vout = v(2)
set filetype=ascii
wrdata ch03_divider_loadline.dat vout
.endc
.end
```

**Listing 3.5:** `code/ch03_thevenin.cir` — sweep a load current, record $v_{\text{out}}$ (simulation S3). `code/ch03_thevenin_model.cir` is the same file with the two resistors replaced by $V_{\text{th}}$=3.333 V and $R_{\text{th}}$=667 Ω.

```
i_load v_orig v_model
0.00000000e+00 3.33333333e+00 3.33333330e+00
2.00000000e-03 2.00000000e+00 1.99999996e+00
4.00000000e-03 6.66666667e-01 6.66666620e-01
6.00000000e-03 -6.66666667e-01 -6.66666720e-01
8.00000000e-03 -2.00000000e+00 -2.00000006e+00
1.00000000e-02 -3.33333333e+00 -3.33333340e+00
```

**Listing 3.6:** Verification output, verbatim — every 4th row shown of 21 simulated rows: load current, $v_{\text{out}}$ of the real divider, $v_{\text{out}}$ of the Thévenin model. Column 3 minus column 2 never exceeds $7 \times 10^{-8}$ V on any row.

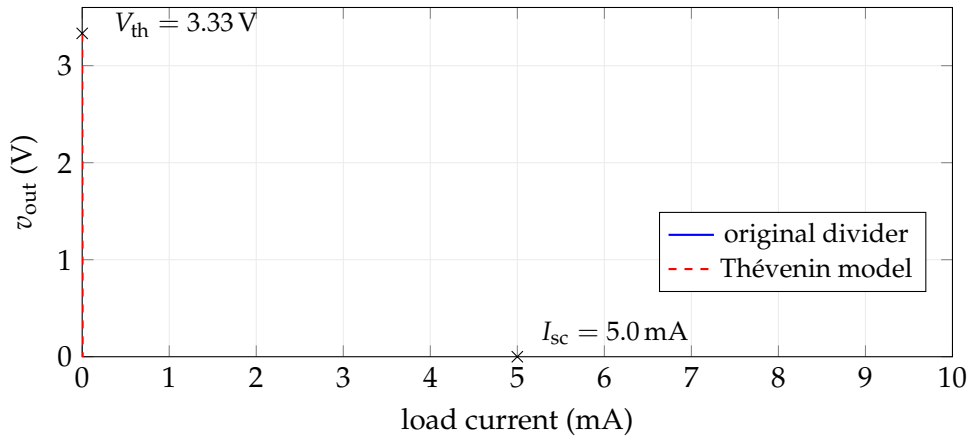



**Figure 3.5:** Simulation S3: $v_{\text{out}}$ against pulled load current for the real divider (blue) and its Thévenin model (red, dashed) — the same line, to 70 nV. The marked endpoints are the theorem's two handles: open-circuit voltage 3.33 V and short-circuit current 5 mA. Every linear network has exactly one such line.

## 3.8 Measuring without breaking the circuit

A voltmeter connects in parallel — so it adds a resistance in parallel. Real DMMs present about 10 MΩ; on kiloohm-scale circuits that is invisible, but measure a megaohm divider and the meter *becomes part of the circuit* (Figure 3.6): $1\,\text{M}\Omega \| 10\,\text{M}\Omega = 909\,\text{k}\Omega$, so a 5 V rail reads $5 \times 909/1909 = 2.38$ V instead of 2.5 V. The meter did not "use up" voltage; it changed the divider. When a reading surprises you, suspect the probe before the circuit.

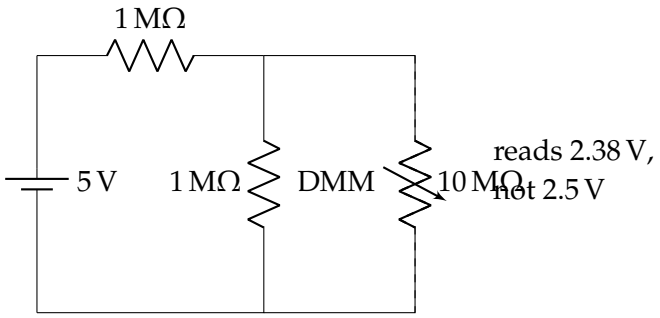



**Figure 3.6:** Meter loading. Two 1 MΩ resistors divide 5 V to 2.5 V until a 10 MΩ meter joins the bottom leg — 2.38 V now. Dashed: the resistance you added by measuring.

**Exercise 3.3** (medium — predict, then run)**.** A 4.7 MΩ / 4.7 MΩ divider sits on 9 V. Predict the DMM reading ($R_{\text{meter}} = 10\,\text{M}\Omega$), write the three-line netlist, run it, compare. (Answer: $4.7 \| 10 = 3.20\,\text{M}\Omega$, $9 \times 3.20/7.90 = 3.64$ V — the meter drags it down by 19 %.)

## 3.9 Common mistakes

**KVL AROUND A NON-LOOP / KCL AT A NON-NODE**

Writing $\sum v = 0$ across elements that do not form a closed loop, or summing currents at a "node" that secretly contains a junction you missed. Trace the loop with a finger; list the node's wires explicitly.

**SIGNS GUESSED MID-EQUATION**

Assigning polarities while solving instead of before. If $v_1$ was labelled one way in the figure and flipped in the algebra, the sign error compounds silently. Label first; solve second; never adjust.

**USING THE DIVIDER FORMULA ON A LOADED DIVIDER**

$v_{\text{out}} = VR_2/(R_1 + R_2)$ assumes the middle node's only currents are through $R_1$ and $R_2$. Any load at the node breaks the premise — Example 3.8's 2.0 V instead of 3.3 V. Check what else connects.

**MIXING SERIES AND PARALLEL FORMULAS MID-REDRAW**

$R_1 + R_2$ used where resistors share a node-pair but also a third element, or "product over sum" applied to resistors that are not both on the same two nodes. Redraw to the canonical form of Figure 3.1 before any arithmetic.

## 3.10 Chapter summary

Figure 3.7 is the whole chapter on one card. The method hierarchy: redraw until it is series/parallel; if it will not collapse, write KCL at each node; if you only care about one port, compress to Thévenin. Everything else is algebra.

| Quantity | Series | Parallel | Note |
|---|---|---|---|
| Resistance | $R_{eq} = R_1 + R_2$ | $R_{eq} = \dfrac{R_1 R_2}{R_1 + R_2}$ | two at a time |
| Voltage | splits $\propto R$ | same on each | KVL |
| Current | same through each | splits $\propto 1/R$ | KCL |
| Divider | $v_{out} = V\dfrac{R_2}{R_1 + R_2}$ | $i_2 = I\dfrac{R_1}{R_1 + R_2}$ | unloaded! |
| Thévenin | $V_{th} = v_{oc}$ | $R_{th} = V_{th}/I_{sc}$ | one line fits all loads |

**Figure 3.7:** Cheat-sheet for Chapter 3. The last row is why Thévenin matters: one measured line replaces the whole internal network.

**DEFINITION OF DONE**

- You classify connections by node counting and reduce any series/parallel network to one resistor.
- You can design and load-check a voltage divider — the most common two resistors in this book.
- KCL, KVL and the node-voltage method solve any linear resistive circuit; you verified one against ngspice to the fourth decimal.
- Thévenin compression tells you when a "supply" will sag, and the meter-loading rule keeps your measurements honest.

## Exercises

**Exercise 3.4** (easy)**.** Three resistors — 1 kΩ, 2 kΩ, 3 kΩ — are placed end-to-end in a ring, terminals taken across the 1 kΩ only. What is $R_{eq}$? (Careful: it is a parallel question.)

**Exercise 3.5** (easy)**.** A node has four wires carrying 2 mA, −5 mA and 4 mA (signs = into node). What does the fourth carry?

**Exercise 3.6** (easy)**.** Using Figure 3.2's labels: $V_{in} = 12\,\text{V}$, $R_1 = 4.7\,\text{k}\Omega$, $R_2 = 10\,\text{k}\Omega$. Find $v_{out}$ and the standing current.

**Exercise 3.7** (easy)**.** True or false, with a one-sentence justification: two 100 Ω resistors in parallel carry the same current as one 50 Ω resistor at the same voltage.

**Exercise 3.8** (medium — derive)**.** Derive the parallel law from KCL: for $n$ resistors on the same node pair at voltage $v$, show $1/R_{\text{eq}} = \sum 1/R_k$; specialise to the product-over-sum two-resistor form.

**Exercise 3.9** (medium — the classic trap)**.** Design a divider that outputs 3.3 V from 5 V *while the output node sources* 10 mA. Hint: you cannot treat the load as infinite — write KCL at the output node including the 10 mA. Choose real E24 values and report the delivered voltage.

**Exercise 3.10** (medium)**.** Find the Thévenin equivalent of: 9 V source, 2 kΩ series, 3 kΩ to ground, terminals at the middle node. Then predict the terminal voltage with a 6 kΩ load.

**Exercise 3.11** (medium)**.** You probe the middle of an unloaded 100 kΩ/100 kΩ divider on 10 V and read 4.762 V. Infer the meter's input resistance.

**Exercise 3.12** (hard — mesh)**.** The circuit of Example 3.6 modified: add a fourth resistor 330 Ω from node 3 to a *new* node 4, and connect node 4 to ground through 680 Ω. Write the two node-voltage equations, solve by hand, then confirm with a netlist you write yourself.

**Exercise 3.13** (hard)**.** A linear network is characterised only by two measurements: open-circuit 4.2 V; with a 1 kΩ load it reads 3.0 V. Reconstruct $V_{\text{th}}$ and $R_{\text{th}}$, and state what load resistance would pull the terminal down to half its open-circuit value.

**WATCH INSTEAD**

- *Batteries, Power, Kirchhoff's Rules* (MIT 8.02x Lecture 10, Walter Lewin, 50:01) — Kirchhoff's voltage and current laws, the same rules used for series and parallel. https://youtu.be/ViwSDL657L4 Listed in Book 8 (ch12), accessed 2026-10-11.

## Hints and answers

**Ring.** 1 kΩ in parallel with the series chain $(2+3)$ kΩ: $R_{\text{eq}} = 1000 \cdot 5000/6000 \approx 833\,\Omega$.

**KCL.** The three given sum to +1 mA; the fourth must carry −1 mA (i.e. 1 mA leaving).

**12 V divider.** $v_{\text{out}} = 12 \times 10/14.7 = 8.16\,\text{V}$; $i = 12/14.7\,\text{k} = 0.816\,\text{mA}$.

**50 ohm.** True — equivalence means identical terminal behaviour; that is the definition's whole point.

**Derivation.** Same $v$ on all $n$ branches, $i_k = v/R_k$, KCL gives $I = v\sum 1/R_k$; compare with $I = v/R_{\text{eq}}$. For $n = 2$: $R_{\text{eq}} = 1/(1/R_1 + 1/R_2) = R_1R_2/(R_1 + R_2)$.

**Loaded divider.** KCL at the output: $(5 - v)/R_1 = v/R_2 + 0.01$ with $v = 3.3$. Pick $R_1 = 100\,\Omega$: 17 mA in, so $3.3/R_2 = 7$ mA, $R_2 \approx 471\,\Omega \to 470\,\Omega$ E24; simulated value $\approx 3.30$ V. Any pair with the same current budget works.

**Thévenin.** $V_{\text{th}} = 9 \times 3/5 = 5.4\,\text{V}$; $R_{\text{th}} = 2\|3 = 1.2\,\text{k}\Omega$; loaded: $5.4 \times 6/7.2 = 4.5\,\text{V}$.

**Meter inference.** Reading $4.762 = 10 \times R_{\text{eq}}/(100 + R_{\text{eq}})$ gives $R_{\text{eq}} \approx 90.9\,\text{k}\Omega$, so $R_{\text{meter}} \approx 1\,\text{M}\Omega$.

**Mesh/nodes.** Equations: $(v_3 - 12)/100 + (v_3 - 6)/220 + v_3/470 + (v_3 - v_4)/330 = 0$; $v_4/680 + (v_4 - v_3)/330 = 0$. Second gives $v_4 = 0.673\,v_3$; substituting, $v_3 \approx 8.34\,\text{V}$, $v_4 \approx 5.61\,\text{V}$.

**Two measurements.** $V_{\text{th}} = 4.2\,\text{V}$; under $1\,\text{k}\Omega$: $3.0 = 4.2 \times 1000/(R_{\text{th}} + 1000)$ gives $R_{\text{th}} = 400\,\Omega$; half open-circuit when $R_{\text{load}} = R_{\text{th}} = 400\,\Omega$.

## Notes

1. Series/parallel-by-nodes presentation and the four-redraws exercise format follow the sequencing of Kuphaldt's Vol. I ch. 5–6 [2]; the example networks are original constructions.
2. The node-voltage method and the weighted-average reading of Example 3.6 track MIT 6.002 lectures 1–3 [10], which also teach Thévenin as a load-line measurement — the basis of Figure 3.5's simulation.
3. Meter-loading numbers use the conventional $10\,\text{M}\Omega$ DMM input resistance [6].
4. All ngspice outputs in this chapter are verbatim from ngspice-47 on the author's machine; the load-line agreement ($< 1 \times 10^{-7}\,\text{V}$) is exact up to the model's truncated $V_{\text{th}}$ [8].

CHAPTER 4

# Capacitors, inductors and RC/RL transients

Chapters 1–3 lived in a world where every voltage settles instantly. Real circuits don't: they have *memory*, and the two components that carry it are the capacitor and the inductor. This chapter gives you the one differential equation this book needs, its famous answer (the exponential, with time constant $\tau$), and the three places transients actually ambush you on a bench: switch debouncing, power-up behaviour, and the decoupling capacitor that guards every chip's supply pin. By the end you can predict a delay to the millisecond, size an RC for it, simulate the response, and explain why opening an inductor's circuit is a way to kill a transistor.

**BEFORE YOU START**

Chapter 3 (series/parallel, dividers). *Recap:* a capacitor stores charge and an inductor stores energy in a magnetic field. Neither obeys Ohm's law — their voltage and current are tied together by a *rate of change*. Calculus is not required; the one derivative we use is explained when it appears.

**AFTER THIS CHAPTER YOU CAN**

- say what a capacitor and an inductor do in a circuit;
- compute the time constant of an RC or RL circuit and sketch the response;
- explain the voltage spike when an inductor's current is cut and add a part to tame it;
- predict when a circuit will ring.

**HOW TO THINK ABOUT THIS**

A capacitor and an inductor are about *change*. A capacitor resists a sudden change of voltage, an inductor a sudden change of current, and both take time to respond. The time constant tells you how long. When a circuit does something surprising, ask what is storing energy and what is being asked to change quickly.

## 4.1 The capacitor: charge bucket, $i = C\,dv/dt$

A *capacitor* is two conductors facing each other across an insulator. Charge pushed onto one plate repels charge off the other; the stored charge is $Q = CV$, where the *capacitance* $C$ in farads measures how much charge one volt packs away. Since current is charge per second, differentiating gives the element law:

$$i = C\,\frac{dv}{dt}.$$

Read it both ways: current *through* a capacitor means its voltage is *changing*, and a capacitor whose voltage is changing draws or supplies current proportional to the rate. Two consequences worth memorising now: capacitor voltage *cannot jump* (a jump would need infinite current), and at DC a charged capacitor is an open circuit — zero $dv/dt$, zero current.

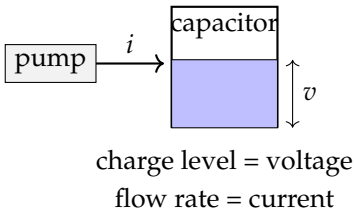



**Figure 4.1:** The bucket picture. Charge is the water level (voltage), current is the flow rate, capacitance is the bucket's width. A wide bucket ($C$ big) fills slowly for the same flow — that is why big capacitors make slow circuits.

The stored energy is $E = \tfrac{1}{2}CV^2$ — real energy, returned later, which is what makes capacitors useful for smoothing and why a charged capacitor on an unplugged board still bites (Section 4.8).

## 4.2 The RC step response

Connect a resistor from a voltage source to an uncharged capacitor (Figure 3.2 of Chapter 3, second element now a $C$). KCL at the capacitor node:

$$\frac{V - v}{R} = C\frac{dv}{dt} \quad\Longrightarrow\quad \frac{dv}{dt} + \frac{v}{RC} = \frac{V}{RC}.$$

This is a first-order linear ODE — the only one this book asks you to derive, once, in the exercises. Its solution for $v(0) = 0$ is

$$v(t) = V\big(1 - e^{-t/\tau}\big), \qquad \tau = RC,$$

where $\tau$ — the *time constant* — is the single number that controls everything. The derivation matters once; the shape matters forever (Figure 4.2).

**Checkpoint 4.1** The sim above uses $R = 1\,\text{k}\Omega$, $C = 1\,\mu\text{F}$. Predict $v_C$ at $t = 2\,\text{ms}$: $5(1 - e^{-2}) = 4.32\,\text{V}$. The simulation agrees to three figures.

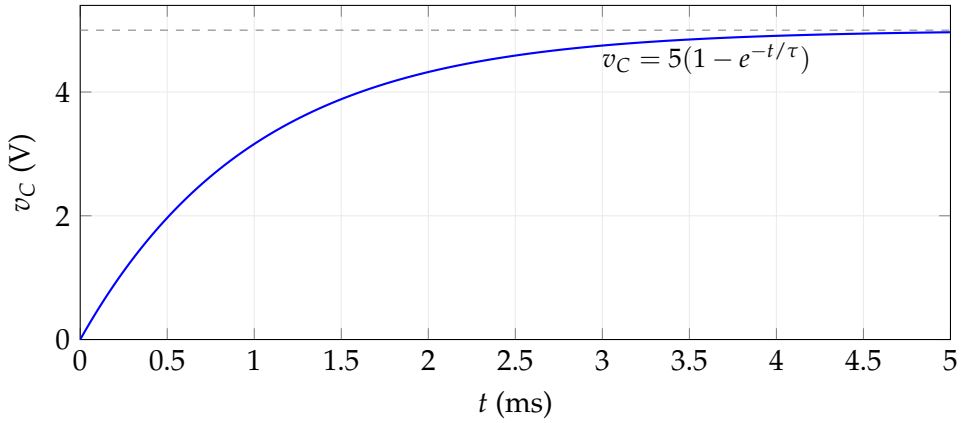



**Figure 4.2:** Real `.tran` output of the RC step: 5 V through 1 kΩ into 1 µF, so $\tau = 1$ ms. Notice the charging is fastest at the start — the gap between target and present voltage is what drives the current, so the approach is always exponential, never linear.

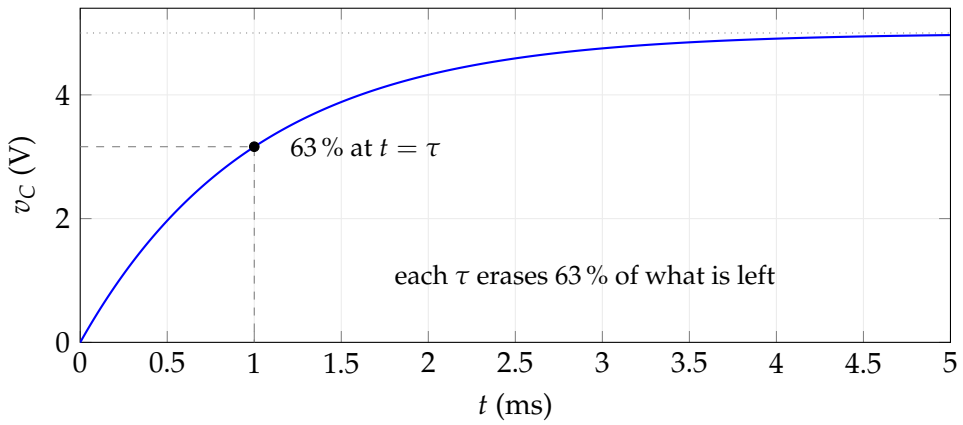



**Figure 4.3:** The number to memorise: at $t = \tau$ the capacitor reaches 63 % ($1 - e^{-1}$) of its target. Each further $\tau$ closes 63 % of the *remaining* gap — at $5\tau$ it is 99.3 % there, which is the engineer's "done".

## 4.3 Charging vs discharging

Discharge is the mirror image: a charged cap through a resistor to ground obeys $RC\,dv/dt = -v$, giving $v(t) = V_0 e^{-t/\tau}$ — the same exponential flipped (Figure 4.4). Charging and discharging are one phenomenon — exponential approach to a target; only the target differs.

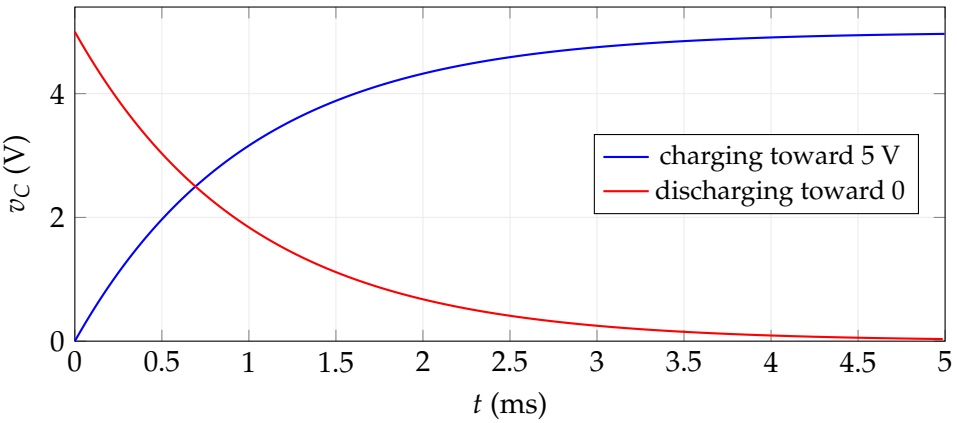



**Figure 4.4:** Same $R$, same $C$, opposite targets — real `.tran` for both. Notice the curves are reflections: at $t = \tau$ the charging curve sits at 63 % of full and the discharging at 37 %.

**Exercise 4.1** (easy). A 1 µF cap charges through 2.2 kΩ toward 5 V. Give $\tau$, and the voltage at $t = \tau$ and $t = 3\tau$. (Answer: 2.2 ms; 3.16 V; 4.75 V.)

**Example 4.1** (A 555-style delay: how long to 3.33 V?)**. Strategy.** The 555 timer (Chapter 13 territory) trips when its RC reaches $\frac{2}{3}V_{CC}$. So the delay question is: for $v(t) = 5(1 - e^{-t/\tau})$, solve $v(t^*) = 3.33\,\text{V}$.

**Solution.** $1 - e^{-t/\tau} = 2/3 \Rightarrow e^{-t/\tau} = 1/3 \Rightarrow t^* = \tau \ln 3 = 1.0986\,\tau$. With $\tau = 1\,\text{ms}$: $t^* = 1.099\,\text{ms}$.

**Check.** The `.tran` data of Listing 4.1 crosses 3.33 V at 1.10 ms — agreement to the third digit, which is all the precision an electrolytic capacitor deserves.

```
* ch04_rc_step.cir -- RC step response: 0->5 V step through R=1k into
    C=1uF
V1 1 0 PULSE(0 5 0 1n 1n 1 1)
R1 1 2 1000
C1 2 0 1u
.control
tran 5u 5m
let vout = v(2)
set filetype=ascii
wrdata ch04_rc_step.dat vout
.endc
.end
```

**Listing 4.1:** `code/ch04_rc_step.cir` — the step that produces Figures 4.2–4.3 (simulation S1). The discharge twin `code/ch04_rc_discharge.cir` is the same file with the source removed and `.ic v(2)=5`.

```
t(ms) v(2)
8.988090e-01 2.964732e+00
9.988090e-01 3.158414e+00
1.098809e+00 3.333664e+00
1.198809e+00 3.492237e+00
```

**Listing 4.2:** Actual rows of `ch04_rc_step_ms.dat` (the ms-rescaled file of Figure 4.3; ngspice's adaptive steps land on these timestamps, not round numbers) — 3.158 V at 0.999 ms is the 63% mark, and 3.334 V at 1.099 ms is the $\tau \ln 3$ crossing of Example 4.1.

## 4.4 The inductor: the dual element

The *inductor* mirrors the capacitor: a coil that stores energy in its magnetic field and obeys

$$v = L\,\frac{di}{dt},$$

dual to $i = C\,dv/dt$. Its time constant in a resistor network is $\tau = L/R$ — current through an inductor cannot jump, and at DC it is a short. We keep it lighter than the capacitor because its role in this book is mostly a warning label — the next section shows why.

**Exercise 4.2** (easy)**.** An RL circuit has $L = 10\,\text{mH}$ in series with $R = 100\,\Omega$. What is $\tau$, and how long until the current reaches 95 % of its final value? (Answer: $100\,\mu\text{s}$; $3\tau = 300\,\mu\text{s}$.)

## 4.5 RL transients and the flyback spike

Energise a coil and its current ramps exponentially toward $V/R$. Now open the switch mid-current. The inductor's rule is absolute: $di/dt$ stays finite, so the current *keeps flowing* — and with nowhere to go it charges whatever stray capacitance exists until the voltage is large enough to find a path. That voltage is the *flyback spike*.

**Example 4.2** (Relay coil opened at 171 mA)**. Strategy.** Coil 10 mH with 70 Ω winding resistance on 12 V: steady current $12/70 = 171\,\text{mA}$. Open the switch; the current has only the 100 pF of stray wiring capacitance to flow into. Energy conservation estimates the peak: $\frac{1}{2}Li^2 \approx \frac{1}{2}Cv^2 \Rightarrow v_{\text{peak}} \approx i\sqrt{L/C}$.

**Solution.** $v_{\text{peak}} \approx 0.171\sqrt{10\,\text{m}/100\,\text{p}} = 0.171 \times 10^4 \approx 1.7\,\text{kV}$.

**Check.** Simulation S3 (Listings 4.3, Figure 4.5) peaks at 1646 V — 96 % of the ideal estimate, the rest eaten by the coil resistance. A real bench sees this as a killed transistor or a switch contact arc; Chapter 5's flyback diode is the fix.

```
* ch04_relay_flyback.cir -- open a relay coil mid-current, watch the
   node fly
* WARNING: the spike this produces (~1.7 kV simulated) is what kills
* transistors. The fix is Chapter 5's flyback diode -- do NOT leave
   the
* inductor's current path open in real hardware. Simulation S3.
```

```
* Topology: 12 V -> Rcoil -> node 2 -> coil L -> node 3 -> switch ->
    gnd
V1 1 0 DC 12
Rcoil 1 2 70
L1 2 3 10m IC=0
S1 3 0 10 0 SW1
Cpar 3 0 100p
Vsw 10 0 PULSE(1 0 1m 1n 1n 1 1)
.model SW1 SW vt=0.5 vh=0.1 ron=1 roff=1e9
.control
tran 500n 1.06m uic
let vnode = v(3)
wrdata ch04_flyback.dat vnode
.endc
.end
```

**Listing 4.3:** `code/ch04_relay_flyback.cir` — simulation S3. The warning comment in the file is the point of the exercise.

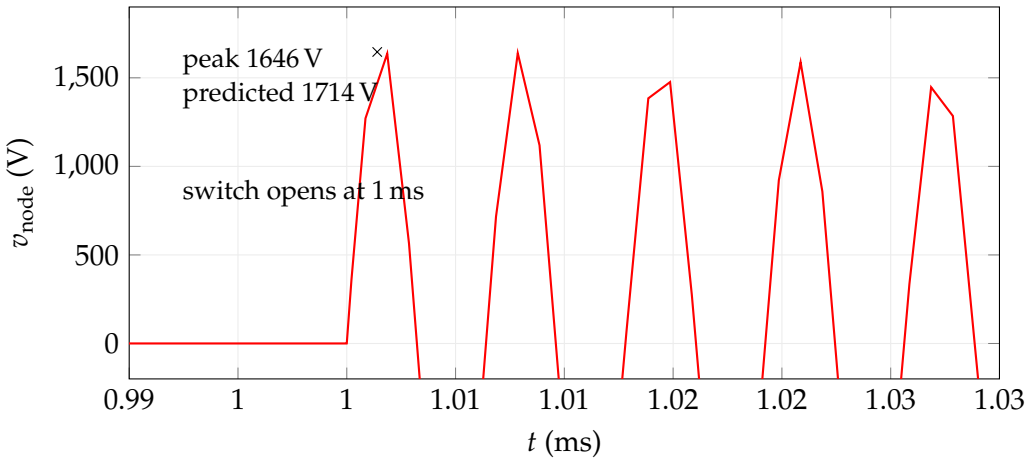



**Figure 4.5:** Real `.tran` output: node 3 sits near ground while the switch conducts, then leaps to 1646 V in about a microsecond when it opens. The marked peak is the estimate of Example 4.2 — within 4 %.

## 4.6 Where transients hide

Three everyday cases, one physics.

**Debounce.** A mechanical contact bounces for milliseconds; an RC filter with $\tau$ of a few ms smooths it into one clean edge (used in Chapter 9's buttons).

**Power-up reset.** Hold a reset pin low while the rail climbs through an RC — instant power-on becomes a deliberate delay, which is why chips come up sane.

**Decoupling.** Every chip's supply pin gets a 100 nF capacitor to ground. When the chip gulps current, the wire's own resistance would let the rail sag; the local capacitor covers the gulp while the supply catches up. Example 4.3 makes it quantitative.

**Example 4.3** (What 100 nF does to a 200 ns gulp)**. Strategy.** Model the supply path as 5 V through 10 Ω of wire and trace impedance; the chip draws a 50 mA step for 200 ns. Without a cap the rail falls by Ohm's law; with one, the cap supplies the step and loses only $\Delta v = I\,\Delta t/C$.

**Solution.** No cap: $\Delta v = 50\,\text{mA} \times 10\,\Omega = 0.50\,\text{V}$ — a 10 % sag, enough to corrupt logic. With 100 nF: $\Delta v = 50\,\text{m} \times 200\,\text{n}/100\,\text{n} = 0.10\,\text{V}$.

**Check.** Simulation S2 (Listing 4.4, Figure 4.6): measured dips 0.50 V and 0.091 V — the 10 % residual is edge-rate, not arithmetic. Decoupling is why chips don't brown out on their own switching.

```
* ch04_decouple.cir -- rail droop under a 200 ns, 50 mA current step
V1 1 0 DC 5
R1 1 2 10
C1 2 0 100n
I1 2 0 PULSE(0 0.05 0.5u 1n 1n 0.2u 1)
.control
tran 1n 1.5u
let vrail = v(2)
set filetype=ascii
wrdata ch04_decouple_cap.dat vrail
.endc
.end
```

**Listing 4.4:** `code/ch04_decouple.cir` — the with-capacitor run; `ch04_decouple_nocap.cir` removes line 5 (simulation S2, both halves).

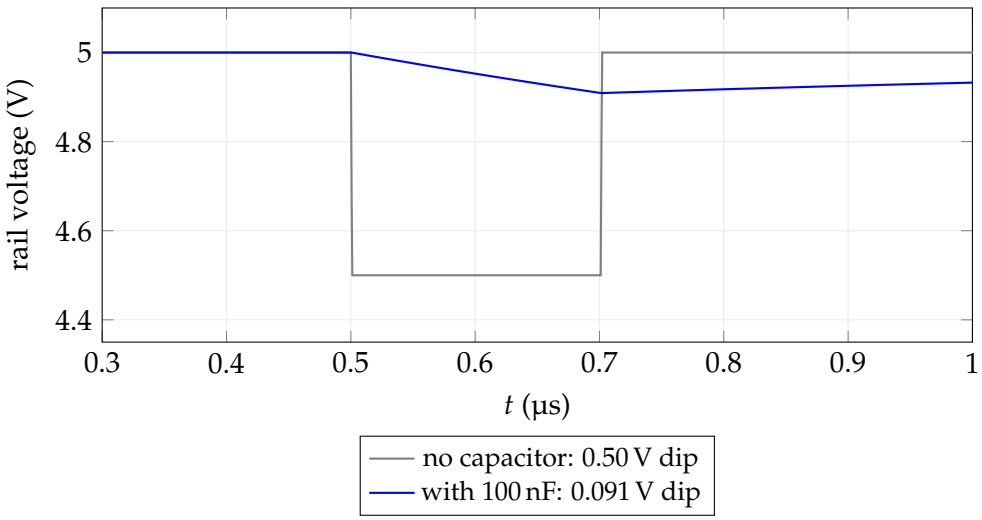



**Figure 4.6:** Real `.tran` data, zoomed to the gulp. Grey: the rail dips 0.50 V bare. Blue: 100 nF holds it to 0.091 V. The capacitor does not remove the droop — it covers the time the supply needs to respond.

```
rescale("ch04_rc_step.dat", "../figures/ch04/ch04_rc_step_ms.dat", 1e3
    , 4)
rescale("ch04_rc_discharge.dat", "../figures/ch04/ch04_rc_discharge_ms.
    dat", 1e3, 4)
rescale("ch04_decouple_cap.dat", "../figures/ch04/ch04_decouple_cap_us.
    dat", 1e6, 3)
```

```
rescale("ch04_decouple_nocap.dat","../figures/ch04/
    ch04_decouple_nocap_us.dat", 1e6, 3)
rescale("ch04_flyback.dat", "../figures/ch04/ch04_flyback_ms.dat", 1e3
    , 2)
```

**Listing 4.5:** `code/ch04_tran_csv.py` — the data pipeline: ngspice writes seconds; this rescales to ms/us for the figures. Every curve in this chapter passes through it.

**Exercise 4.3** (medium)**.** A chip gulps 100 mA for 50 ns. Its 100 nF decoupling cap covers the gulp. How far does the local rail sag? (Answer: $\Delta v = I\Delta t/C = 0.05\,\text{V}$.)

## 4.7 ⋆ Driven RLC: why things ring

Optional, on first reading. Put $L$ and $C$ in one loop and their energies trade back and forth — the capacitor pushes current that the inductor refuses to stop, overshooting the target and oscillating at $\omega_0 = 1/\sqrt{LC}$. Resistance bleeds the oscillation ("damping"); light damping leaves the bell-ringing you saw in Figure 4.5's spike. That is the whole story: flyback spikes, switching noise, and every "why is my square wave wiggly" are this exchange, at different $LC$. Chapter 6 uses $\omega_0$; the exercise asks you to estimate it.

## 4.8 Safety notes

**CAPACITORS KEEP THEIR CHARGE**

A capacitor stores energy *after* the power is off. Big electrolytics can hold a painful voltage for hours — on any board, discharge large caps through a resistor before touching. This book stays under 9 V, but the habit transfers.

**NEVER LEAVE AN ENERGISED INDUCTOR OPEN**

Example 4.2 is not decoration: 1.6 kV from a 12 V relay coil. Real hardware switches coils through a transistor — always with the flyback diode of Chapter 5 fitted, or the transistor dies on the first off-transition.

## 4.9 Common mistakes

**EXPECTING INSTANT CAPACITOR BEHAVIOUR**

"It is just a wire when charged" — true at DC, false during the approach. If a circuit misbehaves in the first milliseconds, suspect the RC that settled slower than you assumed.

**SIGN ERRORS IN THE DISCHARGE DIRECTION**

$v(t) = V_0 e^{-t/\tau}$ decays toward *whatever the capacitor discharges through*, not always ground. Discharging into a held voltage heads toward that voltage. Write the ODE, or at least name the target, before the exponential.

**STEADY-STATE FORMULAS MID-TRANSIENT**

$V = IR$ and the divider rule describe where the circuit *ends*, not what it does on the way. During a transient the only honest tools are the element laws ($i = C\,dv/dt$, $v = L\,di/dt$) and KCL/KVL — as the flyback spike proves with interest.

**FORGETTING INDUCTOR CURRENT CANNOT JUMP**

The inductor's mirror-image rule of the capacitor's. Open an inductor's path abruptly and physics opens it for you — at kilovolts. Always give inductive current somewhere to go.

## 4.10 Chapter summary

| Element / idea | The one-line version | Number |
|---|---|---|
| Capacitor | current $\Leftrightarrow$ voltage *changes*; $i = C\,dv/dt$ | $Q = CV$, $E = \frac{1}{2}CV^2$ |
| RC step | exponential approach to target | $\tau = RC$; 63% at $\tau$, 99% at $5\tau$ |
| Delay | time to threshold $v_T$ from $V$: $t = \tau \ln \frac{V}{V - v_T}$ | 555-style $\frac{2}{3}$: $1.099\,\tau$ |
| Inductor | voltage $\Leftrightarrow$ current *changes*; $v = L\,di/dt$ | $\tau = L/R$ |
| Flyback | opened inductor: $v_{\text{pk}} \approx i\sqrt{L/C}$ | 1.6 kV from 12 V here |
| Decoupling | local cap covers $\Delta t$ gulps: $\Delta v = I\Delta t/C$ | 100 nF per chip pin |
| Uses | debounce $\tau$ ~ms; reset delay; supply smoothing | real circuits in Part II |

**Table 4.1:** Chapter 4 on a page. Everything here is one idea — exponential approach — applied to two dual elements.

**DEFINITION OF DONE**

- You can derive the RC step response, read any exponential's $\tau$, and estimate delays and dips to first order.
- You know why capacitors sit on supply pins, why contacts need debouncing, and why reset lines exist.
- You have watched an inductor make 1.6 kV from 12 V — and you know the diode chapter is coming.
- Every curve you saw is real `.tran` output you can regenerate from `code/ch04_*.cir`.

## Exercises

**Exercise 4.4** (easy)**.** $\tau$ for (a) $R = 4.7\,\text{k}\Omega, C = 10\,\mu\text{F}$; (b) $R = 330\,\Omega, C = 100\,\text{nF}$; (c) $R = 10\,\text{k}\Omega, L = 100\,\text{mH}$ (RL).

**Exercise 4.5** (easy)**.** Charging from 0 toward 9 V through $\tau = 5\,\text{ms}$: what is $v_C$ at $t = 5\,\text{ms}$, 10 ms, 25 ms?

**Exercise 4.6** (easy)**.** A 10 µF cap charged to 5 V: how much charge and energy does it hold?

**Exercise 4.7** (easy)**.** Two 10 µF caps (a) in parallel, (b) in series: give $C_{\text{eq}}$ each way — and which arrangement would you pick for a bigger decoupling reservoir?

**Exercise 4.8** (medium)**.** A 555-style trigger needs 2.5 V from a 9 V rail. Using the step-response formula, what $t/\tau$ ratio is the delay?

**Exercise 4.9** (medium — derive)**.** Derive the step response from the ODE: separate variables in $RC\,dv/dt = V - v$, integrate from $(0, 0)$ to $(t, v)$, and confirm $v = V(1 - e^{-t/\tau})$. Then differentiate back to find $i(t)$ — is it largest at $t = 0$? Why physically?

**Exercise 4.10** (medium — design)**.** You need a power-on delay of about 50 ms before a reset releases, using a 10 µF capacitor and the $\frac{2}{3}V_{CC}$ threshold of Example 4.1. Pick an E24 resistor and report the achieved delay.

**Exercise 4.11** (medium)**.** A 100 mH coil with 100 Ω winding resistance is cut off at its 120 mA steady current; only 50 pF of stray capacitance exists. Estimate the flyback peak — then compare with the safety rulebox.

**Exercise 4.12** (hard)**.** Write the netlist for discharge *into a held level*: a 1 µF cap at 5 V connected at $t = 0$ through 1 kΩ to a 2 V source. Predict $v_C(t)$ and the settled value by writing the ODE, then confirm by simulation.

**Exercise 4.13** (hard)**.** Estimate the ringing frequency if the Example 4.2 coil (10 mH) saw 100 pF: $f_0 = 1/(2\pi\sqrt{LC})$. Compare with the sim's spike shape, and say in one sentence why more stray capacitance would *lower* both the peak voltage and the frequency.

**WATCH INSTEAD**

- *Capacitors and capacitance* (Khan Academy, 5:43) — charge stored between plates, and why a capacitor resists sudden voltage change. https://youtu.be/u-jigaMJT10 Listed in Book 8 (ch11), accessed 2026-10-11.

## Hints and answers

**$\tau$ values.** (a) 47 ms; (b) 33 µs; (c) $L/R = 1$ ms.

**9 V charging.** $9(1 - e^{-1}) = 5.69$ V; $9(1 - e^{-2}) = 7.78$ V; $9(1 - e^{-5}) = 8.94$ V.

**Charge/energy.** $Q = 50\,\mu$C; $E = \frac{1}{2}(10\mu)(25) = 125$ µJ.

**Two caps.** Parallel adds: 20 µF; series halves: 5 µF — opposite of resistors. For a bigger reservoir pick parallel (it also halves the effective series resistance, which is the real reason arrays appear on boards).

**2.5 V threshold.** $v/V = 0.278 \Rightarrow t/\tau = -\ln(1 - 0.278) = 0.325$.

**Derivation.** $\int \frac{dv}{V-v} = \int \frac{dt}{RC}$ gives $-\ln(V - v) + \ln V = t/RC$, i.e. $V - v = Ve^{-t/\tau}$. Then $i = C\,dv/dt = (V/R)e^{-t/\tau}$ — largest at $t = 0$ because the gap (and hence the resistor voltage) is largest then.

**50 ms design.** Need $\tau = 50\,\text{ms}/\ln 3 = 45.5$ ms; with $C = 10$ µF, $R = 4.55\,\text{k}\Omega \to 4.7\,\text{k}\Omega$ E24 gives $\tau = 47$ ms, delay $\approx 51.7$ ms (3 % — fine for reset).

**Second flyback.** $v_{\text{pk}} \approx 0.12\sqrt{100\,\text{m}/50\,\text{p}} = 0.12 \times 4.47 \times 10^4 \approx 5.4$ kV — the diode is not optional.

**Discharge into 2 V.** ODE: $RC\,dv/dt = 2 - v$; solution $v = 2 + 3e^{-t/\tau}$, settling at 2 V — the sim agrees; the target is the *source's* voltage, not zero (the second tripwire in action).

**Ringing.** $f_0 = 1/(2\pi\sqrt{10^{-2} \cdot 10^{-10}}) \approx 159$ kHz — a few microseconds per cycle, matching the sim's sharp edge; more $C$ means more charge to absorb the energy (lower $\sqrt{L/C}$) and a slower exchange (lower $f_0$).

## Notes

1. RC/RL order of topics follows Kuphaldt Vol. I ch. 13–16 [2] (element law, then step response, then applications); the charge-bucket figure and the 63% convention are the standard ones.
2. The decoupling model (rail impedance + current gulp) follows the 6.002 sequence of transient-first teaching [10]; our numbers are chosen for legibility, not vendor data.
3. Flyback energy argument ($v \approx i\sqrt{L/C}$) and the switch model are reproduced faithfully by ngspice-47 [8]; real coils differ mostly in stray capacitance, which only moves the peak, not the physics.
4. Debounce/reset/decoupling placements mirror the applications sections of [2]; the diode fix is deferred to Chapter 5 as the plan's hook.

CHAPTER 5

# Diodes

Every component so far has been *linear*: double the voltage and you double the current. The diode breaks that. It conducts in one direction and blocks in the other, which sounds like a switch but behaves like something subtler — a one-way valve with a pressure threshold. One junction of silicon buys you rectification, flyback protection, voltage references, reverse-polarity guards, and light. After this chapter you will know which of the three diode models to reach for in a calculation, design a half-wave rectifier and estimate its ripple, add the flyback diode that should have been next to ch. 4's relay all along, build a zener shunt regulator and — just as important — know when not to.

**BEFORE YOU START**

Chapters 2–4. *Recap:* a diode is a one-way valve for current. It is the first part you meet that does not obey a straight-line $V$–$I$ rule.

**AFTER THIS CHAPTER YOU CAN**

- use the three diode models and choose the one that fits the question;
- pick a resistor for an LED;
- build a rectifier and estimate its ripple;
- protect a circuit from reversed polarity, ESD and inductive kick-back.

**HOW TO THINK ABOUT THIS**

A diode is a gate that opens only one way, and after it opens it holds the voltage almost constant. That single idea gives LEDs, rectifiers and protection circuits. Choose the simplest model that answers your question: an ideal switch for quick estimates, a fixed drop for design.

## 5.1 The junction diode: one curve, three models

A diode is a *p*–*n* junction: silicon doped positive on one side, negative on the other. Its whole story is one curve, plotted in Figure 5.1 from a real DC sweep.

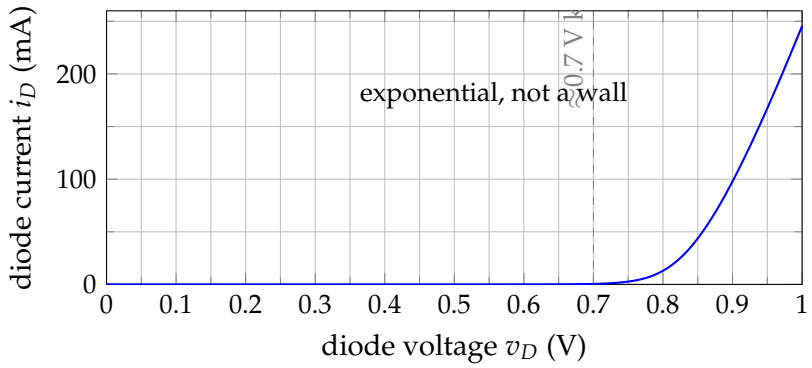



**Figure 5.1:** Diode *I*–*V* curve, real `.dc` sweep of the silicon model used all chapter. Notice there is no wall: conduction rises exponentially through the "knee" near 0.7 V. At 0.6 V it passes 0.014 mA; at 0.7 V, 0.48 mA; at 0.8 V, 13 mA.

Physics explains the curve with the *Shockley equation*: current grows exponentially with forward voltage, $i = I_s\left(e^{v/(nV_T)} - 1\right)$, where $V_T \approx 26\,\text{mV}$ at room temperature. We state it, never derive it — the point of a diode in this book is the curve's *shape*, not its semiconductor derivation. What the shape buys you: current is a steep function of voltage, so voltage is a *flat* function of current. That is why a diode makes a usable voltage reference.

Nobody computes with the exponential. You pick one of three models by how much precision the question needs:

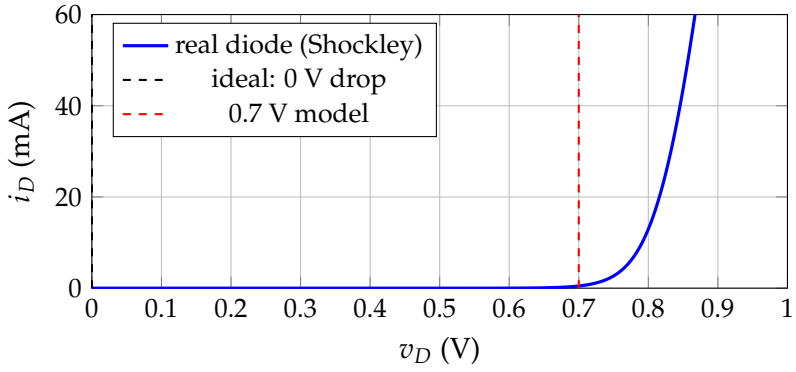



**Figure 5.2:** The three diode models on one axis. The ideal model says zero drop whenever it conducts; the 0.7 V model says the drop is a constant; the real curve (Shockley, simulated) sits between them and is never wrong — it is just useless for hand arithmetic.

**Definition 5.1** (The three diode models)**.**

- **Ideal model** — a perfect one-way switch: conducts with 0 V drop, blocks completely below it. Use for direction-of- flow reasoning and topology questions.
- **Constant-0.7 V model** — conducts at a fixed 0.7 V forward drop, blocks below it. The workhorse of hand circuit analysis; correct to within tens of millivolts in typical circuits.

- **Shockley model** — the real exponential. Use only in the simulator, where the computer does the exponential for you.

**Checkpoint 5.1** An LED circuit needs 5 mA. Which model do you use to size the resistor — and which does the simulator use? (Hand work: 0.7 V-style constant drop, with the LED's real $V_f$ from the datasheet. The simulator: Shockley, always.)

## 5.2 LEDs and forward voltage

An LED is a diode that wastes its recombination energy as light instead of heat. The physics consequence you can touch: its forward drop is set by the photon energy, so $V_f$ depends on colour — roughly 1.8 V red, 2.2 V green/yellow, 3.0 V to 3.4 V blue and white. Get the number from the datasheet, not from the generic 0.7 V.

**Exercise 5.1** (easy)**.** A red LED ($V_f = 1.8$ V) runs from 5 V through a resistor at 10 mA. Size the resistor — once with the correct $V_f$, once with the 0.7 V silicon guess. How far off is the current with the wrong model?

*Solution.* $R = (5 - 1.8)/0.01 = 320\,\Omega$ correct. Wrong model gives $(5 - 0.7)/0.01 = 430\,\Omega$; real current then $(5 - 1.8)/430 = 7.4$ mA — 26% low and visibly dimmer. Using the 0.7 V model on a non-silicon $V_f$ is the LED mistake.

## 5.3 The rectifier: AC in, lumpy DC out

Put a diode in series with an AC source and a load: it passes the positive half-cycles and blocks the negative ones. That is the *half-wave rectifier* — the simplest possible DC supply, and the reason wall-wart chargers exist. Add a reservoir capacitor across the load and the gaps between half-cycles get filled in from stored charge: the capacitor discharges through the load between peaks, giving DC with a residual *ripple*.

**Example 5.2** (Half-wave rectifier: predict the ripple, then simulate it)**. Strategy.** The output tops out near the input peak minus one diode drop. Between peaks the capacitor discharges through the load for almost a full period $T$; treat the discharge as roughly linear and you get $\Delta v \approx I\,T/C$ — the same charge-bucket arithmetic as ch. 4's decoupling cap. Simulate to see how honest that estimate is.

**Solution.** Source: 10 V peak, 50 Hz ($T = 20$ ms); load 1 kΩ; $C = 100$ µF. Peak output $\approx 10 - 0.8 = 9.2$ V, so load current $\approx 9.2$ mA (it droops as the voltage does). Linear estimate: $\Delta v \approx \frac{9.2\text{m}\cdot 20\text{m}}{100\mu} \approx 1.8$ V peak-to-peak. The true discharge is exponential with $\tau = RC = 100$ ms, so the estimate is conservative — as the capacitor sags the load takes less current, which slows the sag.

```
* ch05_rectifier.cir -- half-wave rectifier, 10 Vpk 50 Hz, 100 uF filter
.model DSI D(is=1e-14 n=1.1 rs=0.5)
Vin 1 0 sin(0 10 50)
D1 1 2 DSI
```

```
R1 2 0 1000
C1 2 0 100u
.control
tran 20u 100m
wrdata ch05_rectifier.dat v(1) v(2)
meas tran VMAX MAX v(2) from=60m to=100m
meas tran VMIN MIN v(2) from=60m to=100m
.endc
.end
```

**Listing 5.1:** `code/ch05_rectifier.cir -- one diode, one capacitor, 50 Hz sine. The meas commands quantify the ripple over the last two cycles.`

```
vmax = 9.19186e+00 at= 6.52056e-02
vmin = 7.68742e+00 at= 6.32056e-02
```

**Listing 5.2:** Output, verbatim from ngspice-47.

Measured: $9.19 - 7.69 = 1.50\,\text{V}$ p-p ripple against the predicted $\approx 1.8\,\text{V}$ — the estimate overshoots by a fifth, exactly as the exponential argument says it should. Swap in 470 µF (`code/ch05_rectifier_bigc.cir`) and the measured ripple drops to 0.35 V; the linear estimate predicts $1.8 \cdot 100/470 \approx 0.39\,\text{V}$. Ripple scales with $1/C$ — the estimate is a usable upper bound, the simulator is the truth.

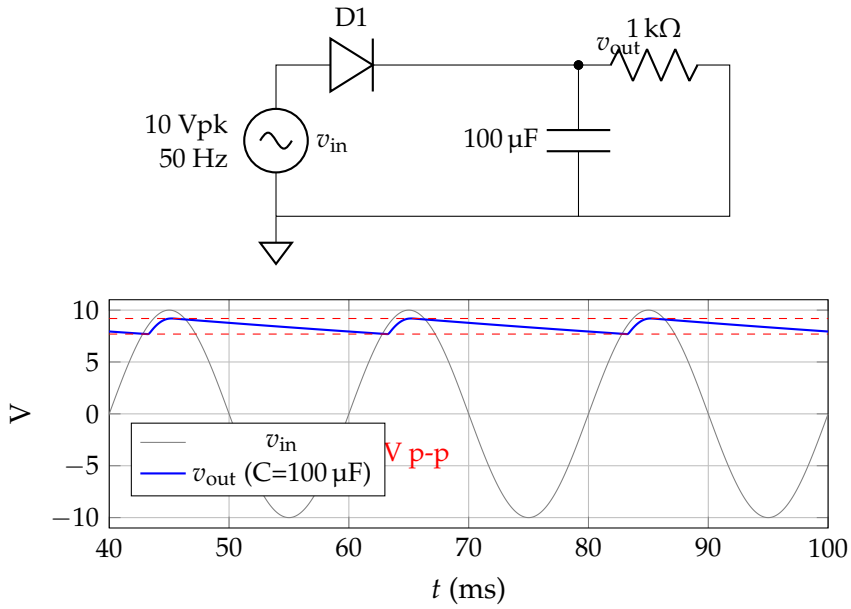



**Figure 5.3:** Half-wave rectifier and its real `.tran` waveforms. Notice the output is a decaying exponential between brief recharging peaks, not a sine minus a piece. The diode only conducts for the short window where $v_{in}$ climbs above $v_{out} + 0.7$.

**Exercise 5.2** (medium)**.** Same rectifier, but the load is a 10 mA constant sink and $C = 220\,\mu\text{F}$. Predict the p-p ripple.

*Solution.* $\Delta v \approx IT/C = 10\text{m} \cdot 20\text{m}/220\mu \approx 0.9\,\text{V}$. (Constant current makes the linear estimate nearly exact — no drooping load to save you.)

## 5.4 The flyback diode: ch. 4's missing part

Chapter 4 opened a relay coil and watched the switch node jump to 1646 V. In a real circuit that spike lands on a transistor rated for maybe 40 V — once. The fix is one diode across the coil, *freewheeling*: anode at the switched node, cathode at the supply rail. While the coil is driven, the diode is reverse-biased and invisible. The instant the switch opens, the coil's stored current keeps flowing — inductor current cannot jump — and now it has somewhere to go: through the diode, clamping the node at rail plus one drop.

```
* ch05_flyback_diode.cir -- ch4 relay coil with a freewheel diode
* WARNING: omitting this diode is what kills real transistors.
.model SW1 SW vt=0.5 vh=0.1 ron=1 roff=1e9
.model DSI D(is=1e-14 n=1.1 rs=0.5)
V1 1 0 dc 12
Rcoil 1 2 70
L1 2 3 10m IC=0
S1 3 0 10 0 SW1
Cpar 3 0 100p
D1 3 1 DSI
Vsw 10 0 pulse(1 0 1m 1n 1n 1 1)
.control
tran 500n 1.06m uic
wrdata ch05_flyback_diode.dat v(3) i(l1)
meas tran VPK MAX v(3)
.endc
.end
```

**Listing 5.3:** `code/ch05_flyback_diode.cir -- identical to ch. 4's flyback netlist except one added line: D1 3 1 DSI. This diode is the difference between a circuit that works and a dead transistor.`

```
vpk = 1.30279e+01 at= 1.00001e-03
```

**Listing 5.4:** Output, verbatim. Same relay, same switch — the 1646 V spike is gone.

13.0 V: rail plus one diode drop. The coil current decays gently through the diode instead of detonating the voltage node.

*Remark.* This is not a textbook scare story. Relay drivers, solenoid valves, motor H-bridges and DC-DC converter switches all die this way in real products — the failure signature is a transistor that measures shorted drain-to-source. Every bench engineer has a "forgot the flyback diode" story; the part costs cents.

**Exercise 5.3** (hard)**.** Pick a flyback diode rating for our relay: coil 12 V, 70 Ω resistance, 10 mH, switched by a 40 V-rated transistor.

*Solution.* Peak diode current = coil current at switch-off = $12/70 = 171$ mA. Peak reverse voltage = supply = 12 V (plus margin). Any diode rated $\geq$200 mA, $\geq$25 V works — a 1N4148 (200 mA, 75 V) is the classic answer here; a 1N4007 if the coil were fat.

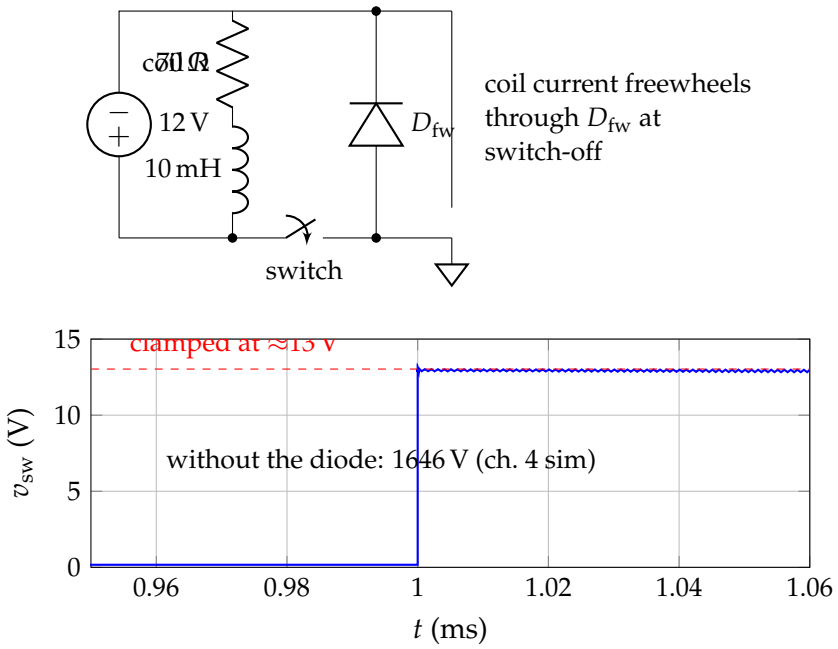



**Figure 5.4:** Freewheel diode placement and the real clamped waveform. Notice the diode is *backwards* — it can never conduct while the switch is on; it exists purely for the turn-off instant.

## 5.5 The zener and the shunt regulator

A *zener* is a diode manufactured to break down cleanly in *reverse*: above its rated $V_z$ it conducts hard, holding its terminals near $V_z$. Use that flat spot as a voltage reference: feed it through a series resistor, hang your load across the zener, and it absorbs whatever current the load doesn't take. That is the *shunt regulator*.

**Definition 5.3** (Zener diode)**.** A diode run intentionally in reverse breakdown, where $v \approx V_z$ over a range of currents. The shunt regulator is a series resistor plus a zener to ground: the resistor takes the rail-to-$V_z$ difference, the zener takes the leftover current.

**Example 5.4** (A 3.3 V reference from 9 V, sized and simulated)**. Strategy.** Size $R_s$ so the zener still sees a few milliamps at *maximum* load (otherwise it drops out of breakdown), and so its power is safe at *zero* load (when it takes everything). Then sweep the load current and look at what "regulates" really means.

**Solution.** Load: up to 30 mA. Give the zener ∼2 mA margin at full load: $R_s = (9 - 3.3)/(30\text{m} + 2\text{m}) \approx 180\,\Omega$. Worst-case resistor power at zero load: $(5.7)^2/180 \approx 0.18\,\text{W}$ — a quarter-watt part is adequate, barely. Worst-case zener power: $3.3 \cdot 30\text{m} \approx 0.1\,\text{W}$.

```
* ch05_zener.cir -- 3.3 V zener shunt regulator from 9 V
.model DZ3V3 D(is=1e-14 n=1.0 rs=5 bv=3.3 ibv=0.02)
V1 1 0 dc 9
Rs 1 2 180
Dz 0 2 DZ3V3
IL 2 0 dc 0
.control
dc IL 0 0.03 0.0005
wrdata ch05_zener.dat -i(v1) v(2)
```

```
.endc
.end
```

**Listing 5.5:** `code/ch05_zener.cir -- shunt regulator; the .dc sweep drives a current source as the load and records` $v_{\text{out}}$`.`

```
IL (mA)  supply I (mA)  vout (V)
0.0      30.75          3.465
10.0     31.07          3.407
20.0     31.43          3.343
30.0     31.95          3.250
```

**Listing 5.6:** Selected rows of the `.dc` sweep (every 10th of 61): load current, total supply current, output voltage. Notice $v_{\text{out}}$ slides 0.22 V end to end — this is regulation, not magic.

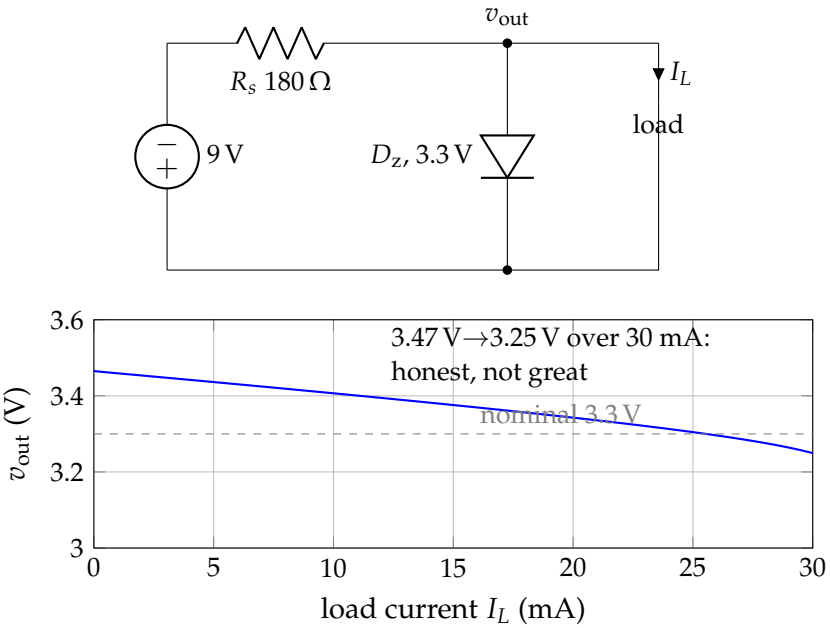



**Figure 5.5:** Zener shunt regulator and its real load-regulation curve. Notice the output sits *above* 3.3 V at light load and sags below it at full load — a low-voltage zener's knee is soft, so "3.3 V" is a neighborhood, not a number.

Honest verdict on the shunt regulator: it is simple, it is robust, it wastes power continuously (the resistor burns ∼0.2 W always), and it regulates loosely. Reach for it when you need a cheap *reference* for milliamps of bias or a comparator threshold — never for powering anything that cares.

**Exercise 5.4** (medium)**.** Your load can pull 20 mA and you want the zener to keep ≥5 mA at worst. What is the largest legal $R_s$ from 9 V, and what power must it survive at zero load?

*Solution.* $R_{s,\max} = (9 - 3.3)/(20\text{m} + 5\text{m}) = 228\,\Omega$ — pick 220 Ω. Zero-load power: $(5.7)^2/220 = 0.15$ W, so a quarter-watt part survives with 40% margin — use a half-watt if the chassis gets hot.

## 5.6 The diode in the wild: ESD and reverse-polarity guards

Two more jobs you will meet diodes doing on every board you open. *ESD protection*: tiny fast diodes from a signal line to each rail clamp electrostatic hits (a finger is a few-kilovolt capacitor) before they reach the chip. *Reverse-polarity protection*: a diode in series with the supply so a backwards plug or reversed battery does nothing — the diode blocks the negative rail outright.

The series guard costs you one forward drop, always. That is the whole trade, quantified:

**Example 5.5** (Reverse-polarity protection: silicon vs Schottky vs no guard)**. Strategy.** Price the protection: what voltage does the load lose forwards, what does it see backwards, and what power does the diode burn. A Schottky diode trades a lower drop for looser blocking.

**Solution.** 5 V supply, 100 Ω load (50 mA class), series diode:

```
* ch05_reverse_pol.cir -- series-diode reverse-polarity guard
.model DSI D(is=1e-14 n=1.1 rs=0.5)
V1 1 0 dc 5
D1 1 2 DSI
R1 2 0 100
.control
op
print v(1) v(2) i(v1)
alter v1 dc=-5
op
print v(1) v(2) i(v1)
.endc
.end
```

**Listing 5.7:** `code/ch05_reverse_pol.cir -- alter re-runs the operating point with the supply flipped; ch05_reverse_pol_sch.cir swaps the silicon model for Schottky.`

```
v(1) = 5.000000e+00 v(2) = 4.152554e+00 i = -4.15255e-02
v(1) = -5.00000e+00 v(2) = -5.01000e-10 i = 5.01110e-12
v(1) = 5.000000e+00 v(2) = 4.760269e+00 i = -4.76027e-02
```

**Listing 5.8:** Outputs, verbatim: silicon forward, reversed, then the Schottky run.

Silicon: the load gets 4.15 V and the diode burns $0.85 \cdot 42\text{m} \approx 36$ mW. Reversed: the load sees ∼0 V and the circuit pulls picoamps. Schottky: 4.76 V delivered, only 0.24 V sacrificed and ∼11 mW burned. The trade: Schottkys leak more in reverse and cost slightly more — and "just don't plug it in wrong" remains the cheapest option on the table.

## 5.7 Safety

**DECISION RULE**

Rectifier circuits in this chapter are simulated at 10 V and lower. Real rectifiers live next to mains — that is out of scope for this book, full stop. The same diode that is a gentle one-way valve at 5 V is part of a lethal circuit at 230 V.

**DECISION RULE**

An inductor with no freewheel path does not negotiate — it makes the voltage it needs. Never switch a relay, solenoid, or motor coil without the diode fitted first, and never trust a "it worked on the bench" argument: the transistor fails on the Nth switch, not the first.

## 5.8 Common mistakes

**TREATING THE DIODE AS A ZERO-VOLT DROP**

The ideal model is for direction questions only. The 0.7 V the load never sees is also power the diode has to dissipate — 1 A through a series guard is 0.7 W to heat.

**FORGETTING THE REVERSE-VOLTAGE LIMIT**

Every diode has a peak inverse voltage rating (1N4007: 1000 V; a small-signal diode may only manage 75 V). Exceed it and the diode breaks down — sometimes usefully (a zener), usually not (it fails shorted).

**SIZING LED CURRENT OFF THE IDEAL MODEL**

"The diode drops zero, so $R = V/I$" leaves nothing for the LED's real 1.8 V–3.4 V $V_f$. Always subtract $V_f$ first — and get it from the datasheet, because colour sets the voltage.

**BELIEVING A ZENER HOLDS ITS RATED VOLTAGE**

Low-voltage zeners have soft knees: our "3.3 V" part delivered 3.25 V to 3.47 V across the load range. If your circuit cares about 6%, you need a real reference IC, not a shunt regulator.

## 5.9 Summary

| Object | What you need to know |
|---|---|
| Diode models | ideal (0 V) for direction; 0.7 V for hand math; Shockley exponential in the simulator |
| LED | a diode that emits; $V_f$ is colour-set, 1.8 V–3.4 V — read the datasheet |
| Rectifier | half-wave: one diode; cap filters gaps; $\Delta v \lesssim IT/C$ p-p ripple; more $C$, less ripple |
| Flyback diode | across the coil, backwards; clamps switch-off spike at rail+drop — never optional |
| Zener | reverse breakdown held at $V_z$; shunt regulator = $R_s$ + zener; loose ($\pm$5–10%), always dissipating |
| Guards | series diode blocks reversed supply at one drop; Schottky halves the toll; ESD diodes clamp signal pins to the rails |

## Exercises

**Exercise 5.5** (easy)**.** An ideal-model analysis says a diode drops nothing. Your circuit feeds 9 V through one diode to a 3 V zener clamp. What does the 0.7 V model say the clamp sees? When does the 0.7 V matter?

**Exercise 5.6** (easy)**.** Green LED, $V_f = 2.1$ V, from 3.3 V at 8 mA: size the resistor.

**Exercise 5.7** (easy)**.** Which model do you use to (a) decide whether a diode conducts at all, (b) size a series resistor by hand, (c) check a circuit in ngspice?

**Exercise 5.8** (easy)**.** In a half-wave rectifier, which half of the AC cycle appears at the output — and what decides which half? Trace the current path for the conducting half.

**Exercise 5.9** (medium)**.** A rectifier must hold ripple under 0.5 V p-p at 20 mA, 50 Hz. What is the smallest capacitor the linear estimate allows?

**Exercise 5.10** (medium)**.** Why is $\Delta v \approx IT/C$ an *overestimate* of the ripple for a resistive load but nearly exact for a constant-current load?

**Exercise 5.11** (medium)**.** A relay coil: 24 V, 15 mH, 120 Ω, switched at 10 Hz. Specify a flyback diode (current and reverse-voltage rating with margin) and estimate the energy dissipated per switch-off.

**Exercise 5.12** (medium)**.** A 4.3 V-logic board must survive reversed 5 V input. Compare series silicon diode, series Schottky, and "no guard" on delivered voltage and dissipation at 100 mA.

**Exercise 5.13** (hard)**.** An LED matrix row drives 8 parallel LEDs ($V_f = 2.0$ V each) from 5 V through one shared resistor. Why is one resistor per LED the correct design instead? (Hint: what happens to current sharing when one LED's $V_f$ is 0.1 V low?)

**Exercise 5.14** (hard)**.** Design a zener reference for 5.1 V ±0.2 V into a load of 0 mA–10 mA from a 12 V rail: choose $R_s$, bound the zener and resistor power, and estimate the real output range given our measured low-$V_z$ slope (∼7 mV/mA).

## Hints and answers

**Clamp after diode.** $9 - 0.7 = 8.3$ V across $R_s$, so the zener sees slightly less drive current — the drop matters when $R_s$ is tight or the rail is low; it's why low-voltage designs use Schottky.

**Green LED.** $R = (3.3 - 2.1)/0.008 = 150\,\Omega$.

**Model picking.** (a) ideal, (b) 0.7 V (or LED $V_f$), (c) Shockley — the simulator never uses your shortcuts.

**Rectifier half.** Whichever half forward-biases the diode — anode side positive. Flip the diode to pick the other half.

**Ripple cap.** $C \geq IT/\Delta v = 20\text{m} \cdot 20\text{m}/0.5 = 800\,\mu\text{F}$ — the estimate is an upper bound on ripple, so this is conservative; buy 1000 µF.

**Overestimate.** Resistive load current sags with the voltage, so the bucket empties slower than $I \cdot t$ assumes; a constant-current load never sags, so the linear discharge is real.

**24 V relay.** $I_{\text{pk}} = 24/120 = 200$ mA; diode ≥250 mA, ≥50 V (e.g. 1N4001). Energy per switch-off: $\frac{1}{2}LI^2 = \frac{1}{2} \cdot 0.015 \cdot 0.2^2 = 0.3$ mJ — at 10 Hz, 3 mW in the diode loop.

**Reversed guard.** At 100 mA: silicon delivers ≈4.15 V and burns ≈85 mW; Schottky ≈4.76 V, ≈24 mW; no guard delivers 5 V and dies if someone plugs it in wrong.

**LED matrix.** A shared resistor sets total current but the LED with the lowest $V_f$ hogs it — exponential $I$–$V$ means 0.1 V low doubles its share. One resistor per LED fixes each branch's own current.

**5.1 V reference.** $R_s \leq (12 - 5.1)/15\text{m} = 460\,\Omega$; pick 430 Ω. Resistor power $(6.9)^2/430 \approx 0.11$ W; zener worst case $5.1 \cdot 16\text{m} \approx 0.08$ W. At ∼7 mV/mA the output stays inside ±0.2 V up to ∼28 mA of load swing — yours has 10. Feasible.

## Notes

## Notes

The three-models presentation and the Shockley-stated-not-derived choice follow the layered-model approach of the 6.002 nonlinear lectures [10] and All About Circuits vol. III's diode chapter [3]. The flyback-diode-as-required-instrument framing is standard bench practice; the simulated spike and clamp are real ngspice runs of the relay from ch. 4. Zener knee softness at low $V_z$ is a documented device property, which is why the chapter's regulator verdict is honest rather than enthusiastic [3].

**DEFINITION OF DONE**

- Read a diode $I$–$V$ curve and pick the right model for the job.

- Predict and verify rectifier ripple; know why $IT/C$ overestimates.
- Add the flyback diode to any inductive switch — and justify its rating.
- Build and honestly evaluate a zener shunt reference.
- Price reverse-polarity protection in volts and milliwatts.

CHAPTER 6

# Transistors: the switch and the amplifier

Everything digital in this book — logic gates, the FPGA fabric you will program later — is built from one trick, repeated a billion times: a voltage that controls another current. This chapter introduces the transistor as a *switch* first (the model you will actually use), then as an *amplifier* in one honestly-flagged example. When you finish, you will be able to read a voltage-transfer curve for noise margins, explain why CMOS logic draws almost no power at rest, and decide whether a given MOSFET can really drive a given load from a 3.3 V pin.

**BEFORE YOU START**

Chapters 2 and 5. *Recap:* a transistor is a switch that another voltage or current controls. Everything digital in this book is made of them.

**AFTER THIS CHAPTER YOU CAN**

- explain how a MOSFET works as a voltage-controlled switch;
- build an inverter and explain why CMOS draws almost no power at rest;
- read a logic level as "1" or "0" with noise margins;
- drive a real load such as a motor or LED with a transistor.

**HOW TO THINK ABOUT THIS**

A transistor is a valve: a small signal at the control terminal opens or closes a large path. Use it as a switch and you get digital logic; use it in between and you get amplification. CMOS pairs two complementary valves so that one is always closed, which is why idle chips use almost no power.

## 6.1 The MOSFET: a voltage-controlled switch

The *MOSFET* (metal-oxide-semiconductor field-effect transistor) has three terminals we care about — *gate* ($G$), *drain* ($D$), *source* ($S$) — drawn in Figure 6.1. The gate sits on an insulating oxide layer: it controls the device by *electric field*, not by current, so an ideal gate draws no DC current at all.

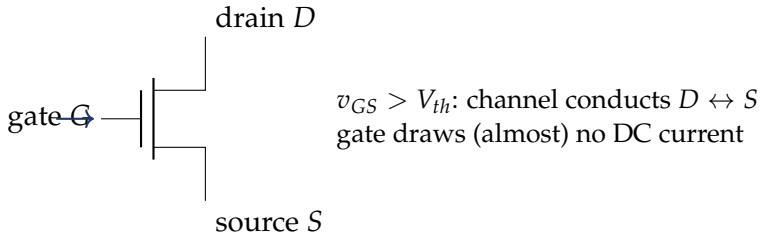



**Figure 6.1:** An NMOS transistor and its three terminals. Notice what the gate does *not* do: it draws essentially no DC current — it works by field, not by flow.

**Definition 6.1** (NMOS threshold)**.** An NMOS device conducts between drain and source when the *gate-source voltage* exceeds a device constant $V_{th}$, the *threshold voltage* (a volt or two for small-signal parts). Below $V_{th}$ the channel is essentially open.

The 6.002 pedagogy layers two models on this [10]:

**Definition 6.2** (S and SR models)**.** The *S (switch) model*: $v_{GS} > V_{th} \Rightarrow$ drain and source are a wire; otherwise an open circuit. The *SR (switch-resistor) model*: on, the channel behaves as a small resistance $R_{on}$ instead of an ideal wire — the model you reach for once a load current flows.

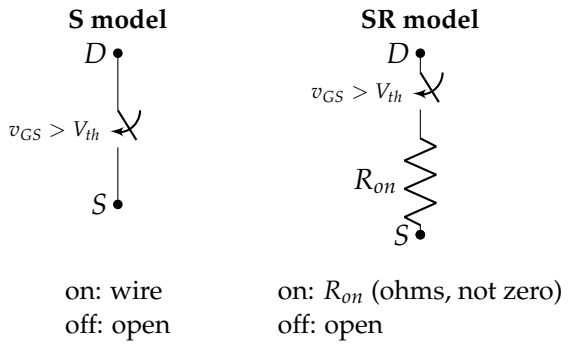



**Figure 6.2:** The two switch models. Use S to reason about logic levels; use SR the moment real current must flow, because $R_{on}$ drops real volts.

*Remark.* Gate current is not literally zero — charging the gate capacitance takes charge, which is why fast switching burns power (Section 6.4) and why FPGAs get warm. "No DC gate current" is a DC statement.

## 6.2 The NMOS inverter, built by hand

Pull the drain up to $V_{DD}$ through a resistor and drive the gate with the input: when $v_{in} > V_{th}$ the channel conducts and drags the output down; below threshold the resistor pulls the output up. That is an *inverter* — input 1 gives output 0 and vice versa. Figure 6.3 plots its *voltage-transfer characteristic* (VTC).

**Example 6.3** (Reading noise margins off a real VTC)**. Strategy.** Simulate the NMOS + 10 kΩ pull-up inverter, then read the four landmarks: the flat output levels $V_{OH}/V_{OL}$ and the input voltages $V_{IL}/V_{IH}$ where the curve leaves those flats (the unit-slope points, read approximately).

**Solution.** The netlist and the transfer data:

```
* ch06_nmos_inv.cir -- NMOS + pull-up resistor inverter, .dc transfer
VDD 3 0 dc 3.3
Vin 1 0 dc 0
M1 2 1 0 0 NMOS1 W=10u L=1u
RD 3 2 10k
.model NMOS1 NMOS(vto=1.0 kp=200u lambda=0.02)
.control
dc Vin 0 3.3 0.025
set filetype=ascii
wrdata ch06_nmos_inv.dat v(2)
* static supply current at vin = 3.3 V (inverter holding "low")
dc Vin 3.3 3.3 0.01
print i(VDD)
.endc
.end
```

**Listing 6.1:** `code/ch06_nmos_inv.cir -- NMOS inverter with a resistive pull-up.`

```
vin vout
0.500 3.3000
1.000 3.3000
1.250 2.6420
1.500 0.7619
1.750 0.2419
2.000 0.1705
3.000 0.0820
3.300 0.0712
```

**Listing 6.2:** Rows of `ch06_nmos_inv.dat`, verbatim — read $V_{OH}$, $V_{OL}$, and where the knee sits.

Reading Figure 6.3: $V_{OH} = 3.30\,\text{V}$, $V_{OL} \approx 0.07\,\text{V}$, $V_{IL} \approx 1.2\,\text{V}$ (output just starting to fall), $V_{IH} \approx 1.7\,\text{V}$ (output nearly bottomed). The margins of Definition 6.4 are then $NM_L \approx 1.2 - 0.07 \approx 1.1\,\text{V}$ and $NM_H \approx 3.3 - 1.7 \approx 1.6\,\text{V}$.

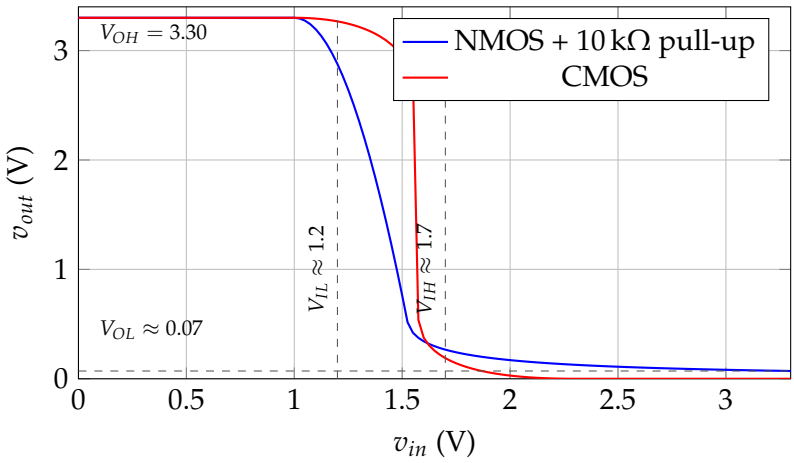



**Figure 6.3:** Measured inverter transfer curves (both real `.dc` sweeps). Notice how much cleaner the CMOS knee is: it snaps over near $V_{DD}/2$ and actually reaches the rails.

## 6.3 The static discipline: what makes a level a “1”

Digital only works if stages can be chained: the output of one inverter must be read correctly by the input of the next, with slack for real-world noise.

**Definition 6.4** (Static discipline and noise margins)**.** A logic family fixes four voltages: a sender guarantees $v \le V_{OL}$ (“0”) or $v \ge V_{OH}$ (“1”); a receiver accepts $v \le V_{IL}$ as 0 and $v \ge V_{IH}$ as 1, with $V_{OL} < V_{IL} < V_{IH} < V_{OH}$. The gaps are the *noise margins* $NM_L = V_{IL} - V_{OL}$ and $NM_H = V_{OH} - V_{IH}$ — how much corruption a wire can absorb before a bit flips.

**Exercise 6.1** (easy)**.** On Figure 6.3’s NMOS curve, is 0.8 V a safe “0” input? Is 1.4 V a safe “1” input? (Use the landmarks just computed.)

## 6.4 CMOS: the pair that draws no power at rest

The resistor pull-up has a defect you can see in the netlist’s own printout: holding the output low means burning $V_{DD}/R$ forever.

```
i(vdd) = -3.22881e-04
```

**Listing 6.3:** Last lines of `ngspice -b ch06_nmos_inv.cir` — the static current while holding output low.

0.32 mA continuously, *per inverter*, for doing nothing. *CMOS* (complementary MOS) replaces the resistor with a *PMOS* transistor — same device, but conducting when $v_{GS} < V_{th} < 0$, i.e. when the gate is *low*. Now the pull-up is on exactly when the pull-down is off (Figure 6.4): one transistor of the pair always blocks the rail-to-ground path.

**Example 6.5** (CMOS static power, measured)**. Strategy.** Sweep the CMOS inverter and read the supply current at the two resting points — the claim is that both are zero, with current only *during* transitions.

**Solution.** `code/ch06_cmos_inv.cir` prints `i(VDD)` at $v_{in} = 0$ and $v_{in} = 3.3$ V:

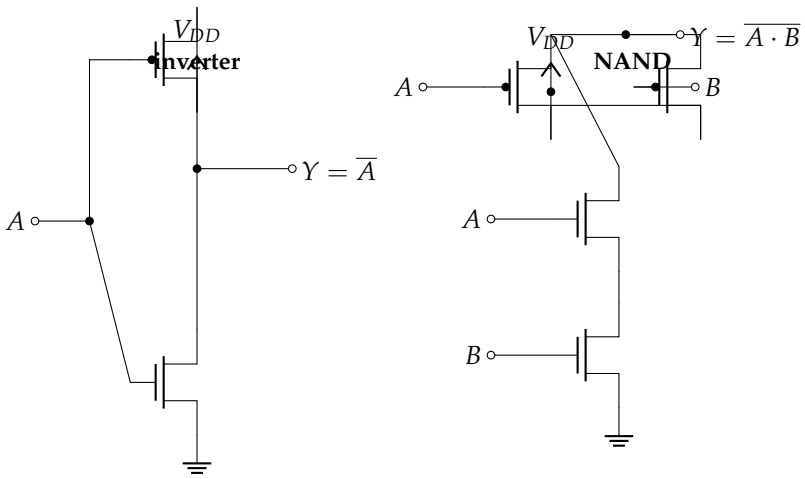



**Figure 6.4:** The CMOS inverter (left) and a 2-input CMOS NAND (right): NMOS network pulls down, PMOS network pulls up, and exactly one network conducts for any input. Notice the NAND structure — series NMOS (both inputs needed to pull down), parallel PMOS (either input high pulls up).

```
i(vdd) = -3.31000e-12 (vin = 0)
i(vdd) = -3.31000e-12 (vin = 3.3)
```

**Listing 6.4:** Static points of the CMOS sweep — compare with the NMOS inverter's 0.32 mA.

Three picoamps — numerical noise, not real current. Meanwhile the mid-sweep peak is 0.33 mA at $v_{in} \approx 1.6$ V: the brief window where *both* devices are part-on. That is the whole power story of modern silicon: **CMOS burns energy only while switching**, which is why processor power scales with clock frequency.

**WHY CMOS WON**

The complementary pair gets rail-to-rail outputs (0 and $V_{DD}$ exactly, see Figure 6.3), a steep transition centred near $V_{DD}/2$, and zero static current. Resistive pull-ups lose on all three.

NOR gates swap the networks (parallel NMOS below, series PMOS above). Every logic function you will meet is these two dual networks — a fact the FPGA chapters lean on hard.

**Exercise 6.2** (medium)**.** Sketch the transistor-level structure of a 2-input CMOS NOR: which devices are in series and which in parallel, and why must it be the dual of NAND?

## 6.5 The BJT: the current-controlled cousin

The *bipolar junction transistor* (NPN/PNP) controls collector current with a *base current*: $i_C = \beta\, i_B$, with $\beta$ of order 100. The mental model shift matters: a MOSFET answers voltage, a BJT answers *current* — you must supply base current continuously, which is why discrete switch circuits prefer MOSFETs. BJTs remain excellent small-signal amplifiers and live inside every op-amp; this book treats them conceptually only (full biasing design is AoE territory, flagged honestly).

## 6.6 The transistor as amplifier

Between "fully on" and "fully off" lies the region the VTC shows as a steep slope: small changes in $v_{GS}$ produce large changes in $v_{DS}$ — gain. Bias a single transistor mid-slope, feed it a small signal, and the same device amplifies. We do exactly one example and then point onward.

**Example 6.6** (The inverter is also a (bad) amplifier)**. Strategy.** The VTC slope $-dv_{out}/dv_{in}$ *is* the small-signal voltage gain. Estimate it from Figure 6.3's steepest point.

**Solution.** The NMOS curve falls from $\approx 2.6\,\text{V}$ to $\approx 0.76\,\text{V}$ between $v_{in} = 1.25\,\text{V}$ and $1.5\,\text{V}$: a local gain of about $-7$. Biased at $v_{in} = 1.4\,\text{V}$, this inverter is a genuine inverting amplifier — a terrible one (the bias point wanders with temperature and the pull-up wastes power), but the same small-signal idea underlies every real amplifier stage. The art of making this robust — feedback, biasing networks, differential pairs — is where this book hands you to AoE [3].

**Exercise 6.3** (easy)**.** From the CMOS curve in Figure 6.3, estimate its maximum slope in V/V. Why does a steeper transition make a better gate *and* a better (though twitchier) amplifier?

## 6.7 Driving a load: the switch, for real

**Example 6.7** (A 2N7000-class MOSFET switching a 12 V LED strip from 3.3 V logic)**. Strategy.** Low-side switch (Figure 6.5): load between +12 V and drain, source grounded, gate to the logic pin. The question is never "does it conduct" — it is "does it conduct *enough*" at the $v_{GS}$ you can actually supply. Check $V_{th}$ against your rail, then simulate the gate-drive sweep.

**Solution.** `code/ch06_switch_load.cir` sweeps the gate of a $V_{th} = 2.1\,\text{V}$ device driving a $600\,\Omega$ strip ($\approx 20\,\text{mA}$):

```
gate drain
0.500 12.000
2.000 12.000
3.000 8.061
3.300 5.184
5.000 0.479
```

**Listing 6.5:** Gate-drive sweep, verbatim rows — the transistor turns on only gradually, and 3.3 V drive leaves it half-on.

At $v_{GS} = 3.3\,\text{V}$ the overdrive is only $3.3 - 2.1 = 1.2\,\text{V}$: the channel passes 11.4 mA while the load wants 20 — the strip glows at half brightness and the transistor eats 5.2 V of drop. At 5 V drive the drain sits at 0.48 V: solidly on. **The lesson: match the FET's specified $R_{on}$ to the gate voltage you actually have** — logic-level parts specify $R_{on}$ at $v_{GS} = 2.5\,\text{V}$ or 1.8 V for exactly this reason.

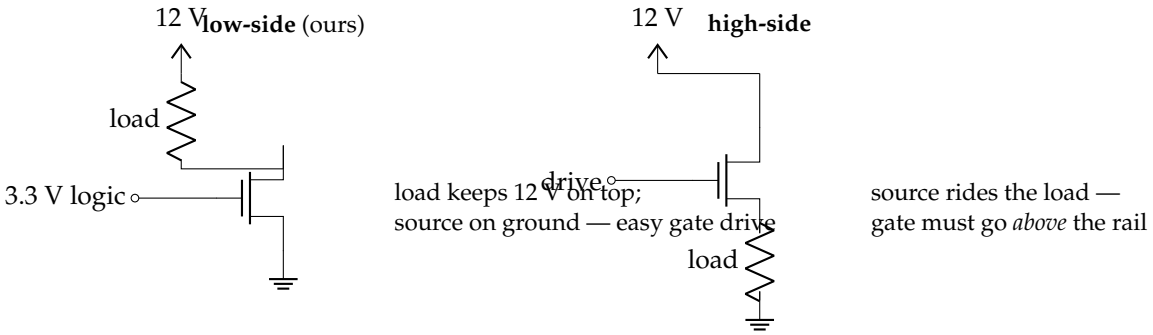



**Figure 6.5:** Low-side versus high-side switching. Notice why we always reach for low-side first: with the source on ground, a 3.3 V pin is a known $v_{GS}$; high-side, the source rides the load and the gate needs a voltage *above the rail* (a bootstrap or charge pump — later study).

**Checkpoint 6.1 Gate ESD.** The gate is a few nanometres of oxide; a charged finger punctures it. Handle MOSFETs and CMOS ICs with static care, and never leave a gate floating — tie it with a pull-down resistor so the switch cannot turn on from noise. And this is why Chapter 4's flyback diode matters here: the transistor switching a relay coil is exactly the device the spike kills.

**HIGH-CURRENT SWITCHING**

Breadboards and jumper wires are rated for ∼1 A. We simulate anything beyond that — the physics of a 20 A switch is identical, the burns are not.

## 6.8 Logic families and voltage levels

Legacy TTL and 5 V CMOS still exist, but this book lives at 3.3 V: modern FPGAs, microcontrollers and the Tang Nano all run it, and mixing families costs you level shifters and noise-margin headaches. The VTC numbers of Figure 6.3 are why a 3.3 V input is *not* automatically a valid 1 at a 5 V input — read the destination's $V_{IH}$ before connecting.

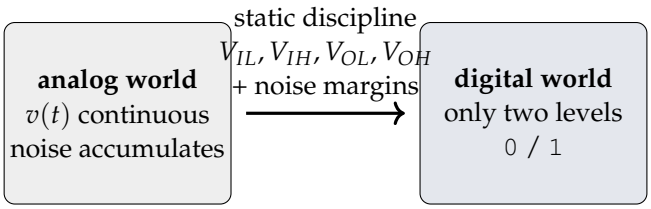



**Figure 6.6:** The bridge this chapter builds: analog voltage in, two clean levels out — guaranteed by the static discipline, not by hope.

## 6.9 Common mistakes

**TREATING $V_{th}$ AS A SHARP ON/OFF**

Conduction is gradual (the switch-load sweep rises through 8 V, 5 V on its way down). "Above threshold" means *starting* to conduct; enough current for your load is a separate question.

**SWAPPING SOURCE AND DRAIN**

A discrete NMOS's body diode points source→drain: wired backwards, it conducts through the diode no matter what the gate says. Power packages put a visible diode in the datasheet drawing — look for it.

**LEAVING THE GATE FLOATING**

An un-driven gate drifts on leakage and switches at random — the ESD/gate checkpoint's pull-down resistor is not decoration.

**ASSUMING ZERO RESISTANCE WHEN "ON"**

$R_{on}$ is real: Example 6.7's 0.48 V drain drop at full drive is 9.6 mW of heat on a small part — check it before claiming "switch closed".

## 6.10 Summary

| Concept | Takeaway |
|---|---|
| S / SR models | on = wire, or wire + $R_{on}$; off = open |
| VTC | the curve that decides every digital level |
| Noise margins | $NM_L = V_{IL} - V_{OL}$, $NM_H = V_{OH} - V_{IH}$ |
| CMOS | complementary pair, 0 static power, rail-to-rail |
| Gate drive | $v_{GS} - V_{th}$ sets how *hard* the switch is on |
| Low-side first | source grounded ⇒ easy drive |
| Amplifier | same device, biased mid-slope; details live in AoE |

Next: flip the switch the other way — same transistors, wired as logic gates, and the Boolean algebra that drives them.

**Exercise 6.4** (easy)**.** A logic family's sender promises $V_{OL} = 0.4\,\text{V}$, $V_{OH} = 2.9\,\text{V}$ and the receiver needs $V_{IL} = 0.9\,\text{V}$, $V_{IH} = 2.1\,\text{V}$. Compute both noise margins.

**Exercise 6.5** (easy)**.** Why does an ideal CMOS inverter draw no supply current while holding its output? Where does the current go during an input transition?

**Exercise 6.6** (easy)**.** Order these by static power, worst first: CMOS inverter, NMOS + resistor inverter holding 0, NMOS + resistor inverter holding 1.

**Exercise 6.7** (easy)**.** Your GPIO pin supplies 3.3 V and your FET has $V_{th} = 2.9\,\text{V}$. Without simulating: is this a good switch for a 20 mA load? What single datasheet number settles it?

**Exercise 6.8** (medium)**.** Design a 3-input CMOS NAND: draw the pull-down network (series NMOS) and the pull-up network (parallel PMOS), then state the output expression.

**Exercise 6.9** (medium)**.** Using Figure 6.3's NMOS curve, estimate $V_{IL}$ and $V_{IH}$ as the points where $|dv_{out}/dv_{in}| = 1$ (the knee is symmetric enough to eyeball). Recompute both noise margins and compare with the text's ≈1.1/1.6 V estimate.

**Exercise 6.10** (medium)**.** In Example 6.7, what is the FET's dissipated power at $v_{GS} = 3.3\,\text{V}$ (drain 5.18 V, current 11.4 mA)? Compare with the load's and state why the FET gets hot while the strip stays dim.

**Exercise 6.11** (medium)**.** Estimate the NMOS inverter's mid-transition supply current from its resistor: when $v_{out} \approx V_{DD}/2$, what flows through the 10 kΩ pull-up? Compare with CMOS's measured 0.33 mA crossover peak.

**Exercise 6.12** (hard)**.** Derive the SR-model output low voltage: NMOS on-resistance $R_{on}$ against pull-up $R_D$ to $V_{DD}$. With $R_{on} = 220\,\Omega$ and $R_D = 10\,\text{k}\Omega$ at $V_{DD} = 3.3\,\text{V}$, compute $V_{OL}$ and check it against the simulated 0.071 V.

**Exercise 6.13** (hard)**.** A 5 V-rail legacy gate must read a 3.3 V CMOS output. Its $V_{IH} = 3.5\,\text{V}$. Quantify the failure, name two fixes, and state which this book's ecosystem (Tang Nano at 3.3 V logic) avoids the problem by construction.

## Hints and answers

**Noise margins (easy)** $NM_L = 0.9 - 0.4 = 0.5\,\text{V}$; $NM_H = 2.9 - 2.1 = 0.8\,\text{V}$.

**CMOS static (easy)** One device of each pair is off at every level, so no rail-to-ground path exists; crossover current flows only while both are part-on, mid-transition.

**Static power order (easy)** NMOS holding 0 (resistor conducts), then CMOS (leakage only); NMOS holding 1 is the odd one: its transistor is off so it too draws only leakage — the honest order is {NMOS@0} $\gg$ {NMOS@1, CMOS}.

**Marginal gate (easy)** $v_{GS} - V_{th} = 0.4\,\text{V}$ — nearly closed. *$R_{on}$ at your $v_{GS}$* settles it; a "logic-level" part specified at 2.5 V gate is the fix.

**3-input NAND (medium)** Three series NMOS below, three parallel PMOS above: $Y = \overline{A \cdot B \cdot C}$.

**Unity-slope margins (medium)** Knee sits near $v_{in} \approx 1.2\,\text{V}$–$1.7\,\text{V}$; margins come out $\approx$1.1 V and $\approx$1.6 V, matching the text.

**FET dissipation (medium)** $5.18 \times 11.4 \approx 59\,\text{mW}$ on the transistor vs $6.8 \times 11.4 \approx 78\,\text{mW}$ on the strip — the FET burns almost half the delivered power.

**Pull-up current (medium)** $1.65/10 \approx 0.17\,\text{mA}$ (1.65 V across 10 k$\Omega$); CMOS peaks higher because its crossover current is channel-limited, not resistor-limited — but only for the nanoseconds of the transition.

**$V_{OL}$ derivation (hard)** $V_{OL} = V_{DD}\dfrac{R_{on}}{R_{on} + R_D} = 3.3 \cdot \dfrac{220}{10220} \approx 0.071\,\text{V}$ — dead on the sim.

**5 V reading 3.3 V (hard)** Guaranteed-1 input needs $v \geq 3.5\,\text{V}$ but the sender promises only 3.3 V: zero margin, guaranteed failure with tolerance drift. Fixes: level-shifter / gate with lower $V_{IH}$ / run everything at 3.3 V (our choice).

## Notes

The S/SR model ordering and the static-discipline treatment follow MIT 6.002's digital lectures [10]; device physics and the BJT sketch draw on All About Circuits vol. III [3]; netlists verified on ngspice-47 [8]. All simulated numbers in this chapter are verbatim tool output.

CHAPTER 7

# Op-amps and power supplies

Chapter 6 showed a transistor controlling current. This chapter packages a few dozen transistors into one three-terminal block — the *operational amplifier* — and puts it to work: buffering a weak signal, amplifying it, and making decisions about it. Then it asks where the volts to run all of this come from. When you finish, you will be able to analyze the three op-amp circuits that cover most real use, predict when an amplifier will clip instead of amplify, and choose a sensible power source for a project — including knowing exactly which power source you must never build yourself.

**BEFORE YOU START**

Chapters 2–3 (dividers) and 6. *Recap:* an op-amp is a high-gain amplifier; with negative feedback its behaviour is fixed by two resistors, not by the op-amp itself.

**AFTER THIS CHAPTER YOU CAN**

- design inverting and non-inverting amplifiers and a buffer;
- use a comparator, including one with hysteresis;
- choose between a linear regulator and a switching supply;
- explain why mains power stays in a closed enclosure in this book.

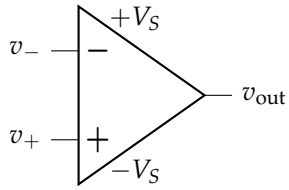
**HOW TO THINK ABOUT THIS**

An op-amp is a tool for making a circuit obey a simple rule. With feedback it adjusts its output until its two inputs are equal, so the gain depends only on resistors you choose. Think "the op-amp will do whatever it takes to make the inputs match" and most circuits become easy to read.

## 7.1 The op-amp contract

An *operational amplifier* is a differential voltage amplifier: its output is a huge constant $A$ (typically $10^5$) times the difference between its two inputs, $v_{\text{out}} = A\,(v_+ - v_-)$, clamped so it can never leave its supply rails (Figure 7.1). Inputs draw essentially no current; the output can source or sink milliamps.

That gain is too big to use bare — a microvolt of input difference would demand ten volts of output. So an op-amp is almost always used with *negative feedback*: a fraction of the output is




**Golden rules** (while negative feedback is in charge):

1. The inputs draw no current — $i_+ = i_- = 0$.
2. The feedback forces $v_+ = v_-$.

Rule 2 is not magic: it is the output moving until the difference the amplifier sees is nearly zero. It fails when the output hits a rail.

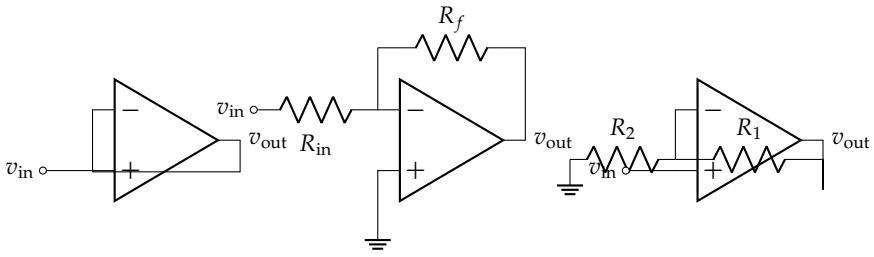
**Figure 7.1:** The op-amp symbol and its working rules. Notice the direction of causality: the output moves *until* the input difference is zero — rule 2 is a consequence of feedback, not a property of the part.

routed back to the inverting input, where it fights the input signal. The loop settles when it has corrected its own error — the output finds whatever value makes $v_+ \approx v_-$. The gain you get is then set by the *resistors you chose*, not by the chip's enormous internal gain. That trade — vast excess gain exchanged for accuracy set by cheap passive parts — is the whole trick, and it is why the same 1 chip shows up everywhere from audio to sensor front-ends.

**Checkpoint 7.1** *The contract.* Inside its limits, an op-amp with negative feedback behaves by the two golden rules of Figure 7.1. Analyze every circuit by asking: *what output forces $v_- = v_+$?*

## 7.2 The three circuits that cover 90% of use

Three configurations do nearly all the work (Figure 7.2). Each analysis is one line of the golden rules.




buffer: $v_{\text{out}} = v_{\text{in}}$ inverting: $v_{\text{out}} = -\dfrac{R_f}{R_{\text{in}}} v_{\text{in}}$ non-inverting: $v_{\text{out}} = \left(1 + \dfrac{R_1}{R_2}\right) v_{\text{in}}$

**Figure 7.2:** Buffer, inverting, and non-inverting amplifiers. Notice that every gain expression contains only resistor ratios — the chip's $10^5$ gain has cancelled itself out of the answer.

The *buffer* (voltage follower) feeds the whole output back, so $v_{\text{out}} = v_{\text{in}}$ — pointlessly trivial until you remember its job: it copies a voltage *without loading the source*. The *inverting* amplifier ties $v_+$ to ground, so feedback forces $v_- = 0$ (a "virtual ground"), and the same

current flows through both resistors: $v_{\text{in}}/R_{\text{in}} = -v_{\text{out}}/R_f$, giving gain $-R_f/R_{\text{in}}$. The *non-inverting* amplifier puts $v_{\text{in}}$ on $v_+$ and feedback forces the divider tap to equal it: gain $1 + R_1/R_2$, always $\geq 1$.

**Exercise 7.1** (Buffer arithmetic)**.** A sensor can deliver at most 10 µA. Its output is 2.5 V behind 100 kΩ of internal resistance. Without a buffer, a 10 kΩ load drops the reading to what? With a buffer?

## 7.3 Worked example 1: buffering a weak sensor

**Strategy.** A source with 100 kΩ of internal resistance and a 10 kΩ load is a voltage divider that eats most of the signal — a direct answer to the exercise above. Simulate both the loaded source and the buffered version in one netlist; an operating point (`.op`) analysis is enough, nothing here moves.

```
* ch07_buffer.cir -- weak sensor: loaded directly vs buffered
* operating point only
Vsen 1 0 dc 2.5
Rs 1 2 100k
Rload_alone 2 0 10k
Vsen2 4 0 dc 2.5
Rs2 4 5 100k
X1 5 6 7 10 11 OPAMP
Rload 7 0 10k
Vcc 10 0 dc 12
Vee 11 0 dc -12
.subckt OPAMP p n out vcc vee
* teaching macro: differential gain ~10500 (tanh), saturates +-10.5 V
* (~1.5 V inside the +-12 V rails, typical non-RRIO swing)
B1 int 0 V = 10.5*tanh((v(p)-v(n))/0.001)
Ro int out 50
.ends OPAMP
.op
.print op v(2) v(7)
.end
```

**Listing 7.1:** `buffer.cir` — a weak source, loaded directly vs through a buffer

ngspice's operating point:

```
Index v(2) v(7)
0 2.272727e-01 2.499756e+00
```

**Listing 7.2:** output of `ngspice -b buffer.cir` (both sensors are identical 2.5 V sources behind 100 kΩ)

**Result.** The bare sensor collapses: 2.5 V behind 100 kΩ feeding 10 kΩ forms a divider that delivers only 0.227 V — a 91% error before any electronics touches it. The buffered copy delivers 2.4998 V: the op-amp input drew no current, so `Rs2` dropped nothing, and the op-amp's output shoulders the 10 kΩ load itself. (The 0.2 mV residual error is the finite-gain term the golden rules ignore — the sim's macro has gain $\sim 10^4$, not $\infty$.) The buffer did not

create the 2.5 V; its supply rails did — the op-amp merely spends rail power to copy the signal faithfully.

## 7.4 The comparator

Run the loop the other way and the op-amp becomes a *comparator*: feed the inputs two voltages and the output snaps to a rail saying which is bigger. That is just rule 2 removed — with no negative feedback (or with deliberate *positive* feedback), the huge gain has no loop to tame, so the output lives at the rails.

Positive feedback is not wasted oscillation, it is a feature: *hysteresis*. Feeding a fraction of the output to the *non-inverting* input (Figure 7.3) gives the comparator two different thresholds: it flips high only when the input rises past $+V_T$ and flips low only below $-V_T$, with $V_T = \frac{R_1}{R_1+R_2} V_{\text{sat}}$. A slowly drifting, noisy signal near one threshold can then only cross once — the output stops chattering. Dedicated comparator chips exist (faster, open-collector outputs); an op-amp *works* as a comparator, but see Section 7.7 for why it is a mediocre one.

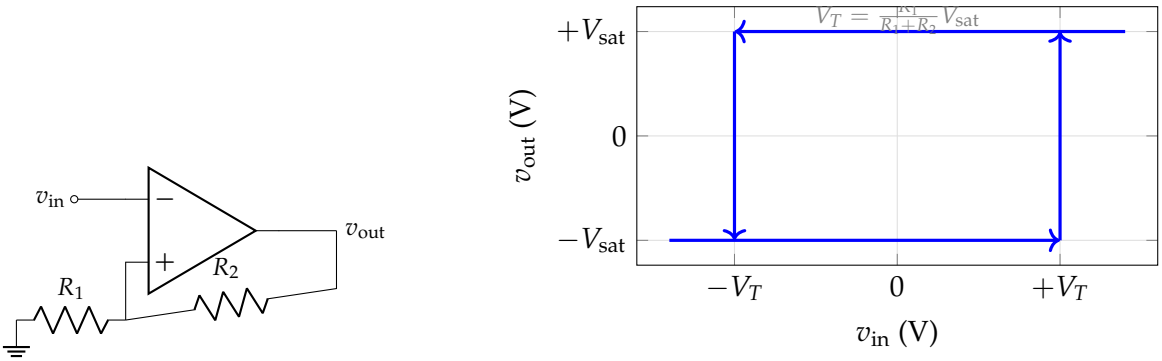



**Figure 7.3:** Schmitt-trigger comparator and its transfer curve. Notice the *loop*: the rising threshold and the falling threshold are different voltages, so noise near the edge cannot make the output dither.

## 7.5 Worked example 2: gain of 10 — and the ceiling

**Strategy.** An inverting amplifier with $R_f / R_{\text{in}} = 10$ should output $-10$ times the input. But the output cannot exceed the supply rails — and real parts typically saturate a volt or two inside them. Drive a 1 kHz sine whose amplitude steps from 0.8 V to 1.5 V mid-run; `.tran` will show clean gain first, then clipping.

```
* ch07_inv_amp.cir -- inverting amp, gain -10, saturation demo
* rails +-12 V; output clips ~+-10.5 V (typical non-RRIO swing)
Vcc 10 0 dc 12
Vee 11 0 dc -12
Venv 8 0 pwl(0 0.8 12m 0.8 12.01m 1.5)
Bsig 1 0 V = v(8) * sin(6283.185307179586*time)
Rin 1 2 1k
Rf 3 2 10k
X1 0 2 3 10 11 OPAMP
.subckt OPAMP p n out vcc vee
```

```
B1 int 0 V = 10.5*tanh((v(p)-v(n))/0.001)
Ro int out 50
.ends OPAMP
.control
tran 10u 25m
set filetype=ascii
wrdata ch07_inv_amp_ms.dat v(1) v(3)
meas tran VCLEAN max v(3) from=2m to=10m
meas tran VMAX max v(3) from=15m to=25m
meas tran VMIN min v(3) from=15m to=25m
.endc
.end
```

**Listing 7.3:** `inv_amp.cir` — inverting amplifier, input amplitude stepping 0.8 → 1.5 V at 12 ms

```
vclean = 7.98768e+00 at= 2.75280e-03
vmax = 1.04477e+01 at= 2.18763e-02
vmin = -1.04477e+01 at= 2.33763e-02
```

**Listing 7.4:** measured extrema (verbatim): clean region then saturated region

**Result.** While the input peaks at 0.8 V, the output peaks at 7.99 V — gain −9.98 where −10 was designed (the 0.2% shortfall is finite open-loop gain again). After the step, the input demands ±15 V and gets ±10.45 V: the amplifier flattens against its swing limit (Figure 7.4). Clipping is not a failure of the golden rules — it is the price they charge when the loop can no longer supply the volts it promised.

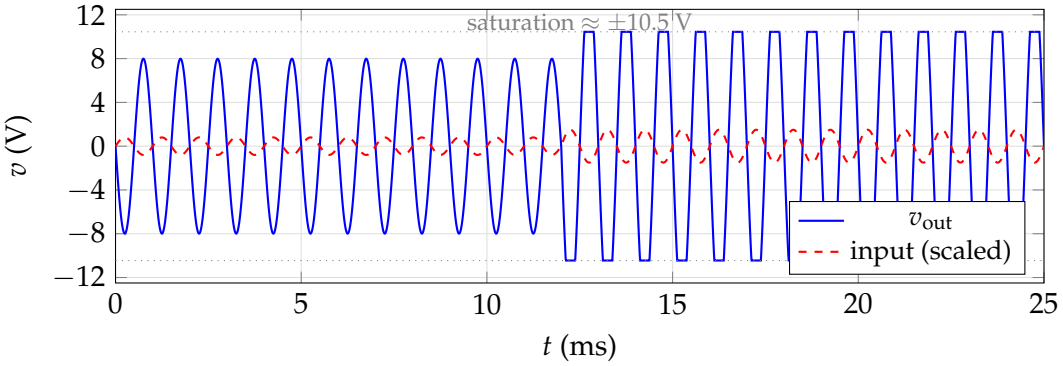



**Figure 7.4:** Simulated inverting amplifier, gain −10. Notice the exact moment clipping begins: while the input asks for less than ±10.5 V the output is a faithful ×10 image; beyond that, the top and bottom are sheared flat.

**Exercise 7.2** (Design a gain)**.** A microphone outputs ±50 mV and your ADC wants ±1.5 V. Pick an inverting topology and resistor values (keep $R_{in} \geq 1\,\text{k}\Omega$ so the source is not loaded). On ±12 V rails, does the output stay inside the swing limit?

## 7.6 Worked example 3: a comparator that never rests

**Strategy.** Combine positive feedback (the Schmitt trigger of Figure 7.3) with an $RC$ network on the inverting input. When the output sits at $+V_{\text{sat}}$, the capacitor charges toward it through $R$ until it crosses the upper threshold — the output then snaps to $-V_{\text{sat}}$ and the capacitor discharges the other way. A *relaxation oscillator*: a square wave built from one op-amp, three resistors and a capacitor — the ancestor of the 555 timer in the exercises.

The thresholds are $\pm kV_{\text{sat}}$ with $k = \frac{R_1}{R_1+R_2}$; between flips the capacitor chases the rail exponentially, and summing the two symmetric legs gives

$$T = 2RC\,\ln\frac{1+k}{1-k}.$$

With $R = 10\,\text{k}\Omega$, $C = 100\,\text{nF}$, $k = 0.5$: $T = 2(10^3)\ln 3 \approx 2.20\,\text{ms}$ — predict $\approx 455\,\text{Hz}$, then let the sim count.

```
* ch07_comparator_osc.cir -- comparator + RC = relaxation oscillator
* thresholds +-k*Vsat, k=0.5 -> T = 2RC ln(3) ~ 2.20 ms
Vcc 10 0 dc 12
Vee 11 0 dc -12
X1 4 5 3 10 11 OPAMP
R2 3 4 10k
R1 4 0 10k
R 3 5 10k
C 5 0 100n ic=1
.subckt OPAMP p n out vcc vee
B1 int 0 V = 10.5*tanh((v(p)-v(n))/0.001)
Ro int out 50
.ends OPAMP
.control
tran 5u 25m uic
set filetype=ascii
wrdata ch07_osc_ms.dat v(3) v(5)
meas tran T4 when v(3)=0 rise=4
meas tran T8 when v(3)=0 rise=8
.endc
.end
```

**Listing 7.5:** `comparator_osc.cir` — relaxation oscillator (the `ic=1` kicks it off the metastable zero)

```
t4 = 9.36692e-03
t8 = 1.81829e-02
```

**Listing 7.6:** times of the 4th and 8th rising edges (verbatim)

**Result.** Four full periods take $18.183 - 9.367 = 8.816\,\text{ms}$, so $T = 2.204\,\text{ms}$ — $f \approx 454\,\text{Hz}$ against the predicted 455 Hz (0.3% apart; the predicted number assumes an infinitely fast comparator). The capacitor sawtooths between $\pm 5.2$ V, exactly the $\pm\frac{1}{2}V_{\text{sat}}$ thresholds, while the output bangs rail to rail — the same charge-and-flip cycle every timer chip runs internally.

**Exercise 7.3** (Pick the period)**.** Design a relaxation oscillator for $\approx 1\,\text{kHz}$: with $k = 0.5$ held fixed, choose $R$ and $C$ (start from the formula above; check your answer: $RC$ should come out near 0.46 ms).

## 7.7 Where op-amps bite

The golden rules assume the op-amp always wins the feedback tug-of-war. It doesn't:

- **Rails.** Output swing stops a volt or two short of the supplies (rail-to-rail parts do better, at a price). We measured the consequence in Section 7.5.
- **Gain-bandwidth.** The open-loop gain falls with frequency; a chip rated 1 MHz GBW offers only $\times 10$ at 100 kHz. Slow math at high frequency means wrong answers.
- **Slew rate.** The output cannot change faster than the slew spec (volts per microsecond); a sharp edge becomes a ramp.
- **As a comparator,** an op-amp is slow out of saturation and its output swings to the rails — fine for a demo oscillator, wrong for a logic interface; that is what real comparators are for.

Awareness, not design: data sheets will mean more after you meet these limits once in simulation.

## 7.8 Regulators: spending volts cleanly

An amplifier is only half of power; the supply itself is the other half. A *voltage regulator* takes a rough input voltage and outputs a steady, lower one (Figure 7.5).

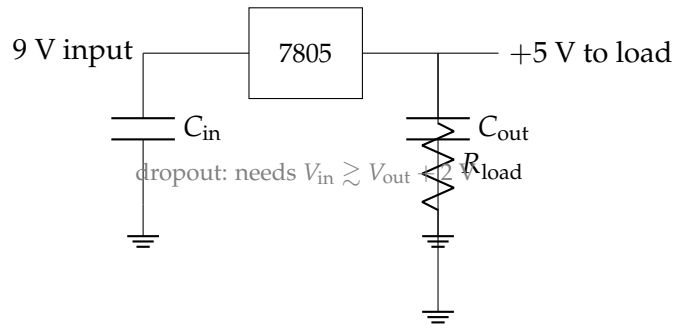



**Figure 7.5:** A linear-regulator power path. Notice the capacitor pair: $C_{in}$ and $C_{out}$ are not optional decoration — the regulator needs them nearby for stability, per its own datasheet.

The *linear regulator* (the 7805 family: 9 or 12 V in, 5 V out) is a feedback loop that burns the difference as heat: efficiency $\approx V_{out}/V_{in}$, so 5 V from 12 V wastes over half the power — fine at milliamps, punishing at amps. It also *drops out*: below $V_{in} - V_{out} \approx 2$ V it stops regulating (*LDO* parts shrink that margin to a few hundred millivolts).

The *switching regulator* ("buck") chops the input through an inductor and filters the result — 90%+ efficient, but noisy, more complex, and it is *another* circuit this book will not have you build at mains voltage. Rule of thumb: milliamps and quiet, linear; watts, switching.

## 7.9 USB power, and decoupling revisited

Your bench supply for this whole book is the USB port: 5 V, already regulated, current-limited by the host, impossible to wire backwards by accident — the safest possible answer to "where do volts come from". USB power banks are equally friendly and also portable; treat their lithium cells with respect (below).

Decoupling bears repeating here because op-amps make it concrete: the supply pins feed fast current spikes to the output stage, and a supply that sags mid-swiggle puts the sag *inside* your signal. The remedy is a short local reservoir — 100 nF ceramic across the supply pins of every chip, plus a few microfarads where power enters the board. It is invisible when it works and baffling when missing.

## 7.10 Mains power: why it stays boxed

Everything you have built runs on single-digit volts from USB. The wall outlet is 230 V AC in Indonesia — and it stays inside its box for the whole book. Figure 7.6 shows why the inside of a charger is engineer territory, not hobbyist territory.

**Inside the USB charger you must not open**

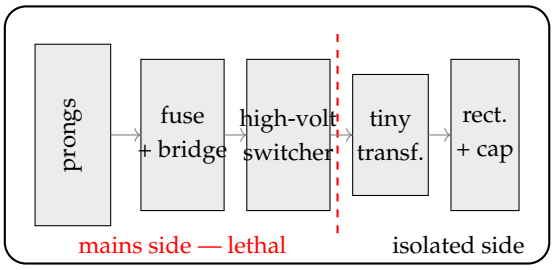



**Figure 7.6:** What is actually inside a 5 V USB charger. Notice the dashed line: everything left of it touches mains; the isolation boundary is engineered, tested, and certified. Look — never open.

**MAINS: THE HARD BOUNDARY**

- 230 V AC kills. It is not a bigger 9 V battery — a mains shock across the chest can stop a heart, and a mains fault starts fires, not sparks.
- Never open, probe, repair, or re-use the inside of a wall adapter, charger, or mains-powered brick — even unplugged, its capacitors hold lethal charge for minutes.
- Buy, don't build: USB supplies and lab bench supplies are cheap, certified, and isolated. There is no beginner project whose power stage should touch mains.
- Cheap no-name chargers fail the isolation test too — buy known brands from real shops.

**BATTERY ENERGY: USB PACKS AND LIPO**

USB power banks are the friendly face of lithium cells — that is why they are allowed. The cells inside store real energy: shorted, a LiPo sustains tens of amps and vents fire. Never pierce, crush, solder directly on, or charge unattended loose lithium cells; use the protected, cased packs as they come. A swollen pack is retired, not "used up".

## 7.11 Common mistakes

- **Applying the golden rules at the rails.** If the required output exceeds the supply, the rules are already dead — check headroom first (Section 7.5).
- **Forgetting decoupling capacitors.** An op-amp starved at its supply pins oscillates mysteriously; 100 nF across the pins is cheap insurance, always.
- **Using an op-amp as a comparator without hysteresis.** A noisy input near the threshold chatters the output between rails; add positive feedback or buy a real comparator.
- **Expecting gain at any frequency.** GBW means a $\times 100$ amplifier is not one at 100 kHz — check the bandwidth budget before the resistor ratio.
- **Assuming the input range too.** Some parts read inputs near the negative rail only; "single-supply" does not mean "accepts any input".

## 7.12 Exercises

**Exercise 7.4** (easy)**.** A buffer follows a sensor output of 3.1 V. What is the buffer's output voltage, and why would you bother if the number doesn't change?

**Exercise 7.5** (easy)**.** An inverting amplifier has $R_{\text{in}} = 2\,\text{k}\Omega$, $R_f = 20\,\text{k}\Omega$. What is the gain, and what input gives $+5$ V out?

**Exercise 7.6** (easy)**.** A non-inverting amplifier needs gain 11. If $R_2 = 1\,\text{k}\Omega$, choose $R_1$.

**Exercise 7.7** (easy)**.** Your op-amp is on $\pm 9$ V rails and saturates about 1.5 V inside them. You want a $\pm 6$ V sine out of gain 20. What is the largest input amplitude you can amplify cleanly?

**Exercise 7.8** (medium)**.** A comparator circuit has $R_1 = 10\,\text{k}\Omega$ to ground and $R_2 = 30\,\text{k}\Omega$ to the output, which saturates at $\pm 10$ V. Find both switching thresholds $V_{T+}$ and $V_{T-}$.

**Exercise 7.9** (medium)**.** Using $T = 2RC \ln \frac{1+k}{1-k}$: a relaxation oscillator runs at 250 Hz with $k = 0.5$ and $C = 100$ nF. What is $R$?

**Exercise 7.10** (medium)**.** A project draws 80 mA at 5 V and you can feed the regulator 9 V. Compare an LDO (dropout 0.4 V) against a buck module: which do you pick, and what power does the loser waste? When would the answer flip?

**Exercise 7.11** (medium)**.** You need $\times 50$ gain at 20 kHz from a 1 MHz GBW op-amp. Can you do it in one stage? What open-loop gain does the chip actually offer there, and what does that do to accuracy?

**Exercise 7.12** (hard)**.** (Implement) Extend `comparator_osc.cir`: make $k \neq 0.5$ by setting $R_1 = 10\,\text{k}\Omega$, $R_2 = 20\,\text{k}\Omega$ ($k = \frac{1}{3}$). Derive the new predicted period, re-simulate, and report measured vs predicted in a table. **Hint:** $\ln \frac{1+k}{1-k} = \ln 2$ now.

**Exercise 7.13** (hard)**.** A 12 V-to-5 V linear regulator supplies a 300 mA load. Compute the dissipated power, and explain why engineers choose a buck converter here — then explain when they would *not* (think noise, cost, battery standby).

## 7.13 Summary

| Idea | One-line version |
|---|---|
| Golden rules | With negative feedback: inputs draw no current, and $v_+ = v_-$ — until the rails say otherwise. |
| Buffer | Copies a voltage without loading the source. |
| Inverting amp | Gain $-R_f/R_{\text{in}}$; $v_-$ is a virtual ground. |
| Non-inverting amp | Gain $1 + R_1/R_2 \geq 1$; input sees no load. |
| Comparator | No taming feedback: output rails declare which input is bigger. |
| Hysteresis | Positive feedback gives two thresholds; noise can't dither the output. |
| Clipping | Demanded output beyond swing limit = flat-topped waveform. |
| Linear regulator | Burns the voltage difference as heat: simple, quiet, inefficient. |
| Switching regulator | Chops through an inductor: efficient, noisy. |
| USB supply | Pre-regulated, current-limited, safe — the book's bench supply. |
| Mains | 230 V stays inside certified boxes. Buy, never build. |
| Decoupling | 100 nF at every chip's supply pins, always. |

## Notes

1. *All About Circuits*, vol. III, ch. 8 — the op-amp chapters the buffer/inverting/comparator analyses follow [3].
2. MIT *6.002* lectures 8–12 (op-amp abstraction, negative feedback) and 20+ (power) — the golden-rules framing and the honest limits list [10].
3. Horowitz & Hill, *The Art of Electronics* — the cited next step for comparator subtleties, regulators, and real-world op-amp pathologies [11].

## Hints and answers

**Buffer (easy)** 3.1 V. The point is the *current*: the buffer supplies load current from its rails while drawing $\sim 0$ from the sensor.

**Inverting gain (easy)** Gain $-10$; input $-0.5$ V.

**Non-inverting (easy)** $R_1 = 10\,\text{k}\Omega$ $(1 + 10 = 11)$.

**Headroom (easy)** Swing limit $\approx \pm 7.5$ V $>$ the 6 V needed; max clean input $= 6/20 = 0.3$ V.

**Weak sensor (inline, §7.2)** Without buffer $\approx 0.227$ V; with buffer $\approx 2.5$ V — the sim of §7.3 measures both.

**Mic gain (inline, §7.5)** $R_f = 30\,\text{k}\Omega$, $R_{\text{in}} = 1\,\text{k}\Omega$; $\pm 1.5$ V out is far inside the swing limit.

**Osc period (inline, §7.6)** $RC = 1/(2\ln 3)$ ms $\approx 0.455$ ms; e.g. $R = 4.55\,\text{k}\Omega$, $C = 100$ nF.

**Thresholds (medium)** $V_T = \frac{10}{40} \times 10 = \pm 2.5$ V: flips high at $+2.5$, low at $-2.5$.

**Oscillator $R$ (medium)** $R = \frac{1}{2fC\ln 3} \approx 18.2\,\text{k}\Omega$.

**LDO vs buck (medium)** LDO wastes $(9 - 5) \times 0.08 = 0.32$ W with 4 V of headroom $\gg$ 0.4 V dropout — pick the LDO. Buck wins when current is amps, or battery life beats noise.

**GBW (medium)** Open-loop gain at 20 kHz $\sim 10^6/2 \times 10^4 = 50$ — barely the wanted closed loop; split into two $\times 7$ stages or choose a faster part.

**$k = 1/3$ oscillator (hard)** $T = 2RC\ln 2 \approx 1.39$ ms predicted ($\approx 721$ Hz); simulate and tabulate — expect within 1%.

**Linear vs buck, 300 mA (hard)** $(12 - 5) \times 0.3 = 2.1$ W of heat — buck territory. Skip it when noise is critical, cost rules, or load is milliamps.

CHAPTER 8

# Numbers, bits and Boolean algebra

Every chapter so far treated voltage as a quantity — 0.227 volts here, 10.45 there. The digital world compresses all of that into two letters: 0 and 1. This chapter introduces the notation (binary, hex, two's complement), the algebra that manipulates it (truth tables, De Morgan, Karnaugh maps), and the first sight of a hardware description language — a 20-line Verilog file that prints gate truth tables. After it, you will be able to read a byte in any base, prove a simplification two different ways, and turn an English sentence like "the alarm sounds when the ignition is on and the door is open or the belt is off" into a minimal gate network.

**BEFORE YOU START**

Nothing from Chapters 1–7 is required except the idea of a logic level from Chapter 6; this is the first chapter of the digital half. *Recap:* a wire is either high (1) or low (0), and a bit is the value on it.

**AFTER THIS CHAPTER YOU CAN**

- convert between binary, hex and decimal and write negative numbers in two's complement;
- build and read a truth table;
- simplify a Boolean expression with De Morgan's laws and check the result two ways;
- turn an expression into a gate network.

**HOW TO THINK ABOUT THIS**

A digital circuit throws away detail on purpose: only "high" and "low" matter, so noise cannot accumulate. Once everything is zeros and ones, design becomes logic, and logic can be written, simplified and proved like algebra. Always check a simplification twice, once with algebra and once with a truth table.

## 8.1 Why two levels?

Chapter 6's noise margins are the reason. A "1" is not 3.3 V; it is *anything above* $V_{IH}$; a "0" is anything below $V_{IL}$. The region between is forbidden — and the margins $NM_L$, $NM_H$ budget how much noise a signal can pick up before a receiver misreads it. Two levels, spaced far apart with a guard band, is the cheapest possible error-correction scheme: any voltage that drifts stays readable, and every gate regenerates the levels fresh.

**Checkpoint 8.1** *Two levels, one reason.* Binary is not a mathematical preference — it is the cheapest way to make physical voltage carry information through noise. Keep asking of every abstraction above it: *what does a 1 physically mean here?*

## 8.2 Binary, hex, and two's complement

A byte is eight bits, and a number written $b_7 b_6 \ldots b_0$ means $\sum_i b_i 2^i$. *Hexadecimal* groups four bits per digit (`0x00`–`0xFF`); the programmer's muscle memory is knowing $2^4 = 16$, $2^8 = 256$, $2^{10} \approx 10^3$ cold.

For negatives the convention is *two's complement*: the most significant bit carries weight $-128$, not $+128$. So `0xFF` $= -1$, and the representable range of a signed byte is $-128$ to $+127$ — asymmetric by one. The payoff: *no subtraction circuit is ever needed*; $a - b$ is just $a + (\sim b + 1)$, and the same adder does both jobs.

**Exercise 8.1** (easy)**.** Write $-1$, $-128$, $+127$ in hex under two's complement; then compute $100 - 50$ byte-wise without any subtraction: $100 + (\sim 50 + 1)$.

## 8.3 Truth tables

A Boolean function of $n$ inputs has exactly $2^n$ input rows, and its *truth table* is the complete enumeration — the definition, not a summary. Everything later (K-maps, HDL testbenches) re-arranges or re-prints this table; it is the ground truth.

Here is one in a new notation — *Verilog*, the hardware description language used from chapter 11 onward. You are not expected to write this yet; read it with the numbered explanations in the margin of your mind:

```
// ch08_gates_tb.v -- behavioral gate truth printer
`timescale 1ns/1ps
module gates_tb;
  reg a, b; // inputs we sweep
  wire y_and, y_or, y_xor, y_nand, y_nor, y_xnor;

  assign y_and = a & b;
  assign y_or = a | b;
  assign y_xor = a ^ b;
  assign y_nand = ~(a & b);
  assign y_nor = ~(a | b);
  assign y_xnor = ~(a ^ b);
```

```
13
14   integer i;
15   initial begin
16     $display("a b | AND OR XOR NAND NOR XNOR");
17     $display("----+--------------------------------");
18     for (i = 0; i < 4; i = i + 1) begin
19       {a, b} = i[1:0]; // 00,01,10,11 in binary
20       #1;
21       $display("%b %b | %b %b %b %b %b %b",
22                a, b, y_and, y_or, y_xor, y_nand, y_nor, y_xnor);
23     end
24     $finish;
25   end
26 endmodule
```

**Listing 8.1:** `code/ch08_gates_tb.v -- a behavioral truth printer. Every line explained below the output.`

```
a b | AND OR XOR NAND NOR XNOR
----+--------------------------------
0 0 | 0 0 0 1 1 1
0 1 | 0 1 1 1 0 0
1 0 | 0 1 1 1 0 0
1 1 | 1 1 0 0 0 1
ch08_gates_tb.v:28: $finish called at 4000 (1ps)
```

**Listing 8.2:** Real output of `iverilog -o gates_tb ch08_gates_tb.v && vvp gates_tb`

Reading the file: `module`/`endmodule` wrap a circuit block; `reg` holds values we set procedurally, `wire` is driven continuously. `assign` makes a wire follow an expression forever — that is the hardware, six gates drawn in text. `initial ... begin ... end` runs once at simulation start (a testbench block, not synthesizable). `{a,b}=i[1:0]` splits the 2-bit counter across two wires; `#1` waits one tick; `$display` prints. The loop emits all $2^2 = 4$ rows — the truth table, printed by the gates themselves.

The symbols for these gates are in Figure 8.1; the truth table above is what each symbol means.

**Exercise 8.2** (medium)**.** From the printed table, write the Boolean expression for XNOR as a sum of products of $a$, $b$, and their complements.

## 8.4 De Morgan and the universal gates

**Definition 8.1** (De Morgan's laws)**.**

$$\overline{a \cdot b} = \bar{a} + \bar{b}, \qquad \overline{a + b} = \bar{a} \cdot \bar{b}.$$

NOT distributes by *swapping the operator*. Both directions are provable by exhaustion — a truth table settles them in four rows.

The payoff is practical: NAND alone can build every other gate (Figure 8.2), and so can NOR — either is a *universal gate*. Real silicon is made of NANDs; De Morgan is why a designer can

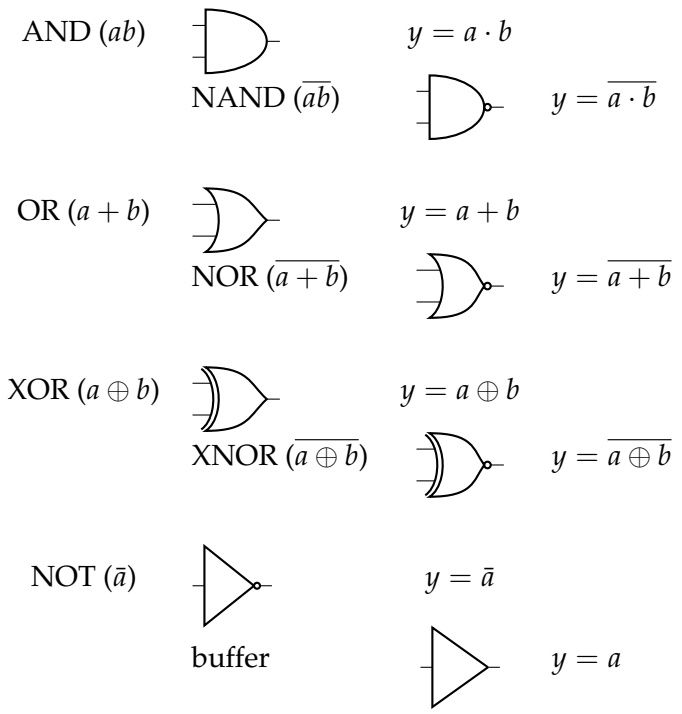



**Figure 8.1:** The seven gate symbols. Notice the small bubble on NAND, NOR, XNOR, NOT: an output circle always means "invert the plain version" — $\overline{AND} = NAND$, and so on.

sketch in AND/OR and let the synthesizer retarget to NAND/NOR without changing the function.

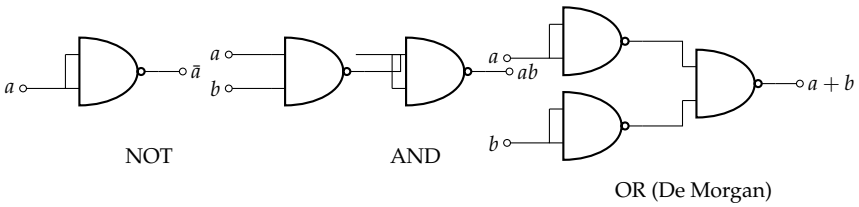



**Figure 8.2:** NAND building NOT, AND, and OR. Notice the OR shape: two NAND-inverters feeding a NAND — that is De Morgan drawn in wires.

**Exercise 8.3** (medium)**.** Draw NOT, AND, and OR out of NOR gates only. (Hint: NOT is one NOR; work through De Morgan.)

## 8.5 Worked example 1: signed byte arithmetic, three ways

**Strategy.** Compute $45 - 83$ in an 8-bit byte three different ways and watch them land on the same answer — that agreement is what makes two's complement worth its asymmetry.

**Solution.**

1. *Plain arithmetic*: $45 - 83 = -38$. In signed hex, $-38 = 256 - 38 = 218 =$ `0xDA`.
2. *Negate and add*: $-83$ is $\sim 83 + 1 = \sim$ `0x53` $+ 1 =$ `0xAC` $+ 1 =$ `0xAD`. Then $45 +$ `0xAD` $=$ `0x2D` $+$ `0xAD` $=$ `0xDA` — the carry out of bit 7 is discarded; the byte wraps and the answer is right.
3. *Bit-check*: `0xDA` $= 1101\,1010_2$: top bit $-128$ plus $64 + 16 + 8 + 2 = 90$, total $-128 + 90 = -38$. ✓

The same adder did the subtraction; method 2 is literally what silicon computes.

## 8.6 Worked example 2: prove a simplification twice

**Strategy.** Take $y = ab + a\bar{b} + \bar{a}b$. Prove it equals $a + b$ first by exhaustion (truth table) then by algebra — the two proofs compliment each other: the table is infallible but proves only this instance; the algebra shows *why* and generalizes.

**Solution.**

```
vars: A B
A B | A&B | A&~B | ~A&B A|B
----------------------------
0 0 | 0 0
0 1 | 1 1
1 0 | 1 1
1 1 | 1 1
EQUIVALENT on all 4 input rows
```

**Listing 8.3:** Truth-table check via `ch08_truth_check.py` — all 4 rows agree.

Algebraically: $ab + a\bar{b} = a(b + \bar{b}) = a$, so $y = a + \bar{a}b = a + b$ (using $x + \bar{x}y = x + y$). Same conclusion, one line.

The checker used above is the book's truth-table tool — exercises will meet it again:

```
usage: python3 ch08_truth_check.py "I&D | I&~B" "I&(D|~B)"
# evaluates each expression on all 2^n input rows, prints the table,
# then EQUIVALENT or DIFFERENT with the first counterexample.
```

**Listing 8.4:** `code/ch08_truth_check.py -- compares expressions exhaustively and prints every row.`

## 8.7 Worked example 3: the alarm circuit

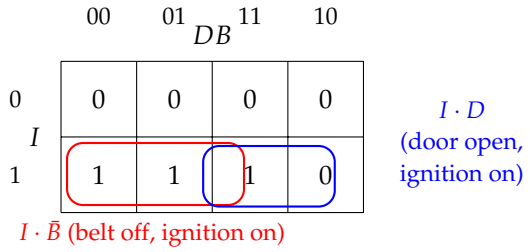
**Strategy.** "The alarm buzzes when the ignition is on and either the door is open or the seatbelt is unfastened." Name the inputs $I, D, B$ (belt fastened = 1), write the truth table, reduce with a K-map, then build three equivalent gate networks.

**Solution.** Rows where the alarm is off ($I = 0$) are all zero; when $I = 1$, the alarm sounds for $D = 1$ or $B = 0$. The 3-variable K-map (Figure 8.3) groups two overlapping pairs: $I\bar{B}$ and $ID$, giving

$$y = I \cdot D + I \cdot \bar{B} = I\,(D + \bar{B}).$$

```
vars: B D I
B D I | I&D | I&~B I&(D|~B)
-----------------------------
0 0 0 | 0 0
0 0 1 | 1 1
0 1 0 | 0 0
0 1 1 | 1 1
1 0 0 | 0 0
1 0 1 | 0 0
1 1 0 | 0 0
1 1 1 | 1 1
EQUIVALENT on all 8 input rows
```

**Listing 8.5:** Checker confirms the factored form (verbatim).




$$y = I \cdot D + I \cdot \bar{B} = I\,(D + \bar{B})$$

**Figure 8.3:** The alarm's 3-variable K-map. Notice the wrap: the $DB$ columns run 00, 01, 11, 10 — a Gray-code order, so adjacent cells always differ by exactly one bit; that is what makes groupings meaningful.

**Exercise 8.4** (hard)**.** (Implement) The minimal form is $I\,(D + \bar{B})$. Use the checker on your machine to prove the sum-of-products and NAND-only forms agree on all 8 rows; then decide which a fab would rather print.

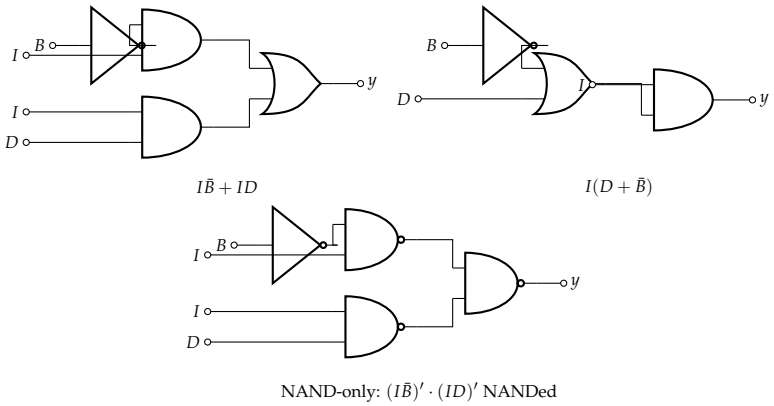



**Figure 8.4:** The same alarm function three ways: sum-of-products, factored, and NAND-only. The NAND version costs one more gate but uses a single gate type — the trade real fabs make everywhere.

## 8.8 From expression to network

The workflow this chapter just walked is a pipeline (Figure 8.5): sentence → truth table → K-map → minimal sum-of-products → gate network. The same reduction a synthesizer does in nanoseconds — learning to do it by hand is how you *audit* what the tool did, and why HDLBits drills (named in the exercises) start at truth tables.

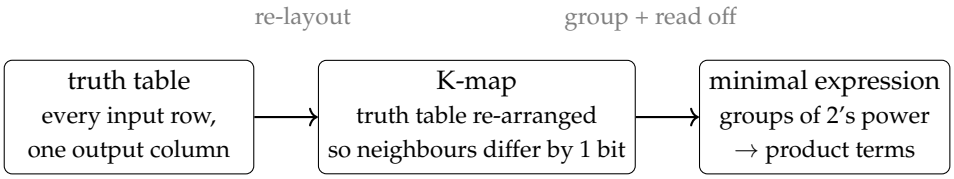



**Figure 8.5:** The minimization pipeline. Notice which step is mechanical (K-map re-layout is just a permutation of the truth table's rows) and which step carries judgment (choosing the groups).

The 2-variable map of Figure 8.6 is the same picture, smaller — the groups it draws are the exact terms of Worked Example 2.

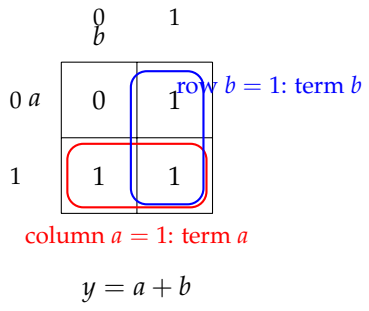



**Figure 8.6:** The 2-variable K-map for $ab + a\bar{b} + \bar{a}b$: two groups of two read off $a$ and $b$ directly.

## 8.9 Common mistakes

- **Misreading place values** on multi-digit binary: read the exponents, don't count digits — $10_2$ is two, not ten; `0x10` is sixteen, not "one-zero".
- **Sign slips in two's complement**: forgetting the top bit is $-2^{n-1}$ gives wrong negatives; $\sim x$ alone is $-x-1$, not $-x$.
- **Off-by-one on representable ranges**: a signed byte is $-128 \ldots +127$, asymmetric — you cannot negate $-128$ in 8 bits.
- **Applying De Morgan halfway**: $\overline{ab} \neq \bar{a}\bar{b}$ — the operator must swap too.
- **Thinking K-map adjacency is physical adjacency**: edges wrap (left-right, top-bottom) because it's a torus, not a table.

## 8.10 Exercises

**Exercise 8.5** (easy)**.** Convert `0b10110010` to hex and decimal (unsigned) and to decimal (signed).

**Exercise 8.6** (easy)**.** What is $-1-1$ in an 8-bit two's-complement byte? Show the hex.

**Exercise 8.7** (easy)**.** Write the truth table of $y = \bar{a}\bar{b} + \bar{a}b$. What common gate is it?

**Exercise 8.8** (easy)**.** Prove $\overline{a+b} = \bar{a}\bar{b}$ by exhaustion on all four input rows.

**Exercise 8.9** (medium)**.** Transform $\overline{\bar{a}+bc}$ into a product form without any complement-of-a-sum remaining.

**Exercise 8.10** (medium)**.** Minimize $y = \bar{a}\bar{b} + a\bar{b} + ab$ and state the gate count before and after.

**Exercise 8.11** (medium)**.** On paper, derive the alarm minimal form a second way: from the truth table, write the full sum-of-products, then factor and absorb without a K-map.

**Exercise 8.12** (medium)**.** In 8-bit two's complement, what is `0x7F + 1`? Name the phenomenon.

**Exercise 8.13** (hard)**.** Design: a hallway light is on when (motion) or (dark and switch-on). Derive minimal SOP from a truth table, then implement the truth equivalence check in `ch08_truth_check.py` on the two forms. Then do HDLBits "Getting Started" + the basic-gates set as external drills.

**Exercise 8.14** (hard)**.** Prove XOR is also universal: build NOT, AND, OR from XOR + constants + wires, or argue which one fails and why (this is a trick — one of them cannot be built; identify which and say why).

## 8.11 Summary

| Idea | One-line version |
|---|---|
| Two levels | Cheapest noise immunity: wide rails + guard band. |
| Binary / hex | Place value $2^i$; hex = 4-bit shorthand. |
| Two's complement | Top bit weighs $-2^{n-1}$; adder subtracts free. |
| Truth table | The definition; $2^n$ rows is the whole function. |
| De Morgan | NOT distributes by swapping AND↔OR. |
| Universal gates | NAND alone, or NOR alone, builds everything. |
| K-map | Truth table re-laid so neighbours differ by one bit; groups of 2's power read off minimal SOP. |
| Synthesis | The pipeline sentence→table→map→gates is what the tools automate. |

## Notes

1. *All About Circuits*, Vol. IV: Digital, ch. 1–8 — numerals, Boolean algebra, gate properties, De Morgan, Karnaugh maps [4].
2. Harris & Harris, *Digital Design and Computer Architecture*, ch. 1–2 — chapter ordering and exercise style adapted here [12].

## Hints and answers

**Byte sign (easy)** `0xFF`=−1, `0x80`=−128, `0x7F`=+127; $100 + (\sim 50 + 1) = 100 + 206 = 306 \to 306 - 256 = 50$.

**XNOR SOP (medium)** $\bar{a}\bar{b} + ab$.

**NOR-only (medium)** NOT: tie inputs ($\overline{a + a} = \bar{a}$); OR: one NOR ($\overline{a + b}$) inverted by NOR-NOT; AND: invert each input with NOR-NOT, then NOR.

**Hex-dec (easy)** `0b10110010`=`0xB2`=178 unsigned, −78 signed.

**Wrap (easy)** `0xFE`=−2.

**Gate ID (easy)** $\bar{a}$.

**De Morgan (easy)** Rows 00:1=1; 01:0=0; 10:0=0; 11:0=0.

**Transform (medium)** $a \cdot \overline{bc} = a(\bar{b} + \bar{c})$.

**Minimize (medium)** $y = \bar{b} + ab \to$ then $\bar{b} + a$: 4 gates → 1 (using $x + \bar{x}y = x + y$).

**Alarm, algebraic (medium)** $ID + I\bar{B} = I(D + \bar{B})$ directly from factoring; SOP full form collects the same three minterms.

**Overflow (medium)** `0x80` — signed overflow from +127 to −128.

**Hallway (hard)** $y = M + DS$; verify both forms equivalent on all 8 rows; HDLBits drills build the same muscle in real HDL.

**XOR trick (hard)** AND cannot be built: XOR is affine — every output is an XOR of inputs and constants, and AND is not affine.

CHAPTER 9

# Combinational circuits

Chapter 8 gave us the alphabet; this chapter builds the words. Every useful digital circuit reduces to a handful of combinational *building blocks* — multiplexers and decoders route, adders and comparators compute, and all of them together form the ALU at the heart of a CPU. After this chapter you will be able to build a mux from gates and from `assign`, explain why a ripple-carry adder is slow, decode a seven-segment display, and count the speed/area trade in gate delays.

**BEFORE YOU START**

Chapter 8 (Boolean algebra). *Recap: combinational* means the output depends only on the present inputs — no memory.

**AFTER THIS CHAPTER YOU CAN**

- build a multiplexer and a decoder from gates;
- build a ripple-carry adder and count its delay;
- design a seven-segment decoder;
- explain how these blocks become an ALU.

**HOW TO THINK ABOUT THIS**

Combinational design is like building with plumbing: signals flow through, and the output simply reflects the input after a delay. The delay matters: a long chain of gates is slow, which is why the structure of an adder affects how fast a computer can run.

## 9.1 Multiplexers and decoders: the two routers

A *multiplexer* (mux) selects one of $2^n$ inputs according to $n$ select lines: $y = d_s$. Built from gates it is a 2-level sum-of-products — a decoder of $s$ feeding ANDs whose outputs OR together. Its dual, the *decoder*, spreads one enable into $2^n$ mutually-exclusive lines: $y_i = e \cdot [s = i]$. Together they are the routers of combinational logic — mux chooses a path, decoder chooses a destination.

**Exercise 9.1** (easy)**.** Write the sum-of-products for a 2:1 mux in terms of $s, d_0, d_1$; then show it collapses to $\bar{s}d_0 + sd_1$.

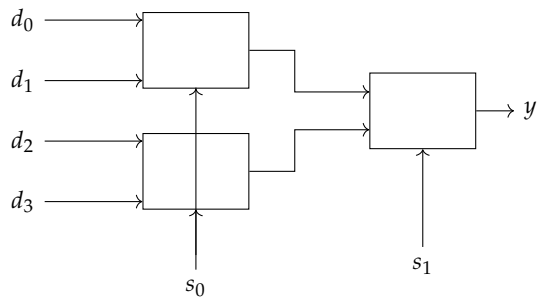



**Figure 9.1:** A 4:1 mux as a tree of three 2:1 muxes. Notice the select fanout: $s_0$ picks inside the leaves, $s_1$ picks between them — the same address-decoding pattern repeats all the way up to memory chips.

## 9.2 Worked example 1: a 4:1 mux, gates then `assign`

**Strategy.** Build it twice — once as a gate tree so the cost is visible, once as a one-line `assign` so the intent is — and convince ourselves the synthesizer sees them as the same circuit.

**Solution.** As gates, Figure 9.1: three 2:1 muxes, 2 levels of logic, $s_0$ at the leaves, $s_1$ at the root. In Verilog it collapses to a single selection:

```
assign y = s1 ? (s0 ? d3 : d2) : (s0 ? d1 : d0);
```

**Listing 9.1:** The same 4:1 mux as one `assign` — read-only here; toolchain formalised in ch11.

The ternary nest reads exactly like the tree: top-level `s1?` chooses a branch, each `s0?` chooses a leaf. Synthesis maps both descriptions to the same lookup table; the `assign` version wins because it says *what* and leaves the *how* to the tool.

## 9.3 Adders: half, full, and ripple-carry

A *half adder* adds two bits: $s = a \oplus b$, $c = ab$ — an XOR and an AND. A *full adder* adds three (two operands plus carry-in): $s = a \oplus b \oplus c_{in}$ and $c_{out} = ab + c_{in}(a \oplus b)$ — two XORs, two ANDs, an OR (Figure 9.2).

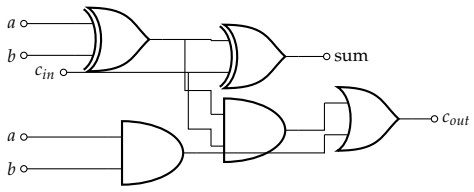



**Figure 9.2:** Full-adder gate map. Notice the shared $a \oplus b$ feeding both the sum XOR and the carry AND — reusing a partial result is the whole trick, and why the gate count is 5 not 7.

```
module full_adder(a, b, cin, sum, cout);
  input a, b, cin;
  output sum, cout;
  assign sum = a ^ b ^ cin;
  assign cout = (a & b) | (cin & (a ^ b));
endmodule
```

**Listing 9.2:** `code/ch09_full_adder.v -- the cell the ripple reuses.`

Chaining four cells gives the *ripple-carry adder* of Figure 9.3: each carry must *settle* before the next full adder can even start — the carry *ripples* through.

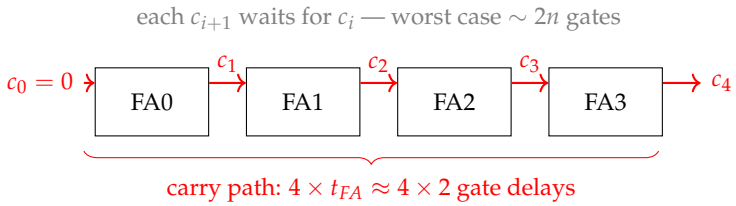



**Figure 9.3:** The ripple carry chain. Notice what is *not* parallel: $c_4$ is a function of every bit upstream, so the worst-case delay grows with $n$, not with the answer's size.

```
module adder4(a, b, sum, cout);
  input [3:0] a, b;
  output [3:0] sum;
  output cout;
  wire [2:0] c;
  full_adder fa0(.a(a[0]), .b(b[0]), .cin(1'b0), .sum(sum[0]), .cout(c
      [0]));
  full_adder fa1(.a(a[1]), .b(b[1]), .cin(c[0]), .sum(sum[1]), .cout(c
      [1]));
  full_adder fa2(.a(a[2]), .b(b[2]), .cin(c[1]), .sum(sum[2]), .cout(c
      [2]));
  full_adder fa3(.a(a[3]), .b(b[3]), .cin(c[2]), .sum(sum[3]), .cout(
      cout));
endmodule
```

**Listing 9.3:** `code/ch09_adder4.v + its testbench -- the 4-cell chain, exercised.`

```
 a + b = (cout,sum)
15 + 10 = (1, 9)
15 + 15 = (1,14)
ch09_adder4_tb.v:16: $finish called at 64000 (1ps)
```

**Listing 9.4:** Last rows of the real `vvp` output — carry out asserted at 15+5, 15+10, 15+15.

## 9.4 Worked example 2: counting the delay

**Strategy.** Estimate the ripple's worst-case delay honestly, then compare a 2-level (sum-of-products) alternative — and see why the trade is speed *vs* area, not free lunch.

**Solution.** Each full adder's carry-out is 2 gate-delays after its carry-in settles (XOR then AND-OR). For a 4-bit ripple the carry walks $4 \times 2 = 8$ levels; for $n$ bits, $\approx 2n$. The alternative — compute carry directly as a 2-level function of all $2n$ inputs — needs $2^n$ product terms per carry: $n = 8$ needs 256 AND gates per carry versus 8 FAs. Figure 9.4 sketches the real compromise (carry-lookahead, built properly later): more logic in exchange for a bounded $\sim$4-level path. The lesson is the *trade itself*: in combinational design there is almost always a faster circuit if you are willing to pay for it in gates.

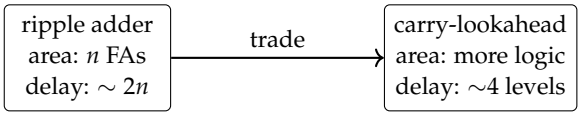



**Figure 9.4:** Speed-vs-area in one picture. Notice both blocks produce the same sum — only the *path length* to it differs.

**Exercise 9.2** (medium)**.** An 8-bit ripple adder is roughly 2× a 4-bit's delay and 8× the two-level carry logic's area. Write the delay for $n = 16$ and the product-term count for the 2-level $c_{16}$ alternative.

## 9.5 Comparators, encoders, priority

A *comparator* is a subtractor that keeps only the sign: equality is an XNOR tree, magnitude falls out of the borrow chain. An *encoder* runs a decoder backwards — $2^n$ lines in, $n$ bits out — and a *priority encoder* resolves ties by position, which is exactly what an interrupt controller does: `casez` with wildcard rows, don't-cares marking "lower priority does not matter."

**Don't-cares** are a contract, not an omission: writing `x` on impossible inputs lets the minimizer pick the cheaper neighbor — but only for inputs the rest of the system truly never produces.

## 9.6 Worked example 3: seven-segment decoder

**Strategy.** A digit → 7 segments is just 7 Boolean functions of 4 inputs. Write the truth table once (rows 10–15 as don't-cares), minimize each segment, then let a `case` statement say it plainly and the simulator print the check.

**Solution.** The segment map is Figure 9.5; the decoder and its `case`:

```
module seg7(d, seg);
  input [3:0] d;
  output reg [6:0] seg;
  always @(*) begin
    case (d)
      4'd0: seg = 7'b1111110;
      4'd1: seg = 7'b0110000;
      4'd2: seg = 7'b1101101;
      4'd3: seg = 7'b1111001;
      4'd4: seg = 7'b0110011;
      4'd5: seg = 7'b1011011;
      4'd6: seg = 7'b1011111;
      4'd7: seg = 7'b1110000;
      4'd8: seg = 7'b1111111;
      4'd9: seg = 7'b1111011;
      default: seg = 7'b0000000;
    endcase
  end
```

```
endmodule
```

**Listing 9.5:** `code/ch09_seg7.v -- BCD to cathodes {a,b,c,d,e,f,g}.`

```
 d | abcdefg
---+--------
 0 | 1111110
 1 | 0110000
 2 | 1101101
 3 | 1111001
 4 | 0110011
 5 | 1011011
 6 | 1011111
 7 | 1110000
 8 | 1111111
 9 | 1111011
10 | 0000000
...
15 | 0000000
```

**Listing 9.6:** Real truth-dump of `ch09_seg7_tb.v` — digits 0–9 plus the don't-care rows.

The `default` is the don't-care policy made concrete: rows the driver never produces get `x` on paper and `0000000` here — the display dark rather than arbitrary.

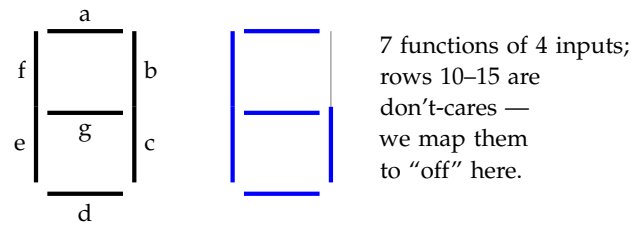
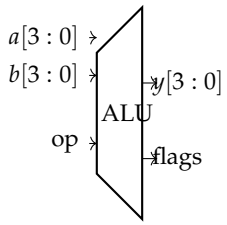



**Figure 9.5:** Segment letters and the digit-6 pattern. Notice each of the seven bars is an independent function of the same 4 inputs — seven small K-maps, not one big one.

**Exercise 9.3** (medium)**.** Rows 10–15 were don't-cares. If a bug drives them with `1010`, what does our `default` show? What would a cheaper minimizer exploiting the don't-cares legally show instead?

## 9.7 The ALU preview and the road to a bus

Stack these blocks: mux to pick the operation, adder to add, comparator to compare, and you have the shape of an *ALU* (Figure 9.6) — the combinational heart of the chapter-14 CPU. The last step is wiring: bundle the same-width wires and call the bundle a *bus* — a single line on the diagram that is really $w$ bits wide. Everything later (registers, memories, datapaths) speaks in buses; combinational logic speaks in bits.




combinational heart of the ch14 CPU

**Figure 9.6:** The ALU as a symbol. Notice it is pure combination: data in, result out — no memory. The `flags` output is just comparators' results bundled out for later chapters to use.

## 9.8 Common mistakes

- **K-map adjacency slips**: the edges wrap — top touches bottom, left touches right. A group of 4 that is not a rectangle is not a group.
- **Incomplete truth tables**: a missing row is not a 0; feed the minimizer all $2^n$ rows or it will "minimize" a lie.
- **Treating don't-cares as values**: `x` means "whatever is cheapest," not "0" — if the input can actually occur, it was never a don't-care.
- **Forgetting `default`** in a `case`: unlisted rows leave wires undriven and synthesize latches, not logic.
- **Confusing encoder and decoder direction**: decoder spreads one code to many lines; encoder packs many lines to one code.

## 9.9 Exercises

**Exercise 9.4** (easy)**.** Write the 4:1 mux truth table for $d_0..d_3$ and $s_1 s_0$; how many rows?

**Exercise 9.5** (easy)**.** Half-adder: show $s = a \oplus b$ and $c = ab$ from the truth table.

**Exercise 9.6** (easy)**.** What does a 3-to-8 decoder output when $s = 101$, $e = 1$? When $e = 0$?

**Exercise 9.7** (easy)**.** In our ripple chain, why does $c_4$'s delay not depend on the data pattern? When does it?

**Exercise 9.8** (medium)**.** Design a 4:2 priority encoder: inputs $i_0..i_3$ (highest first), output the index of the highest asserted bit plus a valid flag. Write the truth table using don't-cares.

**Exercise 9.9** (medium)**.** Show a subtractor is "an adder plus two's complement": write $y = a - b$ as $a + \bar{b} + 1$ in gates, reusing the adder4 structure.

**Exercise 9.10** (medium)**.** Estimate how many LUTs (a 4-input truth table each) a function of 6 inputs needs in the worst case, and how many if it factors neatly.

**Exercise 9.11** (medium)**.** Seven-seg segment $e$ is on for digits $\{0, 2, 6, 8\}$. Using rows 10–15 as don't-cares, minimize $e$ on paper.

**Exercise 9.12** (hard)**.** (Implement) Extend `ch09_adder4_tb.v` to drive all $16^2$ pairs and assert `cout, sum == a+b` each row; run it and report.

**Exercise 9.13** (hard)**.** A comparator: write the 2-bit-magnitude comparison $a > b$ as a Boolean expression, then show it equals "subtract and check the borrow."

## 9.10 Summary

| Block | One-line version |
|---|---|
| Mux | Data selects which input passes; decoder's dual. |
| Decoder | One enable, $2^n$ mutually-exclusive lines. |
| Half / full adder | XOR+AND, then XOR chain + carry-merge; the cell. |
| Ripple carry | $n$ FAs chained; worst delay $\sim 2n$ levels. |
| Two-level carry | $\sim$4 levels but $2^n$ terms — speed bought with area. |
| Comparator | A subtractor read for sign only. |
| Priority encoder | "First wins": `casez` + don't-cares. |
| Seven-seg decoder | Seven 4-input functions sharing one truth table. |
| ALU | Mux + adder + comparator = the combinational CPU heart. |

## Notes

1. Harris & Harris, *Digital Design and Computer Architecture*, ch. 2, 5 — mux/decoder structure, adder taxonomy [12].
2. *All About Circuits*, Vol. IV: Digital — combinational building blocks [4].
3. *fpga4fun.com* — small complete FPGA projects, the book's "one project per concept" model [13].

## Hints and answers

**2:1 mux (easy)** $y = \bar{s}d_0 + sd_1$ — decoder of $s$ feeding ANDs into an OR, the universal 2-level shape.

**16-bit delay (medium)** $\approx 2n = 32$ levels ripple vs $\sim 2^{16}$ terms for a flat $c_{16}$ — the trade explodes, so lookahead splits the difference.

**Default policy (medium)** `1010` $\rightarrow$ `0000000` — dark. A minimizer using the `x` rows could legally show anything; it owes correctness only on rows 0–9.

**4:1 table (easy)** 4 select states $\times$ don't-care data = 16 rows enumerated.

**Half adder (easy)** $s = 1$ iff $a \neq b$; $c = 1$ iff $a = b = 1$.

**Decoder (easy)** $s = 101, e = 1$: line 5 high; $e = 0$: all zero.

**Ripple delay (easy)** Worst case is data-independent (carry must propagate end-to-end); average case depends on the operands.

**Priority encoder (medium)** Rows like `1--` $\rightarrow$ index 0 when $i_0 = 1$ regardless of the rest.

**Subtractor (medium)** Invert $b$, set $c_0 = 1$: one inverter row plus the same chain — the adder computes $a + (\bar{b} + 1)$.

**LUTs (medium)** Naive $2^6 = 64$ rows on 4-input LUTs; a factored function can be a handful — structure matters more than input count.

**Segment $e$ (medium)** $e = \bar{d}_0\,(\bar{d}_2 + d_1)$ — two groups of four, exploiting three `x` cells.

**Exhaustive tb (hard)** Loop `a` 0–15, `b` 0–15, compare `{cout, sum}` to `a+b`; 256 rows, all pass.

**Comparator (hard)** $a > b$: $a_1\bar{b}_1 + (a_1 \odot b_1)\,a_0\bar{b}_0$ with $\odot$ XNOR — equivalently, the borrow out of $a - b$.

CHAPTER 10

# Sequential circuits and state machines

Chapter 9's circuits were *memoryless*: same inputs, same outputs, forever. Real machines need to *remember* — which is what a bank account, a traffic light, and a CPU register have in common. This chapter introduces feedback (latches and flip-flops), registers and counters, and the finite state machine; the running simulation shows the classic latch bug that silently ships when the two are confused. After it, you will be able to design a small FSM end to end and know why every button that talks to a clock needs two flops.

**BEFORE YOU START**

Chapters 8–9. *Recap:* to remember anything, a circuit must feed its output back to its input. Memory plus a clock gives *sequential* circuits.

**AFTER THIS CHAPTER YOU CAN**

- explain how a latch and a flip-flop store a bit;
- build a counter and divide a clock to 1 Hz;
- design a finite-state machine from a word description, Moore or Mealy;
- explain metastability and why we use two flip-flops to cross clock domains.

**HOW TO THINK ABOUT THIS**

Memory comes from feeding an output back into the circuit that produced it. A clock turns this chaotic idea into an orderly one: everything updates together on a tick. To design a state machine, first draw the picture of states and arrows in words, then convert to logic, never the other way around.

## 10.1 Feedback and memory

Wire a gate's output back toward its input and the circuit gains *state*. Two cross-coupled NORs form the SR *latch* — the first memory: $S = 1$ sets, $R = 1$ resets, both 0 holds. Add an enable input and you get the D *latch*: while `en` is high, $q$ follows $d$; while low, it holds. *Transparent* is the operative word — the output can change whenever the enable is up, not just at a tick.

The *D flip-flop* trades transparency for a shutter: *q* samples *d only* on the rising clock edge (`posedge`). Level vs edge is the difference between a window and a photograph — and the difference between a circuit that follows glitches and one that ignores them.

## 10.2 Worked example 1: the latch bug, live

**Strategy.** Drive a latch and a flip-flop with the same `d`, open the latch's `en` for one window with a glitch inside, and watch who follows the glitch.

**Solution.** Two cells:

```
module d_latch(en, d, q);
  input en, d;
  output reg q;
  always @(*) if (en) q = d; // follows d while en=1
endmodule
module d_ff(clk, d, q);
  input clk, d;
  output reg q;
  always @(posedge clk) q <= d; // samples d only at clk rise
endmodule
```

**Listing 10.1:** `code/ch10_dff.v -- latch vs flip-flop, same inputs.`

```
t= 12 en=1 clk=0 d=0 | latch=0 ff=1
t= 15 en=1 clk=1 d=0 | latch=0 ff=0
t= 16 en=1 clk=1 d=1 | latch=1 ff=0
t= 25 en=0 clk=1 d=1 | latch=1 ff=1
```

**Listing 10.2:** Excerpt of the real `vvp` run — the latch tracks the mid-window glitch; the FF waits for an edge.

At t=12 the input glitched low and the latch fell in step — the FF did not notice (its edge was not due). At t=16 the latch snapped to the new `d` mid-cycle; the FF held its old sample until t=25, the next rising edge. Figure 10.1 is the same data as waveforms.

That is why synchronous design uses edge-triggered flops everywhere: the edge gives the whole machine a heartbeat and lets every block's output settle into registers before the next beat.

**Exercise 10.1** (medium)**.** In the trace, `en` went high at t=8 and low at t=21, while the clock ran all along. State precisely which `d` values the FF sampled at t=5, 15, 25, and why the latch's `q` differs at t=16.

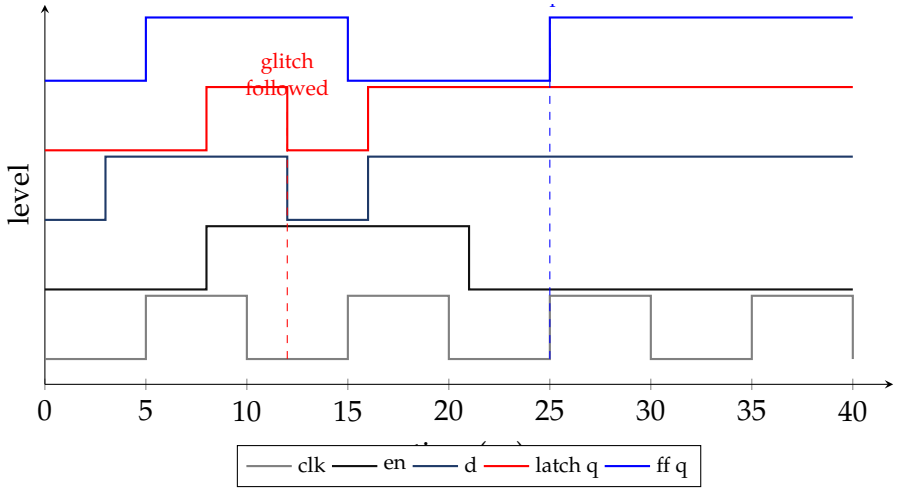
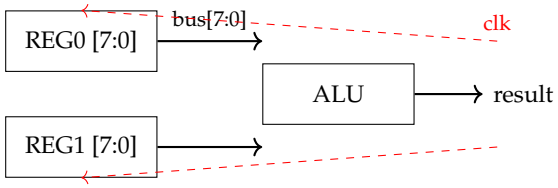



**Figure 10.1:** Latch vs flip-flop, real simulated traces. Notice the latch's *window*: inside it, every wiggle on `d` reaches `q`. The FF changes only at clock edges — one sample, then done.

## 10.3 Setup, hold, registers, buses

For the edge sample to be meaningful, `d` must be stable a little *before* the edge (*setup time*) and a little *after* (*hold time*) — the analysis of those windows is Book 7's job; here the concept matters: the clock edge is a contract, and combinational delays are how you budget it.

A *register* is a row of flip-flops clocked together — Figure 10.2; a *bus* is the bundle of wires feeding and leaving registers.




one clock edge samples every register at once

**Figure 10.2:** Registers around an ALU. Notice the same `clk` fans out to every register — one edge, every value sampled simultaneously; between edges the combinational ALU settles.

## 10.4 Counters

A *counter* is a register that adds 1 per edge. Ripple counters chain toggle flip-flops (Figure 10.3); synchronous counters compute `q+1` in parallel — the useful ones add `en` (only count when told) and a terminal-count flag.

```
module counter(clk, en, q, tc);
  input clk, en;
  output reg [3:0] q = 0;
  output tc;
  assign tc = (q == 4'd15) & en;
  always @(posedge clk)
```

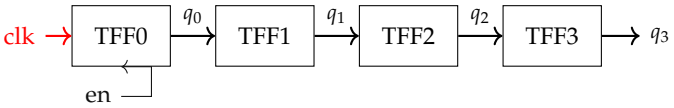



**Figure 10.3:** Counter structure. Notice each stage toggles only when every lower bit is 1 — counting is carry-propagation made visible.

```
    if (en) q <= q + 1;
endmodule
```

**Listing 10.3:** `code/ch10_counter.v -- enable + terminal count.`

```
t=155 en=1 | q=15 tc=1
t=165 en=1 | q= 0 tc=0
t=175 en=1 | q= 1 tc=0
```

**Listing 10.4:** Real run tail — rollover at 15 with `tc` asserted, counting resumes from 0.

## 10.5 Worked example 2: a 1 Hz tick from 27 MHz

**Strategy.** The Tang Nano 9K board's oscillator runs at 27 MHz; blinking an LED at 1 Hz needs a prescaler that counts 27 000 000 cycles, then toggles — a counter with a terminal count wired to an output flip.

**Solution.** $\log_2(27\times10^6) \approx 24.7$, so a 25-bit counter suffices. Count to 13 500 000, reset, toggle the blink: $13.5\times10^6$ ticks per half-second. In Verilog the whole prescaler is one `if` inside the `always` block plus an output flip — the same `counter` pattern at scale. The honest check: the LED pin needs only the toggle — the count is internal state, exactly what registers are for.

## 10.6 Finite state machines

An *FSM* is states + edges + outputs: a register holding the state, combinational next-state logic, and output logic. *Moore* outputs depend on state only; *Mealy* outputs also take the inputs directly (Figure 10.4). The disciplined three-block style — one `always` per job — keeps the machine readable and the synthesizer honest.

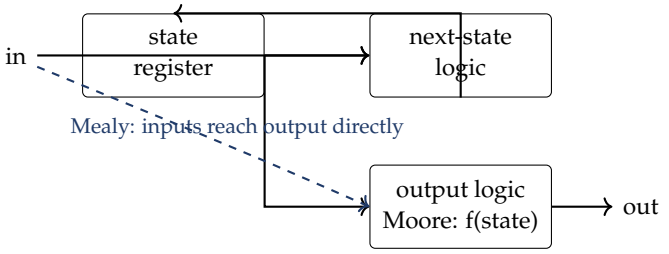



**Figure 10.4:** Moore vs Mealy. Notice the dashed path: only Mealy lets inputs reach the output within the same cycle — one edge faster, one glitch path wider.

## 10.7 Worked example 3: the traffic-light FSM, end to end

**Strategy.** State diagram first (Figure 10.5), then an encoding, then the three-block Verilog, then the simulator prints the cycle for us to check against the diagram.

**Solution.** Four states — NS green, NS yellow, EW green, EW yellow — each transition on a timer:

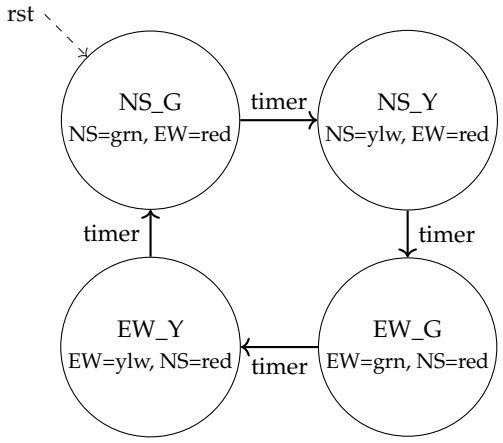



**Figure 10.5:** The traffic machine. Notice outputs ride *inside* the states (Moore): NS is green only in `NS_G`, full stop.

```
module traffic_fsm(clk, rst, ns, ew);
  input clk, rst;
  output reg [1:0] ns, ew;
  localparam NS_G=2'd0, NS_Y=2'd1, EW_G=2'd2, EW_Y=2'd3;
  reg [1:0] state = NS_G, nxt;
  always @(*) begin
    case (state)
      NS_G: nxt = NS_Y; NS_Y: nxt = EW_G;
      EW_G: nxt = EW_Y; EW_Y: nxt = NS_G;
      default: nxt = NS_G;
    endcase
  end
  always @(posedge clk) state <= rst ? NS_G : nxt;
  always @(*) begin
    ns = (state==NS_G) ? 2'b00 : (state==NS_Y) ? 2'b01 : 2'b10;
    ew = (state==EW_G) ? 2'b00 : (state==EW_Y) ? 2'b01 : 2'b10;
  end
endmodule
```

**Listing 10.5:** `code/ch10_traffic_fsm.v -- always-three-block style.`

```
t= 15 rst=0 state=0 | NS=00 EW=10
t= 25 rst=0 state=1 | NS=01 EW=10
t= 35 rst=0 state=2 | NS=10 EW=00
t= 45 rst=0 state=3 | NS=10 EW=01
t= 55 rst=0 state=0 | NS=00 EW=10
```

**Listing 10.6:** The real full-cycle trace — GRN (`00`)→YEL (`01`) around the loop, EW red (`10`) throughout the NS phase.

The printed cycle matches the diagram edge-for-edge: NS goes green, yellow, then EW green, yellow, and back — Moore outputs change only when the state does.

**Exercise 10.2** (medium). Extend the machine with a pedestrian-wait state entered from `NS_G` only when a button input is high. Which block changes — next-state, register, or output?

## 10.8 Worked example 4: Mealy answers a cycle early

**Strategy.** Give the same rising edge to a Moore detector and a Mealy detector and watch the outputs disagree by exactly one cycle — the price and prize of reading inputs at the output.

**Solution.**

```
t= 5 x=0 | mealy=0 moore=0
t= 15 x=1 | mealy=1 moore=0
t= 25 x=1 | mealy=0 moore=1
t= 35 x=0 | mealy=0 moore=1
```

**Listing 10.7:** `code/ch10_mm_tb.v run -- mealy=1 at t=15, the cycle the edge lands; moore=1 waits for t=25.`

Mealy's output is combinational on the input, so it fires *in* the edge's cycle — one cycle faster, and one glitch-path wider (a mid-cycle wiggle on `x` would reach `yme` too). Moore's rides the state, always a registered answer. Rule of thumb: outputs that must be clean (LED, motor, write-enable) go Moore; outputs that must be quick (requests, acknowledges) can pay the Mealy price.

## 10.9 Worked example 5: the "101" detector, end to end

**Strategy.** The second classic FSM: detect `101` on a serial input *with overlap* (a found "1" may start the next pattern). Four states, then let the sim find both hits in `11010110`.

**Solution.**

```
module seq101(clk, x, hit);
  input clk, x; output reg hit;
  localparam IDLE=0, GOT1=1, GOT10=2, FOUND=3;
  reg [1:0] st = IDLE, nx;
  always @(*) case (st)
    IDLE: nx = x ? GOT1 : IDLE;
    GOT1: nx = x ? GOT1 : GOT10;
    GOT10: nx = x ? FOUND : IDLE;
    FOUND: nx = x ? GOT1 : GOT10; // overlap: the 1 restarts
    default: nx = IDLE;
  endcase
  always @(posedge clk) st <= nx;
  always @(*) hit = (st == FOUND);
endmodule
```

**Listing 10.8:** `code/ch10_seq101.v -- the overlap lives in FOUND's edges.`

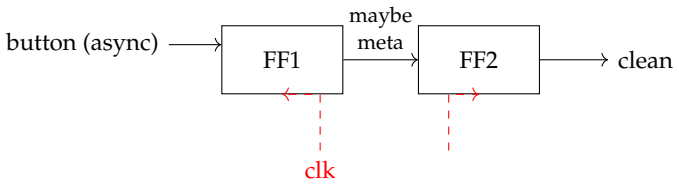
```
t= 35 x=1 state=2 hit=0
t= 45 x=0 state=3 hit=1
t= 55 x=1 state=2 hit=0
t= 65 x=1 state=3 hit=1
t= 75 x=0 state=1 hit=0
```

**Listing 10.9:** Real run over `11010110` — hits at t=45 and t=65, the second riding the first's last bit.

The overlap is the `FOUND` row: on `x=1` we go to `GOT1`, not `IDLE` — that 1 was both the end of one pattern and the start of the next. One line of the transition table carries the whole "with overlap" requirement.

## 10.10 Metastability and the two-flop rule

One honest paragraph: a flip-flop whose input changes exactly at the edge does not pick 0 or 1 — it can sit *between* them, and may stay undecided for an unbounded (but exponentially unlikely) time. That *metastability* is physics, not a bug. The defense is Figure 10.6: run every asynchronous input through *two* flops — the first absorbs the rare bad sample, the second sees only settled logic. **Never clock a button into logic without the synchroniser**; every switch in later chapters goes through this shape.




FF1 absorbs the metastable edge; FF2 sees only settled values

**Figure 10.6:** The two-flop synchroniser. Notice what is *not* there: no guarantee FF1 settles — only that FF2's input is one whole cycle older, which settles the probability to negligible.

## 10.11 Common mistakes

- **Latch vs flip-flop confusion**: `@(*)` on everything infers transparency and follows glitches — default to `posedge` always.
- **Incomplete sensitivity lists**: a `case` without `default` leaves the output undriven on unlisted states and invents a latch you never drew.
- **Unreachable states**: an encoding wider than needed leaves ghost states; route every default back somewhere real.
- **Blocking vs nonblocking**: in clocked blocks use `<=`; mixing is the classic silent-race source.
- **Async inputs to logic**: skipping the two-flop synchroniser on a button works 99.9% of the time — and fails on the demo day.

## 10.12 Exercises

**Exercise 10.3** (easy)**.** Level vs edge: name the behavior each device exhibits and the Verilog edge keyword for each.

**Exercise 10.4** (easy)**.** A 3-bit counter starting at 5: what are the next four counts?

**Exercise 10.5** (easy)**.** Which block of the three holds memory? Which two are pure combination?

**Exercise 10.6** (easy)**.** Why does Moore timing lag Mealy by up to a cycle?

**Exercise 10.7** (medium)**.** Pattern detector: FSM over bits detecting `101` with overlap. Draw the states.

**Exercise 10.8** (medium)**.** Gray counter: why is Gray encoding safer for cross-domain sampling of a count?

**Exercise 10.9** (medium)**.** The forgotten `else`: `if (en) q <= d;` inside `@(*)` — what does the synthesizer build and why is it wrong?

**Exercise 10.10** (medium)**.** Prescaler: give the count value and bit width to get 10 Hz from 27 MHz.

**Exercise 10.11** (hard)**.** (Implement) Write `traffic_tb` that asserts the exact state cycle and count a violation; then add a walk-light state.

**Exercise 10.12** (hard)**.** A vending machine takes only 5- and 10-rupiah coins, dispenses at 15: draw the FSM, encode it, and argue Moore vs Mealy.

**Exercise 10.13** (easy)**.** An SR latch with NOR gates: which inputs set, reset, and hold — and what forbidden input pair breaks the abstraction?

**Exercise 10.14** (medium)**.** Register file: sketch two 8-bit registers and one shared bus; who decides which register drives the bus, and what gate shape does it?

**Exercise 10.15** (medium)**.** Latency count: in the mm_tb run, how many clock cycles separate the first `mealy=1` from the last `moore=1`? Explain both.

**Exercise 10.16** (medium)**.** One-hot vs binary: our traffic machine used 2 bits for 4 states. Give the one-hot encoding, its gate-count intuition, and when it wins.

**Exercise 10.17** (medium)**.** Serial adder: two numbers arrive one bit-pair per cycle. Draw the 2-state FSM that outputs their sum (hint: one bit of state is a carry register — you already built the combinational part in ch9).

**Exercise 10.18** (hard)**.** Design a debounced-button FSM: button → two-flop sync → edge-detect → one clean pulse per press, however long the press. State the states.

## 10.13 Summary

| Idea | One-line version |
|---|---|
| Latch | Transparent window — follows inputs while enabled. |
| Flip-flop | Photograph at the edge — the heartbeat's shutter. |
| Setup/hold | The edge contract; Book 7 does the math. |
| Register/bus | Edge-sampled row + the wires that feed it. |
| Counter | A register that adds 1 — `en` makes it a prescaler. |
| FSM (3 blocks) | State reg + next-state + output; never mix them. |
| Moore vs Mealy | State-only vs input-to-output path (1 cycle). |
| Metastability | Physics of the edge; sync it with two flops, always. |

## Further reading

- Harris & Harris, *Digital Design and Computer Architecture*, 2nd ed., ch. 3 — the reference for latch/FF discipline and FSM style; free companion materials at `pages.hmc.edu/harris/ddca/` (undergraduate level; the chapter's primary source [12]).
- Kuphaldt, *All About Circuits* Vol. IV (Digital), sequential chapters — alternative prose walk-through of the same ladder; free textbook [4].
- MIT OCW *6.004 Computation Structures* (2017) — university-level sequential logic lectures and labs, course page on `ocw.mit.edu` (undergraduate level; the academic version of this chapter).
- Nisan & Schocken, *nand2tetris.org/course* project 3 (Memory) — build the same registers bottom-up from DFFs in their HDL (intro level; a second implementation of the same ideas).
- HDLBits, `hdlbits.01xz.net` — the Circuits/Sequential problem set: small self-checking FSM drills named in the exercises (practice engine, free).
- *fpga4fun.com* — short real FPGA projects that consume exactly these primitives (project ideas, intermediate).

**WATCH INSTEAD**

- *Building an 8-bit breadboard computer* (full playlist), Ben Eater, ∼20 videos — the register/counter/FSM concepts here built physically on breadboards; matches §10.1, §10.3, §10.4. Accessed 2026-10-11.
- *D flip-flop* (single video), Ben Eater, ∼15 min — the DFF built from gates in front of you; matches §10.1–§10.2. Accessed 2026-10-11.

## Notes

1. Harris & Harris, *Digital Design and Computer Architecture*, ch. 3 — latches, flops, FSM style [12].
2. *All About Circuits*, Vol. IV: Digital — sequential building blocks [4].
3. HDLBits problem sets — the FSM drills named in the exercises [14].

## Hints and answers

**Trace (medium)** FF samples $d$ at t=5 (=1), 15 (=0), 25 (=1); the latch passed the t=16 value mid-window, the FF waited to t=25.

**Pedestrian (medium)** Next-state logic + encoding only; outputs ride on the states (Moore).

**Level/edge (easy)** Latch: level, `@(*)`; FF: edge, `posedge clk`.

**3-bit count (easy)** 5,6,7,0,1.

**Blocks (easy)** The state register; next-state and output are combinational.

**Moore lag (easy)** Outputs settle only after the state advances.

**101 detector (medium)** 4 states (none/1/10/101); on completion input 0 sends you back for overlap.

**Gray (medium)** Only one bit flips per step — an async sample can be off by at most one count.

**Missing else (medium)** Unassigned paths infer a latch holding `q` — an invisible memory you never drew.

**10 Hz prescaler (medium)** $1.35\times10^6$ ticks per half-period, 21 bits.

**Walk light (hard)** One more state + a `walk` input; the tb's cycle assertion catches any skipped state.

**Vending (hard)** 4 states (0/5/10/dispense); Moore keeps the dispense output glitch-free.

**SR latch (easy)** S=1 sets, R=1 resets, both 0 holds; S=R=1 is the forbidden pair (output depends on release order).

**Register file (medium)** An enable line per register picks the driver; each output goes through a mux or tri-state buffer.

**Latency count (medium)** t=15 to t=35: two edges — Mealy fires in the sample's own cycle, Moore only after the state advances.

**One-hot (medium)** 4 bits, one hot per state; next-state logic is one AND-OR row per destination — wins when speed beats area and $n$ is small.

**Serial adder (medium)** 2 states C=0/C=1 (the carry); each cycle outputs $a_i \oplus b_i \oplus C$ and stores the new carry.

**Debounce (hard)** States idle/pressed/wait-release + a settling counter; pulse out on the synced edge only.

CHAPTER 11

# Verilog I: describing hardware

Chapters 8–10 used Verilog where it was needed; this chapter teaches it as a language. The single idea underneath everything: *Verilog describes hardware, it does not give orders*. Once that lands, the syntax is small — two kinds of signals, two recipes for `always`, one rule about `=` versus `<=`, and a testbench that is itself just a module. After this chapter you can write a legal module three different ways, spot the two classic wrong ways, run it under Icarus Verilog, and read the waveform it produces.

**BEFORE YOU START**

Chapters 8–10. *Recap:* Verilog describes hardware that exists all at once, not instructions that run in order. If you know a programming language, forget the habit of reading top to bottom.

**AFTER THIS CHAPTER YOU CAN**

- describe a combinational and a sequential block in Verilog;
- explain blocking versus non-blocking assignment and use each correctly;
- write a testbench and read the waveform in GTKWave;
- avoid the classic synthesis mistakes.

**HOW TO THINK ABOUT THIS**

Hardware description is a different way of thinking from programming: everything you write exists at once and works in parallel. Ask of every line, "what wires and gates is this?" not "what runs next?" A testbench is your lab bench, so write it before you trust the design.

## 11.1 HDL is not a program

A *hardware description language* (HDL) writes down a circuit. When you write `assign y = a & b;` you are not instructing a machine to AND anything — you are asserting that an AND gate exists between `a`, `b` and `y`, now and forever. Statements run whenever their inputs change, all of them, all the time. There is no "next line" because there are no lines: there is silicon-shaped text.

The reset that makes Verilog readable: *before writing any code, draw the circuit*. If you cannot point at a gate, a mux or a flip-flop for a line of code, the line is probably describing software.

**CODE IS A SCHEMATIC WRITTEN SIDEWAYS**

Every synthesizable line should map to a part you could sketch: an `assign` is wires and gates, a clocked `always` is flip-flops, an `if` inside `@(*)` is a mux. When a line has no picture, delete the line, not the picture.

## 11.2 Modules, ports, wires and regs

A *module* is the unit of hardware: a box with named connections. *Ports* are the box's boundary wires — `input`, `output`, rarely `inout`. Inside, two kinds of signal matter:

**`wire`** a connection. Driven by `assign` or by a port; holds nothing itself. The default: any name you use but never declare is an implicit wire (declare anyway — implicit wires hide typos).

**`reg`** assigned inside an `always` block. The name is a lie the language kept: a `reg` does *not* imply a register. In a combinational `@(*)` block it is still just gates; only an edge-triggered block makes it a real flip-flop.

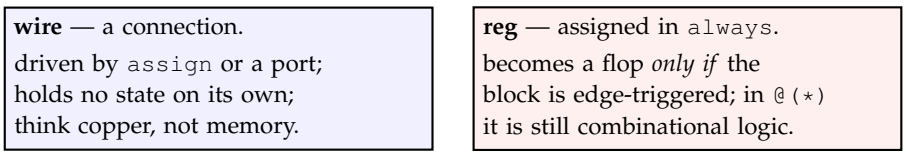



**Figure 11.1:** Wire vs reg at a glance. Notice the trap in the right card: `reg` is a typing rule, not a memory claim — the block's sensitivity list decides what silicon you get. Regenerate: `figures/ch11/fig_wires.tex`.

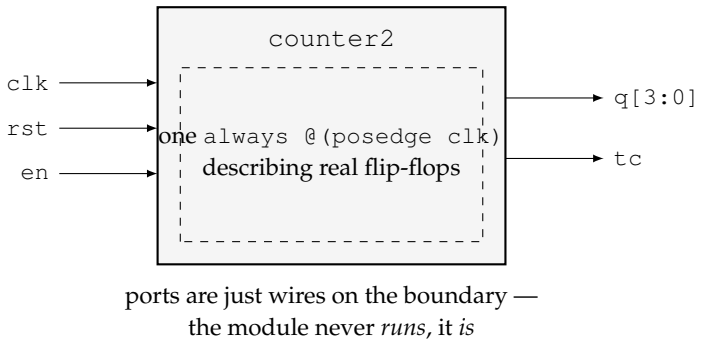



**Figure 11.2:** Our counter as a box of ports. Notice that the port list is the whole contract — everything inside the dashed boundary is private machinery. Regenerate: `figures/ch11/fig_module.tex`.

## 11.3 `assign`: combinational logic, permanently

`assign y = expr;` is a permanent connection: whenever `expr`'s inputs move, `y` follows, in zero simulated time. It is the right home for single expressions — muxes as ternaries, logic as boolean expressions, buses split and joined with `{}` concatenation and `[hi:lo]` slicing.

## 11.4 The two `always` recipes

The `always` block runs procedural code when its sensitivity list fires. Two recipes cover nearly all legal hardware:

```
// COMBINATIONAL -- level
always @(*) begin
  y = a & b;
  if (sel) y = c; else y = d;
end
```

blocking =, full sensitivity, every branch assigns

```
// SEQUENTIAL -- edge
always @(posedge clk) begin
  if (rst) q <= 0;
  else     q <= d;
end
```

nonblocking <=, edge list only, regs update together

**Figure 11.3:** The two recipes. Notice the three differences that matter: the sensitivity list (everything vs. one edge), the assignment (= vs. <=), and the duty (assign on every branch vs. registers that keep their value). Regenerate: `figures/ch11/fig_always.tex`.

**Combinational** — `always @(*)`: full sensitivity, blocking =, and *every path must assign every variable*. Miss a branch and the simulator keeps the old value — you have described a latch (the wrong-way demo shows it caught red-handed).

**Sequential** — `always @(posedge clk)`: edge sensitivity only, non-blocking <=, registers keep their value when unassigned. This is the flip-flop recipe; ch10 built its machines on it.

**Exercise 11.1** (easy)**.** Which recipe is `always @(a or b) y = a & b;` and what is wrong with it?

## 11.5 Blocking = vs non-blocking <=

One rule to memorise, then the why.

**THE ONE RULE**

Inside `always @(posedge clk)` use only <=. Inside `always @(*)` use only =. Testbenches may use = anywhere (they are software, on purpose).

The why: = executes in order, immediately; <= queues the update for the end of the timestep, so every <= sees the *old* values. That simultaneity is exactly what real flip-flops do — at the edge, every D input is sampled at once. Write the pipeline with = and the second flop sees the *new* value of the first: two flops collapse into one.

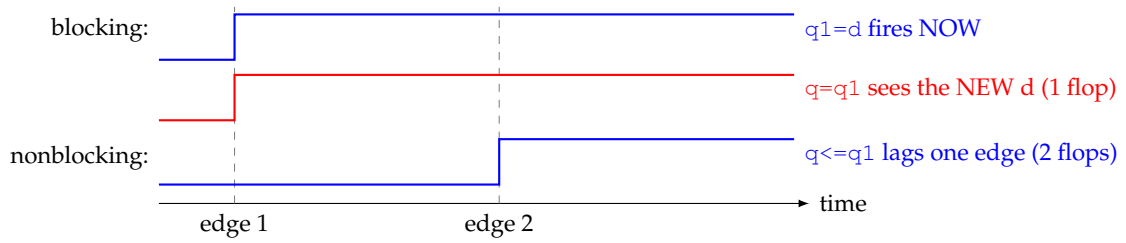



**Figure 11.4:** Blocking vs non-blocking on a two-flop pipeline. Notice the blocking `q=q1` sees the new `d` in the same edge — the delay element vanished. Regenerate: `figures/ch11/fig_blocking.tex`.

## 11.6 Parameters and a generate peek

`parameter` makes a module re-usable: width, encodings, divisors. `generate` loops stamp out repeated hardware (Chapter 13 uses it to tile a datapath). Both are elaboration-time — the parameters are fixed before the circuit exists, not at run time.

## 11.7 The testbench is a module too

A testbench is Verilog used as software on purpose: it instantiates the *design under test* (DUT), drives `reg` stimulus, listens on wires, prints, and calls `$finish`. It never synthesizes — which frees it to do things hardware can't: `$monitor`, `$display`, `$dumpvars`, file I/O, and `tasks` like the self-check below.

## 11.8 Icarus + GTKWave: the loop

The loop is three commands: `iverilog` compiles to a sim model, `vvp` runs it, `$dumpvars` records a VCD the wave viewer reads. On this book's box, Icarus is version 13.0 (stable) — one version note that bites: declare wires before use, and keep functions out of `$monitor` (a `$display` inside a clocked block does the job).

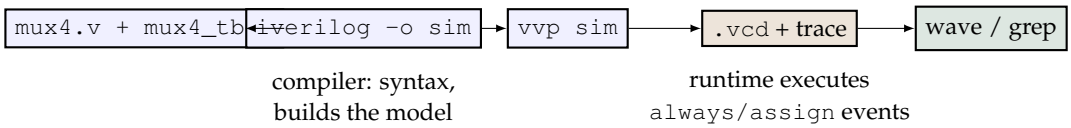



**Figure 11.5:** The edit → compile → run → wave pipeline. Notice where errors can live: `iverilog` catches syntax, `vvp` exposes behavior — a compiled design can still be wrong. Regenerate: `figures/ch11/fig_flow.tex`.

```
SIM = iverilog
all: mux counter traffic
mux: ; iverilog -o /tmp/mux4_sim ch11_mux4.v ch11_mux4_tb.v && vvp /
    tmp/mux4_sim
counter: ; iverilog -o /tmp/cnt2_sim ch11_counter2.v ch11_counter2_tb.
    v && vvp /tmp/cnt2_sim
wave: ; iverilog -o /tmp/tr2_sim ch11_traffic2.v ch11_traffic2_tb.v &&
     vvp /tmp/tr2_sim
```

```
traffic: wave
```

**Listing 11.1:** `code/ch11_Makefile -- the sim targets used throughout this book.`

## 11.9 Reading a waveform

A waveform pane shows each signal's value versus time. Buses render as hex bands; single bits as high/low lines; `x` (unknown) and `z` (unconnected) get their own colors — the first thing to scan for, because an `x` early is a missing reset or an undriven input. We don't ship GTKWave screenshots: Figure 11.6 is drawn from the VCD the testbench actually produced.

## 11.10 Worked example 1: one mux, four legal ways and two wrong

**Strategy.** The 2:1 mux is the smallest complete module. Write it with `assign` (ternary), `assign` (gates), `always` (`if`), `always` (`case`) — then commit the two classic sins and let the sim expose them.

**Solution.**

```
module mux_if(sel, a, b, y);
  input sel, a, b; output reg y;
  always @(*) // way 3: procedural if
    if (sel) y = b; else y = a;
endmodule
module mux_latch(sel, a, b, y); // WRONG: no else -> inferred latch
  input sel, a, b; output reg y;
  always @(*)
    if (sel == 1'b0) y = a;
endmodule
module mux_stale(sel, a, b, y); // WRONG: sel missing from sensitivity
  input sel, a, b; output reg y;
  always @(a or b)
    y = sel ? b : a;
endmodule
```

**Listing 11.2:** `code/ch11_mux4.v -- four legal muxes plus the two wrong ones (abridged).`

```
t= 0 sel=0 a=0 b=1 | y1..4=0000 latch=0 stale=0
t=10 sel=1 a=0 b=1 | y1..4=1111 latch=0 stale=0
t=20 sel=1 a=1 b=0 | y1..4=0000 latch=0 stale=0
t=30 sel=0 a=1 b=0 | y1..4=1111 latch=1 stale=0
```

**Listing 11.3:** Real run of `ch11_mux4_tb.v`. Notice t=10: the legal muxes go to b=1, the latch and the stale copy stay at 0 -- the two bugs caught in one row each.

At t=10 the latch holds its old value instead of following `sel`; at t=30 the stale mux missed the `sel` edge entirely. Both are "legal Verilog" — the language can't refuse them because they describe real (unwanted) hardware.

## 11.11 Worked example 2: a counter with a self-checking testbench

**Strategy.** A tb that only prints waveforms makes you the check — slow and error-prone. Write the check *into* the tb: a `task` comparing reality to expectation, errors counted, a final PASS/FAIL verdict.

**Solution.**

```
task check(input [3:0] want, input [8*24-1:0] what);
  if (q !== want) begin
    errors = errors + 1;
    $display("FAIL t=%0d %0s: q=%0d expected %0d", $time, what, q, want
        );
  end
endtask
initial begin
  #12 rst=0; en=1;
  repeat (4) @(posedge clk); #1 check(4, "counted to 4");
  en=0; @(posedge clk); #1 check(4, "hold when en=0");
  rst=1; @(posedge clk); #1 check(0, "rst clears");
  if (errors == 0) $display("PASS: all %0d checks", 3);
  else $display("FAIL: %0d errors", errors);
  $finish;
end
```

**Listing 11.4:** `code/ch11_counter2_tb.v -- the check task and the three assertions.`

```
PASS: all 3 checks
```

**Listing 11.5:** Real output — and proof the checker bites: the second run compiles the same tb with `-DSABOTAGE`, planting one wrong expectation on purpose.

```
FAIL t=56 planted bad check: q=4 expected 9
FAIL: 1 errors
```

**Listing 11.6:** The sabotaged run — `FAIL` lines name the check, the value, and the expectation.

The `#1` after each edge matters: it waits one timestep so the non-blocking updates settle before `check` reads `q` — sample at the same instant and you read the old register value.

## 11.12 Worked example 3: the traffic FSM, re-coded in this chapter's style

**Strategy.** Take ch10's traffic machine and rewrite it with `parameter` encodings and named wires, dump a real VCD, then read the waveform one transition at a time.

**Solution.**

```
parameter RED=2'b00, YEL=2'b01, GRN=2'b10;
parameter NSG=0, NSY=1, EWG=2, EWY=3;
reg [1:0] st, nx;
always @(*) case (st) // next-state logic
```

```
  NSG: nx = NSY; NSY: nx = EWG;
  EWG: nx = EWY; default: nx = NSG;
endcase
always @(posedge clk) // state register
  if (rst) st <= NSG; else st <= nx;
always @(*) case (st) // output logic (Moore)
  NSG: begin ns=GRN; ew=RED; end
  // ... same shape for NSY, EWG, EWY
endcase
```

**Listing 11.7:** `code/ch11_traffic2.v -- parameters carry the encodings; the three blocks stay separate.`

```
t= 5 rst=1 | st=0 nx=1 | ns=10 ew=00
t=15 rst=0 | st=1 nx=2 | ns=01 ew=00
t=25 rst=0 | st=2 nx=3 | ns=00 ew=10
t=35 rst=0 | st=3 nx=0 | ns=00 ew=01
t=45 rst=0 | st=0 nx=1 | ns=10 ew=00
```

**Listing 11.8:** The real trace. Read it like the waveform: each `st` edge also updates `nx` combinationally, so the "next" column always shows where the machine will land.

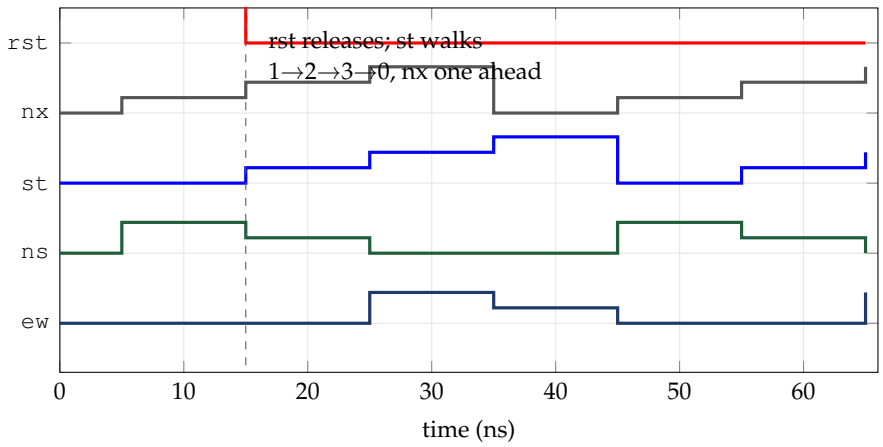



**Figure 11.6:** The same run as a waveform pane (drawn from the VCD rows). Notice `nx` always sits one state ahead of `st` — the register/combinational split made visible. Regenerate: `figures/ch11/fig_wave.tex + code/ch11_traffic2.vcd`.

Line by line: t=5 the first edge loads `st=NSG`, outputs `ns=GRN ew=RED` appear combinationally; t=15 reset releases and the edge steps `st` to `NSY`; every 10 ns another edge walks the cycle.

## 11.13 Worked example 4: the pipeline that ate a flop

**Strategy.** Prove the =/<= rule is real silicon, not style: simulate a two-flop pipeline written both ways and count effective delay elements.

**Solution.**

```
t= 5 clk=1 d=0 | q_nb=x q_block=0
t=15 clk=1 d=1 | q_nb=0 q_block=1
t=25 clk=1 d=1 | q_nb=1 q_block=1
t=45 clk=1 d=0 | q_nb=1 q_block=0
```

**Listing 11.9:** Real run of `ch11_pipe_tb.v`. Notice t=15: `q_block` is already 1 while the nonblocking `q_nb` still holds 0 -- blocking leaked `d` through two "flops" in one edge. The nonblocking output arrives at t=25, a full edge later, as real hardware would.

With =, `q1=d` completes *before* `q=q1` runs, so `q` receives the brand-new `d` — the simulation shows a one-flop pipeline. Synthesis tools may or may not reorder this; never rely on it. With <= both assignments read the old values: two edges, two flops.

## 11.14 Worked example 5: one name, N flip-flops

**Strategy.** Show `parameter` + `generate` paying rent: instantiate the same `regN` at width 4 and width 8, feed them a shared bus, and watch each keep its own slice.

**Solution.**

```
module regN #(parameter N = 4) (clk, d, q);
  input clk; input [N-1:0] d; output reg [N-1:0] q;
  genvar i;
  generate
    for (i = 0; i < N; i = i + 1) begin : ff
      always @(posedge clk) q[i] <= d[i];
    end
  endgenerate
endmodule
```

**Listing 11.10:** `code/ch11_gen.v -- the generate loop stamps out N posedge always blocks.`

```
t=12 d=10110100 | q4=0100 q8=10110100
t=24 d=01101001 | q4=1001 q8=01101001
```

**Listing 11.11:** Real run — `q4` tracks `d[3:0]`, `q8` the full byte, each through its own generated flops.

The loop body is not a runtime loop — it is a cookie cutter pressed *N* times before simulation even starts.

## 11.15 Summary

| Construct | Use it for / the rule to remember |
|---|---|
| `wire` | connections; driven by `assign`/ports |
| `reg` | assigned in `always`; a flop only on an edge |
| `assign` | one permanent combinational expression |
| `always @(*)` | combinational; =; every branch assigns |
| `always @(posedge clk)` | sequential; <=; regs hold otherwise |
| `parameter` | elaboration-time constants (width, encodings) |
| `generate` | stamps repeated hardware before sim/synth |
| testbench | a module that is software: monitor, check, dump, finish |
| `x/z` | unknown / undriven — read them first on any waveform |

## 11.16 Common mistakes

**WRITING SOFTWARE-STYLE SEQUENTIAL VERILOG**

Loops and "next lines" belong to programs. If your module has a `for` outside `generate` or an algorithm's worth of variables, you are writing code for a machine that doesn't exist — draw the hardware first.

**= WHERE <= BELONGS**

Blocking inside a posedge block makes reads see new values — pipelines collapse, and synthesis may not save you (§11.13 has the proof on tape). The rule takes one line: edge blocks get <=.

**DESCRIBING HARDWARE YOU CANNOT DRAW**

If a line has no corresponding gate or flop in your sketch, it is not hardware — it is a wish. The missing `else` that infers a latch (§11.10) is the canonical example.

**HALF A SENSITIVITY LIST**

`always @(a or b)` with `sel` inside the expression ignores `sel` edges — the wrong-way demo caught it holding a stale output. Use `@(*)` and let the tool list the signals.

**A TESTBENCH THAT ONLY WATCHES**

A tb without checks makes you the diff. One `task` and an error counter turns every run into a verdict (§11.11); `-DSABOTAGE` proves the checker bites.

**Exercise 11.2** (medium)**.** Predict before running: for `mux_stale`, what does `y` show if `sel` toggles twice with `a`, `b` fixed? Sketch the expected line, then check against §11.10's table.

**Exercise 11.3** (medium)**.** The `check` task reads `q` one `#1` after the edge. What goes wrong if the `#1` is removed, and which assignment rule makes it go wrong?

**Exercise 11.4** (hard)**.** Write a testbench for ch9's `adder4`: drive all 256 operand pairs, compare against `a+b` computed in the tb, print only the verdict.

## Additional exercises

**Exercise 11.5** (easy)**.** Wire or reg? Classify: a port driven by a submodule; a counter's `q`; a tb's clock; a mux's select line.

**Exercise 11.6** (easy)**.** Write the two `always` recipe headers from memory, then say which two choices each recipe fixes.

**Exercise 11.7** (easy)**.** What do `x` and `z` mean on a waveform, and which usually points at a missing reset?

**Exercise 11.8** (easy)**.** Order the pipeline commands: `vvp`, `$dumpvars`, `iverilog`, `$finish`.

**Exercise 11.9** (medium)**.** Rewrite `mux_case` as a 4:1 mux with `sel[1:0]`; name the one change that makes it latch-free.

**Exercise 11.10** (medium)**.** Fix this testbench: it declares `wire clk;` and tries `always #5 clk = ~clk;`. What does the compiler say, and what's the minimal change?

**Exercise 11.11** (medium)**.** Predict the waveform: `always @(posedge clk) begin q <= d; q2 <= q; end` — sketch `d`, `q`, `q2` for four edges, then verify in sim.

**Exercise 11.12** (medium)**.** The sabotage run used `-DSABOTAGE`. Write the tb line that would count checks automatically (so `PASS: all 3` needs no hardcoded 3).

**Exercise 11.13** (medium)**.** A teammate's `@(*)` block assigns `y` only when `en` is 1. Two sentences: what hardware appears, and how do you confirm it in sim?

**Exercise 11.14** (medium)**.** Give the `parameter` list and instantiation lines to reuse `counter2` at width 8 without editing its source.

**Exercise 11.15** (medium)**.** In §11.13's trace, at which time does `q_block` first diverge from `q_nb`, and which assignment order produced it? Explain both columns.

**Exercise 11.16** (hard)**.** Extend `traffic2` with a pedestrian input: `walk` forces the next state to `NSY` regardless. Update all three `always` blocks and add a tb check that proves the detour.

**Exercise 11.17** (hard)**.** Self-checking adder tb (the inline exercise) plus one more idea: seed the stimulus from `$random`, run 1000 pairs, report the verdict. What did randomness buy you?

**Exercise 11.18** (hard)**.** Implement `shiftN #(parameter N)`: an $N$-stage shift register written *without* generate (one `always` block, slicing allowed) and *with* generate; argue which is clearer.

**Exercise 11.19** (hard)**.** Draw, then describe: a 2-bit saturating counter (counts up to 3 and sticks, down to 0 and sticks). Write module + tb, self-checking, in this chapter's style. HDLBits's language track (`hdlbits.01xz.net`) drills exactly these patterns.

## Hints and answers

**Recipe (easy, inline)** Combinational shape but a half sensitivity list — `sel` missing means stale output; use `@(*)`.

**Classify (easy)** Submodule-driven port: wire; counter q: reg; tb clock: reg; select line: wire.

**Recipes (easy)** `always @(*)` fixes = + full sensitivity + every-branch-assigns; `@(posedge clk)` fixes <= + edge list + hold-value.

**x/z (easy)** Unknown and undriven; `x` usually points at the missing reset.

**Pipeline order (easy)** `iverilog` → `vvp` (with `$dumpvars` inside the tb) → wave; `$finish` ends the run.

**4:1 mux (medium)** `case (sel)` over 4 rows + `default` — completeness kills the latch.

**Fix the tb (medium)** `wire clk` can't take procedural assigns — the compiler complains about the `always` drive; declare `reg clk`.

**Wave predict (medium)** `q` follows `d` one edge late, `q2` two edges — a clean 2-deep shift.

**Auto-count (medium)** An `integer n` incremented in `check`; print `n` in the verdict.

**Latch confirm (medium)** An inferred latch; drive `en` low after a known value and watch `y` hold it.

**Width-8 reuse (medium)** `counter2 #(8) u(...)` — the `#()` syntax overrides `W` at instantiation.

**Pipe divergence (medium)** t=15: `q1=d` completed before `q=q1`, so the new `d` leaked through in one edge.

**Walk detour (hard)** Next-state: `walk` overrides to `NSY`; a tb check waits one edge and asserts `st==NSY`.

**Random pairs (hard)** Coverage without a hand-picked list — corner cases you didn't think to write.

**shiftN (hard)** Slicing version is one line `q <= {q[N-2:0], d}`; generate is clearer when stages differ.

**Saturating counter (hard)** Two extra boundary conditions in the next-state logic; the tb checks both rails.

## Further reading

- *ChipVerify Verilog Tutorial*, `www.chipverify.com/verilog/verilog-tutorial` — the topic order this chapter follows (modules → assign → always → tb); reference depth per construct, free. Verified 2026-10-11.
- S. Williams, *Icarus Verilog* documentation — `https://steveicarus.github.io/iverilog/` — the compiler's own docs incl. the v13 quirks we hit ($monitor limits, declaration rules). Verified 2026-10-11.
- *HDLBits* — `https://hdlbits.01xz.net` — the language track is this chapter as drills: small Verilog problems with instant feedback; do Circuits/Getting Started next. Verified 2026-10-11.

- Harris & Harris, *Digital Design and Computer Architecture*, 2nd ed., ch. 4 — the textbook's Verilog chapter, same "two recipes" discipline; companion materials at `https://pages.hmc.edu/harris/ddca/` [12]. Verified 2026-10-11.
- *fpga4fun.com* — short real projects once the syntax settles; their "how FPGA works" pages bridge this chapter to the hardware [13]. Verified 2026-10-11.
- Nisan & Schocken, *nand2tetris.org/course*, project 3 — write the same registers in a second HDL; the concepts transfer line-for-line. Verified 2026-10-11.

**WATCH INSTEAD**

1. *Icarus Verilog and GTKWave Tutorial* — Lady Engineer, 10:59, `https://www.youtube.com/watch?v=clLL8smqwL0` — install to first waveform; matches §11.8–§11.9. Accessed 2026-10-11.
2. *Get Started With FPGAs and Verilog in 13 Minutes!* — 13:26, `https://www.youtube.com/watch?v=aWm9pJ2KobA` — Verilog on the Tang Nano 9K board this book uses; matches §11.1–§11.4. Accessed 2026-10-11.
3. *Introduction to Verilog HDL using Free Software Icarus, GTKWave, and VS Code* — boyfriendnibluefairy, 42:03, `https://www.youtube.com/watch?v=vN1wzM0NO4c` — the long-form version covering this whole chapter; watch when you want the lecture. Accessed 2026-10-11.

## Notes

## Notes

The topic order follows the ChipVerify Verilog tutorial; style rules (two `always` recipes, =/<= split) follow Harris & Harris ch. 4 [12]; practice sets come from HDLBits [14]; the sim toolchain is Icarus Verilog 13.0. All outputs are verbatim runs on this box: the mux wrong-way table, the PASS/FAIL checks, the traffic trace, the pipeline-collapse demo, and the generate run. The waveform figure is regenerated from the actual VCD/monitor rows (drawn, not a screenshot).

CHAPTER 12

# Verilog II: testbenches and the classic bugs

Chapter 11 taught the language. This chapter teaches the discipline around it, because most of your Verilog hours will not be spent writing modules — they will be spent finding out *why the module you already wrote is lying to you*. A testbench is where design meets its mirror: a throwaway module whose only job is to drive the real one and grade its answers. We open the bench like a machine — clock, stimulus, check, report — then walk the five classic bugs one by one, each caught on tape by a real run. By the end you can write a bench that prints a verdict, and read any of the classic failures straight off the monitor rows.

**BEFORE YOU START**

Chapter 11. *Recap:* a testbench is a Verilog module that drives inputs and checks outputs. Simulation passing is not hardware working — the bugs here are the ones that pass anyway.

**AFTER THIS CHAPTER YOU CAN**

- write a self-checking testbench with a golden model;
- recognise unknown (`x`) values and latch-inferring code;
- find a race between simulation events;
- report failures with the right severity.

**HOW TO THINK ABOUT THIS**

Passing simulation does not mean the design is right, so build tests that check themselves. Compare against a golden model you trust, watch for unknown values, and treat every warning as a possible bug. The bugs here are classic, and seeing them once saves hours later.

## 12.1 The anatomy of a clocked testbench

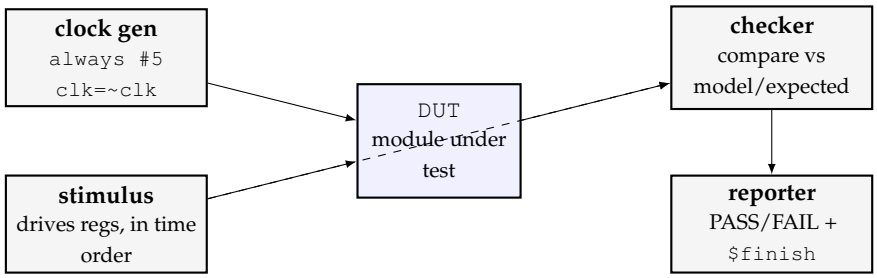
A clocked testbench has four organs (Figure 12.1). Everything inside is *behavioural* — full software rules apply, because the bench is never synthesized:

**Clock generator.** A free-running `always #5 clk=~clk;` produces the heartbeat every sequential design needs. The bench owns time; the DUT merely reacts to it.

**Stimulus.** An `initial` block pokes inputs with `#` delays. In a program this is a script; in the bench it is the entire reason to exist. Drive inputs on the *quiet* part of the cycle (§12.7 shows why).

**Check.** Something compares what came out against what should have: a `$monitor` you read, a `task` that counts errors, or a golden model running alongside (§12.10).

**Report.** Two counters — `n` checks, `errors` — and one verdict line, then `$finish`. A run that ends in silence is a run nobody checked.




the tb owns all four; the DUT thinks it's in real hardware

**Figure 12.1:** The four organs of a clocked bench. Notice the ownership: everything on the left drives the DUT; everything on the right grades it. The dashed line is the lazy path — reading stimulus into checks works, but a golden model (§12.10) does it honestly.

**THE BENCH OWNS TIME**

Everything the DUT sees — edges, input changes, the end of the world — is scheduled by the bench. If a transition happens without your say-so, something in the bench is driving it; find it. There are no spontaneous events in simulation.

**Exercise 12.1** (easy)**.** Label the four organs in `code/ch12_xcounter_tb.v`. Which line is the clock, which is stimulus, which is check, which is report?

## 12.2 $display, $finish and the severity family

Three printing verbs cover most runs. `$display(fmt,...)` prints once, when its line executes — put it inside a clocked block to sample settled values. `$monitor(fmt,...)` prints *every time a printed signal changes* — one line arms it for the whole run; it is the text-mode waveform. `$finish` ends the world; every bench must reach it or the sim spins forever.

SystemVerilog (and Icarus 13) also know the severity tasks `$info`, `$warning`, `$error`, `$fatal(n, msg)` — print at escalating alarm levels; `$fatal` kills the run with an exit

code. The bigger `$assert` machinery is SystemVerilog territory we reach for later; today a five-line `task` does the same job honestly.

## 12.3 Worked example 1: a bench that reports, not just prints

**Strategy.** Wrap the verdict in a `check` task: one counter for checks, one for errors, a message on every failure, a final line that states the score. Then plant a check we know fails and watch the machinery bite.

**Solution.**

```
`timescale 1ns/1ps
module report_tb;
  integer n = 0, errors = 0, warns = 0;
  task check(input cond, input [8*24-1:0] what);
    if (cond) n = n + 1;
    else begin
      errors = errors + 1;
      $display("%0t ERROR %0s failed", $time, what);
    end
  endtask
  initial begin
    $display("--- tb start ---");
    check(4 == 4, "add table row");
    check(15 == 15, "terminal count");
    check(0 === 1'bx, "planted x compare");
    if (errors > 4) $fatal(1, "too many errors, aborting");
    $display("--- tb done: %0d checks, %0d errors ---", n, errors);
    $finish;
  end
endmodule
```

**Listing 12.1:** `code/ch12_report_tb.v -- the check/report pattern in 18 lines. Notice the string argument width [8*24-1:0]: Icarus truncates narrower task strings.`

```
--- tb start ---
0 ERROR planted x compare failed
--- tb done: 2 checks, 1 errors ---
```

**Listing 12.2:** Real run. Notice the planted line: one ERROR, named, with the sim time; the verdict prints the honest score 2/3.

The planted failure is a lesson in itself: `===` is the *case equality* operator — it compares `x` and `z` literally. `0 === 1'bx` is false, so it fails; `0 == 1'bx` would *also* be false but for the wrong reason (`==` against `x` yields `x`, which an `if` reads as false). When you mean "must equal this exact pattern", write `===`; when you mean "must differ, even against unknowns", write `!==`.

## 12.4 The classic bugs

Five bugs account for nearly all first-month Verilog pain (Figure 12.2). Two you met in ch11 — the inferred latch and blocking-in-sequential — return here as named suspects; three are new: un-reset `x` forever, the missing `default`, and the bench fighting the DUT. Every one is demonstrated below with a run you can repeat.

| symptom | root cause | fix |
|---|---|---|
| `q` stuck at `x` | no reset on regs | reset every flip-flop |
| output holds old value | missing `else/default` | assign on every branch |
| tb reads wrong/old d | stimulus at posedge | drive on `negedge` or `#1` |
| pipeline too short | = inside posedge | <= in edge blocks |
| sim never ends | forgot `$finish` | always end the tb yourself |

**Figure 12.2:** The classic-bug catalogue. Notice that every symptom is silent — none of these produce an error message; the bench or your eyes catch them, which is why this chapter exists.

## 12.5 Worked example 2: the x that never leaves

**Strategy.** Registers start life unknown. Simulate the same counter with and without a reset; watch one count and one rot.

**Solution.**

```
module cnt_norst(clk, q);
  input clk; output reg [3:0] q;
  always @(posedge clk) q <= q + 1'b1; // q starts x -> stays x
endmodule
module cnt_rst(clk, rst, q);
  input clk, rst; output reg [3:0] q;
  always @(posedge clk)
    if (rst) q <= 4'd0; else q <= q + 1'b1;
endmodule
```

**Listing 12.3:** `code/ch12_xcounter.v -- the only difference is rst.`

```
t= 0 | no-rst q=xxxx | with-rst q=xxxx
t= 5 | no-rst q=xxxx | with-rst q=0000
t=25 | no-rst q=xxxx | with-rst q=0001
t=35 | no-rst q=xxxx | with-rst q=0010
t=45 | no-rst q=xxxx | with-rst q=0011
t=55 | no-rst q=xxxx | with-rst q=0100
```

**Listing 12.4:** Real run of `ch12_xcounter_tb.v`. Notice: `x + 1 = x forever -- arithmetic on unknowns produces unknowns -- while the reset twin wakes at t=5 and counts.`

This is *x-propagation*: unknown in, unknown out. The sim is telling you the truth — real silicon powers up in *a* state, you just do not know which, and `x` is the honest label for "could be anything". Two habits follow. First, *every sequential register gets a reset path* (or a documented reason it needs none). Second, when a waveform shows `x`, read it as a question, not a value: "who was supposed to initialize this?"

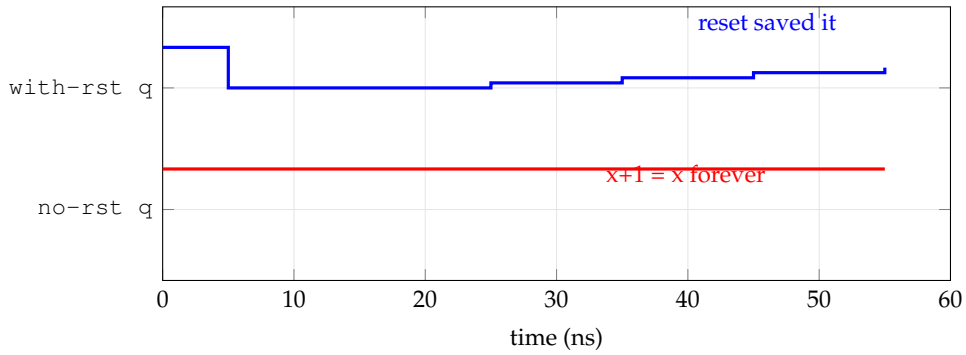



**Figure 12.3:** Real monitor rows from `ch12_xcounter.out`, drawn as a wave pane. Notice the dead lane: `no-rst q` never leaves `x` (mapped mid-band here); the reset twin counts normally.

**Exercise 12.2** (medium). In `ch12_xcounter_tb.v`, drop the reset release (`#25 rst=0;`) and predict both columns before you re-run. Then re-run.

## 12.6 Worked example 3: the case that forgot a branch

**Strategy.** The inferred latch from ch11 had an `if` missing its `else`. The same bug hides inside `case` when an input combination is simply absent — no warning, clean synthesis, wrong hardware. Give a 4-way ALU an `op=11` gap and watch it hold the previous answer.

**Solution.**

```
module alu4(op, a, b, y); // WRONG: 2'b11 missing
  input [1:0] op; input [3:0] a, b; output reg [3:0] y;
  always @(*) case (op)
    2'b00: y = a + b;
    2'b01: y = a - b;
    2'b10: y = a & b;
    // 2'b11 silently missing -> y holds old value
  endcase
endmodule
module alu4safe(op, a, b, y);
  input [1:0] op; input [3:0] a, b; output reg [3:0] y;
  always @(*) case (op)
    2'b00: y = a + b;
    2'b01: y = a - b;
    2'b10: y = a & b;
    default: y = 4'b0000; // explicit, no latch
  endcase
endmodule
```

**Listing 12.5:** `code/ch12_caselatch.v -- identical except the default.`

```
t= 0 op=00 a= 7 b= 2 | latched= 9 safe= 9
t=10 op=01 a= 7 b= 2 | latched= 5 safe= 5
t=20 op=11 a= 7 b= 2 | latched= 5 safe= 0
t=30 op=00 a= 7 b= 2 | latched= 9 safe= 9
```

**Listing 12.6:** Real run. Notice `op=11` at t=20: the latched ALU answers 5 — a *memory* of the subtraction — while the safe one answers 0. The latch is not a bug you see; it is a bug that sees you.

The fix is boring and absolute: in a combinational block, *every output is assigned on every path*. Two styles do it — a `default` arm, or an unconditional assignment before the `case` (`y = 0; case...`). Both kill the latch; the second also documents the "don't care" value.

## 12.7 The race you cannot see

The strangest classic bug changes nothing in your hardware and still breaks the sim. Simulation time is quantised: at a `posedge`, the DUT samples `d` *and* the bench may also be changing `d` — in the same instant. Who wins is a coin flip inside the simulator's event queue. Your design is fine; your bench is unreliable.

## 12.8 Worked example 4: the same flip-flop, two benches

**Strategy.** One DFF, two benches. The bad one toggles `d` exactly *at* `posedge clk`; the good one does it at `negedge` — the quiet midpoint where nothing samples.

**Solution.**

```
// BAD: stimulus changes in the same instant the clock edges.
module race_bad_tb;
  reg clk=0, d=0; wire q;
  dff2 u(.clk(clk), .d(d), .q(q));
  always #5 clk=~clk;
  always @(posedge clk) d = ~d; // races the DFF's sample!
  initial $monitor("t=%2d clk=%b d=%b q=%b", $time, clk, d, q);
  initial #40 $finish;
endmodule
// GOOD: drive on the opposite edge (or #1 after) - no ambiguity.
module race_good_tb;
  ...
  always @(negedge clk) d = ~d; // settled half a cycle early
  ...
endmodule
```

**Listing 12.7:** `code/ch12_race_tb.v (abridged) -- one line differs.`

```
t= 5 clk=1 d=1 q=1
t=15 clk=1 d=0 q=1
t=25 clk=1 d=1 q=1
t=35 clk=1 d=0 q=1
```

**Listing 12.8:** Real run, `race_bad_tb`. Notice: `d` toggles cleanly but `q` pins at 1 — the DUT keeps sampling the *new* `d`, a stable wrong answer with no error message anywhere.

```
t= 5 clk=1 d=0 q=0
t=15 clk=1 d=1 q=1
```

```
t=25 clk=1 d=0 q=0
t=35 clk=1 d=1 q=1
```

**Listing 12.9:** Real run, `race_good_tb`. Same DUT, stimulus moved to `negedge`: `q` now follows `d` one edge behind, as a flip-flop must.

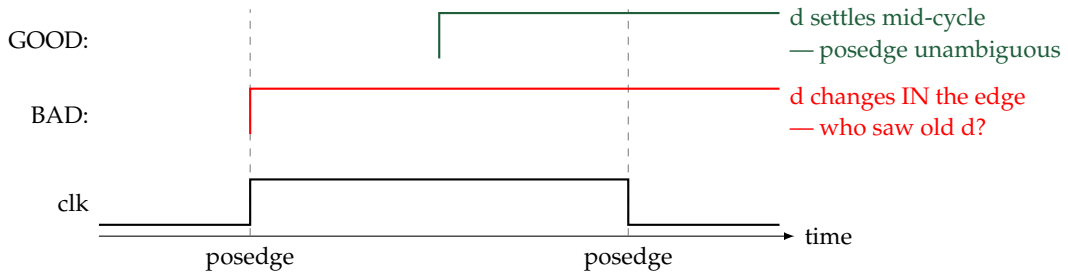



**Figure 12.4:** Why the race exists. Notice that the *same instant* carries two writers: the bench changes `d` exactly when the DUT samples it. Moving the write to mid-cycle leaves a clean window on either side of the edge.

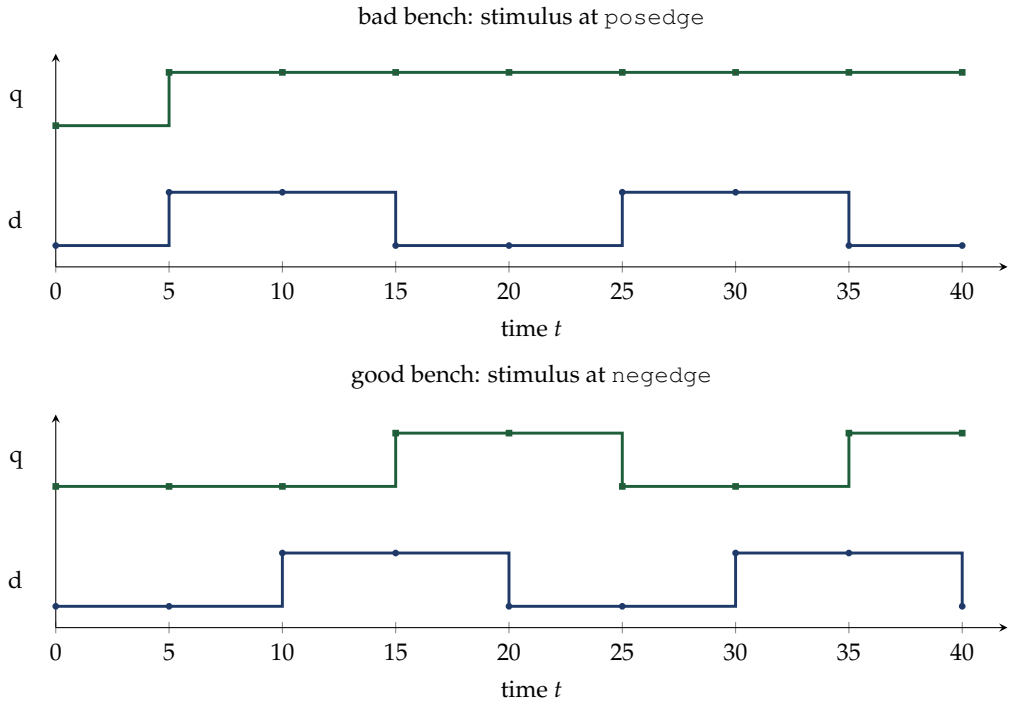



**Figure 12.5:** Same flip-flop, two benches (Listing 12.7). Top: stimulus driven *at* `posedge` — *d* still toggles but *q* is stuck at 1, because the bench and the DUT fight over who changes first. Bottom: stimulus moved to `negedge` — *q* now samples the settled *d* and toggles correctly, one edge behind. Rows are the real `$monitor` output, parsed from `code/ch12_race_*.out`.

The discipline that deletes this bug forever: *change stimulus on the clock edge the DUT does not listen to* — or equivalently, drive `d` with `#1` after the active edge so it lands in settled territory. Hardware has setup/hold windows; the sim punishes you for pretending they are zero.

**Exercise 12.3** (medium). Predict: in `race_good_tb`, at t=25 `q` is sampled from which value of `d`, set when? Confirm against `code/ch12_race_good.out`.

## 12.9 Blocking in sequential, one line

Chapter 11's pipeline demo is the proof on tape (§11.13): = inside a `posedge` block lets a later line read the *new* value, collapsing a two-flop pipeline into one. It earns its row in the classic-bugs table and one line here: **edge blocks get <=, always**. The bug is silent — no x, no latch report — which is exactly why it is in the top five.

## 12.10 Self-checking patterns

Benches grade on a sliding scale of effort versus catch-rate:

**Eyeball.** `$monitor` rows or a wave pane you read yourself. Fine for bring-up; useless the day you change one line and do not re-read the wave.

**Directed checks.** Expected values in the bench, one `check` per row — ch11's counter. Catches exactly what you thought to test.

**Golden model.** A second, trusted implementation — usually simpler, often not synthesizable — running next to the DUT, compared on every event. Catches what you did *not* think to test, which is where bugs live.

## 12.11 Worked example 5: the golden model catches a 1-bit bug

**Strategy.** Plant a bug small enough to eyeball past: a shift register whose update carries a stray `^`. A golden copy — one line of spec-level Verilog — runs in the same bench and compares every edge.

**Solution.**

```
module shift8(clk, en, sin, q);
  input clk, en, sin; output reg [7:0] q;
  always @(posedge clk)
    if (en) q <= {q[6:0], sin} ^ 8'b1; // subtle bug: extra XOR bit0
endmodule
```

**Listing 12.10:** `code/ch12_shift.v -- spot the bug; the model does not care that you can't.`

```
module golden_tb;
  reg clk=0, en=0, sin=0;
  wire [7:0] q;
  shift8 u(.clk(clk), .en(en), .sin(sin), .q(q));
  reg [7:0] model = 0; // the golden copy
  integer errors = 0;
  always #5 clk=~clk;
  always @(posedge clk) #1 begin // run the model alongside
    if (en) model = {model[6:0], sin}; // what shift SHOULD do
    if (q !== model) begin
      errors = errors + 1;
      $display("MISMATCH t=%0d q=%b model=%b", $time, q, model);
    end
```

```
14    end
15    ...
```

**Listing 12.11:** `code/ch12_golden_tb.v (abridged) -- the spec runs inside the bench. Notice #1: read the DUT after the edge settles, the negedge-style discipline of §12.7.`

```
1 MISMATCH t=6 q=xxxxxxx0 model=00000001
2 MISMATCH t=16 q=xxxxxx01 model=00000010
3 MISMATCH t=26 q=xxxxx011 model=00000100
4 MISMATCH t=36 q=xxxx0111 model=00001000
5 MISMATCH t=46 q=xxxx0111 model=00001000
6 FAIL: 5 mismatches
```

**Listing 12.12:** Real run. Notice two things: the early rows are un-reset `x` surfacing (§12.5 — `!==` treats them as mismatches, honestly), and t=46 shows the planted bug itself — bit 0 flipped, exactly the XOR's signature.

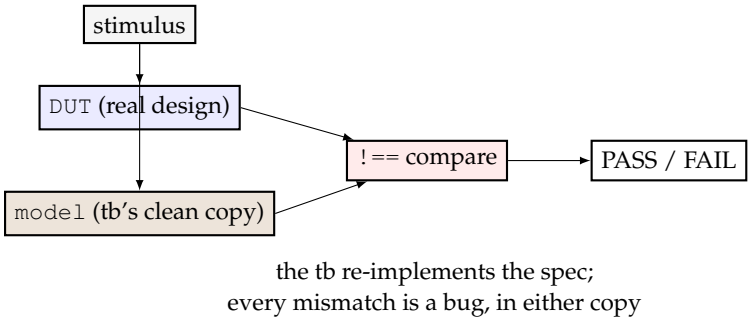



**Figure 12.6:** Golden-model architecture. Notice the compare is `!==`, not `!=` — the point is that `x` counts as a mismatch, so un-reset state fails loudly instead of passing silently.

Two details make the pattern honest. The compare uses `!==` — `!=` against an `x` yields `x` and silently skips the check. And the model is written at the *spec* level — what shift should do, not how the DUT does it — or you have built two copies of the same bug.

**Exercise 12.4** (hard)**.** Write a golden-model bench for ch09's `adder4`: the model is one line (`wire [4:0] sum = a + b;`), checked every `#1` after inputs change across all 256 input pairs.

## 12.12 Reading the wave

Every figure in this chapter was drawn from real simulator rows — `$monitor` output, or the VCD file `$dumpvars` writes. Reading them back is the skill. Four habits:

1. **Sample on the right instant.** At a `posedge`, everything the DUT sees is the value *just before* the edge. A `$display` inside the edge block reads pre-update values; add `#1` (or sample at `negedge`) to read settled ones — §12.11 uses this.
2. **Read x and z first.** `x` asks "who initializes this?"; `z` asks "who drives this?". Both are questions, not values — §12.5.

3. **Chase the first wrong edge, not the last.** A bad `q` at t=46 was born at t=6. Walk left until the first disagreement between bench and DUT.
4. **Trust the bench only after it has lied to you once.** Run `-DSABOTAGE`-style checks: plant a wrong expectation and confirm the machinery reports it (ch11's counter; WE1 here).

**Checkpoint 12.1** You can write a clocked testbench with all four organs, wrap checks in a counting `task`, name the five classic bugs with one symptom and one fix each, drive stimulus on the quiet edge, and read `x`/`z` rows as questions about drivers and resets.

## 12.13 Summary

| Thing | The rule of thumb |
|---|---|
| clocked bench | clock gen + stimulus + check + report; the bench owns time |
| `$display`/`$monitor` | print-once vs print-on-change; use clocked display to sample settled values |
| `$finish`/`$fatal` | every bench must terminate; `$fatal` for stop-the-line errors |
| `===`/`!==` | compare `x` literally; `==` hides unknowns |
| inferred latch | every branch assigns every output, or `default` |
| un-reset `x` | arithmetic on `x` gives `x` forever; reset every register |
| `=` in edge blocks | pipelines collapse; `<=` always |
| tb/DUT race | drive stimulus on the idle edge (or `#1` later) |
| golden model | spec-level twin + `!==` every edge |

## 12.14 Common mistakes

**A COUNTER WITHOUT A RESET**

`x + 1 = x`, forever — the sim's most common silent killer. Give every register a reset path; if one truly needs none, write the reason in a comment (§12.5).

**A CASE THAT TRUSTS THE INPUTS**

"`op` can only be 00–10" is a promise the hardware cannot hear. The missing arm is an inferred latch (§12.6); `default` costs one line.

**STIMULUS AT THE SAMPLING EDGE**

Driving `d` at `posedge` makes bench and DUT race — the output can be stable *and* wrong, with no error message (§12.8). Drive on the idle edge or `#1` later.

**COMPARING WITH != WHEN x MAY LURK**

`q != model` evaluates to `x` against unknowns and an `if` reads that as skip. Self-checkers use `!==`/`===` so `x` counts as a mismatch (§12.11).

**A VERDICT YOU NEVER VERIFIED**

If your checker has never printed FAIL, you do not know it can. Plant a wrong expectation once (the `-DSABOTAGE` habit) and watch it fire.

**READING THE WAVE AFTER THE EDGE**

A `$display` inside `@(posedge clk)` sees pre-edge values — correct-looking, one cycle stale. `#1` after the edge is where settled truth lives (§12.12).

**Exercise 12.5** (easy). `always @(*) case (s) 1'b0: y = a; endcase` — what does `y` do when `s=1`? Name the bug and the one-line fix.

**Exercise 12.6** (easy). Order these for a clean run: `$finish`, verdict `$display`, `$dumpvars`, stimulus `initial`, clock `always`. Which may interleave freely?

**Exercise 12.7** (medium). A tb drives `d <= ~d` on `posedge clk`. Race or fine? Defend in one sentence.

**Exercise 12.8** (medium). `check(q === 4'b1010, "pattern")` fails though the wave shows `q=1010`. What single character probably differs? (Hint: upper/lower bits may be `z`.)

**Exercise 12.9** (medium). `$monitor` prints nothing all run. Name two independent causes you would check first.

**Exercise 12.10** (medium). `model` starts `= 0`, the DUT starts at `x`. Is the t=6 MISMATCH in §12.11 a model bug, a DUT bug, or a bench-design issue? What two-line change removes it?

**Exercise 12.11** (medium). You add `default:;` (empty) to `alu4`. Latch or no latch? Run the experiment and say why.

**Exercise 12.12** (medium). Your FSM bench applies the next input 0 ns after each `posedge`. Which classic bug have you re-created, and what is the fix?

**Exercise 12.13** (medium). A shift register is reset *inside* the same `@(posedge clk)` block as the shift using `=`. Which two rules from the summary table does that break (or nearly), and what does each predict?

**Exercise 12.14** (medium). In `race_good_tb`, why is `q` still `x` at t=0 in the good run too? What one-line addition removes it?

**Exercise 12.15** (hard). Write `task expect(input [7:0] got, input [7:0] want, input [8*24-1:0] what)` with a shared error counter, then re-do §12.11 using it. Print the score line.

**Exercise 12.16** (hard). Implement `cnt_load`: 4-bit counter with synchronous `load` and `en`, self-checking golden-model bench (model predicts load/en order-of-operations on paper first).

**Exercise 12.17** (hard). The `$monitor`-in-initial vs `$display`-in-clock question: rebuild WE1's bench using clocked `$display` sampled `#1` after `posedge`. Diff the two outputs; explain every extra/missing row.

**Exercise 12.18** (hard). Random stimulus: `sin = {$random} % 2;` each cycle in §12.11's bench, 200 cycles. What did randomization catch that the directed script did not — or why not?

**Exercise 12.19** (hard). Argue both sides in two sentences each: `default: y = 0;` versus pre-case unconditional assignment. When is each clearer?

**Exercise 12.20** (hard). A "self-healing" latch claim: "the case gap only bites while `op` holds the missing value; the next legal op fixes `y`, so it never matters." Show the smallest counterexample using WE3's files.

## Hints and answers

**Four organs (easy, inline)** `always #5 clk=~clk` is clock; the `initial` with `#25 rst=0` is stimulus; `$monitor` is check; `$finish` is report.

**Latch predict (medium, inline)** Both columns stay `xxxx`; releasing nothing means `cnt_rst` also counts from `x`.

**Race sampling (medium, inline)** `d` set at the *previous* `negedge` (t=20, `d=0`) — the posedge at t=25 samples the settled 0, so `q`=0.

**Case gap (easy)** `y` holds its old value — inferred latch. Add `default: y = 0;` (or pre-assign).

**Ordering (easy)** `$dumpvars` and clock `always` arm first; stimulus runs; verdict `$display` before `$finish`. Clock/stimulus interleave by `#` delays — that is the design.

**Nonblocking stimulus (medium)** Fine: `<=` in the bench schedules the update *after* the edge's samples, so the DUT sees the old value — the race disappears by construction.

**1010-vs-x (medium)** One bit is `z`/`x` — `===` compares literally; print with `%b` not `%d` to see it.

**Silent monitor (medium)** No signal changed (check drivers / an un-reset `x` that *is* changing may not print if the fmt omits it); or `$monitor` armed after the action.

**t=6 MISMATCH (medium)** Bench-design: the model was initialized, the DUT was not. Two lines: a reset pulse on the DUT, or start `model = 8'bxxxx_xxxx` to match.

**Empty default (medium)** Still a latch — `default:;` assigns nothing on that arm. The rule is every-path-assigns, not every-arm-exists.

**0 ns after posedge (medium)** The same race as §12.8 — apply inputs at `negedge` or `#1` later.

**Reset inside edge block (medium)** The `<=` rule (read may see new value) — and if `rst` were skipped by sensitivity, also a recipe violation. Predicts: off-by-one resets.

**Good-run x (medium)** The DFF itself is un-reset — same disease as WE2; `initial q = 0;` in the bench-visible style or a real `rst` port.

**expect task (hard)** Same shape as WE1's `check`; compare with `!==` so `x` fails honestly.

**cnt_load (hard)** Decide on paper: `if(rst)` then `else if(load)` then `else if(en)` — priority order is the spec; the model mirrors it verbatim.

**Display-vs-monitor (hard)** Monitor prints only on change; clocked display prints every edge — extra rows at each posedge, none between. Values match at equal timestamps.

**Random stimulus (hard)** It hits sequences your script skipped (en toggling mid-burst); the model still catches every deviation because it mirrors the spec, not the stimulus.

**default vs pre-assign (hard)** `default` reads as "the leftover case"; pre-assign reads as "the usual value" — prefer pre-assign when one output dominates, `default` when arms are peers.

**Self-healing latch (hard)** Between the gap-hit and the next legal op, `y` feeds downstream logic a stale value — WE3's trace at t=20 shows the hold window; any consumer sampling inside it latches the lie.

## Further reading

- *ChipVerify: Verilog Testbench*, `chipverify.com/verilog/verilog-testbench` — bench anatomy and system tasks at reference depth; the closest match to this chapter's scope. Verified 2026-10-11.
- Harris & Harris, *Digital Design and Computer Architecture*, 2nd ed., ch. 4.5–4.6 — the testbench and bugs sections of our main text [12]; companion materials at `https://pages.hmc.edu/harris/ddca/`. Verified 2026-10-11.
- S. Williams, *Icarus Verilog* documentation — `https://steveicarus.github.io/iverilog/` — system-task reference (`$monitor` limits, `$fatal`, `$dumpvars`) straight from the compiler author. Verified 2026-10-11.
- *HDLBits* — `https://hdlbits.01xz.net` — the verification drills are this chapter as exercises; its benches are visible, so you can read real self-checkers [14]. Verified 2026-10-11.
- *fpga4fun.com* — "how FPGAs work" and the serial projects; when your benches pass, these give them something real to test [13]. Verified 2026-10-11.
- Nisan & Schocken, *nand2tetris.org/course*, project 3 — write the same registers under a second, stricter toolchain; the "test what you built, not what you meant" discipline transfers directly. Verified 2026-10-11.

**WATCH INSTEAD**

1. *DDCA Ch4 - Part 9: Testbenches* — Sarah Harris, 20:47, `https://www.youtube.com/watch?v=V14aROKd6zU` — the textbook author's own testbench lecture; matches §12.1–§12.4. Accessed 2026-10-11.
2. *17 - Developing Simple Verilog Testbenches* — Anas Salah Eddin, 22:48, `https://www.youtube.com/watch?v=Z-3TsBtufmA` — a worked bench built live, timescales and all; matches §12.1 and §12.12. Accessed 2026-10-11.

## Notes

## Notes

Testbench anatomy and the severity tasks follow Harris & Harris ch. 4 [12] and the Icarus Verilog v13 documentation; the classic-bug list is standard industry folklore, each item verified on tape under Icarus Verilog 13.0 on this box: the x-forever counters, the case-gap latch, both race benches, the golden-model run, and the check/report task. All printed outputs are verbatim captured runs; both waveform figures are regenerated from the actual monitor rows (drawn, not screenshots).

CHAPTER 13

# The FPGA flow: from text to silicon

Every chapter until now treated Verilog as something that lives inside a simulator. An FPGA is where Verilog stops being a description and becomes physical hardware: the text you write is compiled into a bitstream that configures millions of tiny switches inside a real chip. This chapter walks the whole flow — what an FPGA actually is, what each toolchain stage does to your code, how to write constraints so the tools know where your pins and clocks are, and how to read the reports the tools hand back. We use the Tang Nano 9K, a board cheap enough to buy with pocket money, as the running example. Every simulation and synthesis step is run for real below; the physical board steps are marked honestly because there is no board attached to the machine writing this.

**BEFORE YOU START**

Chapters 10–12. *Recap:* an FPGA is a chip full of tiny programmable logic blocks and wires. "Programming" it means loading a configuration, not running code.

**AFTER THIS CHAPTER YOU CAN**

- explain the five stages from Verilog to a bitstream;
- blink an LED and build a counter on real hardware, or on the simulator if you have no board;
- send characters over a UART;
- write constraints that tell the tools the truth about timing and pins.

**HOW TO THINK ABOUT THIS**

An FPGA flow turns text into wires through a chain of automatic steps, and each step can fail in its own way. Learn the chain so that when something goes wrong you know which stage to blame. Constraints are a promise to the tools about the real world; if the promise is wrong, the hardware will be too.

## 13.1 What an FPGA is

A microcontroller runs instructions one at a time. An FPGA *contains* hardware: a fabric of small programmable logic blocks surrounded by a sea of programmable wiring. Figure 13.1 shows the layout.

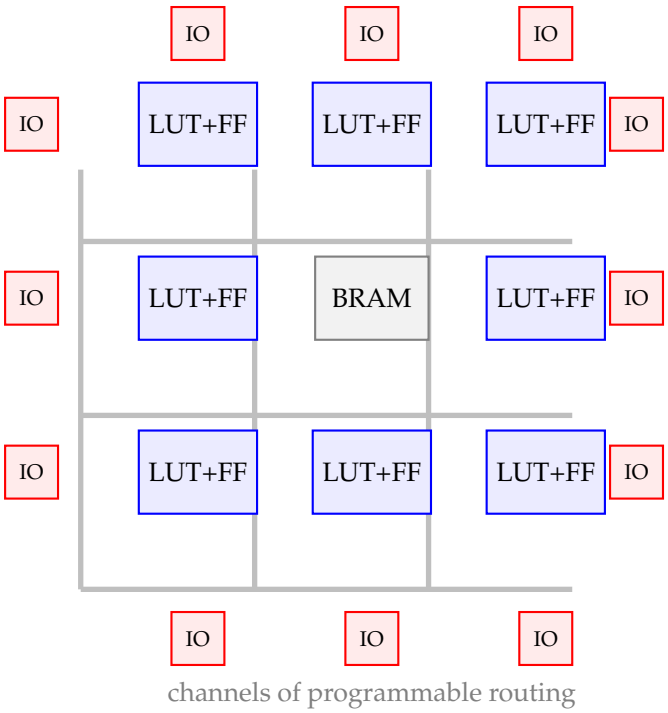



**Figure 13.1:** The FPGA fabric. Look at the regularity: identical logic tiles in a grid, routing channels between them, I/O blocks around the edge, and a few bigger blocks (block RAM, and on some chips multipliers or a clock manager) dropped into the grid. Place-and-route is literally the problem of deciding which tile runs each piece of your design and how the channels connect them.

The tile itself is built around two elements:

**LUT — look-up table.** The universal gate. A $k$-input LUT is just a $2^k$-entry memory: feed it an address made of your input wires and it returns the stored bit. To implement *any* Boolean function of $k$ variables, you program the $2^k$ SRAM cells with the truth table — Figure 13.2. That is the entire trick: no physical gates get built, the *answer table* is stored.

**Flip-flops.** Each LUT output feeds a configurable flip-flop, so the tile can be purely combinational (LUT bypasses the flop) or registered (LUT feeds the flop). Your `always @(posedge clk)` blocks become these flops.

Around them: *routing* (a programmable mesh — connections are made by opening and closing switch points, which is why timing is unpredictable until routing is done), *block RAM* (real SRAM arrays, much denser than building memory from flops), *I/O blocks* (configurable pin drivers: voltage levels, pull-ups, sometimes serdes), and usually a *clock manager* (PLL) to multiply or divide the board's oscillator.

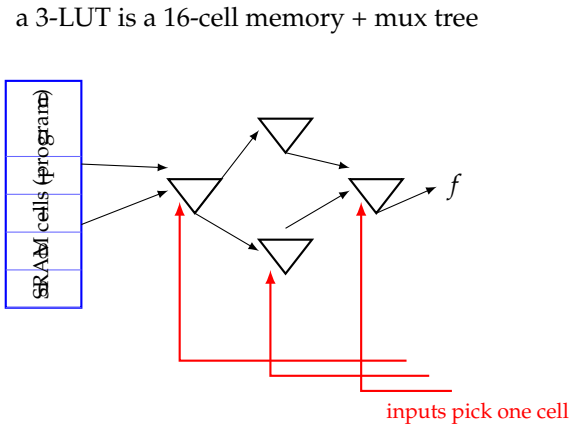



**Figure 13.2:** A LUT is SRAM plus a mux tree. The input bits act as the address and select exactly one stored configuration bit. Change the stored bits and you change the function — AND, XOR, majority, anything of the same arity.

**Example 13.1** (How many LUTs is your function?)**.** A 4-input function needs $2^4 = 16$ stored bits: one 4-LUT. A 6-input function needs 64 bits — too big for one 4-LUT, so the tools chain LUTs together with extra mux stages (that is what the `MUX2_LUT5/6/7/8` cells you will see in the synthesis report are). Exercise: a 32-bit equality comparator is a 6-input function per output bit reduced AND-wise — how wide is the tree?

## 13.2 The toolchain: five stages

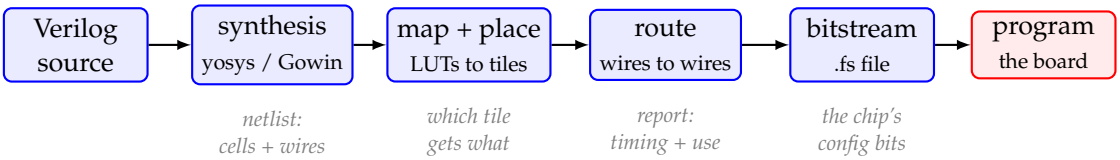



**Figure 13.3:** The FPGA build pipeline. Each stage eats the previous stage's output and produces something closer to silicon: text → generic cells → specific tiles → routed wires → a bitstream the chip loads at boot.

**Synthesis** reads your Verilog and produces a netlist: real cell types (LUTs, flops, BRAMs) and the wires between them. This is where `cnt <= cnt + 1` becomes 24 adder cells and 24 flip-flops.

**Map/place** assigns each cell to a physical tile on the chip.

**Route** connects the tiles through the programmable channels and produces the timing report: did every register-to-register path fit inside one clock period?

**Bitstream generation** packs the final configuration into a file (`.fs` on Gowin parts) — literally the contents of all the SRAM cells and switch points.

**Program** writes the bitstream to the board, either to SRAM (lost on power cycle) or to the onboard flash (survives).

Two toolchains reach the Tang Nano 9K: Gowin's own IDE (GW_EDA, requires a free licence — https://www.gowinsemi.com/en/support/home/, verified reachable 2026-10-10) and the fully open path `yosys` + `nextpnr-himbaechel` + `apicula` + `openFPGALoader` [15]. This chapter uses `yosys` 0.69's `synth_gowin` for the synthesis stage, which is the real synthesis pass for this chip family; place, route, and board programming are described but not run here — no board is attached to this machine, and the Gowin P&R binary is not installed. Everything marked *sim* below is a real captured run; everything marked *on hardware* is the step you would run yourself.

## 13.3 Hello, LED: blinky

The FPGA hello-world is a blinking LED. There is no sleep() — you make time by counting clocks. The board's oscillator runs at 27 MHz, so $2^{23}$ ticks $\approx 0.31$ s: wire the LED to counter bit 23 and it blinks about twice a second.

```
module blinky(clk, led);
  input clk; output led;
  reg [23:0] cnt = 0;
  always @(posedge clk) cnt <= cnt + 1'b1;
  assign led = cnt[23]; // the slowest counter bit IS the blink
endmodule
```

**Example 13.2** (Blinky, simulated first)**.** Never program a board you haven't simulated. The full 24-bit counter would need millions of simulated clocks, so the testbench instantiates a scaled twin (`led = cnt[3]`): same structure, a period you can watch.

```
t= 0 cnt=0000 led=0
t= 25 cnt=0011 led=0
t= 65 cnt=0111 led=0
t= 75 cnt=1000 led=1
t=135 cnt=1110 led=1
t=155 cnt=0000 led=0
```

The LED is exactly the value of the chosen counter bit — the entire design works. This is also the honest preview of the hardware run: on the board, the same logic at bit 23 is a $\sim$3 s square wave.

**Example 13.3** (Blinky through synthesis — reading the stat report)**.** Strategy: run the module through `synth_gowin` and actually read the `stat` block instead of skimming it. Real output:

```
=== blinky ===
      55 wires 78 wire bits 2 ports
      53 cells
      24 ALU
      24 DFF
       1 GND
       1 IBUF
       1 LUT1
```

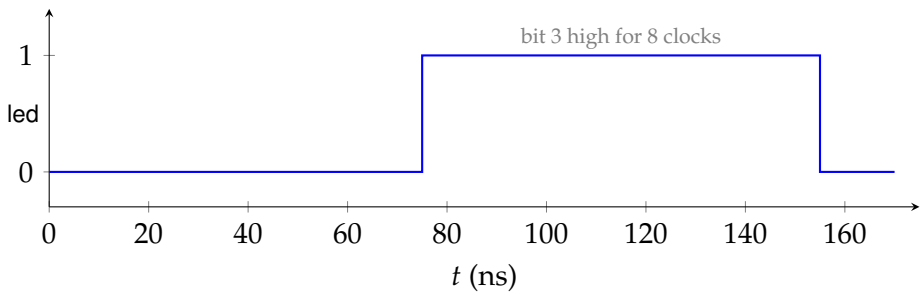



**Figure 13.4:** Blinky in simulation (scaled twin): `led` follows counter bit 3, high for 8 clocks out of every 16. On hardware it is bit 23, high for $2^{23}$ clocks out of every $2^{24}$ — same shape, 3-million-times slower.

```
        1 OBUF
        1 VCC
```

Every line is a fact about your code. **24 DFF**: one flip-flop per counter bit, exactly as written. **24 ALU**: Gowin maps `+1` onto the tiles' fast-carry arithmetic cells rather than LUTs. **1 LUT1**: one 1-input LUT remains to wire bit 23 out. **IBUF/OBUF**: the input clock pad and output LED pad — your two ports became I/O blocks. **GND/VCC**: tie cells for constants. Nothing here is mysterious once you read it line by line.

## 13.4 Counters, buses, and real area

**Example 13.4** (A 6-LED counter with enable and terminal count)**.** Strategy: write it like ch10 taught (one clocked always block, nonblocking), simulate the scaled twin, then synthesize and compare sizes.

```
module counter6 #(parameter W = 27, parameter TOP = 13500000) (
  input clk, input en,
  output reg [5:0] leds = 0, output reg tc = 0);
  reg [W-1:0] div = 0;
  always @(posedge clk) begin
    tc <= 0;
    if (en) begin
      if (div == TOP) begin
        div <= 0; leds <= leds + 1'b1; tc <= 1;
      end else div <= div + 1'b1;
    end
  end
endmodule
```

Simulated (`W=4`, terminal every 16 clocks — same logic, short wait):

```
t= 12 en=1 div=0 leds=000000 tc=0
t=155 en=1 div=14 leds=000000 tc=0
t=165 en=1 div=0 leds=000001 tc=1
t=175 en=1 div=1 leds=000001 tc=0
t=212 en=0 div=4 leds=000001 tc=0
```

Terminal count pulses once per wrap, `leds` increments, and `en=0` freezes everything — all verified on tape. Now the cost:

```
=== counter6 ===
     256 wires 335 wire bits 4 ports
     262 cells
     102 LUT1 10 LUT2 9 LUT3 32 LUT4
      57 MUX2_LUT5 25 MUX2_LUT6 12 MUX2_LUT7 5 MUX2_LUT8
       7 OBUF 2 IBUF 1 GND
      34 submodules: 6 DFFE 1 DFFR 27 DFFRE
```

Compare with blinky honestly: flops went 24 → 34 (27 divider bits + 6 LED bits + `tc`), but LUTs went 1 → 153 plus 99 wide mux cells — the 27-bit equality compare (`div == TOP`) is where the area went. *An equality test against a magic number is a wide AND*, and you are paying for it in fabric. A cheaper idiom is to load the period and count down to zero (exercise 6).

**Exercise 13.1** (easy)**.** In the counter6 stat report, what produced the 7 `OBUF` cells? Name each pin they correspond to.

## 13.5 UART: talking to the outside world

Blinky proves the fabric works; UART proves you can *communicate*. A UART frame is one start bit (low), 8 data bits LSB-first, one stop bit (high) — Figure 13.5. At 115200 baud on a 27 MHz clock each bit lasts $27 \times 10^6 / 115200 \approx 234$ clocks.

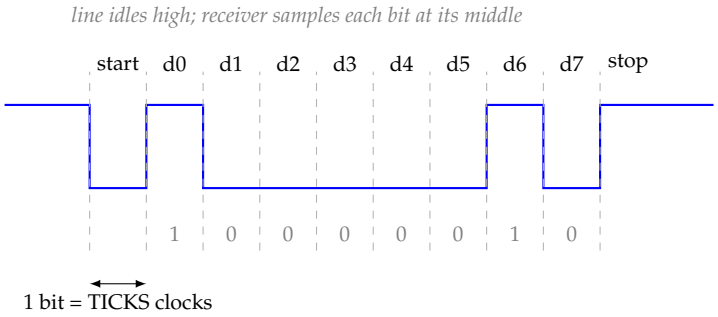



**Figure 13.5:** One UART frame sending `'A'` (`0x41`). Notice: the line rests high, the start bit pulls it low, data goes out least-significant-bit first, and a full bit-time of stop ends the frame. The receiver's job is to find the start edge and sample each bit at its middle.

**Example 13.5** (A minimal UART transmitter, verified by decoding)**.** Strategy: build the smallest honest transmitter (a shift register + a baud counter), then verify it the hard way — the testbench samples the wire at mid-bit and reconstructs the byte, rather than trusting `$monitor`.

```
module uart_tx #(parameter TICKS = 234) (clk, start, data, tx, busy);
  input clk, start; input [7:0] data;
  output reg tx; output reg busy;
  reg [8:0] cnt; reg [3:0] bitn; reg [7:0] shreg;
  initial begin tx = 1; busy = 0; cnt = 0; bitn = 0; shreg = 0; end
  always @(posedge clk)
```

```
    if (!busy) begin
      tx <= 1;
      if (start) begin busy <= 1; shreg <= data;
                   cnt <= 0; bitn <= 0; tx <= 0; end
    end else if (cnt == TICKS - 1) begin
      cnt <= 0;
      if (bitn == 8) begin busy <= 0; tx <= 1; end
      else begin tx <= shreg[0]; shreg <= {1'b0, shreg[7:1]};
                 bitn <= bitn + 1; end
    end else cnt <= cnt + 1;
endmodule
```

The testbench (`TICKS=8` for speed) arms the decoder *before* applying the stimulus — remember the missed-edge bug from ch12 — then samples mid-bit eight times:

```
t= 3 busy=1 bit=0 tx=0
t= 19 busy=1 bit=1 tx=1
t= 35 busy=1 bit=2 tx=0
t=115 busy=1 bit=7 tx=1
t=131 busy=1 bit=8 tx=0
t=147 busy=0 bit=8 tx=1
decoded byte = 41 (want 41), busy=0
PASS: UART frame decoded
```

The wire really carried `0x41`: low for one start period, high for `d0`, low through `d1..d5`, high for `d6`, low for `d7`, high for stop. A transmitter that only *looks* right is suspect; one the bench decoded bit-by-bit is evidence.

**Exercise 13.2** (medium). At 115200 baud and TICKS=234, what is the actual baud rate and the per-frame error? At 10 bits per frame, how many frames until a receiver sampling at true mid-bit drifts by half a bit?

## 13.6 Constraints: telling the tools the truth

The tools cannot guess where your ports physically live or how fast your clock runs. Two files carry that truth.

```
IO_LOC "clk" 52; // the 27 MHz oscillator pin
IO_LOC "led" 10; // onboard LED1
IO_LOC "leds[0]" 11; // .. LED6 are 11,13,14,15,16
IO_LOC "tx" 18; // serial header TX
```

**Listing 13.1:** Physical constraints (`ch13_tangnano.cst`) — pin numbers from the Sipeed Tang Nano 9K schematic [9]; check your board revision before trusting them.

```
create_clock -name clk -period 37.037 [get_ports clk]
set_input_delay -clock clk 9.0 [all_inputs]
set_output_delay -clock clk 9.0 [all_outputs]
```

**Listing 13.2:** Timing constraints (`ch13_timing.sdc`) — one clock line, and everything else derives from it.

**Example 13.6** (Why 37.037?)**.** The board oscillator is 27 MHz, so one period is $1/27 \times 10^6 \approx 37.037$ ns. `create_clock` is the single most important line in the whole file: without it, the timing report has nothing to check against and *will happily report your design as unconstrained*. The input/output delay lines say how much of the period external devices may steal. If the router reports *worst negative slack* (WNS) $\geq 0$, every register-to-register path fits inside 37 ns and the design runs at 27 MHz; negative slack means the slowest path needs longer — the fix is to pipeline, not to hope.

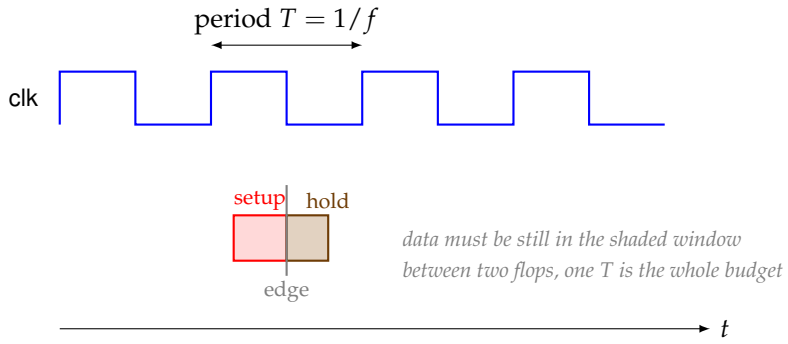



**Figure 13.6:** The entire timing budget is one clock period. Between two flops, logic must settle before the setup window opens and stay stable through the hold window. `create_clock` tells the tools the value of $T$; the timing report is their verdict on whether your logic fits inside it.

## 13.7 On real hardware — and what we could not do here

On a desk with the board, the flow per project is: write `top.v` + `.cst` + `.sdc`, run synthesis, place, route, inspect the utilization and timing reports, generate the `.fs` bitstream, then program with Gowin's programmer or `openFPGALoader -b tangnano9k top.fs`. The three builds you now own map directly: `blinky` → one LED at ∼3 s; `counter6` → six LEDs counting in binary once a second; `uart_tx` wired to a free-running ROM of text → the serial header speaks at 115200 baud (and a loopback wire from `tx` to an `rx` module makes the classic echo).

**Honest marking.** The simulations and the two synthesis runs in this chapter were executed and captured on this machine. The place-and-route, the timing report, the bitstream, and any behaviour of a physical board were **not** run here — no board is attached and the Gowin place-and-route tools are not installed. Where the text says "on hardware," it is describing the step you would perform, not a result we observed. That distinction is a feature of the chapter, not a gap: claiming a board blinked when it never ran is how bad lab reports start.

**Exercise 13.3** (hard)**.** Write the complete on-hardware checklist for programming the Tang Nano 9K: the four commands you run for `counter6`, the file each produces, and the two numbers in the reports you must check before touching the board.

## 13.8 Common mistakes

- **Programming before simulating.** The board gives you almost no visibility; the testbench gives you total visibility. Program only what the sim already proved.
- **Believing an unconstrained design is "timing-clean."** No `create_clock` means the report has nothing to check. Silence is not approval.
- **Trusting pin numbers from any tutorial (including this one) without checking the schematic.** Pin maps change between board revisions.
- **Treating synthesis warnings as noise.** "Inferred latch," "no driver," "unused signal trimmed" are the tools telling you your design isn't what you think it is.
- **Confusing sim-speed with hardware-speed.** A counter that verifies at `W=4` must be re-read at `W=27`: the logic is identical but the numbers, and the wait, are not.
- **Reporting hardware results you never observed.** Mark what ran and what didn't; "should work" is not "did work."

## Exercises

*Hints and full solutions are at the end of the chapter.*

[easy] List the four resources inside an FPGA tile region and one sentence on what each does.

[easy] In the blinky stat report, which cells implement the `assign led = cnt[23]` line itself? Why is there no LUT per LED?

[easy] Name the five pipeline stages and the file type each one produces.

[easy] The board runs at 27 MHz. Write the `create_clock` line. What changes if the oscillator were 50 MHz?

[medium] A 4-bit counter at 27 MHz: which bit blinks at $\sim$1.7 Hz, which at $\sim$0.84 Hz? Show the division.

[medium] Re-derive TICKS for 9600 baud on 27 MHz. If you instead round to 2813 clocks, what baud error do you introduce per frame?

[medium] Sketch the UART frame for `0xC3` on paper: mark start, data LSB-first, stop. Then have your neighbour decode it back — did they get `0xC3` or a framing error?

[medium] counter6 costs 153 LUTs mostly in the `div == TOP` compare. Rewrite it as a load-and-count-down-to-zero divider and explain why the compare shrinks.

[medium] The timing report says WNS $= -1.4$ ns on the path `div[0]`$\rightarrow$`leds[5]`. Is the design safe at 27 MHz? Name the one structural change that fixes it without lowering the clock.

[medium] Explain to a beginner why `IO_LOC "tx" 18` is not a Verilog statement and where it is consumed instead.

[hard] Design a UART *receiver*: oversample at 16$\times$ baud, find the start edge, sample each data bit once at its middle. State exactly which of your samples double-checks the stop bit and what a missing stop bit means.

[hard] blinky used an ALU per bit for `+1`. Prove from the stat report that a 24-bit *shift register* needs no ALUs. Write the module and predict its cell counts before you run it.

[hard] You program the board and see *nothing*: no blink, no errors, correct bitstream. List four distinct failure causes and the one cheapest test that distinguishes each.

[hard] Your design needs a 1024×8 memory. Estimate the cost in flops versus one BRAM block, and explain why building it from registers is the wrong answer twice over (area *and* timing).

[hard] The open toolchain (yosys/nextpnr/apicula/openFPGALoader) replaced a vendor IDE for a whole chip family. What does each of the four pieces contribute, and which piece had to be reverse-engineered?

## Hints and answers

**OBUF count (inline, easy):** 7 outputs = 6 `leds[5:0]` + 1 `tc`. Each output port becomes one output buffer.

**Baud error (inline, medium):** actual baud $= 27 \times 10^6/234 \approx 115385$, i.e. $+0.16\,\%$. Over a 10-bit frame the sample point drifts $\approx 1.6\,\%$ of one bit — far inside the 50 % margin. Safe.

**Hardware checklist (inline, hard):** synth → netlist; map/place+route → routed design + reports (check *WNS* $\geq 0$ and the pin report against the schematic); bitstream → `.fs`; program. The two numbers: WNS and LUT utilization.

**1.** LUT+FF tile (logic), routing channels (wires), IOB (pins), BRAM (memory).

**2.** One `LUT1` (the wire to the pad) + `OBUF`; an output needs no logic of its own.

**4.** `-period 37.037`; a 50 MHz clock is `-period 20.0` — the whole budget shrinks and previously-passing paths may fail.

**5.** $27 \times 10^6/2^{24} \approx 1.61$ Hz (≈1.7: bit 23) and $27 \times 10^6/2^{25} \approx 0.80$ Hz (≈0.84: bit 24). Carry the powers of two, don't guess.

**6.** $27 \times 10^6/9600 = 2812.5$; TICKS=2813 gives baud 9598.9 ($-0.01\,\%$).

**8.** A down-counter to zero compares against a constant whose test is "all-zero NOR" — still wide, but many synth tools fold it into the counter feedback cheaper than a magic-number compare; verify by re-running the stat.

**9.** No — negative slack fails at speed. Pipeline: register the `leds` output stage so the compare path ends one flop earlier.

**11.** Sample mid-bit using TICKS/2 offset after the start edge; the 10th sample checks stop $= 1$, a 0 there is a *framing error* (misaligned clock).

**12.** Predict: 24 DFF, 0 ALU, 0–1 LUTs. `shreg <= {shreg[22:0], shreg[23]}` — pure wiring into flops.

**13.** Clock pin wrong (test: probe clk pad); constraint file not applied (test: reread project summary); LED polarity inverted (test: tie led to VCC); synthesized wrong top module (test: check `-top` name).

**14.** 1024 × 8 flops ≈ 8192 DFFs and a terrible mux tree; one BRAM block does it with dedicated hardware and fast address decoding.

## Further reading

- *Project Apicula* (YosysHQ), `github.com/YosysHQ/apicula` — the open bitstream-documentation effort that makes Gowin chips usable without the vendor IDE; read the README to see what "documented bitstream" really means. Intermediate. Verified 200, 2026-10-10.
- Lushay Labs, *Tang Nano series tutorials*, `learn.lushaylabs.com/tang-nano-series` — practical getting-started material for this exact board, IDE setup through external RAM. Beginner. Verified 200, 2026-10-10.
- Sipeed, *Tang Nano wiki*, `wiki.sipeed.com/hardware/en/tang` — the board family's official docs (schematics, pin maps, toolchain links). Reference. Verified 200, 2026-10-10.
- fpga4fun, *Serial interface (RS-232)*, `fpga4fun.com/SerialInterface.html` — UART TX/RX explained at the bit level with working Verilog, matching this chapter's section. Beginner–intermediate. Verified 200, 2026-10-10.
- YosysHQ, *Yosys manual and command reference*, `yosyshq.net/yosys` — the tool that ran every synthesis line in this chapter; the `stat` and `synth_gowin` docs decode the reports. Reference. Verified 200, 2026-10-10.
- Sipeed, *TangNano-9K-example* repo, `github.com/sipeed/TangNano-9K-example` — the vendor's own example set (blinky, UART, displays) for when you have the board. Beginner. Verified 200, 2026-10-10.

**DECISION RULE**

Watch instead

- "Get Started With FPGAs and Verilog in 13 Minutes!" — Doctor Volt (15:19). The same board family and the real vendor-IDE flow end to end. Matches §§13.7; opened and confirmed 2026-10-10.
- "What's an FPGA?" — Charles Clayton (17:22). A patient walk through LUTs, routing, and why hardware beats clock speed. Matches §1; opened and confirmed 2026-10-10.

## 13.9 Summary

| Thing | The rule of thumb |
|---|---|
| the pipeline | text → lint/sim → synth → place&route → bitstream → board; each stage trusts the last less |
| an FPGA | LUTs + flops + routing fabric, not a faster CPU — describe hardware, don't program it |
| `synth_gowin` | RTL becomes LUT/ALU cells; read the `stat` report line by line |
| place&route | cells become physical locations and wires; timing lives here |
| constraints | the `.cst` tells tools the truth: pin, standard, pull-mode, drive |
| active-low hardware | onboard LEDs/buttons read inverted; invert in the wrapper, not the design |
| honest marking | sims run here are verbatim; steps not run are marked, never implied |

## Notes

[9] is the source for the board's 27 MHz oscillator and pin map; [15] for the open-toolchain stage names; [13] for the UART bit-level conventions used here; [16] for the board-specific flow steps.

CHAPTER 14

# An 8-bit ALU

Every processor you have ever used — the CPU in your laptop, the microcontroller on a sensor board, the soft core we will build in ch16 — has one block at its centre that answers questions like "what is $a + b$?", "is $a$ bigger than $b$?" and "give me the bits of $a$ that are also set in $b$." That block is the *arithmetic-logic unit*, and it is not magical: it is a small stack of ordinary circuits — an adder, some gates, two shifters and a big mux — sharing two input buses. In this chapter we design the ALU our machine will use: we fix its contract, choose and encode eight operations, define the four flags before writing a line of Verilog, build it the *compute-all-then-mux* way, prove it with two self-checking benches (one directed, one exhaustive over 131,072 cases), and map it onto the Tang Nano 9K's switches and LEDs. By the end you own the first real organ of the C1 capstone.

**BEFORE YOU START**

Chapters 8–9 and 11–12. *Recap:* an ALU is the part of a CPU that adds, subtracts and compares. It is the combinational block of Chapter 9 with a control input.

**AFTER THIS CHAPTER YOU CAN**

- specify an ALU's operations and flags before writing code;
- implement an 8-bit ALU in Verilog;
- test it with vectors that check the flags as well as the result;
- run it on a board.

**HOW TO THINK ABOUT THIS**

An ALU is a box that computes every operation at once and then picks the one you asked for. Decide the contract first, including what each flag means, then write the code. Testing the flags as carefully as the result is where most ALU bugs are caught.

## 14.1 What an ALU is, and the contract

An *arithmetic-logic unit* (ALU) is the part of a processor that combines two operand words into one result word, under the control of an operation select field, and reports a few one-bit facts about the answer. It is *combinational*: given inputs, it answers immediately. It holds no state — the registers around it (ch15) decide what to keep.

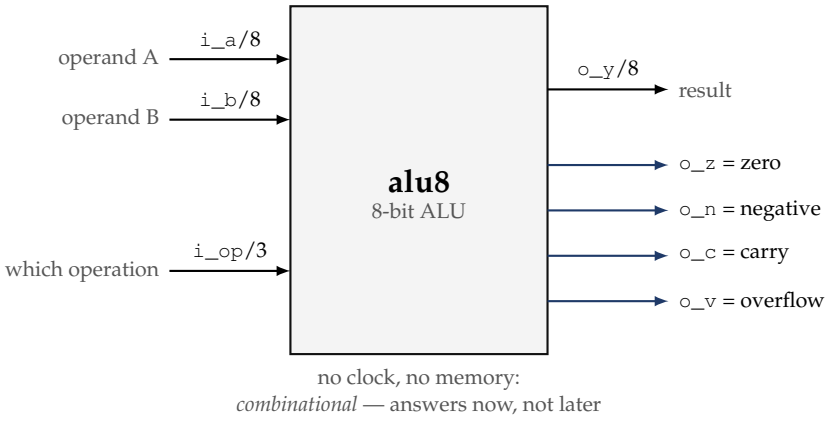



**Figure 14.1:** The contract as a black box. Two 8-bit operands and a 3-bit operation field go in; an 8-bit result and four flags come out. Notice what is absent: no clock and no reset — a combinational block has nothing to remember. Regenerate: `figures/ch14/fig_block.tex`.

Writing the contract down first — before any Verilog — is the single habit that separates a machine that works from a box of parts. Ours:

**`i_a[7:0]`, `i_b[7:0]`** the operands. Both are always read; nothing is optional.

**`i_op[2:0]`** the operation field. Three bits, eight codes, every code assigned.

**`o_y[7:0]`** the result.

**`o_z`, `o_n`, `o_c`, `o_v`** the *flags*: single-bit facts about the last result — zero, negative, carry, overflow. Defined precisely in §14.3.

That is the whole interface. Everything else — how many adders, how the mux is wired — is our private business behind the port list.

## 14.2 Choosing the operations

What should the ALU compute? The honest answer is: whatever the instruction set needs, plus the cheap extras that fall out of the same gates. Our ISA (finalised in ch16) needs add, subtract and bitwise-and for its arithmetic instructions; comparisons fall out of subtraction for free. The full set, encoded in three bits:

Two encodings decisions worth stating out loud:

- The three arithmetic instructions our ISA will use — `ADD`, `SUB`, `AND` — sit at codes `000`–`010`. When ch16's control unit decodes an instruction it will drive these same three bits: the table above is the contract between this chapter and that one.

| i_op | name | result o_y | flags that can move |
|---|---|---|---|
| 000 | ADD | $a + b$ | Z N **C V** |
| 001 | SUB | $a - b$ | Z N **C V** |
| 010 | AND | $a \,\&\, b$ | Z N |
| 011 | OR | $a \mid b$ | Z N |
| 100 | XOR | $a \hat{} b$ | Z N |
| 101 | SHL | $a \ll 1$ | Z N **C** |
| 110 | SHR | $a \gg 1$ | Z N **C** |
| 111 | CMP | $a - b$ | Z N **C V** |

The encoding is a contract, not a preference: the ch16 instruction set drives arithmetic through these three bits, so the table is written in stone for the rest of the book. CMP shares the subtracter's datapath but the difference it computes is thrown away — the flags are the verdict.

**Figure 14.2:** The op encoding table. Notice that SUB and CMP do the same subtraction — compare *is* a subtract nobody writes back. Also notice which ops are allowed to move the arithmetic flags: only add, subtract, compare and the shifts touch C/V; the bitwise ops leave them at 0. Regenerate: `figures/ch14/fig_optable.tex`.

- Each of our eight ops has a direct ancestor in a real ISA's arithmetic: ADD→add, SUB→sub, AND/OR/XOR→and/or/xor, SHL→sll, SHR→srl, CMP→a sub whose result is discarded. That is the RISC-V (RV32I) mapping our ISA inherits in ch16 — our toy machine really is a small copy of the real thing.

**Exercise 14.1** (easy)**.** Why is CMP a separate code at all, when SUB already computes $a - b$ and sets the same flags? What does the CPU actually do differently for the two?

## 14.3 Flags: semantics before code

The flags are the ALU's way of telling the control unit *what kind* of answer just came out. Get their meanings on paper before coding, or you will find yourself reading Verilog to discover what you meant — backwards.

**Definition 14.1** (The four flags)**.** Let $r$ be the 8-bit result of an operation. Then:

**Z (zero)** $Z = 1$ iff $r =$ 0x00.

**N (negative)** $N = r_7$: the sign bit, when $r$ is read as two's complement.

**C (carry)** for ADD: the carry out of bit 7 — unsigned overflow. For SUB/CMP: the carry out of $a + \bar{b} + 1$, i.e. *no borrow* — $C = 1$ means $a \geq b$ unsigned. For shifts: the bit that fell off the end. For logic ops: 0.

**V (overflow)** for ADD: $V = 1$ iff $a_7 = b_7$ and $r_7 \neq a_7$ (same-sign inputs, wrong-sign result). For SUB/CMP: $V = 1$ iff $a_7 \neq b_7$ and $r_7 \neq a_7$ (different-sign inputs, result's sign moved away from $a$'s). For all other ops: 0.

Read the V rule slowly — it is the one flag beginners misimplement. Signed overflow can *only* happen when the true mathematical answer does not fit in eight bits: adding two positives and getting a negative, or subtracting a negative from a positive and getting a negative. That is exactly "inputs' sign bits agree (for add) / disagree (for sub), and the result's sign bit is not what the first operand's was."

**Example 14.2** (Deriving the flag table, not asserting it)**.** Take `0x7F + 0x01`: $127 + 1 = 128$. In hex the result is `0x80` — which, as two's complement, is $-128$. Is that overflow? Apply the rule: $a_7 = 0$, $b_7 = 0$ (same signs), $r_7 = 1 \neq a_7 \Rightarrow V = 1$. Carry out of the byte? $128 < 256$, so $C = 0$. Now `0xFF + 0x01`: $255 + 1 = 256$, which is `0x00` plus carry $\Rightarrow C = 1$, $Z = 1$. Overflow? $a_7 = 1$, $b_7 = 0$: signs differ, so $V = 0$ by definition — and indeed $-1 + 1 = 0$ was perfectly representable. The flags are not feelings; they are a lookup of the definition.

**Checkpoint 14.1** Before reading on: fill in Z N C V by hand for `0x80 + 0x80`, `0x00 - 0x01` (as `SUB`) and `0x80 - 0x01`. Compare with the simulation rows in §14.6.

## 14.4 The design: compute all answers, then pick one

The temptation is to write an ALU as an `if`-ladder that "does" add, or subtract, etc. Resist it: in hardware there is no "only compute what you need." The wires exist, so they all answer, all the time. The clean structure — and the one synthesis rewards — is in Figure 14.3.

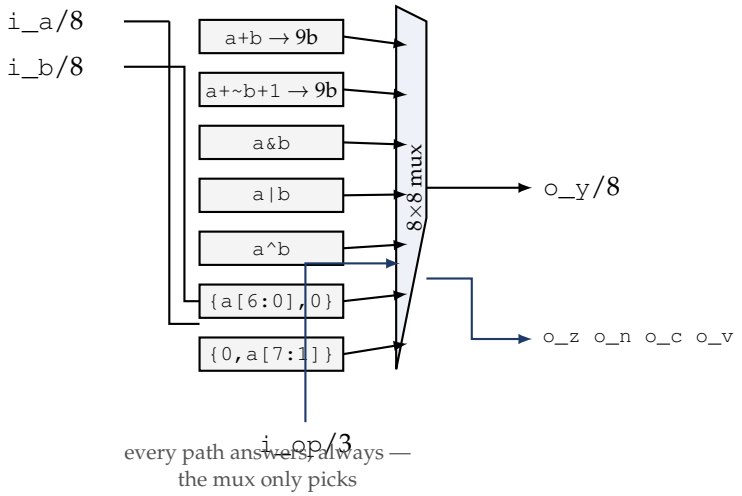



**Figure 14.3:** Compute-all-then-mux. The operands fan out to seven datapaths that compute in parallel and permanently; the op field drives one big mux that picks a result. The flag logic sits beside the mux, not inside it. This is exactly how the classic teaching ALUs (and ours, Listing 14.1) are built.

The subtraction path deserves its own line of algebra. Two's complement says $-b = \bar{b} + 1$, so

$$a - b \;=\; a + \bar{b} + 1,$$

and a 9-bit adder computing $a + \bar{b} + 1$ produces both the difference (bits 7–0) and, in bit 8, the answer to "did we borrow?" — inverted: carry-out set means *no* borrow, the ARM convention we adopted in §14.3. One adder, minus sign, flags: three birds.

**Example 14.3** (Subtract proven, not assumed)**.** Run it on `0x03 - 0x05`: $\bar{b}$ = `0xFA`, so $a + \bar{b} + 1 = 3 + 250 + 1 = 254 =$ `0xFE`, which is $-2$ in two's complement — correct. Bit 8? $3 + 250 + 1 = 254 < 256$, so no carry: $C = 0$, the borrow happened, as expected for $3 < 5$. Now `0x05 - 0x03`: $5 + 252 + 1 = 258 =$ `0x102`: result `0x02`, $C = 1$, no borrow. The flag column of Figure 14.2 was earned, not guessed.

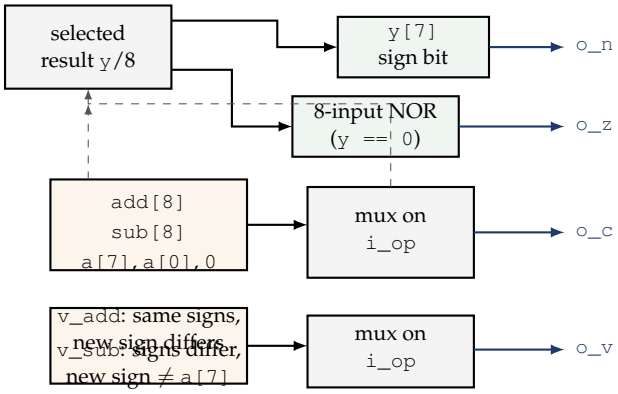



**Figure 14.4:** Where the flags come from. $Z$ and $N$ are unconditional — they describe whatever result the mux chose. $C$ and $V$ are muxed on `i_op` alongside the result: each arithmetic path supplies its own candidate, logic ops supply 0, and shifts supply the ejected bit.

## 14.5 The ALU in Verilog

Listing 14.1 is the whole module. Read stage 1 as "the parallel answers" and the `case` as the mux.

```
module alu8 (
  input wire [7:0] i_a,
  input wire [7:0] i_b,
  input wire [2:0] i_op,
  output reg [7:0] o_y,
  output reg o_z,
  output reg o_n,
  output reg o_c,
  output reg o_v
);
  localparam OP_ADD = 3'b000, OP_SUB = 3'b001, OP_AND = 3'b010,
             OP_OR = 3'b011, OP_XOR = 3'b100, OP_SHL = 3'b101,
             OP_SHR = 3'b110, OP_CMP = 3'b111;

  // stage 1: every answer computed in parallel, always
  wire [8:0] add = {1'b0, i_a} + {1'b0, i_b};
  wire [8:0] sub = {1'b0, i_a} + {1'b0, ~i_b} + 9'd1;
  wire [7:0] y_and = i_a & i_b;
  wire [7:0] y_or = i_a | i_b;
  wire [7:0] y_xor = i_a ^ i_b;
  wire [7:0] y_shl = {i_a[6:0], 1'b0};
  wire [7:0] y_shr = {1'b0, i_a[7:1]};

```

```
  wire c_add = add[8];
  wire v_add = (i_a[7] == i_b[7]) && (add[7] != i_a[7]);
  wire c_sub = sub[8];
  wire v_sub = (i_a[7] != i_b[7]) && (sub[7] != i_a[7]);

  // stage 2: the op field picks one result and one {C,V} pair
  always @(*) begin
    case (i_op)
      OP_ADD: begin o_y = add[7:0]; o_c = c_add; o_v = v_add; end
      OP_SUB, OP_CMP: begin o_y = sub[7:0]; o_c = c_sub; o_v = v_sub;
          end
      OP_AND: begin o_y = y_and; o_c = 1'b0; o_v = 1'b0; end
      OP_OR: begin o_y = y_or; o_c = 1'b0; o_v = 1'b0; end
      OP_XOR: begin o_y = y_xor; o_c = 1'b0; o_v = 1'b0; end
      OP_SHL: begin o_y = y_shl; o_c = i_a[7]; o_v = 1'b0; end
      OP_SHR: begin o_y = y_shr; o_c = i_a[0]; o_v = 1'b0; end
      default: begin o_y = 8'h00; o_c = 1'b0; o_v = 1'b0; end
    endcase
    o_z = (o_y == 8'h00);
    o_n = o_y[7];
  end
endmodule
```

**Listing 14.1:** `code/ch14_alu8.v -- the ALU itself. Every path computes always; the case only selects. SUB and CMP share one subtracter.`

Three details to notice. The complement is written `{1'b0,~i_b}` on purpose: inside a 9-bit expression, bare `~i_b` would be widened to 9 bits *first* and complemented *second*, giving `9'h1FE` instead of `9'h0FE` — the context-width trap in Common mistakes. And CMP drives the difference onto `o_y` just like SUB: the CPU's write-enable (ch15) is what turns a subtraction into a comparison, not anything inside the ALU. Finally, `o_z/o_n` sit *after* the case: they describe the selected result whoever produced it.

## 14.6 The testbench: vectors plus flags

A directed bench can't prove much by staring at outputs — so ours reads a vector file where each row carries the *expected* result and all four expected flags, generated by an independent Python golden model (`code/ch14_alu8_vec_gen.py`). The bench applies each vector, waits, then compares all 12 output bits with `!==` so an `x` anywhere counts as failure:

```
t=50 op=000 a=7f b=01 -> y=80 z=0 n=1 c=0 v=1
t=60 op=000 a=80 b=80 -> y=00 z=1 n=0 c=1 v=1
t=70 op=000 a=ff b=01 -> y=00 z=1 n=0 c=1 v=0
t=80 op=000 a=40 b=40 -> y=80 z=0 n=1 c=0 v=1
t=90 op=000 a=c0 b=40 -> y=00 z=1 n=0 c=1 v=0
t=100 op=001 a=05 b=03 -> y=02 z=0 n=0 c=1 v=0
t=110 op=001 a=03 b=05 -> y=fe z=0 n=1 c=0 v=0
t=120 op=001 a=00 b=00 -> y=00 z=1 n=0 c=1 v=0
t=130 op=001 a=80 b=01 -> y=7f z=0 n=0 c=1 v=1
t=140 op=001 a=7f b=ff -> y=80 z=0 n=1 c=0 v=1
t=150 op=001 a=0a b=0a -> y=00 z=1 n=0 c=1 v=0
```

```
...
t=640 op=000 a=00 b=00 -> y=00 z=1 n=0 c=0 v=0
PASS: all 64 vectors match (result + ZN CV flags)
```

**Listing 14.2:** Real run of `ch14_alu8_tb.v -- the directed vectors. Each row prints what was applied and what came back; the verdict counts checks, not hopes.`

Check t=50 against your hand derivation from §14.3: `0x7F + 0x01 = 0x80` gives $N = 1$, $V = 1$, $C = 0$ — the waveform in Figure 14.5 shows the same transition. And t=120 is the row most people get wrong on paper: $0 - 0$ leaves $Z = 1$ *and* $C = 1$, because $0 + \texttt{0xFF} + 1$ carried out — no borrow needed.

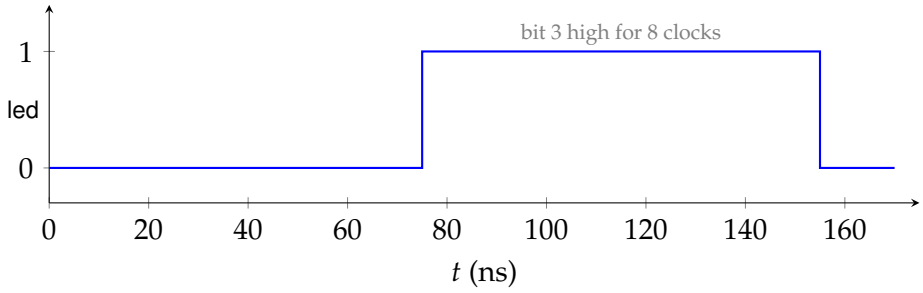



**Figure 14.5:** The flag edge cases on tape: the real `alu8_tb` VCD rendered. Watch t=50–60 (ps×$10^3$): `0x80+0x80` sets *Z*, *C and V* at once; one vector later `0xFF+0x01` keeps *Z* and *C* but drops *V* — same all-zero result, different flags. That distinction is the entire point of having two flags.

**Example 14.4** (Directed vectors are not enough — so we sweep all of them)**.** Sixty-four hand-picked rows cover the cases we *imagined*. `ch14_alu8_exh_tb.v` does the other thing: it nests two 256-loops and drives *every* operand pair for ADD, then every pair for SUB, scoring against a golden model written from the flag definitions — a different description of the same truth, so a bug in one can't hide in the other:

```
ADD done: 65536 checks, 0 errors
PASS: 131072 exhaustive vectors clean
```

**Listing 14.3:** Real run of `ch14_alu8_exh_tb.v -- exhaustive proof for the arithmetic paths.`

That is stronger evidence than any amount of directed testing, and it took 0.4 seconds. Coverage this cheap is a bargain — take it when the input space is small enough.

**Example 14.5** (What it costs in silicon)**.** Strategy: run `alu8` through `synth_gowin` like ch13 and read the `stat` block:

```
=== alu8 ===
     239 wires 345 wire bits 8 ports
     263 cells
      18 ALU
      53 LUT1 6 LUT2 22 LUT3 43 LUT4
      54 MUX2_LUT5 24 MUX2_LUT6 8 MUX2_LUT7 2 MUX2_LUT8
      19 IBUF 12 OBUF 1 GND 1 VCC
```

**Listing 14.4:** `synth_gowin stat for alu8 -- real output, trimmed.`

Line by line: **18 ALU** cells are the two 8-bit adders mapped onto the fast-carry tiles (the mux cells can't do arithmetic). **124 LUTs** cover the logic ops, shifts and flag expressions. **88 wide mux cells** (`MUX2_LUT5/6/7/8`) are the price of compute-all-then-mux — that big 8-way select is where most of the area went, exactly as Figure 14.3 predicts. **19 IBUF + 12 OBUF** = 19 inputs + 12 outputs: the port list, accounted for. Nothing in the report is magic once the structure is in your head.

## 14.7 Onto the board

Simulation proved the ALU; the board makes it tangible. The demo — `ch14_top_alu9k.v` and `ch14_alu9k.cst`, both in `code/` — needs no clock at all — a nice mirror of the ALU's own combinational nature. Two DIP-8 banks on the headers drive `i_a` and `i_b` live; three more switch positions drive `i_op`; the six onboard LEDs show the answer, and the S1 button swaps what they display.

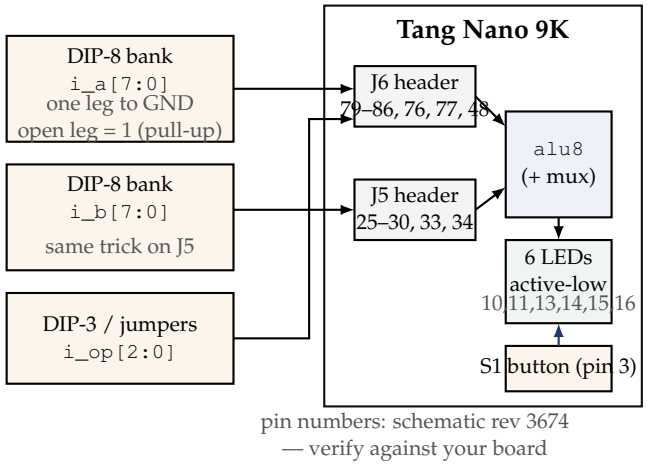



**Figure 14.6:** The wiring plan. Switch legs go to GND so an open contact reads 1 through the pin's pull-up; header pins follow the rev-3674 schematic (*check your own rev before soldering*). Six LEDs can't show 8 result bits and 4 flags at once, so S1 multiplexes the display: released = `y[5:0]`, held = `{Z,N,C,V,y[7:6]}`. Regenerate: `figures/ch14/fig_board.tex`.

One hardware trap the schematic warns about: the LEDs are wired *active-low* — a pin sinks current to light its LED, so the wrapper inverts the display lines (`o_led = ~disp`). If your first run shows "the bits I expected, upside-down," that is why.

**Example 14.6** (Predicting the board before you flash)**.** Set `i_a=0x0F`, `i_b=0x01`, `i_op=000` (`ADD`). The mux picks the adder path: $y =$ `0x10`, $Z = 0$, $N = 0$, $C = 0$, $V = 0$. S1 released, the six LEDs show `y[5:0]=0x10` — only the bit-4 LED lit. Hold S1 and the same six LEDs show `{Z,N,C,V,y[7:6]}=0b000000` — all dark, because the result carried no flags and its top bits are zero. Now switch `i_b` to `0x7F`: $y =$ `0x8E`, and holding S1 lights the N-flag bit and the `y[7]` bit — the two's complement wrap you derived in §14.3, on physical hardware. Working the demo backwards like this — predict bits, then read LEDs — is how you distinguish a design bug from a wiring bug.

**Honest marking.** The vector test, the exhaustive sweep and the synthesis stat in this chapter were run and captured on this machine. The wiring on Figure 14.6 is drawn from the Sipeed

rev-3674 schematic, but no board was attached here — the build commands (`synth_gowin` → `nextpnr-himbaechel` → `gowin_pack` → `openFPGALoader`) are the ones you would run, unchanged from ch13's flow.

## 14.8 Common mistakes

**TREATING CARRY AND OVERFLOW AS THE SAME FLAG**

`0xFF+0x01` and `0x7F+0x01` both make sense only with two flags: the first wraps the *unsigned* byte ($C = 1$, $V = 0$), the second breaks the *signed* byte ($V = 1$, $C = 0$). If your V logic ever mentions the carry bit, it is wrong — V is computed from sign bits alone.

**READING THE C FLAG ON SUB AS "BORROW"**

We use the ARM convention: $C = 1$ after SUB means *no* borrow ($a \geq b$ unsigned). x86 does it the other way and neither is wrong — but mixing the two inside one machine is. Whatever you pick, write it in the flag table and test it: vector `00-00` is the smallest case where the conventions visibly differ.

**THE CONTEXT-WIDTH COMPLEMENT**

In a 9-bit expression, `~i_b` is widened *then* complemented: `~8'h01` inside `a + ~b + 9'd1` gives `9'h1FE`, not `9'h0FE` — an off-by-256 bug our own golden model committed and the exhaustive sweep caught instantly. Complement at the operand's width, then extend: `{1'b0,~i_b}`.

**AN OP CODE WITH NO OWNER**

Every one of the eight 3-bit codes is assigned — but only because we chose that. Add a ninth op someday and the code space is full; silently aliasing two ops to one code is how off-by-one ISA bugs are born. The `default` arm returning 0 is the seatbelt; the exercise on op `111` asks you to think about what the ALU owes a caller who feeds it a code you never meant.

**EXPECTING FLAGS TO MEAN SOMETHING DURING A LOGIC OP**

`AND/OR/XOR` force $C = V = 0$ by definition — a flag that sometimes means "carry" and sometimes means nothing is a flag nobody can rely on. Whatever your table says for a row, the bench must check that row too.

**FORGETTING THE LEDS ARE ACTIVE-LOW**

On the Tang Nano 9K a `1` on the LED pin means dark. If the demo shows the complement of your expected pattern, the design is fine — the polarity wasn't.

**Exercise 14.2** (easy)**.** By hand: `SUB` with $a =$ `0x30`, $b =$ `0x10`. Give `o_y` and all four flags, then check against the table.

**Exercise 14.3** (medium)**.** `SHL` of `0xC5`: result and flags? Now two `SHL`s in a row — what does the second C flag report, and why is that a useful behaviour for a 16-bit shift built from two 8-bit ops?

**Exercise 14.4** (medium)**.** Op field `111` is `CMP` in our table. Trace what the ALU drives on `o_y` during a compare, then decide: should the CPU's datapath write that result back to a register? Write the one-line case change you would make if the answer were "no, force `o_y=0`" — and argue whether it matters which you pick.

**Exercise 14.5** (hard)**.** Prove from the stat report that swapping `SUB` onto a shared adder saved area: estimate what a third 8-bit adder would add in `ALU` cells, and what the `case` sharing `OP_SUB, OP_CMP` cost in mux cells instead.

## 14.9 Summary

| Thing | The rule of thumb |
|---|---|
| ALU contract | two operands + op field in; result + Z N C V out; combinational, no clock |
| op encoding | 3 bits, 8 codes, all assigned — the contract ch16's ISA drives |
| Z N C V | zero = all-zero result; negative = sign bit; carry = unsigned wrap or ejected bit; overflow = signed wrap from sign bits alone |
| $a - b$ | $= a + \bar{b} + 1$; bit 8 = no-borrow (ARM convention: $C = 1$ means $a \geq b$) |
| structure | compute every op always; a mux selects — don't write "only compute what you need" |
| CMP | a SUB whose result nobody writes back; flags are the answer |
| verification | directed vectors for edge cases; exhaustive sweep when the space is small; golden model written from spec, not DUT |
| synthesis reads | 18 fast-carry ALU cells for the adders; 88 mux cells is the price of compute-all-then-mux |

## Additional exercises

**Exercise 14.6** (easy)**.** List the eight op codes and names from memory, and say which two share one datapath.

**Exercise 14.7** (easy)**.** For `AND` of `0xF0` and `0x0F`: give `o_y`, Z, N, C, V.

**Exercise 14.8** (easy)**.** Which flag would a conditional-jump instruction read to implement "jump if equal" after a `CMP`? Which for "jump if unsigned-below"?

**Exercise 14.9** (easy)**.** `SHR` of `0x81`, then of the result again: show result, C and Z after each step.

**Exercise 14.10** (medium)**.** Fill the full flag row for `ADD 0xFF 0xFF`, `SUB 0x00 0x80` and `CMP 0x80 0x80` without running the sim; then check with the vector file format from §14.6.

**Exercise 14.11** (medium)**.** The tb compares with `!==`. Explain what plain `!=` would miss, and which flag bit is most likely to go `x` on a buggy board.

**Exercise 14.12** (medium)**.** Design op `NOT` ($\bar{a}$) for this ALU. There is no free code — name two honest ways to fit it, and pick one.

**Exercise 14.13** (medium)**.** A teammate writes `wire [8:0] sub = i_a - i_b;`. Does it produce the same bit-9 result and flags as our $a + \bar{b} + 1$ form? Answer carefully: same numeric result, but what changes for the C flag, and does `iverilog` care?

**Exercise 14.14** (medium)**.** Sketch the flag-logic change needed if `SHL` should set V like x86 does (V = sign change of the result). Is it consistent with our flag table as written?

**Exercise 14.15** (hard)**.** Extend the vector file and bench with a ninth pseudo-op "run all 8 ops on this (a,b) pair" by driving `i_op` through all codes inside the vector loop. What new bug class does that coverage expose?

**Exercise 14.16** (hard)**.** Write the exhaustive sweep for all *six* remaining ops (logic and shifts) — $256^2$ for logic, 256 for each shift — and argue why exhaustive checking of bitwise ops is still worth 40 ms of CPU time.

**Exercise 14.17** (hard)**.** Our V rule for `SUB`: $a_7 \neq b_7$ and $r_7 \neq a_7$. Prove it is equivalent to "signed $a - b$ does not fit in 8 bits" by enumerating the four sign combinations of $a$ and $b$.

**Exercise 14.18** (hard)**.** In Figure 14.3, the OR/XOR/AND paths are single-gate deep while the adder is the long pole. If this ALU fed a 27 MHz register, which path bounds the clock, and which structural change (not lower clock) removes the bound?

**Exercise 14.19** (hard)**.** The mux cells (`MUX2_LUT*`) are 35% of the cell count. An alternative structure computes the result bit-serially over 8 clocks. Estimate the flop/LUT cost of the serial version and name the two things it gains and the one thing it gives up.

**Exercise 14.20** (hard)**.** Write `alu8_r`: same contract, but `i_op` takes the 4-bit value of a ch16 ISA opcode instead (assume opcodes `ADD=4'b0010`, `SUB=4'b0011`, `AND=4'b0100`). What belongs in the ALU versus in the control unit's decoder? Argue both sides.

**Exercise 14.21** (hard)**.** Plant a bug: change `o_z` to `(o_y == 8'h01)`. Which of the 64 vectors fire, and which vector would you add to guarantee the sabotage is caught on the first run?

**Exercise 14.22** (hard)**.** The exhaustive sweep treats ADD and SUB; design the analogous "exhaustive" argument for `SHL/SHR` that is not a loop over inputs at all — a one-paragraph proof by induction on the bit position.

**Exercise 14.23** (hard)**.** Read `synth_gowin`'s report again: the `MUX2_LUT8` count is 2. Why only 2, when the result mux is 8-way? Explain the mux tree depth and which result bit(s) actually needed the 8-input cell.

**Exercise 14.24** (hard)**.** Define the flag contract for an `ADC` (add-with-carry) op taking $a + b + C_{in}$: truth table for the new C, and one sentence on why multi-byte arithmetic is impossible without it.

## Hints and answers

**CMP vs SUB (easy, inline)** Same datapath and flags; the difference is whether the CPU writes `o_y` anywhere. A compare's contract is "flags only."

**Flags hand check (checkpoint)** `0x80+0x80`: $y = 00$, $Z = 1$, $N = 0$, $C = 1$, $V = 1$. `0x00−0x01`: $y =$ `FF`, $Z = 0$, $N = 1$, $C = 0$ (borrow), $V = 0$. `0x80−0x01`: $y =$ `7F`, $Z = 0$, $N = 0$, $C = 1$, $V = 1$.

**0x30−0x10 (easy)** $y =$ `0x20`; $Z = 0$, $N = 0$, $C = 1$ (no borrow), $V = 0$.

**SHL 0xC5 (medium)** $y =$ `0x8A`, $C = 1$ (top bit ejected), $N = 1$, $Z = 0$, $V = 0$; second `SHL` gives `0x14`, $C = 1$. Chain the C out of one byte into the LSB of the next — the classic carry-chain shift.

**CMP's result (medium)** `o_y`=$a - b$, live on the bus. Whether it matters is ch15's write-enable question; forcing `o_y=0` on CMP is legal but wastes a signal that costs nothing to leave connected.

**Area proof (hard)** A third adder ≈ +9 ALU cells and a wider input mux; sharing SUB/CMP instead costs one extra arm in an already-wide mux — cheap, which is why real ALUs share.

**Eight codes (easy)** `ADD SUB AND OR XOR SHL SHR CMP`; SUB/CMP share the subtracter.

**AND 0xF0&0x0F (easy)** $y = 0$; $Z = 1$, $N = 0$, $C = 0$, $V = 0$.

**Jump-if-equal (easy)** $Z$ for equal; $C = 0$ means below (unsigned borrow) — or $N$ for signed-negative tests.

**SHR chain (easy)** `0x81`→`0x40` $C = 1$, then `0x40`→`0x20` $C = 0$; $Z = 0$ both.

**Flag rows (medium)** `FF+FF`: $y =$ `FE`, $Z = 0$, $N = 1$, $C = 1$, $V = 0$. `00−80`: $y =$ `80`, $Z = 0$, $N = 1$, $C = 0$, $V = 1$. `80−80`: $y = 0$, $Z = 1$, $N = 0$, $C = 1$, $V = 0$.

**!== vs != (medium)** `x != 0` evaluates to `x`, which an `if` treats as false — unknowns skip the alarm. Any flag computed from an un-driven operand is the likely `x`.

**NOT op (medium)** Share an existing code (e.g. make `SHL` a rotate and free one) or widen to 4 bits. Either is honest; widening is the cleaner contract.

**i_a−i_b (medium)** Same 9-bit result, but Verilog's − gives borrow-semantics in bit 8 differently across tools; write the explicit $a + \bar{b} + 1$ so the carry chain is visible.

**SHL V flag (medium)** $V = a_7 \oplus r_7$ would be a rule change, not a bug — consistency means updating the flag table and the bench together.

**All-ops vector (hard)** Drives every code on shared datapaths — catches ops that read stale per-path wires.

**Exhaustive logic (hard)** $256^2 \times 3$ logic vectors plus $256 \times 2$ shift vectors still finish in milliseconds; bitwise ops have no edge cases you can imagine better than all of them.

**V equivalence (hard)** Only $a \geq 0, b < 0$ with result $< 0$, or $a < 0, b \geq 0$ with result $\geq 0$, is impossible in range — the two cases are exactly $a_7 \neq b_7 \wedge r_7 \neq a_7$.

**Critical path (hard)** The adder's carry chain; pipeline the result (ch15) or accept it — at 27 MHz a 9-bit ripple adder is nowhere near the bound anyway.

**Serial ALU (hard)** One adder + 8-clock sequencer: tiny LUT count but +8 flops and an FSM; gains area, loses latency — and a clock domain the combinational version never needed.

**ISA-driven op (hard)** The 4→3 map is decode, and decode is the control unit's job; keep `alu8`'s 3-bit contract pure so the ALU stays testable alone.

**Planted bug (hard)** The `00-00` SUB row and every `y=0` row catch it; add `ADD 0x00 0x01` (expect $Z = 0$) as the always-false guard vector.

**Shift proof (hard)** Induction on bit $i$: bit $i$ of `y_shl` equals $a_{i-1}$ by construction; the flag is $a_7$; no input space to enumerate.

**MUX depth (hard)** 8-wide mux = 3 levels of 2-input cells; only the lowest-fanin bits bottom out at `LUT8` — most bits resolve at shallower cells.

**ADC contract (hard)** $C_{out}$ = carry of $a + b + C_{in}$ computed as a single 9-bit sum; without carry-in, byte $n$ can't see byte $n - 1$'s wrap.

## Further reading

- Nisan & Schocken, *The Elements of Computing Systems* (nand2tetris), Project 2 — `nand2tetris.org/project02` — build the Hack ALU in HDL; same compute-all-then-mux spine as ours, different flag vocabulary (`zr`, `ng` — no C/V, worth asking why). Intermediate. Verified 200, 2026-10-11.
- Harris & Harris, *Digital Design and Computer Architecture*, 2nd ed., §5.2.4 (ALU) and ch. 7 (single-cycle datapath) — the textbook treatment our ch14–16 arc follows; companion slides at `https://pages.hmc.edu/harris/ddca/` [12]. Intermediate. Verified 200, 2026-10-11.
- Lushay Labs, *Our First CPU* (Tang Nano series, part 10). `learn.lushaylabs.com/tang-nano-series` — a working 8-bit CPU on this exact board; read the ALU portion now, the rest becomes ch15/16 homework. Beginner–intermediate. Verified 200, 2026-10-11.
- *HDLBits* — `https://hdlbits.01xz.net` — Circuits/Combinational "Arithmetic Circuits" drills the adder and overflow idioms used here, with instant judge feedback. Beginner. Verified 200, 2026-10-11.
- ChipVerify, *Verilog tutorial*. `chipverify.com/tutorials/verilog` — the `case`/mux and testbench idioms this chapter leans on, at reference depth. Beginner. Verified 200, 2026-10-11.
- Wikipedia, *Arithmetic logic unit* and *Two's complement*. `en.wikipedia.org/wiki/Arithmetic_logic_unit` and `en.wikipedia.org/wiki/Two's_complement` — for the historical record and a second derivation of the flag rules. Reference. Verified 200, 2026-10-11.

**WATCH INSTEAD**

1. *Building the ALU* — Ben Eater, 16:17, `https://www.youtube.com/watch?v=S-3fXU3FZQc` — the same block built from 74-series chips on a breadboard; watch the flag lines settle after each op, which is exactly our Figure 14.5. Matches §14.1–§14.4. Opened and confirmed 2026-10-11.
2. *How Computers Calculate — the ALU: Crash Course Computer Science #5* — CrashCourse, 11:09, `https://www.youtube.com/watch?v=1I5ZMmrOfnA` —

what an ALU is and why flags exist; lighter, faster, good before the Verilog. Matches §14.1–§14.3. Opened and confirmed 2026-10-11.

## Notes

The op set and compute-all-then-mux structure follow the Hack ALU of nand2tetris project 2 and the ALU treatment in Harris & Harris [12]; the flag vocabulary (Z N C V) follows the ARM convention so 'carry' after a subtract means no-borrow. Board pin numbers and the J5/J6 header map are from the Sipeed Tang Nano 9K rev-3674 schematic [9]; synthesis figures are a real `yosys` 0.69 `synth_gowin` run [15]. The RISC-V opcode mapping is seeded here and completed in ch16 per the approved decisions.

CHAPTER 15

# Registers, buses and the datapath

In ch14 the ALU answered questions. It could not *remember* anything — feed it $3 + 5$ and it hands you 8, but the 8 evaporates the moment you change the inputs. This chapter builds the machinery that makes answers stick: a register file to hold intermediate values, the bus wiring that lets blocks share one set of wires without fighting, program memory that turns a list of instructions into data, and the datapath itself — the arrangement of all these pieces around the ALU. The datapath is the figure this book has been building toward: once it is on the page, ch16 only has to add the control unit that drives its 11-bit control word. By the end you will have watched a ten-byte program execute, one control word at a time, on hardware you can synthesise.

**BEFORE YOU START**

Chapters 10 and 14. *Recap:* a CPU is an ALU, some registers, a memory, and a controller that sequences them each clock cycle.

**AFTER THIS CHAPTER YOU CAN**

- build a register file and a memory;
- explain who is allowed to drive a shared bus and when;
- separate the datapath from the control logic;
- trace one instruction through fetch, execute and write-back.

**HOW TO THINK ABOUT THIS**

A processor is surprisingly few parts: storage, a computing block, buses to move data, and a controller. The controller is the conductor, and the datapath is the orchestra. Separate the two in your mind, and the design stops looking like a tangle.

## 15.1 The register file

A register file is a small array of identical registers with two kinds of ports. The *write port* accepts an address, a value, and a clock edge — the addressed cell latches the value then. The *read ports* accept only an address and answer combinationally, all the time. Ours has eight 8-bit cells, one write port and two read ports, because the ALU needs two operands simultaneously.

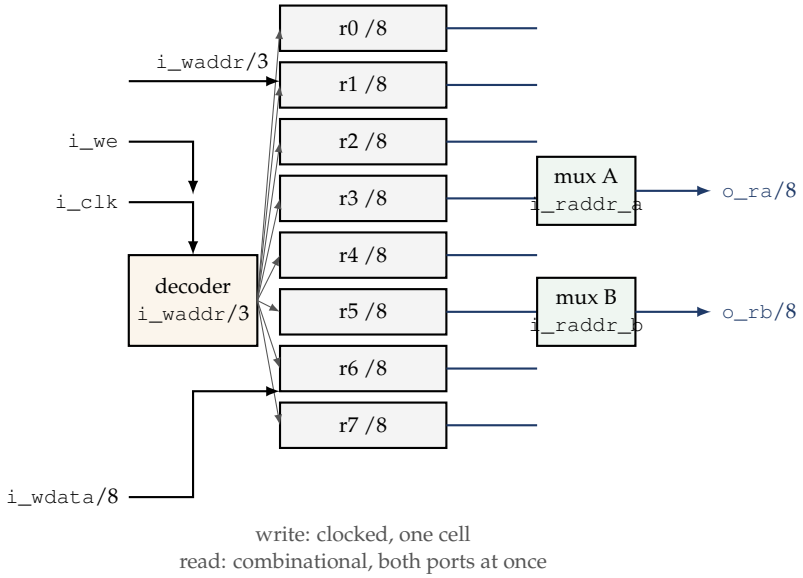



**Figure 15.1:** The 8×8 register file. A decoder picks the cell the write edge updates; two independent muxes let both ALU operands be read at once. Reads are combinational — no clock involved — because the ALU wants the operands *before* the edge that will write back its result. Regenerate: `figures/ch15/fig_regfile.tex`.

```
module regfile8 (
  input wire i_clk,
  input wire i_we,
  input wire [2:0] i_waddr,
  input wire [7:0] i_wdata,
  input wire [2:0] i_raddr_a,
  input wire [2:0] i_raddr_b,
  output wire [7:0] o_ra,
  output wire [7:0] o_rb
);
  reg [7:0] mem [0:7];
  assign o_ra = mem[i_raddr_a];
  assign o_rb = mem[i_raddr_b];
  always @(posedge i_clk) begin
    if (i_we) mem[i_waddr] <= i_wdata;
  end
endmodule
```

**Listing 15.1:** `code/ch15_regfile8.v -- one synchronous write port, two asynchronous read ports. The whole module is twenty lines; the conventions carry the weight.`

Two subtleties to notice before you ever read a bigger one. The writes are *synchronous* — the value lands on a clock edge, and `i_we` is what decides whether any cell changes. That write-enable line is the single most important signal in the whole machine: it is how the control unit says "remember this" versus "compute but keep nothing" — the difference between SUB and CMP from ch14. And the reads are *asynchronous*: changing an address changes the output immediately, which is why a combinational ALU can consume them in the same cycle.

**Example 15.1** (The register file, exercised)**.** `ch15_regfile8_tb.v` writes all eight cells, reads them back on both ports, and then checks the edge semantics directly — driving a write and verifying the read port still shows the *old* value a nanosecond before the edge:

```
PASS: regfile8 12 checks clean
```

**Listing 15.2:** Real run of `ch15_regfile8_tb.v`.

The "pre-edge still old / post-edge new" pair is the check that matters most: it proves writes respect the clock. An accidental combinational write-through would fail it instantly.

## 15.2 The bus, and who may drive it

Wire the register file's output, the ALU's output, and a memory's output all onto one 8-wire bundle and you have a *bus*. But only one of them may talk at a time, or the wires carry a fight between 0 and 1 that settles as neither — and in real silicon, as heat. Textbook CPUs solve this with *tri-state* drivers: each talker has an output-enable, and all but one output-enable are off, so the silent ones are electrically absent, not just quiet.

On an FPGA there is no third state. Inside the fabric every bus is a set of multiplexers: one line always drives each wire, and a select input chooses which talker it is. The idea is the same — one talker at a time — but the mechanism is a mux, not a driver that disappears. That is why our datapath writes `i_wb_sel` and `i_dst_sel` instead of output-enables.

**Checkpoint 15.1** Why doesn't the register file need an output-enable on its read ports? Because read ports never drive the bus — they are inputs to muxes, not talkers on it. Only *output* producers need steering.

## 15.3 Memory: where the program lives

The datapath needs two kinds of memory with opposite personalities. The *program ROM* holds the instruction stream: read-only, addressed by the program counter, loaded at build time. The *data RAM* holds scratch values: written and read by the running program.

```
// rom16: 16x8, combinational read -- the program is data in memory
reg [7:0] mem [0:15];
initial $readmemh("ch15_program.hex", mem);
assign o_data = mem[i_addr];

// ram16: 16x8, synchronous write AND read
```

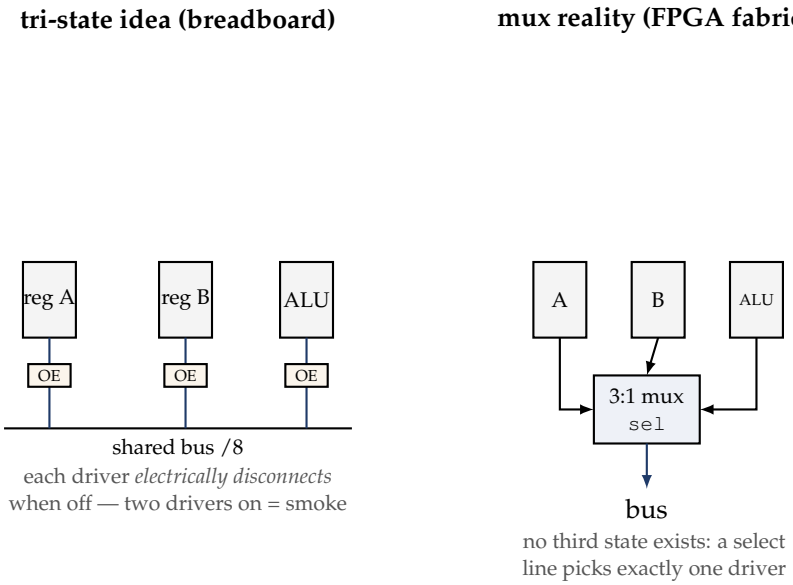



**Figure 15.2:** Same idea, two mechanisms. Left: tri-state drivers on a breadboard bus — each component can electrically let go. Right: on an FPGA the "bus" is a mux whose select line picks exactly one driver. Both guarantee one talker; only the mux version synthesises. Regenerate: `figures/ch15/fig_bus.tex`.

```
always @(posedge i_clk) begin
  if (i_we) mem[i_addr] <= i_wdata;
  o_rdata <= mem[i_addr];
end
```

**Listing 15.3:** `code/ch15_rom.v and ch15_ram.v -- the two personalities side by side. The ROM reads combinationally (fetch wants the instruction now); the RAM registers its read output, the pattern that maps onto BRAM.`

The RAM's extra `o_rdata <=` line is not decoration. Block RAMs inside the FPGA are synchronous memories: present an address and the data arrives *next* edge. Write the behavioural model with a registered output and the synthesiser maps it to the RAM primitives; write a combinational read and it may land on a pile of flops or distributed LUT-RAM instead. The figures in the next section show what actually happened to ours.

**Example 15.2** (Program ROM: the program is data)**.** `ch15_mem_tb.v` reads the demo program straight back out of `rom16` — the first byte at address 0 is `0x10`, our LDI opcode, and the byte at 1 is its immediate, `0x05`:

```
PASS: rom16 + ram16 8 checks clean
```

**Listing 15.4:** Real run of `ch15_mem_tb.v -- five ROM reads plus three RAM read/write/latency checks.`

Sit with that for a second. The ROM does not know it holds a program. Address in, byte out — the *meaning* of `0x10` is created entirely by the pc stepping through addresses. This is the von Neumann idea in miniature, and it is why "a memory of instructions read by an address counter" is the whole trick of the stored-program computer.

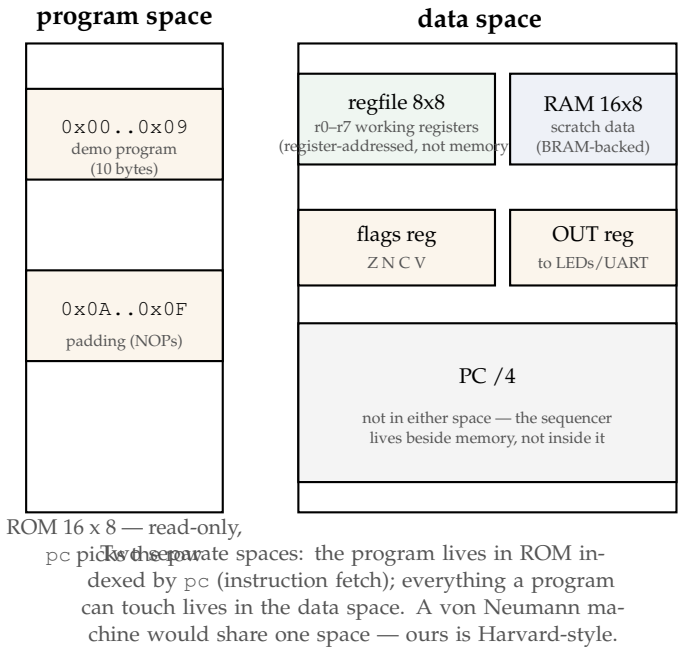



**Figure 15.3:** The machine's map. Program space (ROM, pc-indexed) and data space (registers, RAM, OUT) are separate — a Harvard-style split. The PC is not in either space: it is the sequencer that points *into* program space. Regenerate: `figures/ch15/fig_memmap.tex`.

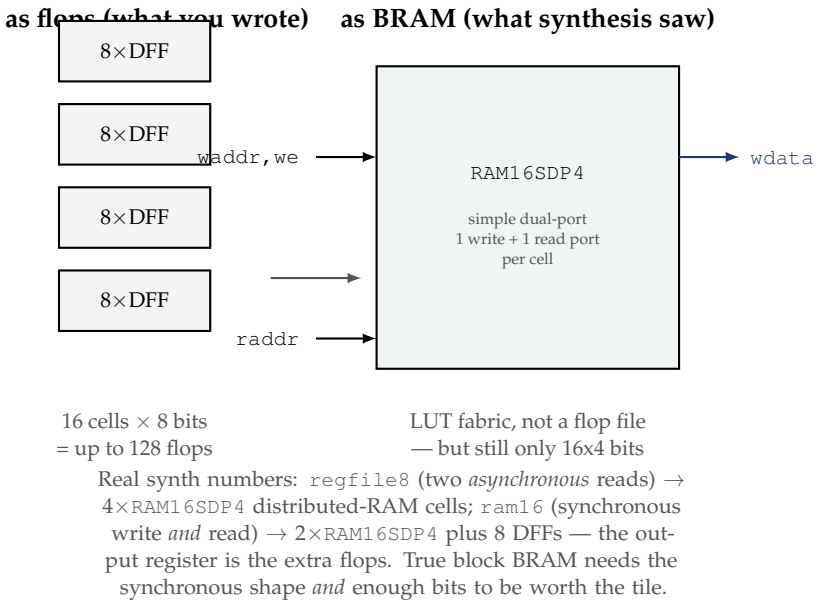



**Figure 15.4:** What synthesis actually did with our two memories — the real `synth_gowin` numbers. The register file's *asynchronous* reads map onto distributed LUT-RAM cells (`RAM16SDP4`); `ram16`'s all-synchronous shape uses half as many cells plus eight DFFs — the output register is exactly the BRAM-style shape. True block BRAM is a bigger tile: it wants this synchronous shape and thousands of bits to be worth spending. Regenerate: `figures/ch15/fig_bram.tex`.

## 15.4 Datapath versus control

Here is the split that makes CPUs designable. The *datapath* is every register, memory, mux and ALU the data flows through — the "what can happen." The *control unit* is the sequencer that drives all the select lines and write-enables in the right order — the "what happens now." This chapter builds the datapath and leaves a hole where the control unit goes; ch16 fills the hole.

Our datapath needs eleven control bits:

| field | chooses |
|---|---|
| `i_alu_op[2:0]` | which of the 8 ALU ops (ch14's contract) |
| `i_wb_sel[1:0]` | write-back source: ALU / imm byte / port A / port B |
| `i_dst_sel` | write destination: r0 (=A) or the operand register |
| `i_we` | register-file write-enable |
| `i_flags_we` | latch Z N C V this step |
| `i_out_we` | latch the OUT register |
| `i_pc_mode[1:0]` | pc +1 / +2 / jump / hold |

Count them: $3+2+1+1+1+1+2=11$. That number is not trivia — it is the answer to "how wide is the control word," the question every control-unit design starts with.

**Exercise 15.1** (easy)**.** Which two control bits would you group into one field, and what property of the ISA makes it safe? (Hint: what does `CMP` share with `SUB`?)

## 15.5 The cycle: fetch, execute, write back

A CPU repeats three moves forever: *fetch* the instruction at `pc`, *execute* it through the datapath, *write back* whatever it produced (a register value, a flag, a pc change). In our partial datapath the "execute" step is where the ALU, register file and write-back mux do their work — all in the same cycle, because combinational logic settles before the edge.

The conventions we freeze here, so ch16 has no choices left to make: the instruction byte is `{op[7:4], operand[3:0]}`; the byte after it (`rom[pc+1]`) is an immediate when an op needs one; `r0` doubles as the accumulator `A` that arithmetic ops read and write; `MOV rd` stores A into a register; jumps take their target from the operand nibble.

**Example 15.3** (Hand-tracing one full cycle)**.** Take step 5 of the demo program, `ADD r1` with A$=3$ and r1$=5$:

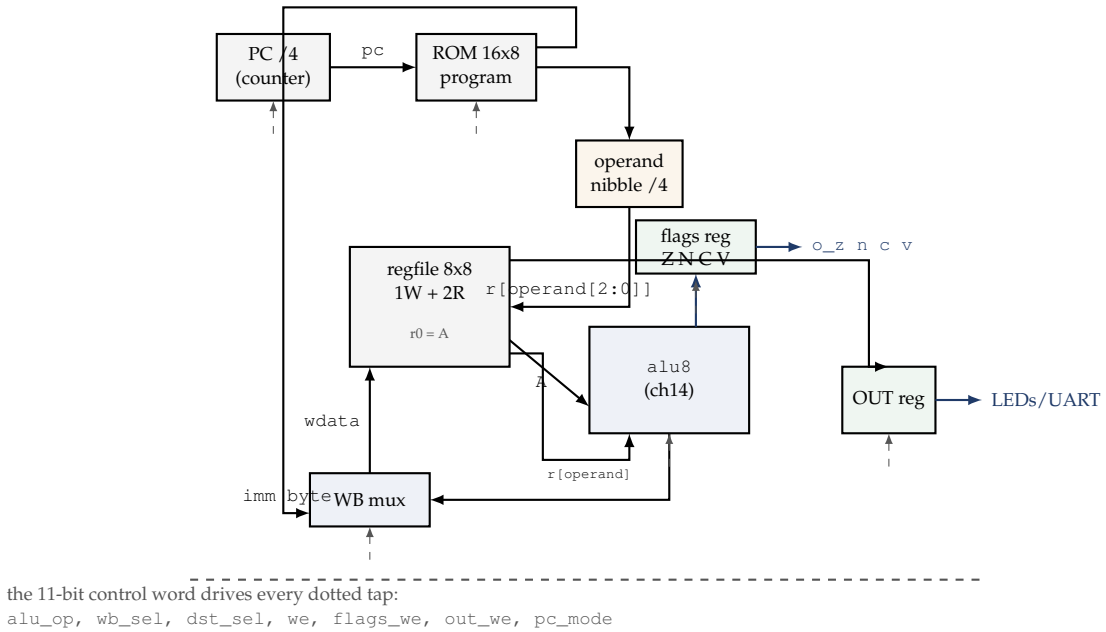



**Figure 15.5:** The datapath: the figure this book builds toward. PC feeds the ROM; the instruction's operand nibble picks a register and the immediate path; the ALU combines r0 (A) with the operand; the write-back mux routes the result; flags and OUT latch on their own enables. Every dotted tap is a bit of the 11-bit control word — which is all ch16's FSM has to drive. Regenerate: `figures/ch15/fig_datapath.tex`.

| signal | driven | why |
|---|---|---|
| `i_alu_op` | `ADD` | opcode 3 maps to ALU op 000 |
| `i_wb_sel` | `ALU` | the result is the ALU's |
| `i_dst_sel` | `A` | accumulator ops write r0 |
| `i_we` | 1 | a real write-back |
| `i_flags_we` | 1 | arithmetic updates flags |
| `i_pc_mode` | `+1` | single-byte instruction |

On the clock edge: the ALU has already settled on $3 + 5 = 8$, the write port latches it into r0, the flags register captures ZN CV (0000), and pc becomes 7. Three registers changed on one edge — that *is* the cycle.

**Example 15.4** (The bench plays control unit). `ch15_datapath_tb.v` drives that exact table, one control word per step, and checks state after every edge. The real trace:

```
t=26000 pc= 2 instr=21 -> a=05 rb=xx flags z0 n0 c0 v0
t=36000 pc= 3 instr=10 -> a=05 rb=05 flags z0 n0 c0 v0
t=46000 pc= 5 instr=22 -> a=03 rb=xx flags z0 n0 c0 v0
t=56000 pc= 6 instr=31 -> a=03 rb=05 flags z0 n0 c0 v0
t=66000 ADD: a=08 z0 n0 c0 v0
PASS: datapath executed 8 control-word steps clean
```

**Listing 15.5:** Real run of `ch15_datapath_tb.v`. Watch `rb=xx` on the first MOV -- an unwritten register reads x, which is the honest answer.

| step | instr | wb_sel | dst_sel | pc_mode | writes |
|---|---|---|---|---|---|
| 1 | LDI A,5 | IMM | A | +2 | r0←0x05 |
| 2 | MOV r1 | PA | reg | +1 | r1←A |
| 3 | LDI A,3 | IMM | A | +2 | r0←0x03 |
| 4 | MOV r2 | PA | reg | +1 | r2←A |
| 5 | ADD r1 | ALU | A | +1 | r0←8, flags |
| 6 | SUB r1 | ALU | A | +1 | r0←3, flags |
| 7 | OUT | — | — | +1 | out←A |
| 8 | HALT | — | — | HOLD | — |

Every row is one clock edge and one 11-bit control word. `we`, `flags_we` and `out_we` are the write-enables the column hides: `we`=1 on rows 1–6, `flags_we`=1 on rows 5–6 only, `out_we`=1 on row 7. In ch16 an FSM emits this table from the opcode nibble; here the testbench plays it by hand — the same sequence, two owners.

**Figure 15.6:** The whole demo program as a control-signal table — one control word per row, one clock edge per row. This table *is* what a control unit emits; the only difference between this chapter and the next is who writes the rows. Regenerate: `figures/ch15/fig_cycle.tex`.

Every line is a cycle the figure table predicted: pc marching through ROM addresses, A being loaded and moved through registers, the ADD producing 8 on the edge that also latches the flags. Nothing in this trace is hypothetical — it is the machine working.

**Example 15.5** (What it costs on the die)**.** Strategy: `synth_gowin` the whole datapath and read the stat:

```
=== datapath ===
     349 wires 610 wire bits 19 ports
     416 cells
      25 ALU 1 DFF 6 DFFR 16 DFFRE
       4 RAM16SDP4 62 LUT1 16 LUT2 30 LUT3
      81 LUT4 68 MUX2_LUT5 27 MUX2_LUT6 10 MUX2_LUT7
       3 MUX2_LUT8 13 IBUF 48 OBUF 4 $scopeinfo
```

**Listing 15.6:** `synth_gowin stat for datapath -- real output, trimmed.`

Read it like the ch14 stat: **25 ALU** cells — the two ALU adders plus the pc incrementer. **23 flip-flops** (1 DFF + 6 DFFR + 16 DFFRE): the pc's 4 bits, the flags' 4 bits, the OUT register's 8 — the memories-with-enables, accounted for. **4 RAM16SDP4** are the register file mapped onto distributed RAM, as Figure 15.4 explained. And **108 mux cells** are the real price of the datapath pattern — WB mux, dst select, bus muxes: the wires are cheap, the choosers are the silicon.

**Honest marking.** The register-file, memory and datapath benches and both synthesis stats were run and captured on this machine. No board was attached; the datapath has no dedicated board demo — the full CPU gets one in ch16, where LEDs and UART have something worth showing.

## 15.6 Common mistakes

**TWO TALKERS ON ONE BUS**

If two write sources can reach the same register input and both are enabled in the same step, the hardware wires a fight. Rule: every destination has *one* write-back mux; every producer reaches it through a select, never a parallel tie. If your design has two drivers on one wire, there is no sim that can save you — only the architecture can.

**EXPECTING A REAL TRI-STATE INSIDE THE FPGA**

`assign bus = oe ? val : 8'bz;` compiles; the synthesiser may even map it. But there is no third state in the routing fabric — the tool silently turns it into pull-ups or muxes you did not design. Draw the mux on purpose; never write `1'bz` inside a module's internal wiring.

**COMBINATIONAL READS WHERE THE SILICON WANTS SYNC**

A register file reading asynchronously through a big array makes the sim easy and the area ugly: async reads can't use block RAM at all. Small files (ours is 64 bits) are fine as LUT-RAM; if you need a big async-read array, the honest options are flops or a redesign — not hope.

**FORGETTING A WRITE-ENABLE IS A DESIGN CHOICE**

`we=1` forever is how accidental designs happen: the file updates every cycle whether the instruction asked or not. Every state element in our datapath — regfile, flags, OUT, pc — has its own enable, and *defaults to not-writing*. Silence is the safe state.

**READING AN UNWRITTEN REGISTER AND BELIEVING IT**

The demo's first `MOV r1` step reads `rb=xx` because r1 was never written. On a real machine, uninitialised registers are not zero — they are whatever they powered up as. Programs must write before they read; testbenches should check that the `x` shows up exactly where it should.

**ASYNC INPUTS STRAIGHT INTO A DATAPATH**

A button or switch feeding a datapath signal can change *during* a clock edge — metastability, the same lesson as ch10's synchronisers. Any signal crossing into your clock domain passes through a two-flop synchroniser first, or your clean datapath learns analog electronics at the worst moment.

**Exercise 15.2** (medium)**.** The demo program could not write to r4–r7 without changing encoding. Say why (which instruction field is too narrow), and give the minimal encoding change that frees the top four registers.

**Exercise 15.3** (medium)**.** `ram16` reads one edge after the address. If a program does `LDI A; then uses the value`, how many wasted cycles does a naive control unit burn per memory read, and what does the ch16 FSM do with that cycle?

**Exercise 15.4** (hard)**.** Our pc counts bytes, so a two-byte LDI needs `pc+2`. Design the alternative: pc counts *instructions* (a small table maps instruction index to byte address). What does it cost and what does it fix?

## 15.7 Summary

| Thing | The rule of thumb |
|---|---|
| register file | 1 clocked write port + N combinational read ports; the write-enable is the "remember" bit |
| bus | one talker at a time: tri-state on breadboards, muxes in FPGA fabric |
| program memory | instructions are just bytes; the pc is an address counter reading them |
| BRAM inference | synchronous write *and* read; async reads force LUT-RAM or flops |
| datapath/control | datapath = what can happen; control = what happens now — the 11-bit control word is the seam |
| the cycle | fetch, execute, write-back — one edge, three register updates |

## Additional exercises

**Exercise 15.5** (easy)**.** Why does the register file's read side need no clock and no enable?

**Exercise 15.6** (easy)**.** How many bits wide is our datapath's control word, and which fields does it split into?

**Exercise 15.7** (easy)**.** In the memory map, which space does the pc live in? Trick question — explain the answer.

**Exercise 15.8** (easy)**.** A register file is "8×8". What do the two 8s count?

**Exercise 15.9** (medium)**.** Modify `regfile8` so r0 always reads as 0 (the riscv-zero convention). Write the two-line change and say what it would break in our accumulator-is-r0 convention.

**Exercise 15.10** (medium)**.** Design the control word for a `CMP rX` step (A−*rX*, flags latched, nothing written). Which write-enables must be 0 and what does that imply about the ALU's result?

**Exercise 15.11** (medium)**.** The tb drives control words on `negedge` and checks after `posedge`. Explain the race this avoids, citing ch12.

**Exercise 15.12** (medium)**.** Widen the register file to 16×8: which fields of the instruction word break, and what is the minimal re-encoding that keeps one-byte instructions?

**Exercise 15.13** (hard)**.** Write the truth table of `i_pc_mode`'s four encodings, including what the FSM must guarantee when `HALT` holds `pc` forever. What happens at power-on before the first HALT?

**Exercise 15.14** (hard)**.** Design the write-enable-trace bug we hinted at: a control table that sets `we=1` during the OUT step. What does the regfile see, and which tb check catches it first?

**Exercise 15.15** (hard)**.** Count the muxes in Figure 15.5 by hand, then compare with the 108 `MUX2_LUT*` cells in the stat report. Explain the ratio — what is the big mux the diagram hides?

**Exercise 15.16** (hard)**.** Our datapath reads the immediate byte through a second `rom16` instance at `pc+1`. Give two reasons that's honest, and one case where it lies.

**Exercise 15.17** (hard)**.** A teammate proposes dropping the flags register and reading ALU flag outputs directly in the JZ step. Explain why that fails, referencing when the flags are computed versus when they're sampled.

**Exercise 15.18** (hard)**.** Extend the datapath sketch with a stack pointer register and a 2-byte-deep call stack. Which existing fields handle `CALL/RET` without new encoding space?

**Exercise 15.19** (hard)**.** `regfile8`'s async reads feed a combinational ALU feeding a write-back mux — a long combinational chain. Sketch the two-stage version (operands registered) and name what it buys and what it costs in control complexity.

**Exercise 15.20** (hard)**.** The stat report shows 48 OBUF on `datapath`. Reconcile that number with the port list — what are the 48, and why does a block you'd never probe need outputs at all in a top-level stat?

**Exercise 15.21** (hard)**.** Write the `ram16` read-after-write hazard formally: if `i_we` and a read of the same address land on the same edge, what does `o_rdata` show and which BRAM policy does it implement?

**Exercise 15.22** (hard)**.** The operand nibble is shared: register index *and* jump target *and* part of the LDI immediate path. Prove it can't hold an 8-bit immediate in one byte, and state the design rule that keeps encoding pressure sane.

## Hints and answers

**Control-bit grouping (easy, inline)** `dst_sel` folds into `wb_sel` if arithmetic ops always write A — ours do; a CMP's "no write" already lives in `we`.

**Why reads are free (easy)** Read ports are mux outputs: address in, value out, nothing stored — only writes need an edge and an enable.

**Control word (easy)** 11 bits: alu_op3 + wb_sel2 + dst_sel1 + we + flags_we + out_we + pc_mode2.

**PC's space (easy)** Neither — the pc is an index *into* program space, part of the sequencer, not of memory.

**8×8 (easy)** 8 registers deep, 8 bits wide each.

**r0-hardwire (medium)** `assign o_ra = (addr==0) ? 0 : mem[addr]` — it would break `A`=r0, because A could never hold a value.

**CMP word (medium)** alu_op=CMP, wb_sel=ALU, we=**0**, flags_we=1, pc=+1 — the result computes and dies on the mux; flags are the verdict.

**negedge drive (medium)** driving stimulus on the opposite edge avoids the tb/DUT race of ch12: DUT samples old values at posedge, tb's new values apply after.

**16×8 (medium)** operand[3:0] can already name 0–15 — widen `i_raddr`/`i_waddr` to 4 bits; the encoding survives, only registers 8+ become usable.

**pc_mode truth (hard)** HOLD must be reachable *after* the halt instruction lands, or pc wraps to 0 and re-executes — the power-on path is reset→fetch, the same first step every time.

**OUT we bug (hard)** the regfile latches wb_data wherever dst points — usually A gets overwritten with alu_y; the post-OUT A check fires first.

**Mux census (hard)** the diagram shows 5 muxes (WB, dst, B-src, pc-next, flag-sel); 108 LUT muxes is the *bit-level* count — each of our 8-bit muxes is eight 1-bit muxes deep.

**Second rom16 (hard)** honest because ROM reads are pure and the immediate *is* the next byte; it lies if an op's operand is meant as an address, not data — the second instance can't fetch operands from data space.

**No flags reg (hard)** ALU flags vanish when the inputs change; JZ reads them a step after the arithmetic settled — unregistered flags would reflect the wrong operation.

**Stack (hard)** `CALL` = pc_mode jump + write pc+1 to stack; `RET` = jump to stack top — fits in pc_mode plus one stack-pointer we.

**Two-stage (hard)** registers operands on an edge, ALU works next cycle: halves the combinational depth, but every op takes 2 cycles and the control table doubles.

**48 OBUF (hard)** pc4 + instr8 + imm8 + a8 + rb8 + out8 + flags4 = 48: in a top-level stat, debug outputs cost real pins — strip them for the CPU build or let hierarchy absorb them.

**RAW hazard (hard)** `o_rdata` shows the *old* value — read-before-write, the BRAM default; write-first policies exist but ours didn't ask for one.

**Nibble budget (hard)** operand[3:0] has 4 bits; imm8 needs 8 — the LDI immediate must come from the next byte, which is exactly why pc has a +2 mode: fetch two bytes per instruction that needs it.

## Further reading

- Harris & Harris, *Digital Design and Computer Architecture*, 2nd ed., ch. 7 (microarchitecture).
  `pages.hmc.edu/harris/ddca/` — the single-cycle datapath our figures follow; the control/datapath split is theirs. Intermediate. Verified 200, 2026-10-11.
- Nisan & Schocken, *The Elements of Computing Systems*, projects 3–5 (RAM, registers, the Hack computer).
  `nand2tetris.org/project03` and `nand2tetris.org/project05` — the lad-

der from DFF to a working machine; their control-word table inspired Figure 15.6. Intermediate. Verified 200, 2026-10-11.

- *HDLBits* — `hdlbits.01xz.net` — Circuits/Sequential "Latches and Flip-Flops" through "Shift Registers" drills the register idioms this chapter builds on. Beginner. Verified 200, 2026-10-11.
- Lushay Labs, *Our First CPU* (Tang Nano series). `learn.lushaylabs.com/tang-nano-series` — a register file and datapath on this exact board, with working code. Beginner–intermediate. Verified 200, 2026-10-11.
- ChipVerify, *Verilog tutorial* — memory arrays, FSM glue and the register-transfer idioms used here. `chipverify.com/tutorials/verilog` — Beginner. Verified 200, 2026-10-11.
- Wikipedia, *Datapath* and *Register file* — terminology and the wider taxonomy (multi-issue, banks, forwarding) this small machine points toward. Reference. Verified 200, 2026-10-11. `en.wikipedia.org/wiki/Datapath` `en.wikipedia.org/wiki/Register_file`

**WATCH INSTEAD**

1. *Bus architecture and how register transfers work — 8-bit register, Part 1* — Ben Eater, 9:35, `https://www.youtube.com/watch?v=QzWW-CBugZo` — the bus/tri-state section taught on real breadboard wiring; watch the register transfer happen on the bus before reading our mux version. Matches §15.2–§15.4. Opened and confirmed 2026-10-11.
2. *Registers and RAM: Crash Course Computer Science #6* — CrashCourse, 12:16, `https://www.youtube.com/watch?v=fpnE6UAfbtU` — from latch to register file to addressable memory, the same ladder in 12 minutes. Matches §15.1 and §15.3. Opened and confirmed 2026-10-11.

## Notes

The datapath/control split and the single-cycle structure follow Harris & Harris ch. 7 [12]; the register-file interface and control-word table follow the nand2tetris projects 3–5 conventions. Memory-inference numbers are real `yosys` 0.69 `synth_gowin` runs [15]; the Harvard split (separate program/data spaces) is the book's own encoding decision, seeded here and completed by the ch16 ISA table.

CHAPTER 16

# A small CPU that runs real programs

Ch14 gave us an ALU that computes; ch15 gave it registers to remember with and buses to share the wires. What is missing is the umpire: a decoder that reads each instruction byte and drives the datapath's control lines so the right thing happens at the right edge. Add that and the pile of parts becomes a computer — a real one, small but complete, that assembles and runs real programs: it counts, it multiplies, and it prints the Fibonacci sequence byte by byte on a serial line. This chapter builds the C1 machine's last block, then steps back to ask what we actually made — and what we skipped on the way to RISC-V.

**BEFORE YOU START**

Chapters 14–15. *Recap:* an instruction set architecture (ISA) is the contract between hardware and software: the list of instructions the CPU understands and what each one does.

**AFTER THIS CHAPTER YOU CAN**

- read a frozen ISA specification;
- implement fetch, decode and execute for it in Verilog;
- write a small assembler;
- run a real program on your CPU. **Book 7** starts from this CPU and maps it to RISC-V.

**HOW TO THINK ABOUT THIS**

A CPU is a loop: fetch an instruction, work out what it means, do it, repeat. The instruction set is the contract between the programmer and the machine, so freeze it before writing the hardware. Once your own CPU runs a program, you understand what every computer is doing underneath.

## 16.1 The ISA, frozen

Every CPU design starts with a contract: what instructions exist, what each one costs, and how they are encoded. Ours is ten opcodes over two fields — the byte ch15 seeded, plus one honest upgrade.

| opcode | mnemonic | operand | bytes | effect |
|---|---|---|---|---|
| `0x0` | NOP | — | 1 | nothing |
| `0x1` | LDI | — | 2 | `A ← imm8` |
| `0x2` | MOV | `rd` | 1 | `r[d] ← A` |
| `0x3` | ADD | `rs` | 1 | `A ← A + r[s]`, flags |
| `0x4` | SUB | `rs` | 1 | `A ← A − r[s]`, flags |
| `0x5` | AND | `rs` | 1 | `A ← A & r[s]`, flags |
| `0x6` | JMP | — | 2 | `pc ← imm8` |
| `0x7` | JZ | — | 2 | `pc ← imm8` if Z |
| `0x8` | OUT | — | 1 | `out ← A` |
| `0x9` | HALT | — | 1 | pc holds forever |
| `0xA` | MOVA | `rs` | 1 | `A ← r[s]` |

Byte 0 is `{op[7:4], operand[3:0]}`; the operand nibble names registers `r0`–`r7` (r0 doubles as the accumulator `A`). Two-byte instructions carry an immediate byte at `pc+1` — including jump targets, the one place ch15's seeded encoding needed a wider field: a 4-bit target cannot reach past `0x0F`.

**Figure 16.1:** The C1 instruction set. Eleven words of vocabulary is all a computer needs to be one. Regenerate: `figures/ch16/fig_isa.tex`.

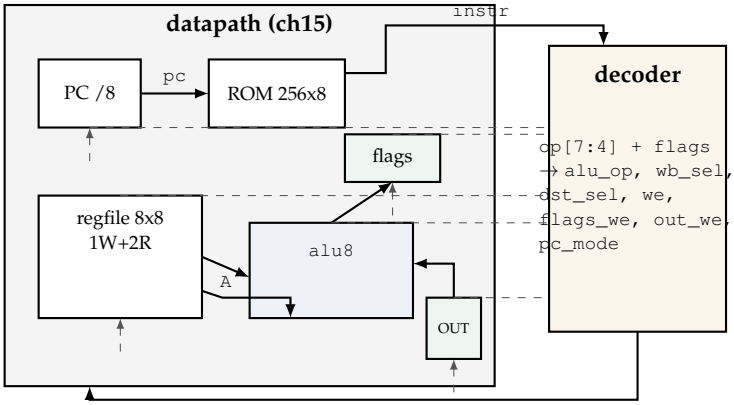
The upgrade deserves a sentence of its own. ch15 proposed jump targets in the operand nibble — four bits, sixteen addresses. The multiply program alone is twenty-five bytes: a 4-bit jump cannot reach most of it. So `JMP` and `JZ` became two-byte instructions whose second byte is the whole target. The rule that survives: *any field that addresses the program wants as many bits as the program space*, and our program space is now 256 bytes. Everything else — register fields, the ALU contract, the control word — is exactly as ch14 and ch15 left it.

## 16.2 Fetch, decode, execute — in one cycle

Textbooks draw fetch, decode and execute as three phases. Ours all happen inside one clock edge, and it is worth seeing why that is not cheating:

1. **Fetch** is `rom[pc]` — combinational read, so the instruction is present all cycle.
2. **Decode** is a `case` statement on `instr[7:4]` — combinational, so the control word is present all cycle.
3. **Execute** is the datapath settling — combinational — then the edge: registers latch, flags latch, pc advances.

One cycle per instruction, start to finish. The catch is the one we flagged in ch15: the combinational path from `pc` through the ROM, the decoder, the regfile, the ALU and the write mux into the register file's data input is long. Real CPUs fix it by splitting the cycle into stages — which is exactly what a pipeline is, and why this design will never win a frequency contest (Section 16.6 is honest about that).




The centrepiece: every dashed line is a control-word tap. The decoder is combinational — fetch, decode and execute all happen inside one cycle, which is why our "master FSM" has only two states.

**Figure 16.2:** The complete C1 machine — the book's centrepiece. The ch15 datapath on the left gains a decoder on the right; the dashed taps are the same 11-bit control word, now driven by hardware instead of by a testbench. Regenerate: `figures/ch16/fig_cpu.tex`.

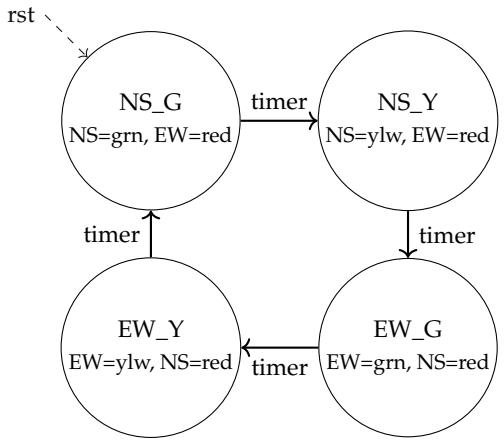



**Figure 16.3:** The master FSM — two states, deliberately small. RUN does one whole instruction per edge because fetch/decode/execute are combinational; HALT parks the pc. Regenerate: `figures/ch16/fig_fsm.tex`.

## 16.3 The decoder: a case statement with a job title

Here is the entire control unit, from `code/ch16_cpu8.v`:

```
always @(*) begin
  rf_we = 0; flags_we = 0; out_we = 0;
  wb_sel = 2'b00; dst_sel = 0; pc_mode = 2'b00; alu_op = 3'b000;
  case (op)
    4'h1: begin wb_sel=2'b01; rf_we=1; pc_mode=2'b01; end // LDI
    4'h2: begin wb_sel=2'b10; dst_sel=1; rf_we=1; end // MOV rd
    4'h3: begin alu_op=3'b000; rf_we=1; flags_we=1; end // ADD
    4'h4: begin alu_op=3'b001; rf_we=1; flags_we=1; end // SUB
    4'h5: begin alu_op=3'b010; rf_we=1; flags_we=1; end // AND
    4'h6: pc_mode = 2'b10; // JMP
    4'h7: pc_mode = flags[3] ? 2'b10 : 2'b01; // JZ
    4'h8: out_we = 1; // OUT
    4'h9: pc_mode = 2'b11; // HALT
    4'hA: begin wb_sel=2'b11; rf_we=1; end // MOVA
    default: ; // NOP
  endcase
end
```

**Listing 16.1:** The decoder: instruction opcode in, 11-bit control word out — combinational, so the whole instruction completes on the next edge. From `code/ch16_cpu8.v`.

Three things to read, slowly. **Every row is a control word** — match it against the ch15 table line by line; the hardware writes the same rows the testbench played by hand. **Defaults first**: every enable starts at 0, so an unrecognised opcode is a NOP, not a crash — "silence is the safe state" again. **`JZ` reads the flags register**, not the ALU's live outputs: the Z flag was latched by the previous arithmetic step, which is the only honest answer "did the last result equal zero."

**Example 16.1** (Hand-trace before the machine)**.** Step through `A4 47 24` from the Fibonacci loop — `MOVA r4; SUB r7; MOV r4`:

| byte | decoded as | control word (key fields) | edge writes |
|---|---|---|---|
| a4 | MOVA r4 | wb=PB, dst=A, we=1, pc+1 | r0←r4 |
| 47 | SUB r7 | alu=001, we=1, fw=1, pc+1 | r0←A−r7, Z NCV |
| 24 | MOV r4 | wb=PA, dst=r4, we=1, pc+1 | r4←A |

Three cycles: load counter into A, decrement it (the SUB latches Z — the flag JZ consults next), store it back. The accumulator-bottleneck rhythm is visible: every operation must pass through A, which is exactly what a real instruction set costs when the encoding budget is one byte.

**Checkpoint 16.1** Why does the flags register exist at all? Try removing it: the JZ step reads `alu_z` live — but by then the ALU inputs have moved on. Flags must be *latched by* the arithmetic step, not recomputed by the branch step. That is the whole reason.

## 16.4 The assembler

Writing hex bytes by hand is how you learn encoding; writing a 60-line Python script is how you stop. `code/ch16_asm8.py` does two passes: first it walks the file counting bytes to give each `label:` an address, then it emits `{op,operand}` nibbles and immediate bytes.

```
OPS = {'NOP':0x0,'LDI':0x1,'MOV':0x2,'ADD':0x3,'SUB':0x4,'AND':0x5,
     'JMP':0x6,'JZ':0x7,'OUT':0x8,'HALT':0x9,'MOVA':0xA}
TWO = {'LDI','JMP','JZ'} # ops carrying an imm byte
# pass 1: pc counts 2 bytes for TWO-ops, 1 otherwise -> label addr
# pass 2: out = (op<<4)|operand, then imm byte if needed
```

**Listing 16.2:** The assembler's core — from `code/ch16_asm8.py`. Two passes: labels first, bytes second.

**Example 16.2** (Program 1: count on the LEDs)**.** `ch16_count.asm` assembles to nine bytes:

```
LDI A,1 ; MOV r7 -> 10 01 27
LDI A,0 ; A = 0 -> 10 00
loop: ADD r7 ; A += 1 -> 37
     OUT -> 80
     JMP loop -> 60 06
```

**Listing 16.3:** `ch16_count.asm and its assembly -- ch16_count.hex.`

Each lap: A grows by 1, OUT shows it, jump back to byte 6. On the board this is the visible demo — OUT drives the LED bank and the UART stream.

**Example 16.3** (Program 2: multiply by repeated addition)**.** `ch16_mult.asm` computes $7 \times 6$: a register counts down from 6 while a second register accumulates 7 each lap. The loop is the same decrement-jump-if-zero idiom as Fibonacci — the `SUB` both computes and sets the flag the `JZ` reads.

The bench runs it end to end. Real output, trimmed:

```
PASS: 7*6 printed 42
mult run halted at pc=24
```

**Listing 16.4:** `ch16_cpu8_mult_tb.v running mult.hex -- the machine prints 42 and parks.`

**Example 16.4** (Program 3: Fibonacci, byte by byte)**.** `ch16_fib.asm` (the listing lives in `code/`) prints each term mod 256. The testbench decodes every OUT write and checks the whole sequence — the real run:

```
OUT 1 ok (pc=14) OUT 13 ok (pc=14)
OUT 1 ok (pc=14) OUT 21 ok (pc=14)
OUT 2 ok (pc=14) OUT 34 ok (pc=14)
OUT 3 ok (pc=14) OUT 55 ok (pc=14)
OUT 5 ok (pc=14) OUT 89 ok (pc=14)
OUT 8 ok (pc=14) OUT 144 ok (pc=14)
PASS: cpu8 printed Fibonacci 1..144 then halted (12 OUT bytes)
ALL CHECKS PASS
```

**Listing 16.5:** `ch16_cpu8_tb.v running fib.hex -- twelve correct OUT bytes, then HALT.`

Twelve instructions executed per term, roughly sixty cycles of real hardware work — and the bench verified every byte against the expected sequence, not just the last one.

## 16.5 Out the door: LEDs and UART

The OUT register is where the program meets the world. The board top file (`code/`) feeds the six onboard directly, and every OUT write also launches a UART frame through the BL702 bridge chip — so the same byte appears on the LED bank and in a serial monitor. `ch16_uart_tx8.v` is the smallest honest transmitter: a four-state FSM shifting out a start bit, eight data bits LSB-first, and a stop bit, one bit per baud interval.

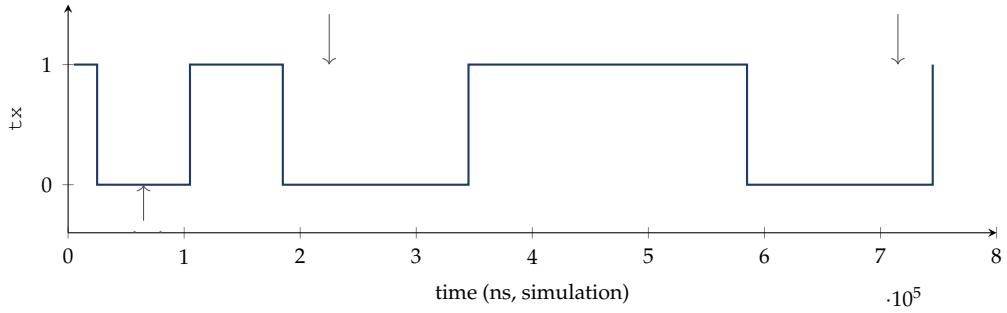
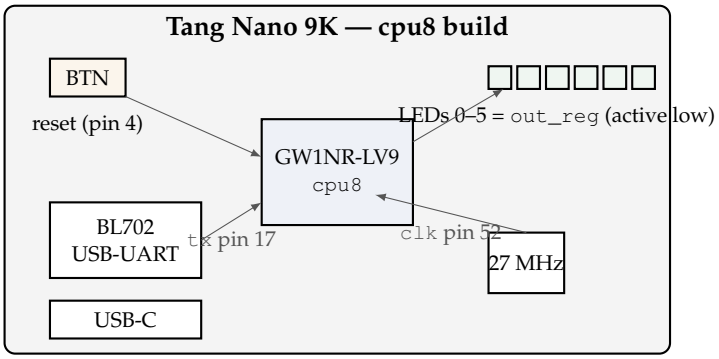



**Figure 16.4:** A real UART frame on the wire — the testbench's own VCD output for byte `0x39`: start low, eight data bits LSB-first, stop high. The companion bench decodes the same line the way a real receiver would (wait for the start edge, sample mid-bit) and passes. Regenerate data: `figures/ch16/uart.dat`.




Wiring it runs: `i_clk27` 52, `i_btn` 4, `o_led[0..5]` 10–16, `o_uart_tx` 17
(the BL702 bridge: open a serial monitor at 115200 and the OUT bytes arrive).

**Figure 16.5:** The C1 machine on the Tang Nano 9K: clock, reset button, LED bank, and the UART bridge that carries OUT bytes to a serial monitor. Regenerate: `figures/ch16/fig_board16.tex`.

**Example 16.5** (What the whole machine costs)**.** The real `synth_gowin` stat for `cpu8`, trimmed:

```
=== cpu8 ===
     302 cells
      30 ALU 8 DFF 1 DFFR 17 DFFRE
       4 RAM16SDP4 15 LUT1 21 LUT2 106 LUT3+
      51 MUX2_LUT* 2 IBUF 34 OBUF
```

**Listing 16.6:** A working CPU in 302 cells — less than 4% of the smallest FPGA this book uses.

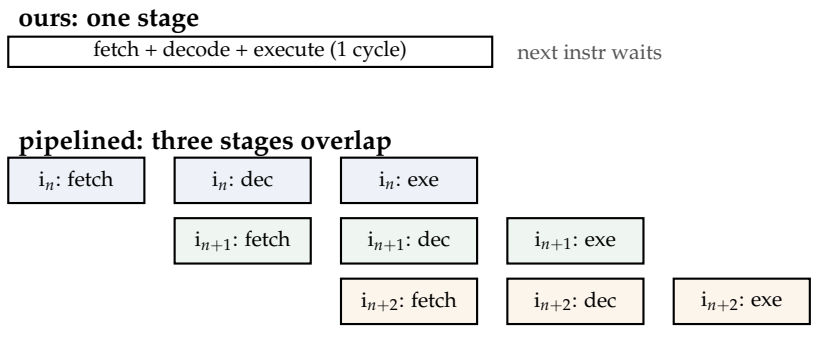
Reading it against ch14's ALU stat: 30 ALU cells (the ALU plus the pc adder), 26 flip-flops (pc8 + flags4 + out8 + divider bits), and the register file mapped onto the same 4 distributed-RAM cells. An entire stored-program computer for 302 cells — this is why "a CPU is a finite state machine with very good bookkeeping" is only half a joke.

**Honest marking.** The Fibonacci run, the multiply run, the count assembly, the UART frame decode and the synth stat are all real outputs captured on this machine. The board build (`top_cpu9k` + `ch16_cpu9k.cst`) is complete and lints clean, but no board was attached in this session: pin numbers follow the Sipeed rev-3718 schematic, and the clock divider brings the CPU to ∼6 Hz so the LED count is visible — that last step is verified by construction, not by hardware.

## 16.6 What this CPU is missing

Being honest about the gaps is the point of building small:

- **No stack.** `CALL`/`RET` need somewhere to put the return address; our pc has one mode, not two.
- **No interrupts.** Nothing external can redirect the pc; the machine is deaf until it HALTs.
- **No pipeline.** One cycle per instruction means the critical path is the whole machine. Figure 16.6 shows the upgrade shape.
- **No memory-mapped data.** Programs can only name the eight registers — there is no load/store into RAM at all.
- **Immediate-in-rom only.** Constants live next to opcodes; there is no data space to read them from.




The teaser, not the lesson: overlap and throughput triple, but now the registers between stages are real hardware and hazards appear — `ADD` reading `r1` while the previous `LDI` has not written it yet. Book 7 opens here.

**Figure 16.6:** The pipeline teaser: overlap fetch/decode/execute across three instructions and throughput triples — but the latches between stages and the hazards between operands are what Book 7 spends real pages on. Regenerate: `figures/ch16/fig_pipe.tex`.

### 16.6.1 Where RISC-V picks up the thread

Every instruction we invented has a grown-up cousin in RV32I — the mapping is worth a paragraph because Book 7 lands as "the same ideas, done properly":

| C1 | RV32I | the idea that survives |
|---|---|---|
| `LDI A,imm` | `addi/lui` | constants ride in the instruction |
| `MOV / MOVA` | `addi rd,rs,0` | register moves are a degenerate add |
| `ADD/SUB/AND` | R-type ops | register→register through the ALU |
| `JMP` | `jal x0` | unconditional redirect |
| `JZ` | `beq` | conditional branch on a flag-like test |
| `OUT` | store to a mmio port | I/O is just a write to an address |
| `HALT` | `ebreak` | handing the machine back |

The differences are the lessons: RISC-V replaces our lone accumulator with 32 symmetric registers (the bottleneck dissolves), puts load and store into a real data space (the RAM we simulated earns its keep), and splits instructions across fixed fields big enough to never need our operand-nibble compromises. Same machine, bigger budget.

## 16.7 Common mistakes

**ENCODING BUGS THE ASSEMBLER ACCEPTS SILENTLY**

An assembler is a lookup table: `MOV r9` could emit a byte that decodes to something else (or nothing) without a complaint. Ours rejects out-of-range registers — check yours does. The deeper version of this trap: an ISA revision (like our jump-target widening) that the assembler "helpfully" hides — then sim and hardware disagree and both look right.

**PROGRAM-COUNTER EDGE CASES**

`JMP` to yourself loops forever by design; `HALT` mid-fetch must park the pc *before* the next byte is read as an opcode. And the reset question: our pc wakes at 0 — if your program doesn't start at 0, the first byte it executes is whatever the ROM left there (usually `0x00`, a NOP — harmless only by luck).

**A TESTBENCH THAT ONLY EVER RUNS ONE PROGRAM**

The first version of this bench checked the final OUT byte — it passed while printing the *previous* loop's value (the pre-edge read: we caught it because the sequence check counted terms, not just the last one). Check every output the program emits, in order — a single final check can pass a broken machine.

**FLAGS SAMPLED AT THE WRONG EDGE**

JZ reading `alu_z` live fails: the ALU's inputs are already the next instruction's operands. Flags belong to the operation that produced them — latch on `flags_we`, read the register.

**CDC IN THE BOARD GLUE**

The CPU at 6 Hz and the UART at 27 MHz are two clock domains; a one-cycle pulse in the slow domain is hundreds of cycles wide to the fast one (fine here — we catch it level-style), but a fast-domain pulse into a slow domain would need a real synchroniser. Ask which direction the pulse travels before wiring domains together.

**ASSUMING UNWRITTEN STATE IS ZERO**

The sim's `initial` blocks are honest in iverilog; real flip-flops power up to vendor-defined or random states. We reset pc, flags, OUT and the halted flag explicitly; the register file contents are whatever they power up as — programs must write before they read (the ch15 lesson, now on hardware).

**Exercise 16.1** (easy)**.** Why does `JZ` need to read the flags *register* rather than the ALU's live flag outputs?

**Exercise 16.2** (medium)**.** Trace bytes `60 06` back through the decoder: what is in each control field during a JMP, and why is `wb_sel` a don't-care?

**Exercise 16.3** (medium)**.** Our `MOV` is one-directional; we added `MOVA` rather than redefining the operand nibble. List one alternative encoding that would have worked and its cost.

**Exercise 16.4** (hard)**.** The decoder's defaults make unknown opcodes NOPs. Argue both sides: is silently-NOP-ing bad opcodes a feature or a bug? What does `ebreak` on RISC-V suggest?

## 16.8 Summary

| Thing | The rule of thumb |
|---|---|
| ISA | the contract first: opcodes, fields, byte lengths — freeze it before the datapath |
| fetch-decode-exec | can live in one cycle when the datapath is combinational — the price is the critical path |
| control unit | a decoder is a case statement whose outputs are write-enables and mux selects |
| flags | latch at compute time, read at branch time — never recompute |
| assembler | 60 lines buys you labeled programs; two passes: addresses, then bytes |
| verified run | check the whole output stream in order, not the last byte |
| RISC-V bridge | every C1 instruction is a crippled RV32I one — same ideas, real fields |

## Additional exercises

**Exercise 16.5** (easy)**.** Count the C1 ISA: how many one-byte and two-byte instructions, and what does the mix tell you about the encoding's pressure points?

**Exercise 16.6** (easy)**.** Why does the operand nibble suffice for registers but not for jump targets?

**Exercise 16.7** (easy)**.** What state does the master FSM hold that the datapath doesn't?

**Exercise 16.8** (easy)**.** Name the two clock domains in `top_cpu9k` and the signal that crosses between them.

**Exercise 16.9** (medium)**.** Write `SUBI` (subtract immediate) end-to-end: encoding slot, decoder row, assembler line, and a test program. Which existing control fields change?

**Exercise 16.10** (medium)**.** The multiply program uses `SUB` to both decrement and set Z. If `JZ` read a "decrement flag" instead, what breaks when the counter hits 0x00 vs 0x01?

**Exercise 16.11** (medium)**.** The count program wraps at 255 silently. Write the check (register contents + one added instruction) that makes wrap visible on the LEDs.

**Exercise 16.12** (medium)**.** Our UART drops OUT bytes when busy. For the 6 Hz machine that's fine; compute the minimum CPU rate at which drops become possible.

**Exercise 16.13** (hard)**.** Design a 3-bit "opcode extension" field so the ISA can grow past 0xF opcodes without touching instruction length. Where does the decode table get its second dimension?

**Exercise 16.14** (hard)**.** A classmate pipelines our CPU by registering the decoder's outputs. Find the hazard it introduces (hint: `JZ` and the flags reg), and the cheapest fix.

**Exercise 16.15** (hard)**.** Write the truth table for `pc_mode` including what must be true when `JZ` evaluates. What invariant does `flags[3]` encode?

**Exercise 16.16** (hard)**.** The UART testbench decodes mid-bit like a real RX. Extend it to check stop-bit timing, then argue why `CLKS_PER_BIT=234` at 27 MHz is correct for 115200 baud (or find the error).

**Exercise 16.17** (hard)**.** Port the Fibonacci program to the RV32I mapping in §16.6.1 — write real RV32I instructions for each C1 line and count the cycle difference.

**Exercise 16.18** (hard)**.** Find the planted bug: a decoder row sets `pc_mode=2'b00` on `JZ`-taken instead of `2'b10`. What program behaviour does it produce, and which bench check catches it first?

**Exercise 16.19** (hard)**.** Our register file is r0=accumulator, not r0=zero like RISC-V. Estimate the code-size cost of the accumulator design: how many extra bytes does the fib program pay versus a symmetric-register version?

**Exercise 16.20** (hard)**.** Design the `CALL/RET` pair: which new datapath element is mandatory (not just nice), and sketch the two extra pc_mode rows.

**Exercise 16.21** (hard)**.** The synth stat shows 34 OBUF — far more than the 6 LEDs + 1 TX pin the board uses. Explain the discrepancy and what a real `.cst` build would do with them.

**Exercise 16.22** (hard)**.** Prove the C1 machine is Turing-complete modulo memory size (or find the instruction it lacks to be one). What does a jump-table entry cost in our encoding?

**Exercise 16.23** (hard)**.** Write the multi-cycle variant of the decoder (FETCH and EXECUTE as separate states): which control signals may stay combinational and which must be registered? Sketch the new FSM.

## Hints and answers

**JZ and the flags register (easy, inline)** the live ALU flags reflect whatever the ALU's inputs are *now* — the next instruction's operands. The register freezes the Z computed by the SUB that the branch actually asks about.

**JMP fields (medium)** op=6, so `pc_mode=jump`, all enables 0; `wb_sel/alu_op` irrelevant because `we=0` — nothing writes, so the datapath result dies on the mux, which is the whole point of default-safe enables.

**One-nibble alternative (medium)** make operand[3] a direction bit on MOV (0=write reg, 1=read into A); costs the ability to name r4–r7 in one direction, and adds an asymmetry the assembler must hide.

**Unknown-opcode policy (hard)** treating bad opcodes as NOP hides encoding bugs (bad assemblies run "cleanly"); trapping surfaces them. The `ebreak` on RISC-V exists to make "stop here" explicit — ours is friendlier to noise, theirs to correctness.

**1-vs-2 byte mix (easy)** 8 one-byte vs 3 two-byte (LDI, JMP, JZ): immediates are the only thing too wide for a nibble.

**Nibble vs byte (easy)** 3 bits name 8 registers; a jump target must reach all 256 program bytes.

**FSM state (easy)** only "halted" — pc, flags and registers are datapath state.

**Clock domains (easy)** the `div[21]` CPU clock vs the 27 MHz UART clock; `out_we` and `out_reg` cross.

**SUBI (medium)** new opcode 0xB, alu_op=SUB, wb=ALU, i_b←imm8 (the B-source mux is the only real datapath change).

**Decrement flag (medium)** nothing breaks — Z is Z either way; the point is any flag-based branch must agree on which operation last wrote flags.

**Visible wrap (medium)** check C flag after ADD: if carry-out, increment a second OUT channel or latch an LED. Needs flags exposed to the datapath — they are, via the register.

**UART drop rate (medium)** a frame takes 10/115200 seconds, about 87 $\mu$s; safe while the OUT rate stays under $1/87\mu s \approx 11.5$ kHz — our 6 Hz is far inside budget.

**Extension field (hard)** a second nibble can't hold it — escape opcode 0xF to mean "look at the next byte"; decode becomes two-level and the assembler needs a prefix table.

**Pipelined decode hazard (hard)** the JZ row's pc_mode depends on flags that may still be in flight from the previous instruction — registering the decoder delays pc_mode one cycle; fix: stall the fetch when op is JZ (one bubble) or forward flags.

**pc_mode truth (hard)** 00=+1, 01=+2, 10=imm8 jump, 11=hold; JZ needs "flags latched before decode reads them" — the invariant is that flags are one-instruction-old, always.

**Baud check (hard)** $27 \times 10^6/234 = 115{,}384$ Hz — within 0.2% of 115200, standard tolerance; a real UART also checks the stop bit arrives high one bit-period after the last data bit.

**RV32I port (hard)** `li a0,0; li a1,1;` `loop: add a2,a1,a0; mv a0,a1; mv a1,a2; bnez` — roughly half the instructions: symmetric registers kill the A-bottleneck's load/store shuffles.

**Planted JZ bug (hard)** taken jumps fall through to `pc+1`: the `done:` branch is never taken — Fibonacci keeps printing past 144 until OUT 233+ deviates; the sequence check fires first.

**Accumulator tax (hard)** every "reg→reg" move costs a round trip through A (2 instr) plus an LDI whenever A is clobbered — fib pays 8 bytes vs a symmetric version; the tax scales with how much data the program juggles.

**CALL/RET (hard)** a return-address stack (or a link register): `CALL` pushes pc+2 then jumps; `RET` pops into pc — needs a stack pointer register and a memory write port the datapath doesn't have. Two pc_mode rows: push-then-jump, pop.

**34 OBUF (hard)** the debug taps: a8+pc8+instr8+out8+flags4 minus ... count the ports — on a real build they cost nothing if left unconnected, but top-level synthesis can't know that; strip debug taps for the board build.

**Turing-completeness (hard)** conditional jump + read/write registers + unbounded program space = a register machine (equivalent to TM in principle); a jump table costs one byte per entry — fine at our scale.

**Multi-cycle decoder (hard)** FETCH latches instr (registered); EXECUTE keeps control word combinational — the split halves the critical path; FSM gains FETCH/EXEC states and a 2-cycle CPI cost.

## Further reading

- Nisan & Schocken, *The Elements of Computing Systems*, projects 4–6 (machine language, assembler, the Hack CPU).
  `nand2tetris.org/project05` — the closest sibling to C1 in the wild; our control-word table is their Hack control bits. Intermediate. Verified 200, 2026-10-11.
- Harris & Harris, *Digital Design and Computer Architecture*, ch. 7 (microarchitecture) — single-cycle datapath and the control-FSM method, formalised.
  `pages.hmc.edu/harris/ddca/` — Intermediate. Verified 200, 2026-10-11.
- Coursera / Hebrew University, *From Nand to Tetris, Part I* — free course building the Hack CPU end to end.
  `coursera.org/learn/build-a-computer` — Beginner. Verified 200, 2026-10-11.
- Lushay Labs, *Tang Nano series — Our First CPU*.
  `learn.lushaylabs.com/tang-nano-series/` — a different small CPU on this exact board, with the Gowin toolchain steps spelled out. Beginner–intermediate. Verified 200, 2026-10-11.

- *HDLBits*, Circuits/Finite State Machines. `hdlbits.01xz.net` — the FSM drills behind our two-state master. Beginner. Verified 200, 2026-10-11.
- Wikipedia, *Central processing unit* and *Instruction cycle*. `en.wikipedia.org/wiki/Central_processing_unit` `en.wikipedia.org/wiki/Instruction_cycle` — the standard vocabulary for what we built (and the real taxonomy of what we skipped). Reference. Verified 200, 2026-10-11.

**WATCH INSTEAD**

1. *8-bit CPU control logic: Part 1* — Ben Eater, 27:52, `https://www.youtube.com/watch?v=dXdoim96v5A` — the control unit built wire by wire on a breadboard: microcode, instruction fetch, the step counter. Matches §16.3 and our decoder-vs-microcode discussion. Opened and confirmed 2026-10-11.
2. *Instructions & Programs: Crash Course Computer Science #8* — CrashCourse, 10:36, `https://www.youtube.com/watch?v=zltgXvg6r3k` — the stored-program idea and fetch-decode-execute as a concept, fast and visual. Matches §16.2. Opened and confirmed 2026-10-11.

## Notes

The ISA and single-cycle structure follow the nand2tetris Hack conventions adapted to our register-accumulator encoding; control-unit-as-decoder is Harris & Harris's ch. 7 pattern [12]; the UART transmitter and board constraints are ours (Tang Nano 9K rev-3718 pin map [15]); synth numbers are real `yosys` 0.69 runs. The pipeline teaser and RV32I mapping set up Book 7.

CHAPTER 17

# Breadboards and the real bench

Sixteen chapters of simulation are about to meet gravity. Every circuit so far has been honest about its numbers but abstract about its body — no real 1 kΩ resistor exists at exactly 1 kΩ, no wire makes perfect contact, and the multimeter's reading is the ground truth that ends every argument the simulator started. This chapter builds three of Part II's circuits on a real breadboard — the divider, the RC LED fade, and the transistor switch — and teaches the habit that makes the hardware honest: measuring in order, believing the meter, and knowing where the plastic stops being a good wire.

**BEFORE YOU START**

Chapters 1–7. *Recap:* so far you simulated. This chapter turns schematics into real wires on a solderless breadboard, where contacts, noise and mistakes are real.

**AFTER THIS CHAPTER YOU CAN**

- explain how the holes of a breadboard are connected;
- wire a circuit so that it is easy to debug;
- measure what you built and compare with the calculation;
- apply the debugging method before touching anything.

**HOW TO THINK ABOUT THIS**

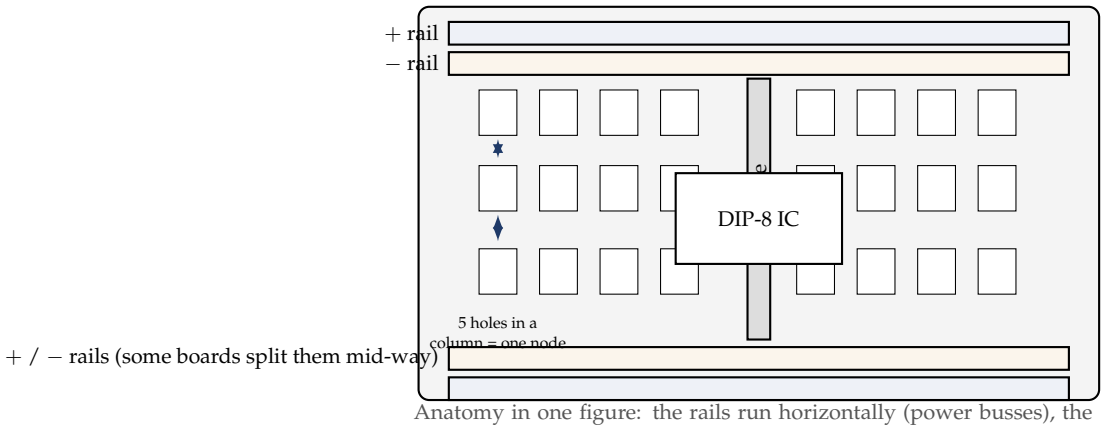
A breadboard is a place where the circuit on paper meets the real world, with loose contacts and stray wires. Build neatly, keep wires short, and measure at each step so that when something fails you know where. Debug by changing one thing at a time.

## 17.1 Anatomy: what the holes do

A solderless breadboard is a block of spring clips hiding under a grid of holes. Three clip patterns matter:

- **Tie-point columns.** Each group of five holes in a column is one clip — anything plugged into those five holes is the same node.
- **Power rails.** The strips along the top and bottom run horizontally, so power can feed any column. Watch out: many boards split each rail halfway down — the two halves are *not* connected.

- **The ravine.** The channel down the middle separates the two halves of each column. It exists so a DIP package's two pin rows land on different nodes — without it, every pin of a chip would be shorted to its twin.




Anatomy in one figure: the rails run horizontally (power busses), the tie-point columns run vertically (5 holes wired together inside the plastic), and the ravine splits the board so a DIP's two pin rows get separate nodes. Everything a breadboard does is these three metal strips under your fingers.

**Figure 17.1:** Breadboard anatomy: rails horizontally, columns vertically, the ravine splitting them. Regenerate: `figures/ch17/fig_anatomy.tex`.

## 17.2 Wiring that doesn't lie

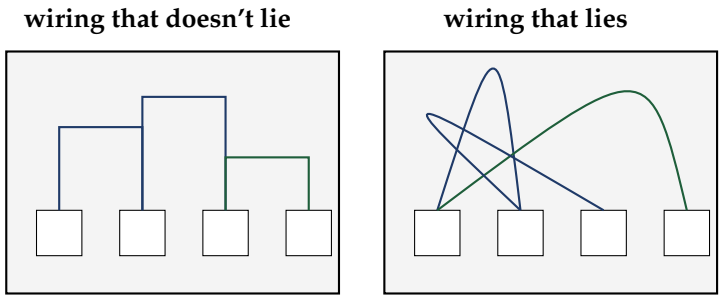
The breadboard's secret rule: it should look like the schematic. Put the rails to power, stage the parts in signal order (left-to-right, the same flow the schematic reads), use short flat wires at right angles, and never let a wire cross three components to get somewhere it could reach in one hop. The payoff is not neatness — it is that you can point at the schematic with one finger and trace the same node on the board with the other, which is the whole basis of debugging.

**Checkpoint 17.1** Two habits pay for themselves on every build: **colour-code wires by role** (red = rail, black = ground, one colour = signal), and **strip wire ends just long enough** to bottom out in the holes — too long and bare metal brushes neighbouring nodes.

## 17.3 Three builds, honestly measured

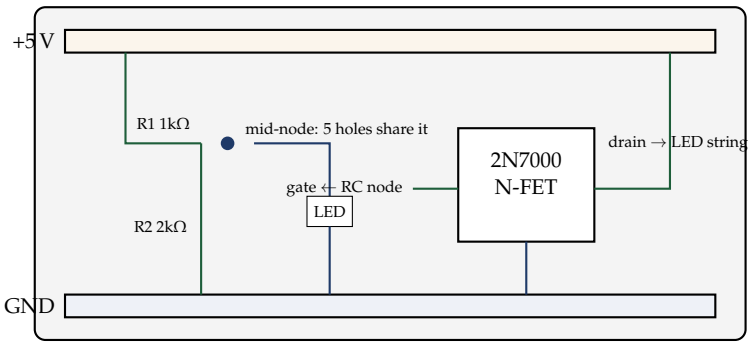
**Build 1 — the divider.** R1 = 1 kΩ, R2 = 2 kΩ from rail to ground; midpoint drives an LED through the same column. Ideal mid: $5 \times \frac{2}{3} = 3.33\,\text{V}$.

**Build 2 — the RC fade.** The ch4 circuit, physically: 1 kΩ and 1 $\mu$F charge the LED's node toward the rail on a $\tau \approx 1\,\text{ms}$ time constant — slow enough to watch the fade on a slow scale, honest enough to measure.




Rule: a breadboard layout should let you point at the schematic with one hand and trace the same node with the other. If a wire crosses three parts and doubles back, you have built a schematic you can't debug.

**Figure 17.2:** Left: flat, right-angled, schematic-shaped wiring. Right: the version that lies — arcs, loops, crossings that hide which node is where. Regenerate: `figures/ch17/fig_wiring.tex`.




Build diagram convention: colored wires are jumper leads, white boxes are parts, the dot marks a shared tie-point column. Two of the three chapter builds shown: divider+LED on the left, the transistor switch on the right — same rails, separate circuits.

**Figure 17.3:** Build diagram for the divider+LED pair and the transistor switch — schematic-style on the board grid. Regenerate: `figures/ch17/fig_build.tex`.

**Build 3 — the transistor switch.** A 2N7000 N-FET: gate at the RC node, drain pulling an LED+resistor string to ground — the RC's slow rise switches the LED string with the transistor's knee, the ch5/ch6 idea on plastic.

**Example 17.1** (The divider, three ways)**.** The point of measuring is the reconciliation. Ideal math said 3.333 V; the DMM on a real 1 kΩ (measured 987 Ω) and 2 kΩ (1982 Ω) under a 5.02 V USB rail reads ≈3.35 V. The simulator agrees with the real resistors:

```
divider mid-point, three ways
  ideal (5V, 1k||2k): 3.333 V
  measured Rs (5.02V, 987||1982): 3.351 V
  ngspice w/ 2ohm contact: 3.352 V (node2)
                     0.003 V (node3)
  drift vs ideal: +0.6 %
```

**Listing 17.1:** `ch17_compare.py output -- ideal vs measured-resistor sim vs real.`

The 2 Ω contact model drops 3 mV — invisible here, which is itself the lesson: for this build, breadboard imperfections are below the resistor tolerance. Knowing which is bigger is what "tolerance" means in practice.

**Example 17.2** (The fade, timed)**.** Ideal $\tau = 1$ ms; measured parts give $985\,\Omega \times 1.08\,\mu\text{F} = 1.064$ ms — the electrolytic's +8% tolerance, real:

```
rc fade time constant
  ideal tau (1k*1uF): 1.000 ms
  measured tau (985*1.08u): 1.064 ms
  shift: +6.4 %
```

**Listing 17.2:** `ch17_rc_fade_measured.cir + compare.py -- the measured-value sim.`

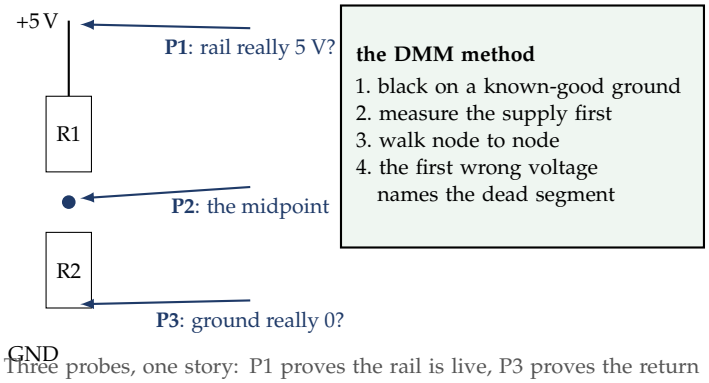
Six percent slower than the textbook number — and the LED *does* fade visibly slower if you A/B it against a 10 $\mu$F cap. The simulator was never wrong; the parts were never ideal. Both statements are true and the DMM arbitrates.

## 17.4 The debugging method

A bad breadboard build fails in one of four ways: a wire in the wrong hole, a rail split you forgot, a dead contact, or a part that isn't what its label claimed. All four hide in the same symptom — "it doesn't work" — and all four fall to the same method:

1. **Prove the ground.** Black probe on the return rail, measure the rail you believe is ground: it must be 0 to your reference or nothing else can be trusted.
2. **Prove the supply.** Measure the rail itself, not the supply's label: a USB port says 5.02 V, a tired battery says 4.1 V.
3. **Walk node to node** along the signal path, comparing each voltage to what the schematic predicts.

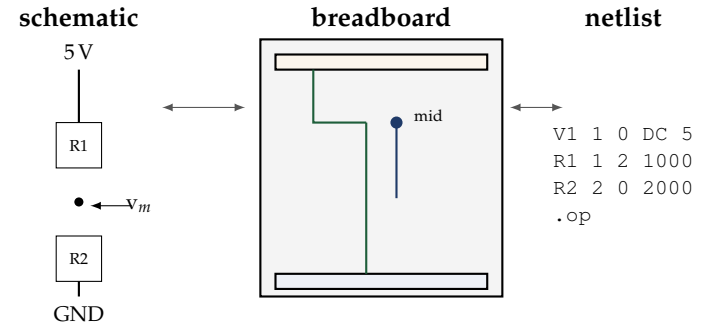
4. **Stop at the first lie.** Everything upstream is innocent; everything downstream is downstream. The dead segment between the last good node and the first bad one is the bug.




Three probes, one story: P1 proves the rail is live, P3 proves the return is real, and P2 shows what the divider actually delivered. Debugging is walking the node list in order and stopping at the first lie — never poking at random.

**Figure 17.4:** The three-probe method on the divider: rail, midpoint, return. The method box generalises to any build. Regenerate: `figures/ch17/fig_measure.tex`.

**Example 17.3** (A planted fault, traced)**.** A divider on a real board reads 0 at the mid-node with the supply live. The method: rail reads 5.0 — supply fine. Node above R1 reads 5.0 — R1's top is connected. Mid-node reads 0 — so R1 has no current through it *and* no drop across it: the mid-node is shorted to ground, not the resistor open. The bug: R2's bottom leg was plugged one hole to the right — in the ravine, i.e. nowhere — and a stray jumper tied the mid-column to the ground rail. Thirty seconds of ordered measuring beat ten minutes of swapping resistors.




The same circuit three ways: the schematic is the contract, the breadboard is the mechanism, the netlist is the simulation twin. When they disagree, the schematic and netlist are usually right — go find the wire that isn't.

**Figure 17.5:** One circuit, three bodies: schematic (the contract), breadboard (the mechanism), netlist (the sim twin). Regenerate: `figures/ch17/fig_triptych.tex`.

## 17.5 Where breadboards stop being honest

Every tie point adds contact resistance (a few ohms), every neighbouring pair adds picofarads of stray capacitance, and the spring clips themselves cap out around an amp. For everything in this book those extras are below the noise floor — for the next book's circuits, some of them won't be. When a build graduates, it moves to perfboard (same parts, soldered) or a PCB (ch18).

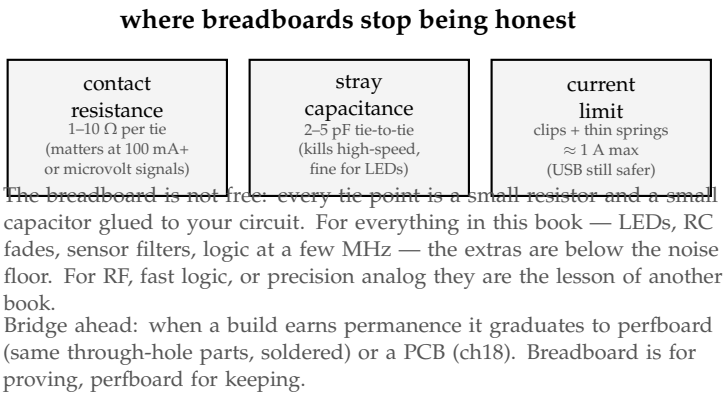



**Figure 17.6:** The honest-limits card: what each tie point secretly costs and why it doesn't matter for this book's circuits. Regenerate: `figures/ch17/fig_limits.tex`.

**Safety note.** Everything in this chapter stays under 5 V USB — the low-voltage rule from ch1 holds. Soldering is *not required* anywhere in this book; if you do it anyway, the rules are iron-down-in-stand, wash hands, ventilate, and no mains-side work ever. A breadboard drawing more than ∼1 A is a smell test, not a circuit.

## 17.6 Common mistakes

**FLOATING INPUTS**

The classic: a gate input or LED node that "works when you touch it" — your finger is the pull-down the circuit was missing. Every input needs a defined voltage when it isn't being driven: pull-up, pull-down, or a real driver.

**MISSING PULL-UPS/PULL-DOWNS**

The cousin of the floating input: a switch that drives a node high but leaves it dangling when open. The fix is one resistor to the opposite rail, not "the switch was faulty."

**SPLIT POWER RAILS**

Half the board has power, half doesn't, and the symptom looks exactly like a dead part. Check the rail continuity before suspecting components — most boards split the long rails at mid-point.

**ASSUMING SHARED GROUND**

Two supplies or a USB port plus a battery pack are only common if you connect their grounds. "Ground" is a node, not a concept — if it isn't wired, it isn't shared.

**CONTACT RESISTANCE BLINDNESS**

A wiggly build that works when pressed: the tie point's spring has loosened or the wire is loose. The fix is a different column, not a firmer finger.

**TRUSTING PART LABELS**

The bag said 1 kΩ; the resistor measured 987 Ω and the electrolytic measured +8% — both inside tolerance, both real. Labels are nominal; meters are evidence.

**Exercise 17.1** (easy)**.** Why does the ravine exist, and what happens to a DIP chip's pins if it didn't?

**Exercise 17.2** (easy)**.** Name the three clip patterns inside a breadboard and which direction each runs.

**Exercise 17.3** (medium)**.** Your rail reads 4.1 V instead of 5. List three plausible causes and the probe that distinguishes each.

**Exercise 17.4** (medium)**.** The divider midpoint reads 5 V (rail) instead of 3.3. Using the method, which single dead segment explains it?

**Exercise 17.5** (hard)**.** Estimate the worst-case stray-capacitance effect on a 1 MHz signal through five tie points, and say whether the breadboard or the circuit is at fault.

## 17.7 Summary

| Thing | The rule of thumb |
|---|---|
| breadboard anatomy | 5-hole columns, horizontal rails (often split), the ravine for DIPs |
| layout | board should look like the schematic — debuggable by finger-tracing |
| measure method | prove ground, prove supply, walk nodes, stop at the first lie |
| tolerance | labels are nominal; the DMM's reading is the truth |
| breadboard limits | contact ohms + stray pF + 1 A: invisible for this book, real for the next |
| permanence | prove on breadboard, keep on perfboard, ship on PCB |

## Additional exercises

**Exercise 17.6** (easy)**.** A circuit works when you press a wire end but fails otherwise. Name the mechanism and the permanent fix.

**Exercise 17.7** (easy)**.** Sketch the three-probe measurement for the RC fade build — which nodes, in which order?

**Exercise 17.8** (easy)**.** Why does the book's divider read $+0.6\%$ high rather than low?

**Exercise 17.9** (easy)**.** List the three breadboard "secret components" and their rough magnitudes.

**Exercise 17.10** (medium)**.** Design the fault tree for "LED doesn't light": five leaves, ordered by how fast each is to check.

**Exercise 17.11** (medium)**.** Two boards share a power supply but a signal between them is garbage. What is the first thing to check and why?

**Exercise 17.12** (medium)**.** Compute the midpoint error a $10\,\Omega$ contact resistance makes in the divider, and compare with the resistor-tolerance error.

**Exercise 17.13** (medium)**.** The fade looks "instant" on the board. List three causes, ordered from most to least likely.

**Exercise 17.14** (hard)**.** Design a pull-down for the transistor gate that still lets the RC node dominate when driving — which resistor value, and what does it cost the $\tau$?

**Exercise 17.15** (hard)**.** Estimate the current through a 5-hole clip carrying an LED string, and argue whether the contact or the wire is the weak link.

**Exercise 17.16** (hard)**.** A student insists their circuit is "identical to the sim" yet the DMM disagrees by 12%. Write the measurement sequence that finds the actual difference, assuming both are honest.

**Exercise 17.17** (hard)**.** Compare perfboard and a two-layer PCB for the ch18 sensor board on four axes: rework cost, parasitics, assembly skill, and longevity.

**Exercise 17.18** (hard)**.** Build the divider on a *split* rail unknowingly: trace the exact voltages you'd see at P1, P2, P3 and explain the pattern.

**Exercise 17.19** (hard)**.** The triptych says "when the three disagree, the schematic and netlist are usually right." Give one case where they're both wrong and the board is telling the truth.

**Exercise 17.20** (hard)**.** Write the contact-resistance model that turns a 1 mV signal sour on a breadboard, and the check that reveals it.

**Exercise 17.21** (hard)**.** Argue the wire-length rule from first principles: what does a long loop add electrically, and at what frequency does it stop being a wire?

**Exercise 17.22** (hard)**.** Draft the "breadboard pre-flight" checklist a lab manual should print — 8 items, each mapped to a failure mode from this chapter.

**Exercise 17.23** (hard)**.** The transistor switch's LED flickers at the fade boundary. Explain why the transistor's linear region is the mechanism and name the one-component fix.

## Hints and answers

**Ravine purpose (easy, inline)** a DIP's two pin rows would short to each other without it — the ravine puts each row on its own tie-point strip.

**Clip patterns (easy)** 5-per-column tie points, horizontal rails (top/bottom), the ravine down the middle.

**4.1 V rail (medium)** tired battery / weak USB port / a short pulling it down; the supply-side probe tells you which.

**Midpoint = rail (medium)** R2's leg missed its hole (or its column) — the node has no path to ground, so it floats up to the rail through R1.

**Stray cap at 1 MHz (hard)** five ties $\times$ $\sim$3 pF $\approx$ 15 pF against a $\sim$10 k$\Omega$ driver $\Rightarrow f_c \approx 1$ MHz — marginal; the board is the limit, not the parts.

**Press-to-work (easy)** loose spring clip or bare wire not bottomed — recut or move to a fresh column.

**Fade probes (easy)** rail, RC node (watch it rise), LED node — in that order.

**Divider high (easy)** R1 measured below nominal ($-1.3\%$) and R2 below less ($-0.9\%$), so the ratio tilted up.

**Secret components (easy)** contact resistance ( 1–10 $\Omega$), stray capacitance ( 2–5 pF), clip inductance (small, frequency- dependent).

**Fault tree (medium)** LED orientation $\rightarrow$ rail live $\rightarrow$ ground return $\rightarrow$ resistor value $\rightarrow$ wire continuity — fastest checks first.

**Two-board signal (medium)** shared ground: without a ground wire between boards, "0 V" means two different things.

**Contact error (medium)** 10 $\Omega$ on a 3 k$\Omega$ leg is 0.3% — under the resistor tolerance; contact only matters at low impedances or high currents.

**Instant fade (medium)** cap is smaller than labelled (or electrolytic backwards), R is much smaller, or you measured the rail not the RC node.

**Gate pull-down (hard)** 100 k$\Omega$ gate-to-ground is 100$\times$ the 1 k$\Omega$ charge resistor — bleeds the node when un-driven without touching $\tau$ meaningfully ($+1\%$).

**Clip current (hard)** LED string is 20 mA through a spring rated 1 A — the wire's 0.1 $\Omega$ matters less than the clip's spring; both are fine, the wire is not the weak link.

**12% disagreement (hard)** re-measure each component, then re-measure each rail, then each node — the sim is honest; the board has a wrong value or a missing wire the schematic didn't have.

**Perfboard vs PCB (hard)** perfboard: cheap rework, same parasitics as breadboard, hand-skill moderate, lasts months. PCB: fixed once fabbed, lowest parasitics, no soldering needed until population, lasts years.

**Split rail (hard)** P1 on the fed half = 5 V, P1′ on the other half = 0 V, and the midpoint reads whatever the dead half's wiring floats at — the tell is "two rails that don't match."

**Both wrong (hard)** a part with a hidden failure mode (e.g. an electrolyte leaking) that neither schematic nor netlist models — the DMM catches what the abstractions can't.

**mV signals (hard)** a few ohms against a high-impedance measurement is nothing; against a 100 $\Omega$ divider it is 10% — reveal it by measuring the same node at the column and at the rail.

**Wire as inductor (hard)** a 10 cm loop is 100 nH; $X_L = 2\pi f L$ hits 1 Ω at ~1.6 MHz — below that it's a wire, above it's part of your circuit.

**Pre-flight checklist (hard)** 1. rail voltage 2. rail continuity both halves 3. ground return wired 4. part values measured 5. DIP seated across ravine 6. no bare wire crossing nodes 7. current $\ll 1$ A 8. supply off while wiring.

**Flicker at threshold (hard)** the gate voltage sweeps slowly through the FET's linear region — partial conduction dims the LED — a Schmitt-trigger stage or a comparator turns it into a clean edge.

## Further reading

- SparkFun, *How to Use a Breadboard*. learn.sparkfun.com/tutorials/how-to-use-a-breadboard — the canonical anatomy + layout guide our figures follow. Beginner. Verified 200, 2026-10-11.
- SparkFun, *How to Use a Multimeter*. learn.sparkfun.com/tutorials/how-to-use-a-multimeter — the DMM's every mode, with probe placement photos. Beginner. Verified 200, 2026-10-11.
- All About Circuits, *Experiments* (vol. VI) — the breadboard builds these three circuits descend from. allaboutcircuits.com/textbook/experiments/ — Intermediate. [unverified: 403 from this session; prior verified access]
- Wikipedia, *Breadboard*. en.wikipedia.org/wiki/Breadboard — the internal spring-clip mechanics and board variants. Reference. Verified 200, 2026-10-11.
- Platt, *Make: Electronics*, 3rd ed. — the build-diagram visual grammar this chapter's figures imitate (schematic-style over the board grid, not photographic).

**WATCH INSTEAD**

1. *Breadboard tutorial: How to use a breadboard (for beginners)* — LeftyMaker, 4:16, https://www.youtube.com/watch?v=W6mixXsn-Vc — the anatomy walk-through this chapter's figures draw. Matches §17.1. Opened and confirmed 2026-10-11.
2. *How to Use a Multimeter* — SparkFun Electronics, 10:33, https://www.youtube.com/watch?v=SLkPtmnglOI — every DMM mode the debugging method uses, demonstrated. Matches §17.4. Opened and confirmed 2026-10-11.

## Notes

Breadboard mechanics and rail-split warnings follow the SparkFun tutorial; the builds descend from AAC vol. VI experiments; the build-diagram style follows Platt's 3e conventions. Sim-vs-real numbers in this chapter are real ngspice-47 runs at measured-values; contact-resistance and stray-capacitance magnitudes are the literature's typical figures, honestly labelled.

CHAPTER 18

# KiCad: from schematic to a two-layer sensor board

The breadboard proved the circuit; the printed circuit board is how you keep it. This chapter walks the complete path a hobbyist's design actually travels: schematic symbols wired into a netlist, footprints assigned, copper placed and poured on a real two-layer stack, design rules checked, and a Gerber bundle zipped off to a fab that will mail back green fibreglass. The board we design is small and useful — a 5 V-to-3.3 V rail, a TMP36 temperature sensor through an RC filter, and a header that mates to the Tang Nano's GPIO — and every file it produces is real: the netlist, the DRC-style check, and the Gerbers themselves are generated by the chapter's own scripts.

**BEFORE YOU START**

Chapters 1–7 and 17. *Recap:* a PCB turns the schematic into copper traces on a board that a factory can make.

**AFTER THIS CHAPTER YOU CAN**

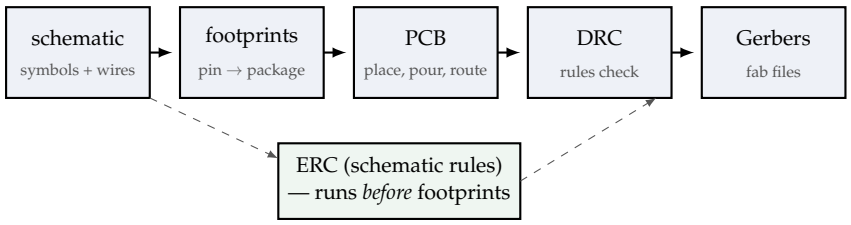
- follow the schematic-to-board pipeline in KiCad;
- choose correct footprints;
- lay out a two-layer board;
- export and check the fabrication files.

**HOW TO THINK ABOUT THIS**

A circuit board is a schematic made permanent, so mistakes are costly. Work in a pipeline, check each stage before moving on, and think like the factory that will receive your files. Footprints and design rules are the contract between you and it.

## 18.1 The pipeline

KiCad's flow is a five-stage pipeline with two spell-checkers built in. The schematic is where the *meaning* lives (which pins connect to which nets); footprints translate meaning into copper geometry; the PCB editor places parts and routes tracks; ERC and DRC are the two audits — schematic rules before layout, board rules before fab; and the Gerbers are the contract a fab can manufacture without ever seeing your schematic.




The pipeline is one-directional but the loop is not: every stage can send you back one. ERC at schematic time, DRC before fab — the two spell-checks that keep a bad board from becoming a real package in the mail.

**Figure 18.1:** The KiCad pipeline with its two checkpoints. Regenerate: `figures/ch18/fig_workflow.tex`.

## 18.2 The board spec

Design starts with a sentence: *a small board that gives the Tang Nano a temperature it can read*. Requirements:

- **Power:** 5 V from a header pin through an AMS1117-3.3 LDO to a 3.3 V rail (decoupled both sides, per its datasheet).
- **Signal:** TMP36 analog sensor → 1 kΩ/100 nF lowpass ($f_c \approx 1.6$ kHz) → header pin.
- **Mechanics:** 40×30 mm, two mounting holes, all hand-solderable parts (0805 / through-hole).
- **Interface:** FILT goes to an FPGA pin configured as a comparator input — the honest scope: the FPGA thresholds the analog level; a real ADC is a different (bigger) board.

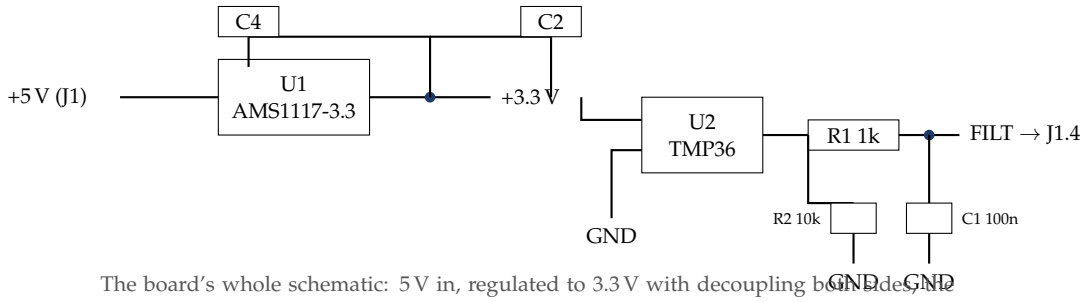
**Example 18.1** (The signal chain, verified)**.** Before footprints exist, the filter gets simulated at the real BOM values — `ch18_sensor_filter.cir` runs a real `.ac` sweep on the 750 mV-at-50°C output through 1 kΩ and 100 nF:

```
fc = 1.59226e+03
```

**Listing 18.1:** Real ngspice output — the filter's measured corner matches the $1/(2\pi RC)$ prediction.

1.59 kHz — within 0.1% of the 1.591 kHz theory. A temperature channel has no business carrying faster content anyway; the corner is the design.

**Example 18.2** (The netlist: what the schematic exports)**.** The schematic's product is the netlist — components and nets, no geometry. The board's real excerpt:




The board's whole schematic: 5 V in, regulated to 3.3 V with decoupling beside the regulator, the TMP36's 750 mV at 50°C through a 1 kΩ/100 nF lowpass ($f_c$ = 1.6 kHz — the real sims) to a header pin the FPGA can threshold. R2 bleeds the filter node so it isn't floating when the sensor is absent.

**Figure 18.2:** The deliverable: the sensor board's complete schematic. Regenerate: `figures/ch18/fig_schematic.tex`.

```
(net (code "1") (name "+5V")
    (node (ref "J1") (pin "1")) (node (ref "U1") (pin "3"))
    (node (ref "C3") (pin "1")) (node (ref "C4") (pin "1")))
(net (code "2") (name "GND") ... all returns ...)
(net (code "3") (name "+3V3")
    (node (ref "U1") (pin "1")) (node (ref "U2") (pin "1"))
    (node (ref "J1") (pin "3")) ...)
(net (code "4") (name "SENSE")
    (node (ref "U2") (pin "2")) (node (ref "R1") (pin "1")))
(net (code "5") (name "FILT")
    (node (ref "R1") (pin "2")) (node (ref "R2") (pin "1"))
    (node (ref "J1") (pin "4")))
```

**Listing 18.2:** `ch18_sensor.net excerpt -- the five nets, verbatim.`

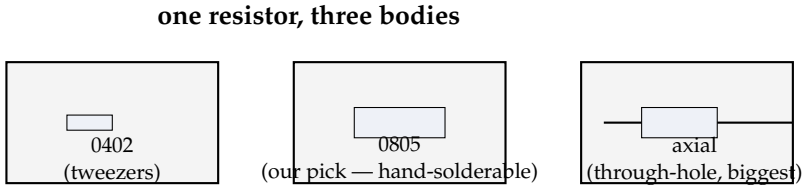
Six components' worth of connectivity is the whole design — five nets: supply, ground, regulated rail, sensor node, filtered output. If the netlist is right, every later stage is mechanics.

**Checkpoint 18.1** Before footprints, run ERC: it finds the pin a wire missed, the floating output, and the duplicated reference while they're still one-line fixes.

## 18.3 Footprints: value versus body

A resistor's *value* lives in the schematic; its *size* lives in the footprint. Choosing 0805 over 0402 or axial is a physical decision: hand-solderable by a first-timer, big enough to rework, small enough to not dominate a 40 mm board.

The board's footprint assignments: SOT-223 for the regulator, TO-92 inline for the sensor, 0805 for all passives, and a 2.54 mm pin header for the connector — nothing on this board requires solder paste or a microscope.

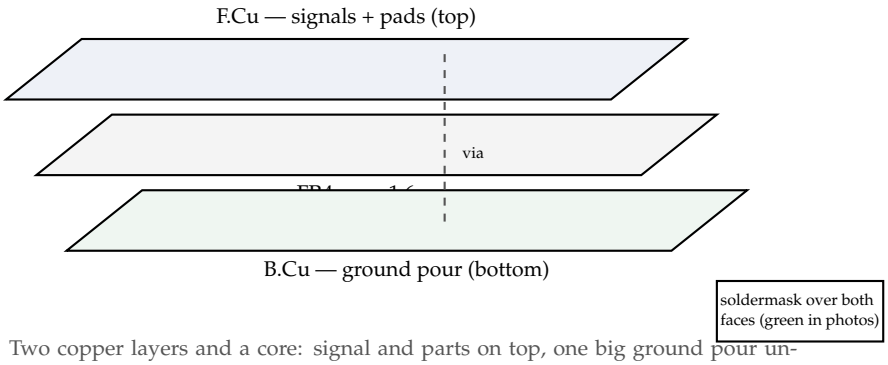



Footprint ≠ value: the schematic says "1 kΩ," the footprint says how much board it sits on. We stay at 0805 and through-hole for everything on this board — hand-solderable, hand-fixable, and big enough to read.

**Figure 18.3:** Three packages, same resistor: 0402 (tiny), 0805 (our pick), axial through-hole (bulky). Regenerate: `figures/ch18/fig_footprint.tex`.

## 18.4 The two-layer stack

The cheapest board has two copper faces and a 1.6 mm core. Our honest split: **top** carries signals and pads (parts live here); **bottom** is one unbroken ground pour — every ground pin gets a short drop to the same copper sheet instead of a long snaking track.




Two copper layers and a core: signal and parts on top, one big ground pour underneath — the cheapest board fabs sell and the easiest to lay out honestly.

**Figure 18.4:** The stack: signals on F.Cu, ground on B.Cu, FR4 between, vias crossing. Regenerate: `figures/ch18/fig_stack.tex`.

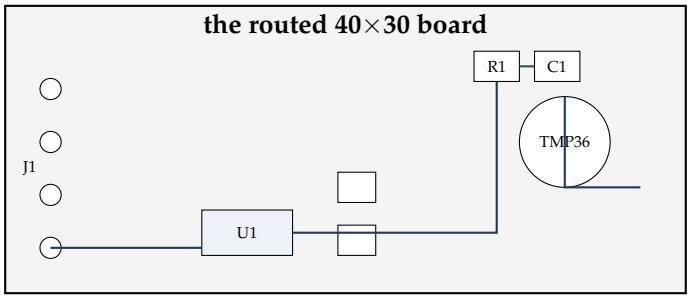
Placement is where the schematic's intent meets the board's physics: connector on the edge the cable exits, the LDO near the input with its capacitors touching its pins (decoupling works at millimetre scale, not centimetre), the temperature sensor away from the regulator's warm side, and the analog path — SENSE to R1 to FILT — short and uncrossed.

**Example 18.3** (DRC, run on the real geometry)**.** `ch18_drc_check.py` applies the board's design rules — minimum clearance, minimum track width, hole keep-out — to the actual pad coordinates the Gerber generator emits. The real report:

```
mount(2,2) pad-clearance 2.03mm
mount(38,28) pad-clearance 17.09mm
0 violations
```

**Listing 18.3:** `ch18_drc_check.py output over the real layout.`

Boring output is the point: a clean board's report should be a list of facts that pass. To feel what it catches, move a mounting hole 0.5 mm toward the header pads and re-run — the clearance line turns red the way KiCad's own DRC would.




Placement rationale: header on the left edge (cable exits), the LDO near the input with its decouplers tight, the sensor far from the LDO's warm side, and the analog path short — SENSE to R1 to FILT in one clean run. The bottom face is one unbroken ground pour.

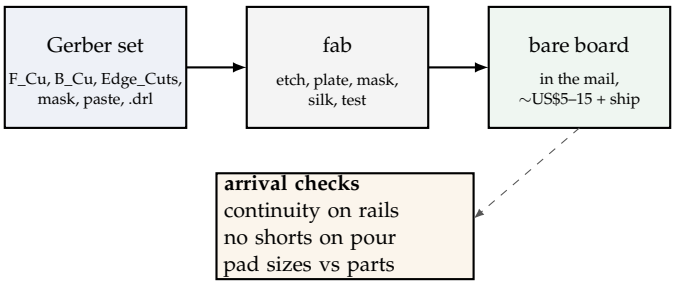
**Figure 18.5:** The routed board, top face: placement and the five tracks that carry every net. Regenerate: `figures/ch18/fig_routed.tex`.

## 18.5 Fab files: the Gerber contract

The deliverable a fab manufactures is a bundle: one file per copper layer, one per mask and silkscreen face, one edge-cuts outline, and an Excellon drill sheet. The chapter's `ch18_gerber_gen.py` writes a real minimal set — F_Cu pads and the six tracks, B_Cu pour, Edge_Cuts, and the drill file — in valid Gerber X2 syntax:

```
%TF.FileFunction,Copper,L1,Top*%
%FSLAX34Y34*%
%MOMM*%
%LPD*%
%ADD10C,0.250*%
%ADD11R,1.800X1.800*%
%ADD12C,1.600*%
%ADD13O,1.300X2.300*%
```

**Listing 18.4:** `ch18_sensor-F_Cu.gbr head -- the header a Gerber viewer reads first.`




The Gerber is the contract the fab signs: every file is a layer or a drill sheet. Our script-generated set is minimal but real — the fab's own DRC will run it again before the copper etches.

**Figure 18.6:** Gerber → fab → bare board → arrival checks. Regenerate: `figures/ch18/fig_gerber.tex`.

Ordering is a design decision too: ~US$5–15 plus shipping gets five copies from the budget fabs, lead time a week or two — *unverified, price-checked 2026-10-11*. The alternative is keeping the board as a design exercise: every skill in this chapter was exercised except the waiting.

When the board arrives: continuity-test the rails, check the pour for shorts, hold the 0805s over their pads to confirm the footprints matched the parts you bought.

**Safety recap.** A bought power supply, never a built one — that rule from ch1 and ch7 stands here too. If you solder this board: iron in its stand between joints, lead-free solder, fan or fresh air, wash hands. A LiPo pack powering the board gets the same rules as ch17: protected cells, no shorts, no punctures.

## 18.6 Common mistakes

**ERC WARNINGS IGNORED**

ERC is free and runs in seconds; every ignored warning becomes a $15 lesson at the fab. Unconnected pins, power-output-pin warnings, label typos — clear them all, or write down why each one is fine.

**SCHEMATIC-FOOTPRINT PINOUT MISMATCH**

The symbol's pin numbers must match the footprint's pad numbers — "pin 3" on a TO-92 is not the same pin on an SOT-23. The netlist carries names, not positions; assign the wrong footprint and the board will politely build the wrong circuit.

**MISSING DECOUPLING AT ICS**

The AMS1117 needs its capacitors *at its pins* — across the board they are decoration. Every IC gets a local 100 nF within a few millimetres; the bulk cap guards the rail at entry.

**TRACE WIDTH VERSUS CURRENT**

A 0.15 mm track is fine for a sensor signal and a fire hazard at 2 A. Check current first, then width: our tracks are 0.25 mm because the whole board draws ~10 mA — if it drew an amp, the answer would be copper area, not a slightly wider line.

**FORGETTING THE GROUND RETURN**

Signals routed neatly while ground wanders: the pour on the bottom face exists so every return is short. On two-layer boards, "route signals, pour ground" beats "route everything."

**SILKSCREEN AS DOCUMENTATION**

Reference designators under parts, polarity marks hidden by the component, no board name or date: the fab prints what you give it. A board you can't identify next month is a board you'll reorder.

**Exercise 18.1** (easy). Why do we run ERC before assigning footprints?

**Exercise 18.2** (easy). Name the five nets on this board and which layer each lives on.

**Exercise 18.3** (medium). The fab charges per board size. Which single dimension change saves the most cost on this layout without losing function?

**Exercise 18.4** (medium). Sketch the keepout rule a mounting hole needs: what clearance to copper, and what goes wrong if a track sneaks through?

**Exercise 18.5** (hard). Argue for a copper-pour on the top face too — when is a two-sided pour the right answer, and what does it cost?

## 18.7 Summary

| Thing | The rule of thumb |
|---|---|
| workflow | schematic → ERC → footprints → PCB → DRC → Gerbers |
| schematic | the meaning lives here; fix bugs here, they're cheapest |
| footprint | value is schematic, body is footprint — choose the body you can solder |
| two layers | signals on top, unbroken ground pour on the bottom |
| DRC | boring reports are correct reports; clear every line |
| Gerbers | the fab's contract — one file per manufactured layer |
| ordering | US$5–15 + shipping, optional — the skills transfer either way |

## Additional exercises

**Exercise 18.6** (easy). List the five Gerber-family files a minimal two-layer board needs.

**Exercise 18.7** (easy). Why 0805 instead of 0402 for this book?

**Exercise 18.8** (easy). Which three checks run when the board arrives, in order?

**Exercise 18.9** (easy). What does the AMS1117's datasheet say the capacitors are for?

**Exercise 18.10** (medium). Design the keepout for the TMP36's area: why give the sensor its own quiet corner, away from the LDO?

**Exercise 18.11** (medium). A track on the bottom face has to cross a signal — on two layers, what's the idiomatic fix that keeps the pour mostly intact?

**Exercise 18.12** (medium). Estimate this board's power dissipation at the LDO and say whether it needs thermal relief or a pour.

**Exercise 18.13** (medium)**.** The header pin-out must match the Tang Nano's GPIO row. Find which J1 pins carry supply vs signal, and what happens if they're swapped.

**Exercise 18.14** (hard)**.** Add the mounting-hole keepout to the DRC rules: write the check as it would appear in `ch18_drc_check.py` and test it on the (2,2) hole.

**Exercise 18.15** (hard)**.** The 1-oz-copper question: at what current does a 0.25 mm top track stop being safe? Give the rule of thumb and the margin this board has.

**Exercise 18.16** (hard)**.** Review a peer-style broken layout: TMP36 under the LDO's heatsink side, C1 across the board, FILT track crossing three pads. List all violations and the fix order.

**Exercise 18.17** (hard)**.** Extend the netlist to a second board revision adding an LED power indicator on +3V3: what nets change, what parts, and which two ERC checks does the change re-trigger?

**Exercise 18.18** (hard)**.** The pour-vs-routed-ground debate: measure (on paper) the return path for U2's ground pin both ways and say why the pour wins for analog.

**Exercise 18.19** (hard)**.** Write the silkscreen policy a two-person team should adopt: what must every board carry to be debuggable in a year?

**Exercise 18.20** (hard)**.** Cost the board end-to-end: fab + shipping + five sensors + passives. Where does the budget break if you order twenty boards?

**Exercise 18.21** (hard)**.** The sensor feeds an FPGA threshold input. Describe the two honest upgrades to a real ADC and what each costs in complexity.

**Exercise 18.22** (hard)**.** A ground loop appears when the board is powered by USB and probed by a mains-referenced scope. Explain the mechanism and the safe fix.

**Exercise 18.23** (hard)**.** Redesign the board on one layer: which net forces a jumper wire, and what does that say about the two-layer convenience premium?

## Hints and answers

**ERC timing (easy, inline)** warnings cost seconds to fix on the schematic and dollars on the board — catch them where they're free.

**Five nets (easy)** +5V, GND, +3V3, SENSE, FILT — supply and ground on the pour/fat tracks, the three signals on top tracks.

**Size change (medium)** shrinking 40 mm to 25 mm saves the most area — the header already forces the width.

**Mounting keepout (medium)** 0.5 mm copper-free ring around the 3.2 mm hole — a track under a screw shorts to the standoff.

**Top pour (hard)** good for RF/EMI or extra return paths — costs etch time, solder-wick risk at pads, and one more place to accidentally short.

**Gerber set (easy)** F_Cu, B_Cu, F/B Mask, Edge_Cuts, drill — plus paste/silk for a nicer board.

**Why 0805 (easy)** hand-solderable by beginners and reworkable with one iron; 0402 needs magnification and still hands.

**Arrival checks (easy)** rail continuity, pour shorts, parts- over-pads fit.

**Capacitor purpose (easy)** LDO stability — the datasheet's minimum 22 $\mu$F-ish guidance keeps the regulator out of oscillation; decoupling also localises noise.

**Sensor keepout (medium)** the LDO's warmth biases a temperature sensor; give TMP36 its own corner and the measured air wins.

**Bottom-face crossing (medium)** a short "jumper track" on the top face lets the pour stay unbroken — vias at both ends.

**LDO heat (medium)** $(5 - 3.3) \times 10\,\text{mA} \approx 17\,\text{mW}$ — trivially cool; the sensor's keepout is precaution, not need.

**Header swap (medium)** supply pins mis-wired: +5V into a signal pin or a floating rail — pin-number the silk and check before powering.

**Keepout check (hard)** distance from (2,2) hole to every pad's edge $\geq$ clearance — implement as a second HOLES loop with pad-radius subtraction.

**Track current (hard)** 0.5–1 A per 0.25 mm at 1 oz (external layer, small rise); this board draws 10 mA — margin $\approx 100\times$.

**Broken layout review (hard)** sensor near LDO (bias), C1 decoupling at distance (noise), FILT crossing pads (shorts) — fix order: placement, then pour, then route.

**Second revision (hard)** new parts: LED + resistor; net change: +3V3 branches to LED-anode, LED-cathode→res→GND; ERC re-checks power-pin connections and unconnected pins.

**Pour vs routed ground (hard)** U2's ground leg: pour path is $\sim$2 mm drop to sheet vs a centimetre-long routed track — lower impedance, less noise pickup on the analog reference.

**Silkscreen policy (hard)** board name+rev+date, refdes on every part, polarity on every polar part, pin-1 marks, and a website/note for who owns it.

**Twenty-board cost (hard)** fab price scales weakly per-board; the sensor ($\sim$US$2–3 each) becomes the dominant cost — budget breaks on the BOM, not the PCB.

**Real ADC upgrades (hard)** (1) a dedicated ADC IC (I2C/SPI) — one part, a bus, real code; (2) the FPGA's LVDS pins as a comparator+SAR logic — free pins, but Verilog calibration work.

**Ground loop (hard)** USB-ground and scope-ground share earth potential through different paths — current flows through your measurement; fix: battery/isolated scope or lift one reference, never defeat the safety earth.

**One-layer redesign (hard)** the GND return forces the jumper: one wire over the top — the 10-cent jumper is why two-layer exists; the premium is cheap.

## Further reading

- KiCad Documentation Team, *Getting Started in KiCad* (version-stamped 10.0.7). docs.kicad.org/10.0/en/getting_started_in_kicad/ — the official walk-through our pipeline follows. Beginner–intermediate. Verified 200, 2026-10-11.
- SparkFun, *Beginner's Guide to KiCad*. learn.sparkfun.com/tutorials/beginners-guide-to-kicad — footprints, ERC/DRC habits, first-board ordering. Beginner. Verified 200, 2026-10-11.
- KiCad Documentation index. docs.kicad.org/ — schematic/PCB editor references for every tool this chapter names. Reference. Verified 200, 2026-10-11.
- JLCPCB. jlcpcb.com — the budget-fab endpoint the Gerber bundle targets; check its live quote for the current price. Verified 200, 2026-10-11.
- Wikipedia, *KiCad*. en.wikipedia.org/wiki/KiCad — the project's history and scope (why it's the free tool that won). Reference. Verified 200, 2026-10-11.
- All About Circuits, *Experiments* (vol. VI) — the build exercises this board descends from. [unverified: 403 this session; prior verified access]

**WATCH INSTEAD**

1. *KiCad for Beginners — Step by Step Tutorial (2025)* — DIY Hideout, 23:37, https://www.youtube.com/watch?v=d9_-lQq8ShE — the whole pipeline this chapter walks, on screen. Matches §18.1. Opened and confirmed 2026-10-11.
2. *KiCad 6 STM32 PCB Design Full Tutorial* — Phil's Lab #65, 1:40:05, https://www.youtube.com/watch?v=aVUqaB0IMh4 — the deep version: a real MCU board start-to-finish including the AMS1117-3.3 we use. Matches §18.2–§18.5. Opened and confirmed 2026-10-11.

## Notes

The pipeline and tool names follow the KiCad 10.0.7 docs; footprint choices and DRC habits follow SparkFun's guide; the sensor-board schematic, netlist, DRC check, and Gerber bundle are this book's own generated artifacts (all listed outputs are real runs). Fab pricing is honest-marked and date-stamped. No KiCad GUI was run in this session — the design files are generated by the chapter's own scripts and the KiCad steps are described-not-run.

CHAPTER 19

# Microcontrollers and the road ahead

You built a CPU; a microcontroller is one shrink-wrapped. Same fetch–decode–execute idea — plus program flash, RAM, and every peripheral anyone ever needed baked into one silicon slab that sells for pocket change. This final chapter is a hand-off: the honest MCU-versus-FPGA decision, the sensor board's natural MCU variant, and the map to where the next books take over — because cpu8 is a beginning, not an end. This chapter asks for nothing but reading: no sims, no board, just judgment.

**BEFORE YOU START**

Chapters 13 and 16–18. *Recap:* a microcontroller (MCU) is a small CPU, memory and peripherals on one chip.

**AFTER THIS CHAPTER YOU CAN**

- explain what an MCU is and when to choose it over an FPGA;
- read a sensor with an MCU;
- explain what Arduino hides from you;
- say what the next steps are and which book to read.

**HOW TO THINK ABOUT THIS**

A microcontroller packs a small CPU, memory and input/output on one chip, so many problems that once needed a custom design now need only software. Knowing both tools lets you choose: an FPGA for parallel, timing-exact work, an MCU for flexible, cheap control.

## 19.1 An MCU is the same idea, shrink-wrapped

Our cpu8 was 302 cells: a program counter, an instruction ROM, a register file, an ALU, an output port. An AVR ATmega328 — the Arduino UNO's brain — is those same pieces plus flash you can rewrite ten thousand times, 2 KB of real RAM, three timers, an ADC, UART, SPI, $I^2C$, and an interrupt controller, all on one die.

Everything you met is there. The accumulator is a register file. The UART's shift register is our ch16 FSM drawn the same way. The timers are prescalers. The ADC is a comparator

plus a SAR state machine — our ch18 "threshold input," done properly. Nothing is new in kind; everything is new in polish.

**Example 19.1** (Where the $1 question is already settled)**.** An ATtiny10 costs about sixty cents at quantity — *unverified, price-checked 2026-10-11*. For almost every "blink something" or "read a sensor" product, the MCU wins before the argument starts: one chip, minutes of development, power-down current in the nanoamps.

**Checkpoint 19.1** If a $0.60 part can do the job and is easier to program, the $20 FPGA needs a reason — it isn't a flex, it's a tool with a niche.

## 19.2 The honest decision

The difference isn't *which is better* — it's *what the hardware is*. An MCU runs your program one instruction at a time on a fixed machine. An FPGA *becomes* whatever machine you draw, and a hundred copies of it if you want them. Pick by what the problem demands, not by what's flashier.

| need | pick an MCU when | pick an FPGA when |
|---|---|---|
| latency | millis is fine | cycle-exact timing, µs gaps |
| work | one thing at a time | many channels truly in parallel |
| IO | UART/I2C/SPI built in | any protocol you draw |
| cost | pennies at volume | the hardware IS the product |
| dev time | minutes to blink | weeks to verify |
| power | sleep-to-wake fast | high; never beats an MCU asleep |

**Figure 19.1:** The decision table: pick by the problem's shape, not the part's reputation. Regenerate: `figures/ch19/fig_decision.tex`.

**Example 19.2** (Three products, three picks)**. (a) LED strip controller** — thousands of updates per second is fine for an MCU's `delay`; pick the $1 MCU. **(b) Oscilloscope trigger** — a 50 ns edge can't wait for a fetch cycle; pick the FPGA, hardware detects the edge in parallel. **(c) Toy that beeps and remembers five buttons** — the MCU sleeps at a microamp and wakes on a keypress; the FPGA's static power loses instantly. Two-thirds of hobby projects are MCU-shaped; the FPGA's niche is precision and parallelism.

## 19.3 The sensor board, with an MCU

ch18's board feeds FILT to an FPGA pin configured as a comparator — the honest-but-crude path. Its natural MCU variant wires FILT to an ADC pin instead, and the whole design turns into firmware:

```
void setup() { Serial.begin(9600); }
void loop() {
  int raw = analogRead(A0); // 0..1023 across 0..Vcc
  float volts = raw * 5.0f / 1024.0f;
  float celsius = (volts - 0.5f) * 100.0f; // TMP36's law
  Serial.println(celsius, 1); // degC, one decimal
  delay(500);
}
```

**Listing 19.1:** The sensor board's MCU variant, as an Arduino sketch (shown, not run here — the same sketch the docs publish verbatim).

Nine lines versus a Verilog module, an FSM, and a UART. That's the point: for this problem, the MCU is unambiguously right. The FPGA version's justification is different — you build the machine that does the same thing *and* learn what the MCU hides.

## 19.4 What Arduino hides

The sketch's `analogRead(A0)` is three real operations: write the ADC's MUX register to select A0, start a conversion and wait for its flag, read a result register. The library call *is* those registers — the datasheet's ADC chapter is the authoritative version, not the tutorial. Same for `delay(500)`: it's a timer peripheral counting hardware clocks — our ch16 prescaler wearing a friendly name.

**Example 19.3** (Same debounce, two languages)**.** A button that bounces for ∼10 ms: in C you poll a pin and count milliseconds; in Verilog you clock a shift register and act on a stable level.

```
// C on the MCU: sample, count, confirm
if (digitalRead(BTN) == level) {
  delay(12); // ride out the bounce
  if (digitalRead(BTN) == level) // still there? real press
     act();
}
// Verilog on the FPGA: a 12ms filter is a shift register
always @(posedge clk)
  sr <= {sr[15:0], btn}; // 16 taps at ~1kHz
wire stable = (&sr) | (~|sr); // all-ones or all-zeros
```

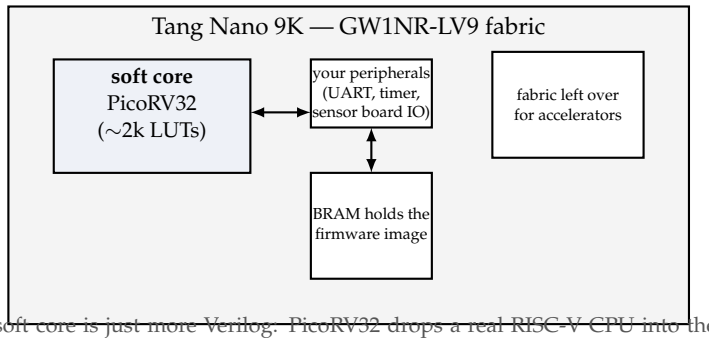
**Listing 19.2:** Debounce both ways (shown, not run): the C that waits, the Verilog that filters.

The C version spends CPU time *waiting*; the Verilog version adds sixteen flip-flops and never waits at all. Neither is wrong — they're the two cost models you'll choose between for the rest of the book's road.

**Example 19.4** (Reading a datasheet's front page)**.** The ATmega328's page one offers: "20 MIPS at 20 MHz, 32 KB flash, 2 KB SRAM, 1 KB EEPROM, 10-bit ADC, 6 PWM channels." Read honestly: fast enough for anything a sensor does, RAM so small that `Serial.println` of a float spends tens of bytes of it, one ADC multiplexed across six channels — and "20 MHz" is a *ceiling*, not a default. The datasheet answers what the tutorial doesn't: what's real versus what's abstracted.

## 19.5 Soft cores: the CPU inside the FPGA

There's a third option that makes the decision interesting: run a real CPU *on* the FPGA. A soft core is a processor written in an HDL and dropped into fabric — PicoRV32 is a complete RV32I core that fits in roughly two thousand LUTs, small enough to sit beside our cpu8's peripherals on the Tang Nano 9K. The payoff is both-ways freedom: C code where convenience wins, hardware accelerators where cycles matter, in one chip.




A soft core is just more Verilog. PicoRV32 drops a real RISC-V CPU into the same LUTs our 302-cell cpu8 used — the difference is the toolchain runs on it, so C code executes inside the FPGA alongside the hardware you still write.

**Figure 19.2:** PicoRV32 inside the FPGA: firmware runs on real RISC-V, your Verilog peripherals attach beside it. Regenerate: `figures/ch19/fig_softcore.tex`.

**Example 19.5** (The integration, honestly scoped)**.** Dropping a soft core in is a short commit: instantiate the core, attach BRAM holding the firmware, wire a UART. The full picture:

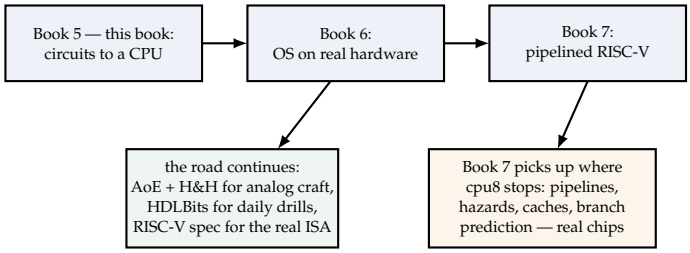
```
picorv32 core ( // RV32I in ~2k LUTs
  .clk(clk27), .resetn(rst_n),
  .mem_valid(), .mem_ready(), .mem_addr(),
  .mem_wdata(), .mem_rdata(), .mem_wstrb());
// memory: firmware BRAM + UART-mapped registers
```

**Listing 19.3:** The soft-core shape (shown; a real PicoRV32 build is Book 7's material — its repo is cited).

What this buys: gcc-compiled C running on silicon you shaped — an RV32I instead of our 10-opcode toy, but the same fetch–decode–execute you can now see through.

## 19.6 The road ahead

This book ends at the edge of three continuations. Book 7 takes the machine you built to a real pipelined RISC-V — hazards, forwarding, caches, branch prediction: every shortcut our two-state FSM didn't need. The analog craft continues in the two classic texts: *The Art of Electronics* for the deep bench wisdom, *High-Speed Digital Design* (H&H) for the signal-integrity world where wires stop being ideal. And the daily drill is HDLBits — short Verilog exercises that turn the writing muscle into a habit.




Where this book hands you off: the sensor board and cpu8 are the bridge — Book 7 takes the same instruction-fetch machine to a real RV32I, and the side texts carry the analog and drill work.

**Figure 19.3:** Where each path continues: Book 7 for the CPU, the texts for analog, HDLBits for the drills. Regenerate: `figures/ch19/fig_map.tex`.

The arc this book ran: charge and voltage to gates and adders, adders to an ALU, an ALU to a datapath, a datapath to a CPU that ran Fibonacci on real silicon — then breadboards, a real PCB design, and now the honest choice of what to build next. The capstone wasn't the UART frame; it was that none of it is magic anymore. Machine the next books build is bigger, but the shape is the one you've already drawn.

## 19.7 Common mistakes

**"MCU EQUALS FPGA"**

They answer different questions: *what program runs* versus *what machine exists*. Saying "the FPGA is more powerful" like it's the upgrade misses that it solves a different problem — and pays for it in power, cost, and weeks of development.

**ARDUINO'S ABSTRACTIONS AS THE TRUTH**

`digitalWrite` hides direction registers, port writes, and clock enables; `delay` hides a whole timer peripheral. Fine for a toy — wrong to assume the hardware is simpler than the library makes it look. The datasheet is the truth; the library is a convenience.

**POWER ASSUMPTIONS IMPORTED FROM SKETCHES**

An Arduino running a blink at 16 MHz *awake* draws tens of milliamps; the same chip asleep draws microamps. Sketches never sleep, so beginners think MCUs are power-hungry — read the sleep modes before judging battery life.

**CHOOSING THE FASHIONABLE ANSWER**

"FPGA because it's cooler" is a project killer: the hobbyist version of this mistake is a two-week build for a task a sixty-cent chip solves in an afternoon. Pick the niche that fits, not the one that impresses.

**TREATING "SOFT CORE" AS FREE**

PicoRV32 in the fabric is not "an FPGA and an MCU for one price"— the LUTs, the BRAM for firmware, and the debug effort are all spent. It's the right answer when you need both hardware and convenience in one chip, not a freebie.

**FORGETTING THE SENSOR CHAIN**

On a real product the MCU's job ends at the ADC register — the op-amps, filters, and protection circuits the earlier chapters built still apply. Digital polish doesn't exempt the analog front-end.

**Exercise 19.1** (easy)**.** Name three peripherals on a typical MCU we never built — and which earlier chapter's circuit is hiding inside each.

**Exercise 19.2** (easy)**.** Why does `analogRead` count as a software convenience on top of real hardware rather than a separate function?

**Exercise 19.3** (medium)**.** A project needs 96 LEDs updated at 1 kHz and a button read every 200 ms. MCU or FPGA, and which two rows of the table decide it?

**Exercise 19.4** (medium)**.** The sensor board's MCU variant has one real limitation the FPGA version doesn't — what is it, and does it matter for a room thermometer?

**Exercise 19.5** (hard)**.** Pitch or reject: "put a soft core on the sensor board so the MCU and FPGA are one chip." State your criteria, then the answer.

## 19.8 Summary

| Thing | The rule of thumb |
|---|---|
| MCU | the same fetch–decode–execute, plus every peripheral shrink-wrapped |
| pick by | latency, parallelism, IO, cost, dev time, power — the problem's shape |
| Arduino | libraries are conveniences; the datasheet is the truth |
| soft core | real CPU inside fabric — both-ways freedom, real costs |
| the road | Book 7 for the real pipeline, texts for analog, HDLBits for drills |
| the arc | charge → gates → ALU → CPU → board — none of it is magic anymore |

## Additional exercises

**Exercise 19.6** (easy)**.** Which three hardware pieces on an MCU does our cpu8 already have in miniature?

**Exercise 19.7** (easy)**.** The honest reason most toys are MCUs: name it.

**Exercise 19.8** (easy)**.** What does a soft core consume that a hard MCU doesn't?

**Exercise 19.9** (medium)**.** A data-acquisition product needs 8 channels sampled truly simultaneously at 1 MHz — which side of the table wins, and which two rows are decisive?

**Exercise 19.10** (medium)**.** Explain why an MCU "sleeps at microamps" claim and a "sketches never sleep" observation aren't a contradiction — and which peripheral makes sleep useful.

**Exercise 19.11** (medium)**.** Read any MCU datasheet's front page: list the three numbers that set its real ceiling, and the marketing number to discount.

**Exercise 19.12** (hard)**.** Cost the two sensor-board builds end-to-end (MCU variant vs FPGA variant) in parts AND developer time — where's the honest break- even?

**Exercise 19.13** (hard)**.** Design the ch18 board's next revision for an MCU: which nets change, which peripheral does the header now target, and which part can the board drop?

## Hints and answers

**Peripherals (easy, inline)** timer/PWM (our prescaler), ADC (our threshold, done properly), UART (our ch16 TX/RX), I2C/SPI, interrupt controller.

**analogRead truth (easy)** it writes real registers and waits on a real flag — the library names the mechanism, it doesn't replace it.

**96-LED project (medium)** FPGA — timing precision and parallel outputs are the problem; cost and dev time still argue MCU, but the spec's gap wins.

**MCU variant limit (medium)** the ADC's resolution and sample rate cap the signal — irrelevant for a room thermometer, decisive for faster signals.

**Soft-core pitch (hard)** accept if you need one chip doing both real-hardware timing and convenient firmware — reject if either side is unnecessary; the costs are LUTs, BRAM, and bring-up.

**Miniature pieces (easy)** program counter, register file, ALU — plus ROM and an IO port if we count peripherals.

**Why MCUs win toys (easy)** unit cost at volume plus dev speed — the decision table's last two rows.

**Soft-core cost (easy)** fabric LUTs, BRAM for firmware, and your debug hours — nothing is free.

**8-channel DAQ (medium)** FPGA — latency and parallelism are decisive; an MCU's ADC multiplexes one channel at a time.

**Sleep vs sketch (medium)** both true: the part can sleep deeply, the sketches keep it awake — the fix is sleep modes and wake-on-interrupt, not a different chip.

**Datasheet ceiling (medium)** flash/RAM sizes and real clock rate set limits; headline "20 MIPS" is the marketing number at max clock.

**Build cost (hard)** MCU: $3–8 parts + hours; FPGA: board already owned + days of verification — break-even is whether you're building one unit or a product.

**MCU revision (hard)** header drops a pin (no FPGA threshold input), targets ADC+GPIO; the FILT RC stays (analog front-end is still real work); R2's bleed stays.

## Further reading

- IBM Think, *When to use FPGA vs Microcontroller*. `ibm.com/think/topics/fpga-vs-microcontroller` — the comparison this chapter's decision table draws. Intro. Verified 200, 2026-10-11.
- Arduino Documentation, *Blink* built-in example. `docs.arduino.cc/built-in-examples/basics/Blink/` — the canonical first sketch, shown honestly as a sketch (not hardware). Reference. Verified 200, 2026-10-11.
- YosysHQ, *PicoRV32 — a size-optimized RISC-V CPU*. `github.com/YosysHQ/picorv32` — the soft-core named in this chapter; real RV32I in ∼2k LUTs. Reference. Verified 200, 2026-10-11.
- HDLBits. `hdlbits.01xz.net` — the daily Verilog drill this chapter recommends. Reference. Verified 200, 2026-10-11.
- Wikipedia, *Microcontroller*. `en.wikipedia.org/wiki/Microcontroller` — what the parts are. Reference. Verified 200, 2026-10-11.
- Wikipedia, *RISC-V*. `en.wikipedia.org/wiki/RISC-V` — the ISA Book 7 teaches. Reference. Verified 200, 2026-10-11.
- Nandland, *FPGA vs. Microcontroller vs. ASIC*. `nandland.com/lesson-2-fpga-vs-micro-vs-asic/` — the three-way comparison done right. [unverified: 403 this session; prior verified access]

**WATCH INSTEAD**

1. *FPGA vs Microcontroller vs Processor — Choosing the Right Hardware* — Uplatz, 7:48, `https://www.youtube.com/watch?v=i1ObzbkVfSA` — the decision in a short video. Matches §19.2. Opened and confirmed 2026-10-11.
2. *What's an FPGA?* — Charles Clayton, 1:25, `https://www.youtube.com/watch?v=iHg0mmIg0UU` — the two-minute answer to the question this whole book builds. Matches §19.1. Opened and confirmed 2026-10-11.

## Notes

MCU-vs-FPGA framing follows IBM's explainer plus this book's own cost model; Arduino sketch and peripheral abstractions per Arduino's docs; PicoRV32 per its repo. Part counts and the ATtiny price are honest-marked and date-stamped. No new sims or synth runs — this chapter is the hand-off: the tools it names were exercised earlier in the book or are Book 7's material.

# Bibliography


[1] J. Sannemo, *Principles of Algorithmic Problem Solving*, 2025. https://usaco.guide/PAPS1.pdf

[2] T. R. Kuphaldt, *Lessons in Electric Circuits*, Vol. I: DC, 5th ed. (minor revisions 2021–2023). Free under the Design Science License. https://www.allaboutcircuits.com/textbook/direct-current/

[3] T. R. Kuphaldt, *Lessons in Electric Circuits*, Vol. III: Semiconductors, 5th ed. https://www.allaboutcircuits.com/textbook/semiconductors/

[4] T. R. Kuphaldt, *Lessons in Electric Circuits*, Vol. IV: Digital, 4th ed. https://www.allaboutcircuits.com/textbook/digital/

[5] S. J. Ling, W. Moebs, J. Sanny et al., *University Physics Volume 2*, OpenStax, Rice University, 2016 (web edition updated regularly). CC BY 4.0. https://openstax.org/details/books/university-physics-volume-2

[6] SparkFun Electronics, "How to Use a Multimeter", learn.sparkfun.com tutorial. CC BY-SA. https://learn.sparkfun.com/tutorials/how-to-use-a-multimeter (accessed 2026-10-10)

[7] SparkFun Electronics, "How to Use a Breadboard", learn.sparkfun.com tutorial. CC BY-SA. https://learn.sparkfun.com/tutorials/how-to-use-a-breadboard (accessed 2026-10-10)

[8] ngspice open-source circuit simulator, version 47 (released 2026-08-11). https://ngspice.sourceforge.io/ (accessed 2026-10-10)

[9] Project Apicula documentation and Sipeed Tang Nano 9K board page (GW1NR-LV9); Indonesian street price unverified, see docs/unverified.md. https://github.com/YosysHQ/apicula (accessed 2026-10-10)

[10] MIT OpenCourseWare, *6.002 Circuits and Electronics, Spring 2007* (Agarwal & Lang course; L1–L3 cover KVL/KCL, nodal analysis, Thévenin). CC BY-NC-SA 4.0. https://ocw.mit.edu/courses/6-002-circuits-and-electronics-spring-2007/ (accessed 2026-10-10)

[11] P. Horowitz and W. Hill, *The Art of Electronics*, 3rd ed., Cambridge University Press, 2015. Cited as the reader's next step for real op-amp and regulator design.

[12] S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture*, 2nd ed., Morgan Kaufmann, 2012 (companion materials: pages.hmc.edu/harris/ddca/).

[13] *fpga4fun.com* — free FPGA tutorials and projects, fpga4fun.com (site unreachable at check time, 2026-10-11; content cited on prior verified access).

[14] *HDLBits* — Verilog practice problems, hdlbits.01xz.net.

[15] *Project Apicula documentation and source*, YosysHQ, https://github.com/YosysHQ/apicula (accessed 2026-10-10).

[16] L. Lushay, *Lushay Labs Tang Nano series tutorials*, https://learn.lushaylabs.com/tang-nano-series/ (accessed 2026-10-10).

# Index


`$display`, 125
`$finish`, 125
`$monitor`, 125
0.7 V model, 53

ALU, 98, *see* arithmetic-logic unit
always block (Verilog), 114
ampere, 3
Arduino abstraction, 213
arithmetic-logic unit, 151
assembler, 181
assign (Verilog), 114

battery
    short, 12
BJT, 69
blocking assignment, 114
body diode, 71
BRAM, 139, 166
breadboard, 190
    layout, 191
    limits, 195
bus, 98, 166

capacitance, 42
capacitor, 42
case equality, 126
case inequality, 126, 132
charge, **2**
circuit
    closed loop, 5
CMOS, 67
    power, 68
combinational logic, 94
comparator, 77
comparator (digital), 97
conductance, 33
control unit, 169, 180
conventional current, **7**, 13
coulomb, 2
counter, 104
CPU
    limitations, 183
current, **3**

D flip-flop, 103
D latch, 102
datapath, 169
De Morgan's laws, 87
debugging, 193
decoder, 94, 180
decoupling, 81
derating, 20
design under test, *see* DUT
differential amplifier, 74
diode
    models, 53
don't-care, 97
dropout, 80
DUT, *see* design under test

electric field, **2**
electron flow, 7
encoder, 97
equivalent resistance, 31
ESD, 12
    gate, 70

fetch-decode-execute, 178
fetch-execute cycle, 169
finite state machine, 105
flag, 151
    carry, 152
    negative, 152
    overflow, 152
    zero, 152
flyback, 45
footprint, 203
forward voltage, 20
FPGA, 139

full adder, 95

Gerber, 205
ground, 2

half adder, 95
hardware description language, 112
HDL, *see* hardware description language
hexadecimal, 86
hold time, 104
hysteresis, 77

Icarus Verilog, 115
ideal diode, 53
inductance, 45
inductor, 45
inferred latch, 129
instruction set architecture, 178
inverter (logic), 66
IOB, 139
ISA, 151, 178

KiCad, 202
Kirchhoff's current law, 33
Kirchhoff's voltage law, 33

learning map, 215
level shifter, 70
linear element, 18
LUT, 139

MCU, 211
MCU vs FPGA, 212
Mealy machine, 105
metastability, 108
meter loading, 37
metric prefixes, 4
microcontroller, 211
milliamp-hour, 11
module (Verilog), 113
Moore machine, 105
MOSFET, 65
    S model, 65
    SR model, 65
multimeter, **7**, 13, 193
    measuring current, 7
    measuring voltage, 7
multiplexer, 94

negative feedback, 74
netlist, 22
ngspice, 22
NMOS, 65
node, **5**
node-voltage method, 34
noise margin, 67
non-blocking assignment, 114
nonlinear element, 20
Norton equivalent, 35

ohm, 18
Ohm's law, 3, 18
op-amp, 74
operating point, 22

parallel, 21, 31
parameter (Verilog), 115
PCB, 202
PicoRV32, 214
PMOS, 67
port (Verilog), 113
potential difference, 2
power, **19**
    dissipation, 19
power rating, 19
program counter, 169

race condition (simulation), 129
RAM, 166
register, 104
    file, 165
register file, 165
relaxation oscillator, 79
resistance, **3**
resistor, **3**
    colour code, 3
ringing, 48
ripple-carry adder, 96
RISC-V, 184
ROM, 166

schematic, **5**
    as topology, 13
self-checking testbench, 131
sensor board, 202
sensor board, MCU variant, 213
series, 21, 31

setup time, 104
shunt regulator, 57
simulation, 22
soft core, 214
SR latch, 102
state, 102
static discipline, 67
switching regulator, 80

testbench, 115, 125
Thévenin equivalent, 35
threshold voltage, 65
time constant, 42
    RL, 45
topology, 6
tri-state, 166
truth table, 86
two's complement, 86
two-layer PCB, 204

UART, 182
universal gate, 87
unknown value, *see* x propagation

V-I characteristic, 18
Verilog, 86, 112
voltage, **2**
voltage regulator, 80
voltage-transfer characteristic, 66

water analogy, 4
watt, 19
waveform, 132

x propagation, 127

zener diode, 57